HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

BÁO CÁO TIỂU LUẬN FINAL

ĐỀ TÀI: NGHIÊN CỨU VÀ TRIỂN KHAI CÁC HỆ THỐNG THÔNG MINH – TỪ HỌC MÁY, CNN, RNN ĐẾN NEO4J KNOWLEDGE GRAPH, RAG, HỆ ĐA TÁC TỬ VÀ MẠNG NƠ-RON ĐỒ THỊ

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 11/2026

Mục lục

Danh mục Hình vẽ **v**

Danh mục Bảng biểu **x**

Mở đầu **1**

1 Lịch sử phát triển của trí tuệ nhân tạo **3**
1.1 Tiền đề trước năm 1956 4
1.2 Dartmouth và AI ký hiệu (1956 – 1974) 4
1.3 Các trường phái tư tưởng 4
1.4 Hai mùa đông AI (1974 – 1993) 5
1.5 Học máy thống kê (1993 – 2011) 6
1.6 Cuộc cách mạng học sâu (2012 – 2017) 6
1.7 Transformer và mô hình ngôn ngữ lớn (2017 – 2024) 7
1.8 Kỷ nguyên tác tử (2025 – 2026) 7
1.9 Bài học từ lịch sử 8

2 Các kỹ thuật học máy cơ bản **9**
2.1 Cơ sở lý thuyết 9
2.1.1 Học có giám sát và quy trình chống rò rỉ 9
2.1.2 Hồi quy tuyến tính và hồi quy logistic 9
2.1.3 Cây quyết định và rừng ngẫu nhiên 11
2.1.4 Máy vector hỗ trợ 11
2.1.5 Gradient boosting 12
2.1.6 MLP và lan truyền ngược 12
2.1.7 Độ đo đánh giá 13
2.2 Thiết lập thực nghiệm 14
2.3 CDC Diabetes: sàng lọc nguy cơ tiểu đường 14
2.3.1 Nguồn và kích cỡ 15
2.3.2 Phân bố và nhận xét 15
2.3.3 Tiền xử lý chống rò rỉ 16
2.3.4 Kết quả và phân tích lỗi 16
2.4 Vietnam Housing: ước lượng giá nhà 18
2.4.1 Nguồn và kích cỡ 18
2.4.2 Phân bố và nhận xét 18

2.4.3 Tiền xử lý chống rò rỉ . . . 19
2.4.4 Kết quả và phân tích lỗi . . . 19
2.5 UIT-VSFC: cảm xúc phản hồi sinh viên . . . 22
2.5.1 Nguồn và kích cỡ . . . 22
2.5.2 Phân bố và nhận xét . . . 23
2.5.3 Tiền xử lý chống rò rỉ . . . 23
2.5.4 Kết quả và phân tích lỗi . . . 23
2.6 MLP trên NumPy, Keras và PyTorch . . . 26
2.7 Tổng kết và triển khai . . . 28

3 Mạng nơ-ron tích chập 29
3.1 Cơ sở lý thuyết . . . 29
3.1.1 Phép tích chập . . . 29
3.1.2 Pooling và vùng cảm nhận . . . 30
3.1.3 Chuẩn hóa theo lô . . . 30
3.1.4 Kết nối tắt . . . 30
3.1.5 Tích chập tách biệt chiều sâu . . . 31
3.1.6 Các kiến trúc thực nghiệm . . . 31
3.1.7 Giải thích dự đoán bằng Grad-CAM . . . 31
3.2 Dữ liệu . . . 33
3.2.1 SVHN . . . 33
3.2.2 EuroSAT RGB . . . 34
3.2.3 PathMNIST . . . 35
3.3 Hiện thực trên ba nền tảng . . . 37
3.4 Đánh giá và so sánh . . . 40
3.4.1 Thiết lập . . . 40
3.4.2 Đường cong học . . . 41
3.4.3 Đánh đổi độ đúng và chi phí . . . 41
3.4.4 Phân tích lỗi . . . 43
3.4.5 Giải thích bằng Grad-CAM . . . 44
3.4.6 So sánh nền tảng . . . 44
3.5 Triển khai . . . 46

4 Mạng nơ-ron hồi quy 47
4.1 Cơ sở lý thuyết . . . 47
4.1.1 Tensor chuỗi và RNN cơ bản . . . 47
4.1.2 BPTT, tiêu biến và bùng nổ gradient . . . 47
4.1.3 LSTM và GRU . . . 48
4.1.4 BiLSTM và recurrent dropout . . . 50

4.2 Dữ liệu và tiền xử lý . . . 50
4.2.1 Quy trình chung, không rò rỉ . . . 50
4.2.2 VN30: FPT, VCB, HPG, VIC . . . 51
4.2.3 Thời tiết Hà Nội theo giờ . . . 51
4.2.4 Household Power Consumption . . . 53
4.3 Hiện thực . . . 53
4.4 Đánh giá . . . 56
4.4.1 Kết quả một bước . . . 56
4.4.2 Gradient trên mô hình thật . . . 57
4.4.3 Dự báo nhiều bước và khoảng dao động . . . 58
4.4.4 Phân tích lỗi . . . 61
4.5 Triển khai . . . 61

5 Knowledge Graph, Neo4j, RAG và Agent 63
5.1 Biểu diễn tri thức bằng đồ thị . . . 63
5.2 RAG và phân mảnh văn bản . . . 65
5.3 GraphRAG và tác tử ReAct . . . 66
5.4 Thiết kế đồ thị trong Neo4j . . . 67
5.5 Ba bộ dữ liệu . . . 70
5.5.1 Hetionet: tiểu đồ thị đái tháo đường . . . 70
5.5.2 VN30: công ty, ngành, chỉ số và văn bản . . . 71
5.5.3 HotpotQA: truy xuất đa bước . . . 73
5.6 Cài đặt truy xuất trên Neo4j . . . 74
5.7 Đánh giá truy xuất . . . 76
5.8 Đánh giá sinh với ba LLM . . . 78
5.9 Tác tử ReAct và so sánh ba hệ . . . 80
5.10 Triển khai và kết luận chương . . . 83

6 Hệ đa tác tử 85
6.1 Mô hình hình thức của hệ đa tác tử . . . 85
6.1.1 Dec-POMDP . . . 85
6.1.2 Markov game và cân bằng Nash . . . 86
6.1.3 Kiến trúc nhận thức BDI . . . 86
6.2 Kiến trúc phối hợp và giao thức . . . 87
6.2.1 Bốn cấu trúc phối hợp . . . 87
6.2.2 Giao thức trao đổi . . . 87
6.2.3 Ba framework phổ biến . . . 88
6.3 Dữ liệu . . . 89
6.4 Cài đặt . . . 91

6.4.1 Tác tử là generator 91
6.4.2 Ba kiến trúc 91
6.5 Đánh giá 94
6.5.1 Thiết lập đo 94
6.5.2 Phân tích 94
6.6 Kịch bản minh họa: hội chẩn đa chuyên khoa 97
6.7 Tóm tắt chương 98

7 Deep Graph và mạng nơ-ron đồ thị 99
7.1 Cơ sở toán học 99
7.2 Ba đồ thị thực nghiệm 101
7.3 Hiện thực: GCN từ đầu và PyTorch thuần 103
7.4 Đánh giá ba mô hình trên ba đồ thị 106
7.4.1 So sánh với đường cơ sở 106
7.4.2 Over-smoothing theo độ sâu 108
7.4.3 Biểu diễn học được: t-SNE 109
7.5 Chống rò rỉ dữ liệu trên đồ thị 109
7.6 Dự đoán liên kết trên Hetionet từ Neo4j 111
7.7 Triển khai và tóm tắt 112

8 Tác tử học tăng cường và tác tử dựa trên luật 114
8.1 Mô hình hóa 114
8.2 Thí nghiệm A–E 115
8.3 Trả lời mười câu hỏi 117
8.4 Thay đổi môi trường 118
8.5 Tác tử cải tiến 119

9 Sản phẩm Web App tổng hợp 121
9.1 Kiến trúc 121
9.2 Các trang chức năng 122
9.3 Kiểm thử 126

Kết luận 127

Tài liệu tham khảo 129

Danh mục Hình vẽ

1.1 Mười giai đoạn phát triển của AI. Chiều cao cột chỉ minh họa định tính mức kỳ vọng và đầu tư, không phải số liệu đo. 3
1.2 Năm trường phái của AI: ý tưởng trung tâm, các hệ thống kinh điển và xu hướng hội tụ trong các hệ hiện đại. 5
1.3 Chu kỳ kỳ vọng dành cho AI từ 1950 đến 2026. Đường cong là minh họa định tính; năm của các sự kiện là thật. 6
1.4 Vòng lặp của một tác tử: LLM lập kế hoạch, gọi công cụ, nhận quan sát rồi tiếp tục cho đến khi đạt mục tiêu. 8

2.1 Quy trình bảy bước áp dụng cho cả ba dataset; val và test chỉ đi qua biến đổi đã fit trên train 10
2.2 Trái: đường hồi quy và phần dư (đoạn đỏ) mà MSE cực tiểu. Phải: sigmoid và ngưỡng quyết định 10
2.3 Một cây quyết định và rừng tổng hợp nhiều cây trên các mẫu bootstrap 11
2.4 SVM lề mềm: siêu phẳng, hai biên lề, vector hỗ trợ và một mẫu lọt vào lề 12
2.5 Gradient boosting: mỗi cây nông học phần dư $r^{(m)}$ của tổng các cây trước 12
2.6 MLP hai tầng ẩn và hai chiều truyền: thuận tính đầu ra, ngược nhân chuỗi đạo hàm 13
2.7 CDC Diabetes: nguy cơ tăng theo tuổi, phân bố BMI của nhóm bệnh lệch phải 15
2.8 CDC Diabetes: so sánh mô hình và các đặc trưng XGBoost dựa vào nhiều nhất 17
2.9 CDC Diabetes: bốn đường ROC gần trùng nhau; ở ngưỡng đã chọn, gần một nửa ca dương vẫn bị bỏ sót 18
2.10 Vietnam Housing: giá trải trong khoảng hẹp và có dấu hiệu bị cắt ở hai đầu 19
2.11 Vietnam Housing: so sánh mô hình và các đặc trưng XGBoost dựa vào nhiều nhất 21
2.12 Lỗi của XGBoost: dự đoán co về giữa, nhà rẻ bị đoán cao, nhà đắt bị đoán thấp 22
2.13 UIT-VSFC: lớp trung tính rất hiếm, câu ngắn 23
2.14 UIT-VSFC: so sánh mô hình và lỗi của XGBoost 25
2.15 Mất mát validation theo epoch của seed cuối trên ba nền tảng . . . 27

3.1 Ví dụ tính tích chập không padding: kernel Sobel phản hồi mạnh ở nơi cường độ đổi theo chiều ngang . . . 29
3.2 Vùng cảm nhận lớn dần theo độ sâu; ba con số tính từ đúng dãy tầng của các mô hình trong chương . . . 30
3.3 Tích chập chuẩn và tích chập tách biệt chiều sâu, cùng kích thước đầu vào và đầu ra . . . 31
3.4 Khối dựng của CNN cơ bản, ResNet-18 và MobileNetV2 . . . 32
3.5 ResNet-18 trên ảnh SVHN: mỗi lát là một kênh đặc trưng, kích thước ghi đúng theo mô hình . . . 32
3.6 Luồng tính Grad-CAM; ô bản đồ nhiệt ở đây chỉ để minh họa . . . 33
3.7 Ảnh thật của SVHN, mỗi cột một chữ số. Nhiều ảnh lẫn chữ số bên cạnh ở hai mép . . . 34
3.8 Phân bố lớp và cường độ điểm ảnh của SVHN . . . 34
3.9 Ảnh thật của EuroSAT: lớp phân biệt nhau bằng kết cấu nhiều hơn hình dạng . . . 35
3.10 Phân bố lớp và cường độ điểm ảnh của EuroSAT . . . 35
3.11 Ảnh thật của PathMNIST: chín lớp mô cùng tông hồng tím của thuốc nhuộm . . . 36
3.12 Phân bố lớp và cường độ điểm ảnh của PathMNIST . . . 36
3.13 F1 macro trên test: khoảng cách lớn nhất nằm giữa mô hình tuyến tính và CNN . . . 41
3.14 Độ đúng và loss theo epoch. SVHN và EuroSAT theo dõi trên test (không dùng để chọn mô hình), PathMNIST theo dõi trên val . . . 42
3.15 Chi phí của từng kiến trúc . . . 42
3.16 Ma trận nhầm lẫn của mô hình tốt nhất, chuẩn hóa theo hàng (%) . 43
3.17 Lỗi trên PathMNIST tập trung vào vài lớp . . . 43
3.18 Ảnh test bị mô hình tốt nhất đoán sai với độ tin cao nhất, ghi lớp thật→lớp đoán (xác suất). Ba hàng lần lượt là SVHN, EuroSAT, PathMNIST . . . 44
3.19 Grad-CAM trên ảnh test được đoán đúng; mỗi hàng một bộ, hai cụm ảnh, mỗi cụm gồm ảnh gốc và bản đồ của ba kiến trúc . . . 45
3.20 Trang phân loại ảnh trên Web App . . . 46

4.1 Tensor đầu vào $B \times T \times D$ và đầu ra dự báo trực tiếp H bước. . . 47
4.2 RNN ở dạng gập và dạng mở theo thời gian; trọng số dùng chung ở mọi bước. . . . 48
4.3 Tích Jacobi co hoặc giãn theo khoảng cách thời gian, cùng hướng khắc phục. . . . 48
4.4 Ô LSTM: đường ô nhớ phía trên và ba cổng. . . . 49

4.5 Ô GRU: hai cổng, một trạng thái. 49
4.6 Trái: BiLSTM trong cửa sổ quá khứ. Phải: mặt nạ dropout thường và mặt nạ recurrent dropout. 50
4.7 Chia theo thời gian, khớp thống kê trên train và cửa sổ trượt. . . . 51
4.8 VN30: giá chuẩn hóa về 1 ở phiên đầu, lợi suất ngày gộp bốn mã, ACF của FPT. 52
4.9 Thời tiết Hà Nội: trung bình ngày, phân bố, ACF tới 168 giờ. . . . 52
4.10 Công suất hộ gia đình: trung bình ngày, phân bố, ACF tới 168 giờ. 53
4.11 Mức giảm RMSE so với Persistence. Hai trục khác thang đo vì độ lớn chênh nhau nhiều lần. 57
4.12 Dự báo một bước cuối tập test của mô hình được chọn. 58
4.13 MSE chuẩn hóa theo epoch: nét liền là validation, nét chấm là train. 59
4.14 Độ nhạy đầu ra theo độ trễ, chuẩn hóa về 1 tại bước cuối. 59
4.15 Chuẩn của δ_t trong BPTT NumPy khi lùi về đầu cửa sổ, RNN trên FPT. 59
4.16 RMSE test theo bước dự báo. 60
4.17 Một lần dự báo trực tiếp giữa tập test, kèm khoảng 90 %. 60
4.18 MAE một bước theo nhóm của mô hình được chọn và Persistence. 61
4.19 Trang dự báo chuỗi thời gian trên Web App. 62

5.1 Cùng một dữ kiện theo RDF và theo đồ thị thuộc tính 64
5.2 Năm chiến lược phân mảnh trên cùng một văn bản 65
5.3 Năm thế hệ RAG, từ truy xuất đơn lẻ tới tác tử 66
5.4 Luồng GraphRAG cục bộ trên Neo4j 67
5.5 Vòng lặp ReAct có bộ lọc an toàn đặt trước LLM 67
5.6 Ba miền tri thức và tầng chunk trong cùng một đồ thị Neo4j . . . 68
5.7 Phân bố bậc của ba đồ thị, trục log – log 70
5.8 Đồ thị con đọc từ Neo4j bằng `subgraph()` 71
5.9 Số cạnh theo kiểu trong tiểu đồ thị Hetionet 71
5.10 Ma trận tương quan lợi suất ngày của 30 mã, xếp theo tương quan trung bình 72
5.11 Độ dài câu hỏi và đoạn văn của HotpotQA 73
5.12 Recall@5 theo miền và bộ truy xuất 76
5.13 EM của ba LLM theo miền 79
5.14 EM của ba hệ theo miền 82
5.15 Web App: trang đồ thị và trang trợ lý 83

6.1 Dec-POMDP: mỗi tác tử chỉ thấy quan sát riêng, cùng tác động lên trạng thái ẩn và chia chung phần thưởng. 86

6.2 Chu trình BDI: nhận thức cập nhật niềm tin, cân nhắc chọn ý định, ý định sinh hành động. 87
6.3 Bốn cấu trúc phối hợp; chương này cài (a), (b), (d). 88
6.4 Phân bố trên tập con dùng đo: số bước lời giải GSM8K, nhãn PubMedQA và nhãn ca tổng hợp. 90
6.5 Các tầng của hệ đa tác tử tự cài. 91
6.6 Độ đúng và khoảng tin cậy Wilson 95%. 95
6.7 Đánh đổi giữa token và độ đúng. 96

7.1 Bốn cách nhìn một đồ thị 5 nút: cấu trúc, ma trận kề, ma trận bậc và Laplacian chuẩn hóa. 100
7.2 Truyền thông điệp qua ba lớp: cùng một đồ thị, nút đỏ gộp thông tin láng giềng ở lớp trước. 100
7.3 GCN hai lớp cho phân loại nút: $Z = \hat{A}\,\mathrm{ReLU}(\hat{A}XW^{(0)})W^{(1)}$. . . 101
7.4 Tính hệ số chú ý của GAT và ghép nhiều đầu chú ý. 101
7.5 Phân bố lớp (hàng trên) và phân bố bậc thang log-log (hàng dưới). Elliptic chỉ đếm nút có nhãn. 102
7.6 F1 macro của năm phương pháp trên ba đồ thị, thanh sai số là độ lệch chuẩn giữa các seed. 107
7.7 Trái: số giao dịch illicit theo bước, nét đứt ngăn train, val, test. Phải: F1 illicit trên test theo bước, trung bình các seed. 107
7.8 Phân tích lỗi: độ đúng test theo tỷ lệ láng giềng cùng nhãn của nút (seed 0). 108
7.9 Over-smoothing trên Cora: độ đúng theo số lớp, và MAD của $\hat{A}^k X$ không qua huấn luyện. 108
7.10 t-SNE của đặc trưng thô (trên) và biểu diễn ẩn GCN (dưới), màu là nhãn thật. 109
7.11 Ba cách dựng phép chia: xuyên đồ thị, quy nạp và theo thời gian. . 110
7.12 Quy trình dự đoán liên kết từ Neo4j tới AUC, AP. 111

8.1 Một bước tương tác: hai tác tử chọn hành động đồng thời, thị trường tính xác suất chuyển đổi, doanh số và phần thưởng. 114
8.2 Bảng Q của lượt tham chiếu: ở cả bốn trạng thái, cột giữ giá cao hơn cột giảm giá. 116
8.3 Thưởng đánh giá theo α và theo lịch khám phá; chấm đen là từng seed, nét đỏ là mức tối ưu 35. 116
8.4 Trái: đường học lượt tham chiếu, trung bình trượt 100 episode. Phải: thưởng đánh giá trên mười seed. 117

8.5 Theo mức giảm δ: thưởng của RL, của luật và mức tối ưu; tỉ lệ giảm giá của RL khi đánh giá. 119
8.6 Tác tử dựa trên mô hình: thu kinh nghiệm, cập nhật ba mô hình đếm tần suất, quy hoạch động ngược, chọn hành động. 119
8.7 Regret mỗi bước của ba tác tử (trung bình mười seed; thấp hơn là tốt hơn). 120

9.1 Kiến trúc ba tầng của Đồng Tâm AI Lab. 121
9.2 Trang tổng quan và trang theo dõi thực nghiệm. 123
9.3 Đồ thị Neo4j và trợ lý truy xuất. 123
9.4 Đấu trường tác tử và phòng thử mô hình. 124
9.5 Bản đồ dữ liệu và nhật ký trực tiếp. 124
9.6 Trang tổng quan trên khung điện thoại 390 px. 125

Danh mục Bảng biểu

1.1 Sự chuyển dịch phương pháp qua các giai đoạn 8

2.1 Tổng quan ba dataset (số liệu gốc trước làm sạch) 14
2.2 Năm hồ sơ thực của CDC Diabetes (một phần các cột) 15
2.3 CDC Diabetes trên test, ngưỡng chọn trên validation; F1 macro kèm khoảng tin cậy 95 % 17
2.4 Năm tin rao thực (PN: phòng ngủ; HĐMB: hợp đồng mua bán; dấu gạch: tin không ghi) 18
2.5 Vietnam Housing trên test, đơn vị tỷ đồng; RMSE kèm khoảng tin cậy 95 % 21
2.6 Năm câu thực trong tập test của UIT-VSFC 22
2.7 UIT-VSFC trên test; F1 macro kèm khoảng tin cậy 95 % 24
2.8 Từ đặc trưng của từng lớp, đọc từ hệ số LogReg 25
2.9 Bốn câu XGBoost đoán sai, lấy nguyên văn từ test 25
2.10 MLP trên ba nền tảng; chỉ số chính là F1 macro (CDC, VSFC) hoặc RMSE (Housing), trung bình theo seed 27

3.1 Tổng quan ba bộ dữ liệu ảnh 33
3.2 Kết quả trên tập test của ba bộ dữ liệu 40
3.3 So sánh ba nền tảng: huấn luyện PyTorch và Keras, NumPy suy luận bằng trọng số PyTorch, và hàng cuối là NumPy tự huấn luyện mạng hai tầng 45

4.1 Mẫu thực của FPT (VNĐ, đã điều chỉnh). 51
4.2 Mẫu thực của chuỗi thời tiết. 52
4.3 Mẫu thực của chuỗi điện theo giờ. 53
4.4 NumPy và Keras so với PyTorch cùng kiến trúc trên FPT. 54
4.5 Tập test, dự báo một bước trên FPT (VNĐ), nhiệt độ (°C), công suất (kW). In đậm là RMSE thấp nhất của chuỗi. 56
4.6 Bốn mã VN30: RMSE một bước theo kiến trúc (VNĐ) và mô hình được chọn. Cột Đa số là tỉ lệ của hướng phổ biến hơn trong test. . 56
4.7 Sai số ở bước cuối H và khoảng 90 % (độ phủ trên test, độ rộng trung bình theo đơn vị gốc). 60

5.1 Ontology của đồ thị, số nút đọc từ Neo4j 68
5.2 Quy mô đồ thị theo miền, đọc từ Neo4j 69

5.3 Bộ câu hỏi đánh giá theo miền và loại 77
5.4 Chất lượng truy xuất của bốn phương pháp 77
5.5 Recall@5 theo loại câu hỏi 78
5.6 Chất lượng và độ trễ sinh của ba LLM trên ba miền 79
5.7 Naive RAG, GraphRAG và Agent trên cùng LLM Qwen2.5 7B . . 80

6.1 Đặc điểm đã kiểm chứng từ README của ba framework. 89
6.2 Ba dataset của Chương 6. Hai tập công khai có giấy phép MIT. . . 89
6.3 Mẫu đầu ra thực của từng dataset (đã cắt ngắn). 90
6.4 So sánh các kiến trúc trên ba dataset (qwen2.5-7b). 95
6.5 Kiểm định cặp McNemar; thắng là số câu kiến trúc bên phải đúng còn bên trái sai. 96

7.1 Kích cỡ và tính chất đồ thị. Bậc, đồng dạng tính sau khi đối xứng hóa cạnh; đồng dạng trên Elliptic chỉ tính cạnh có nhãn ở hai đầu. 102
7.2 Nút mẫu thật của Cora và Amazon Photo: nút đầu tiên của hai lớp đầu. 103
7.3 Giao dịch mẫu thật của Elliptic. Đặc trưng đã z-score bằng thống kê các bước 1 đến 29. 103
7.4 Phân loại nút, trung bình trên các seed. Elliptic đo bằng chia thời gian, cột F1 illicit là thước đo thật của bài toán. 106
7.5 Đo rò rỉ. Cora, Photo: mức 1 là xuyên đồ thị, mức 2 là quy nạp. Elliptic: mức 1 là chia theo thời gian, mức 2 là chia ngẫu nhiên. . 110
7.6 Dự đoán liên kết, 3 seed, âm cùng kiểu. Dòng cuối cố ý để cạnh test lọt vào đồ thị truyền tin khi đo. 111

8.1 Lợi nhuận kỳ vọng một bước theo (d, b); b là hành động hiện tại của đối thủ. 115
8.2 Thí nghiệm C, D, E: 10 seed, đánh giá tham lam; "Đổi CS" là số lần chính sách đổi khi huấn luyện. 116
8.3 Các biến thể môi trường (10 seed; đánh giá luôn theo lợi nhuận). . 118
8.4 Ba tác tử trong năm kịch bản. 120

9.1 Giá trị tốt nhất trên tập kiểm thử của từng bộ dữ liệu, chỉ để đối chiếu; mô hình đại diện của mỗi chương được chọn trên tập kiểm định. 127

Mở đầu

Bối cảnh

Một hệ thống thông minh ngày nay hiếm khi chỉ là một mô hình. Nó đọc dữ liệu bảng, ảnh và chuỗi thời gian, tra cứu tri thức có cấu trúc, gọi mô hình ngôn ngữ lớn (LLM) và phối hợp nhiều tác tử để hoàn thành một nhiệm vụ. Mỗi hình thái thông tin cần một họ kỹ thuật riêng, và chọn sai họ kỹ thuật thường tốn kém hơn chọn sai tham số. Lịch sử AI cho thấy điều đó: mỗi thế hệ phương pháp giải được điểm yếu của thế hệ trước rồi lộ ra điểm yếu của mình [1].

Tiểu luận này đi qua toàn bộ chuỗi đó, từ học máy cơ bản, CNN, RNN đến đồ thị tri thức, RAG, hệ đa tác tử và mạng nơ-ron đồ thị. Mỗi chương gắn lý thuyết với dữ liệu thật, chạy trên máy cá nhân, đo bằng chỉ số rõ ràng và triển khai thành một phần của cùng một Web App.

Mục tiêu

1. Trình bày có hệ thống lịch sử AI và cơ sở lý thuyết của từng họ kỹ thuật, với công thức và hình minh họa.
2. Với mỗi chương thực hành, chọn nhiều bộ dữ liệu từ nguồn gốc, mô tả nguồn, kích cỡ, mẫu thực, phân bố và tiền xử lý.
3. So sánh ba mô hình trên từng bộ dữ liệu, có đường cơ sở, bằng cùng phép chia và cùng chỉ số; phân tích lỗi.
4. Xây đồ thị tri thức thật trên Neo4j, RAG và tác tử chạy với LLM cục bộ, so sánh tác tử đơn với đa tác tử bằng số đo về độ đúng, token và độ trễ.
5. Đóng gói các mô hình đã huấn luyện vào một Web App chạy ở localhost.

Phương pháp

Mọi thực nghiệm đi theo một quy trình: lấy dữ liệu từ nguồn gốc, khảo sát và tiền xử lý, huấn luyện trên tập huấn luyện, chọn mô hình trên tập kiểm định, chỉ dùng tập kiểm thử một lần. Hai nguyên tắc được giữ xuyên suốt. Thứ nhất, mọi con số trong báo cáo được đọc tự động từ tệp kết quả do mã sinh ra, và mã in trong báo cáo được trích nguyên văn từ mã đã chạy. Thứ hai, không gửi dữ liệu lên dịch vụ đám mây: LLM và cơ sở dữ liệu đồ thị đều chạy cục bộ. Dữ liệu tổng hợp được ghi rõ là tổng hợp; kết quả về y tế và đầu tư chỉ mang tính học thuật, không phải lời khuyên.

Máy thực nghiệm dùng CPU Ryzen 9 7945HX 32 luồng, GPU RTX 4060 8 GB, RAM 40 GB, Windows 11. PyTorch và LLM chạy trên GPU, với LLM mặc định là Qwen2.5-7B qua Ollama; scikit-learn, XGBoost, NumPy, Keras và Neo4j chạy trên CPU.

Bố cục báo cáo

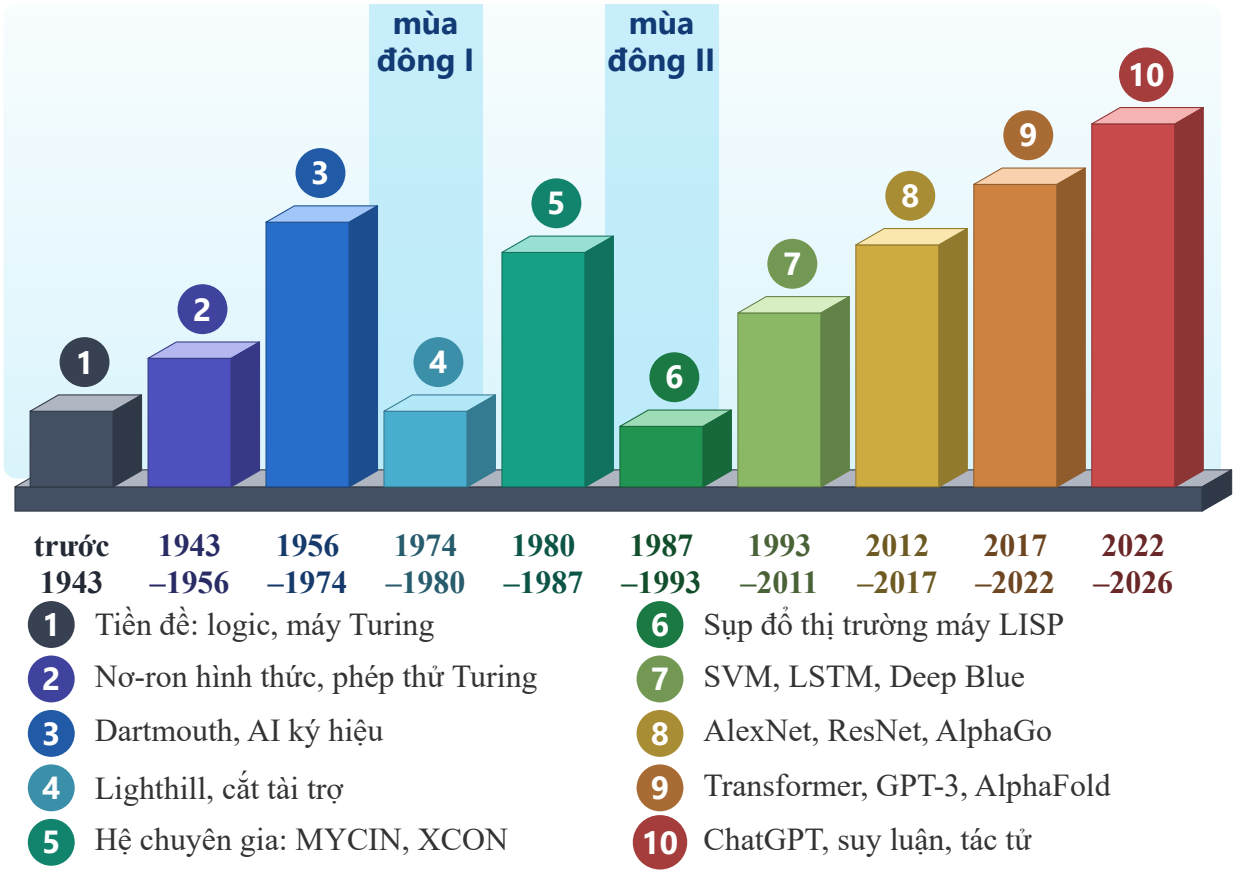
Ngoài Mở đầu và Kết luận, báo cáo gồm tám chương chuyên môn và một chương sản phẩm:

- **Chương 1. Lịch sử phát triển AI:** từ tiền đề trước 1956 đến tác tử giai đoạn 2025–2026.
- **Chương 2. Các kỹ thuật học máy cơ bản:** phân loại và hồi quy trên dữ liệu y tế, nhà đất và văn bản tiếng Việt.
- **Chương 3. Mạng nơ-ron tích chập:** SVHN, ảnh vệ tinh EuroSAT, mô bệnh học PathMNIST; Grad-CAM.
- **Chương 4. Mạng nơ-ron hồi quy:** cổ phiếu VN30, thời tiết Hà Nội, điện năng hộ gia đình.
- **Chương 5. Knowledge Graph, Neo4j, RAG và Agent:** đồ thị thật, ba cách truy xuất, ba LLM cục bộ, tác tử ReAct.
- **Chương 6. Hệ đa tác tử:** so sánh tác tử đơn, Supervisor và Debate.
- **Chương 7. Deep Graph và GNN:** GCN, GraphSAGE, GAT và dự đoán liên kết.
- **Chương 8. Tác tử học tăng cường và tác tử luật:** Q-learning trong thị trường giảm giá hai cửa hàng.
- **Chương 9. Sản phẩm Web App:** ghép các mô hình thành một ứng dụng.

Chương 1

Lịch sử phát triển của trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence, AI) nghiên cứu cách xây dựng hệ thống tính toán làm được những việc mà ở con người ta gọi là thông minh: nhận biết, suy luận, học từ kinh nghiệm, lập kế hoạch và hành động có mục đích. Giáo trình hiện đại lấy *tác tử duy lý* làm khung chung, tức chủ thể chọn hành động tối đa hóa kết quả kỳ vọng theo một thước đo hiệu năng [2]. Lịch sử của lĩnh vực không phải đường thẳng đi lên mà là chuỗi chu kỳ: lời hứa lớn, thành công trong phạm vi hẹp, thất vọng khi mở rộng, cắt tài trợ, rồi phục hồi dưới một phương pháp mới [1]. Hình 1.1 tóm tắt mười giai đoạn của tiến trình đó.



Hình 1.1. Mười giai đoạn phát triển của AI. Chiều cao cột chỉ minh họa định tính mức kỳ vọng và đầu tư, không phải số liệu đo.

1.1 Tiền đề trước năm 1956

Nền móng của AI có trước máy tính điện tử. Năm 1936, Turing định nghĩa khái niệm tính toán bằng một cỗ máy trừu tượng và chứng minh tồn tại máy vạn năng mô phỏng được mọi máy khác [3]. McCulloch và Pitts (1943) mô tả nơ-ron như một đơn vị ngưỡng tính được các hàm logic [4], ý tưởng đầu tiên nối sinh học với tính toán. Năm 1948, lý thuyết thông tin của Shannon [5] và điều khiển học của Wiener [6] đem lại ngôn ngữ toán học để nói về tín hiệu và phản hồi. Năm 1950, Turing thay câu hỏi "máy có thể suy nghĩ không?" bằng một phép thử kiểm chứng được: nếu người thẩm vấn trò chuyện qua văn bản mà không phân biệt được máy với người thì máy được xem là có hành vi thông minh [7].

1.2 Dartmouth và AI ký hiệu (1956 – 1974)

Hội thảo mùa hè 1956 tại Dartmouth, do McCarthy, Minsky, Rochester và Shannon đề xuất, đặt tên cho lĩnh vực với giả định rằng mọi khía cạnh của trí thông minh đều mô tả được chính xác để máy mô phỏng [8]. Cũng tại đây, Newell và Simon giới thiệu Logic Theorist, chương trình chứng minh được các định lý trong *Principia Mathematica* [9]. Hướng ký hiệu xem trí thông minh là thao tác trên ký hiệu theo luật; tìm kiếm và logic là công cụ chính.

Song song là hướng học. Rosenblatt (1958) thêm khả năng cập nhật trọng số cho nơ-ron, gọi là Perceptron [10]:

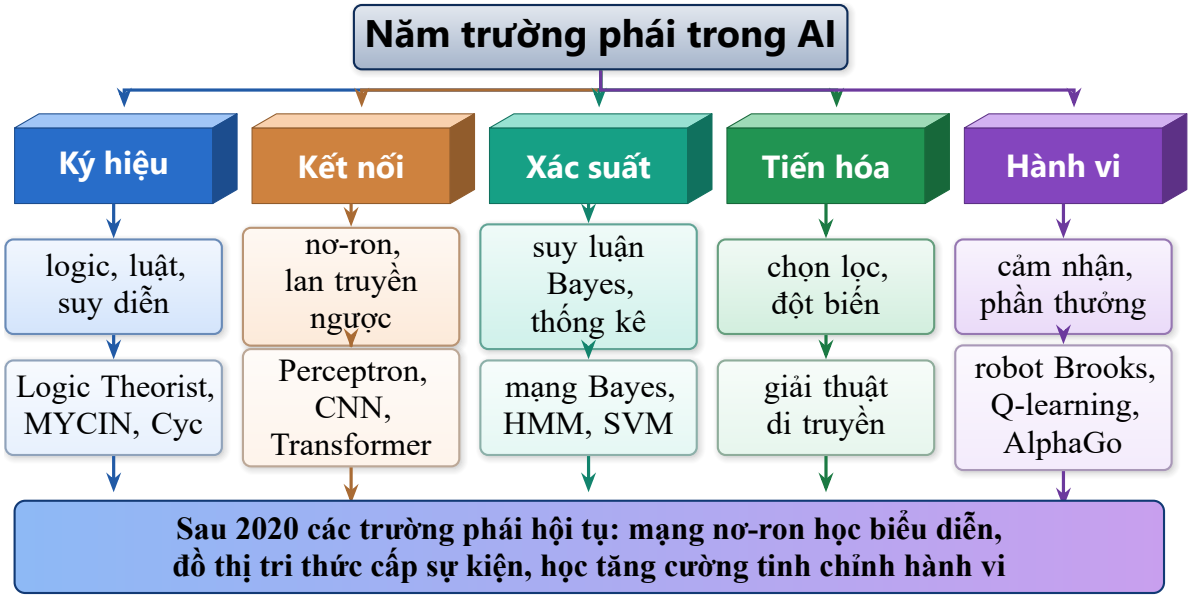
$$\hat{y} = \operatorname{sign}(\mathbf{w}^{\top}\mathbf{x} + b), \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}. \tag{1.1}$$

Samuel (1959) cho chương trình cờ đam tự cải thiện qua các ván đấu và phổ biến thuật ngữ học máy [11]. ELIZA (1966) mô phỏng hội thoại chỉ bằng so khớp mẫu, nhưng nhiều người dùng vẫn gán cho nó sự thấu hiểu [12]. Những kết quả này trong các thế giới thu nhỏ dẫn tới dự đoán quá lạc quan về thời điểm máy bắt kịp con người.

1.3 Các trường phái tư tưởng

Khác biệt giữa các hướng tiếp cận nằm ở câu trả lời cho câu hỏi: trí thông minh sinh ra từ đâu? Hình 1.2 xếp năm câu trả lời chính, mỗi trường phái có một thời kỳ dẫn đầu. Hướng ký hiệu thắng thế tới thập niên 1980 vì chạy được trên máy tính yếu; hướng kết nối cần dữ liệu và phần cứng nên phải chờ tới sau năm 2010; hướng xác suất đem lại cách xử lý bất định có nền tảng [13]; hướng hành vi, với

Brooks, cho rằng robot không cần biểu diễn tri thức tường minh [14]. Các hệ thống lớn ngày nay thường là hệ lai của nhiều trường phái.



Hình 1.2. Năm trường phái của AI: ý tưởng trung tâm, các hệ thống kinh điển và xu hướng hội tụ trong các hệ hiện đại.

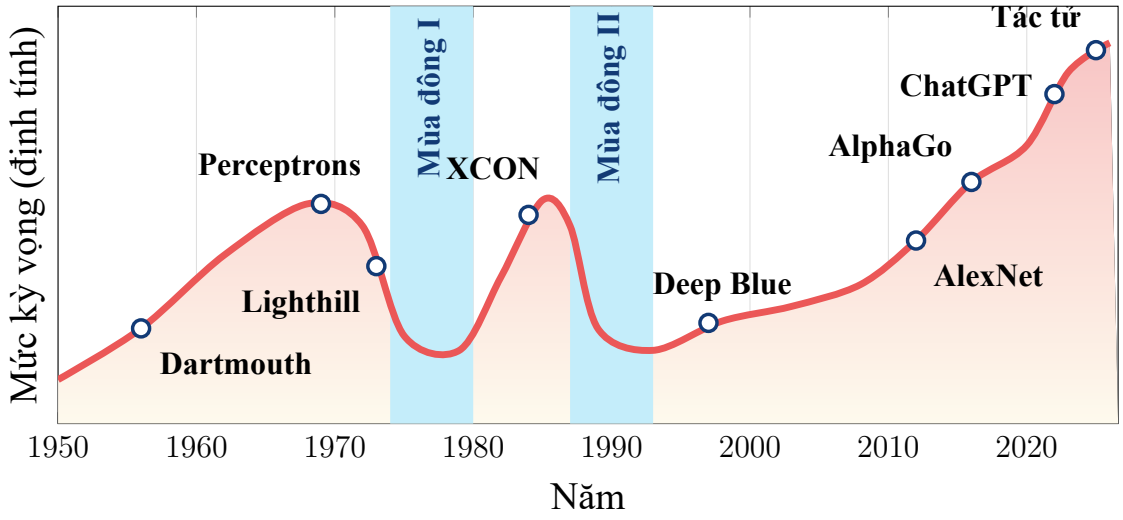
1.4 Hai mùa đông AI (1974 – 1993)

Mùa đông thứ nhất. Báo cáo ALPAC (1966) kết luận dịch máy không đạt kỳ vọng [15]. Minsky và Papert (1969) chứng minh Perceptron một tầng chỉ biểu diễn được hàm tách tuyến tính, nên không học nổi XOR [16], và nghiên cứu mạng nơ-ron gần như ngừng lại. Báo cáo Lighthill (1973) phê phán AI không vượt được bùng nổ tổ hợp ngoài các bài toán đồ chơi [17], dẫn tới cắt tài trợ ở Anh; DARPA tại Mỹ cũng chuyển sang chỉ cấp vốn cho dự án có mục tiêu ứng dụng. Ba nguyên nhân kỹ thuật là năng lực tính toán thấp, bùng nổ tổ hợp và thiếu tri thức thường thức.

Hệ chuyên gia. Hướng đi mới là giới hạn miền và mã hóa tri thức chuyên gia thành luật `NẾU--THÌ`. MYCIN chẩn đoán nhiễm khuẩn huyết [18], R1/XCON cấu hình máy VAX cho DEC và là một trong các hệ thương mại thành công đầu tiên [19]. Nhật Bản mở dự án Máy tính thế hệ thứ năm (1982), kéo theo đầu tư cấp quốc gia ở Mỹ và châu Âu. Cùng lúc, mạng Hopfield [20] và **lan truyền ngược** [21] mở lại đường cho mạng nhiều tầng, vì quy tắc chuỗi tính được đạo hàm theo mọi trọng số, nhờ đó vượt giới hạn XOR.

Mùa đông thứ hai. Năm 1987 thị trường máy LISP sụp đổ khi máy trạm phổ thông rẻ và mạnh hơn. Hệ chuyên gia lộ điểm yếu: đắt bảo trì, giòn khi ra ngoài

miền đã mã hóa, khó thu thập tri thức. Dự án thế hệ thứ năm không đạt mục tiêu và tài trợ lại bị cắt. Hình 1.3 khái quát các chu kỳ này.



Hình 1.3. Chu kỳ kỳ vọng dành cho AI từ 1950 đến 2026. Đường cong là minh họa định tính; năm của các sự kiện là thật.

1.5 Học máy thống kê (1993 – 2011)

AI trở lại bằng toán học, thực nghiệm và tiêu chuẩn đánh giá khách quan. SVM tìm siêu phẳng có lề lớn nhất [22], rừng ngẫu nhiên kết hợp nhiều cây quyết định [23], học tăng cường được hệ thống hóa [24]. LSTM (1997) giải quyết việc nhớ xa trong chuỗi [25], LeNet-5 (1998) đọc chữ số trên séc ngân hàng bằng mạng tích chập [26]. Hai sự kiện công chúng biết đến là Deep Blue thắng Kasparov năm 1997 [27] và Watson thắng *Jeopardy!* năm 2011 [28], cả hai đều là kỹ thuật chuyên biệt hơn là trí tuệ tổng quát. Giới hạn của thời kỳ này là đặc trưng do con người thiết kế: chất lượng mô hình phụ thuộc vào công sức làm đặc trưng cho từng bài toán.

1.6 Cuộc cách mạng học sâu (2012 – 2017)

Học sâu bùng nổ khi ba điều kiện gặp nhau: dữ liệu lớn như ImageNet [29], GPU tính song song và các cải tiến thuật toán từ huấn luyện từng tầng [30] đến ReLU, dropout, chuẩn hóa theo lô [31]. Tại ILSVRC 2012, AlexNet đạt lỗi top-5 khoảng 15,3%, thấp hơn hẳn mức 26,2% của đội thứ hai [32]. ResNet (2015) dùng kết nối tắt để huấn luyện mạng 152 tầng, đưa lỗi xuống 3,57% [33]. Mạng tự học đặc trưng nên bước làm đặc trưng thủ công biến mất. Năm 2016 AlphaGo thắng Lee Sedol [34], rồi AlphaGo Zero tự học hoàn toàn từ tự chơi [35]; GAN [36] mở đầu cho mô hình tạo sinh hiện đại.

1.7 Transformer và mô hình ngôn ngữ lớn (2017 – 2024)

Với chuỗi, seq2seq [37] và attention [38] đưa mạng hồi quy vào dịch máy. Transformer (2017) bỏ hẳn hồi quy, chỉ dùng tự chú ý [39]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{1.2}$$

nhờ đó xử lý song song được và nắm được quan hệ xa. GPT-3 (2020) cho thấy mở rộng quy mô làm tăng chất lượng [40]. AlphaFold 2 đưa học sâu vào dự đoán cấu trúc protein [41]. ChatGPT cuối năm 2022, xây trên kỹ thuật học từ phản hồi của người dùng [42], đưa AI tới đại chúng; mô hình khuếch tán [43] đem lại bước nhảy cho sinh ảnh; LLaMA [44] mở hướng mô hình trọng số mở.

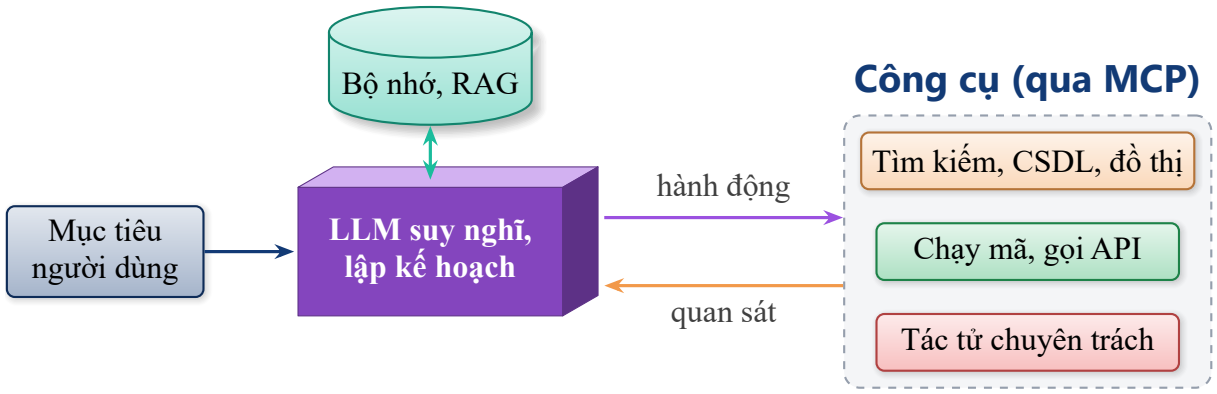
Mô hình cũng đi theo chiều ngược lại: các LLM nhỏ trọng số mở, chỉ vài tỉ tham số, đủ chạy trên GPU của máy xách tay. Đây là loại mô hình Chương 5 và 6 sử dụng, với Qwen2.5-7B qua Ollama làm mặc định, vì dữ liệu không rời khỏi máy.

Từ năm 2024, các mô hình suy luận như o1 [45] và DeepSeek-R1 [46] dành thêm bước tính toán để "suy nghĩ" trước khi trả lời. Nobel năm 2024 ghi nhận chặng đường này: giải Vật lý cho Hopfield và Hinton [47], giải Hóa học có Hassabis và Jumper vì AlphaFold [48]. Tuy vậy LLM còn giới hạn nội tại: tri thức đóng băng tại thời điểm huấn luyện, có thể sai với giọng tự tin và không tự dẫn nguồn. RAG [49] bù điểm yếu đó bằng cách đưa bằng chứng bên ngoài vào lúc suy luận (Chương 5).

1.8 Kỷ nguyên tác tử (2025 – 2026)

Tác tử (agent) đặt LLM vào một vòng lặp: suy nghĩ, gọi công cụ, quan sát kết quả rồi quyết định tiếp (Hình 1.4). Khung ReAct đã mô tả vòng này từ 2022 [50]. Điều đổi mới ở giai đoạn 2025–2026 là hạ tầng và độ dài nhiệm vụ. Giao thức Model Context Protocol chuẩn hóa cách tác tử nối với công cụ và dữ liệu [51]; Agent2Agent chuẩn hóa cách các tác tử nói chuyện với nhau [52]; các khung như AutoGen tổ chức nhiều tác tử theo hội thoại [53], và các khảo sát bắt đầu phân loại cơ chế cộng tác của chúng [54]. Một đo lường của METR cho thấy độ dài nhiệm vụ phần mềm mà mô hình hoàn thành được đang tăng theo hàm mũ [55]. Chương 6 thử nghiệm câu hỏi đang được tranh luận: phối hợp nhiều tác tử có đáng với chi phí tăng thêm không.

Năng lực tăng kéo theo yêu cầu quản trị. Liên minh châu Âu ban hành Đạo luật AI năm 2024 [56]; Việt Nam có Chiến lược quốc gia về AI theo Quyết định 127/QĐ-TTg [57]; Báo cáo quốc tế về an toàn AI 2025 tổng hợp các rủi ro đã thấy



Hình 1.4. Vòng lặp của một tác tử: LLM lập kế hoạch, gọi công cụ, nhận quan sát rồi tiếp tục cho đến khi đạt mục tiêu.

và chưa chắc chắn [58]. AI tổng quát (AGI) vẫn là chủ đề tranh luận, vì ngay cách đo trí thông minh cũng chưa có đồng thuận [59].

1.9 Bài học từ lịch sử

Bảng 1.1 tóm tắt sự chuyển dịch phương pháp. Ba bài học đáng giữ. Mỗi lần phục hồi đều gắn với thay đổi về dữ liệu, phần cứng hoặc thuật toán, không phải với lời hứa mới. Phương pháp bị bỏ rơi có thể trở lại khi điều kiện chín muồi, như mạng nơ-ron sau hai lần bị gạt đi. Và mỗi thế hệ đều bị giới hạn bởi điểm yếu của mình: hệ luật giòn, học máy cần đặc trưng, học sâu cần dữ liệu, LLM cần bằng chứng và giám sát. Các chương sau đi qua đúng trình tự đó.

Bảng 1.1. Sự chuyển dịch phương pháp qua các giai đoạn

Giai đoạn	Phương pháp chính	Hệ thống tiêu biểu	Giới hạn
1956–1974	Tìm kiếm, logic ký hiệu	Logic Theorist, ELIZA	Bùng nổ tổ hợp
1980–1987	Hệ chuyên gia, luật	MYCIN, XCON	Giòn, khó bảo trì
1993–2011	Học máy thống kê	SVM, Deep Blue, Watson	Đặc trưng thủ công
2012–2017	Học sâu	AlexNet, ResNet, AlphaGo	Cần nhiều dữ liệu nhãn
2017–2024	Transformer, LLM	GPT-3, ChatGPT, DeepSeek-R1	Sai tự tin, tri thức cũ
2025–2026	RAG, tác tử, đa tác tử	Tác tử gọi công cụ qua MCP	Chi phí, an toàn, kiểm soát

Chương 2

Các kỹ thuật học máy cơ bản

Theo Mitchell [60], chương trình *học* từ kinh nghiệm E đối với nhiệm vụ T và độ đo P nếu hiệu năng trên T, đo bằng P, tốt lên nhờ E. Chương này tóm tắt các thuật toán nền tảng rồi kiểm chứng chúng trên ba bài toán khác dạng: phân loại nhị phân dữ liệu y tế, hồi quy giá nhà Việt Nam và phân loại cảm xúc tiếng Việt. Mọi con số trong chương do mã chạy thật sinh ra.

2.1 Cơ sở lý thuyết

2.1.1 Học có giám sát và quy trình chống rò rỉ

Tập dữ liệu gồm n cặp $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$. Huấn luyện là cực tiểu rủi ro thực nghiệm, thường kèm hạng chính quy hóa:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i\big) + \lambda R(\boldsymbol{\theta}). \tag{2.1}$$

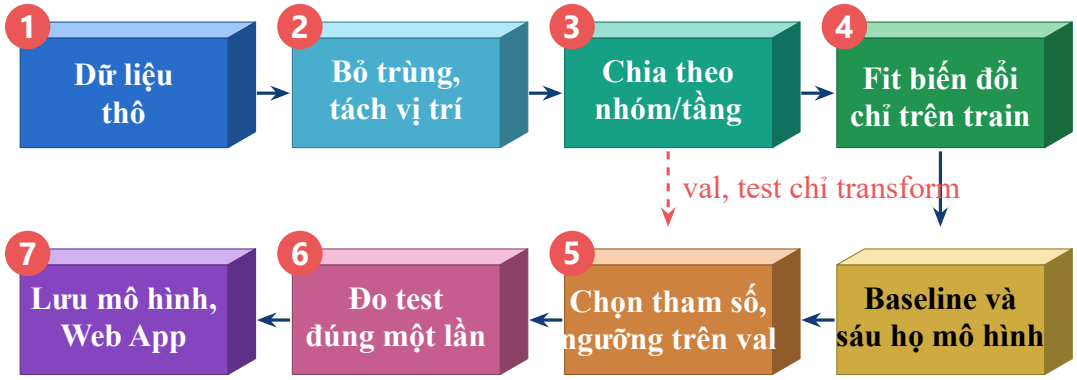
Khi không có nghiệm đóng, tham số được cập nhật theo hạ gradient $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J$. Sai số kỳ vọng phân rã thành thiên lệch bình phương, phương sai và nhiễu không giảm được [61], nên mô hình quá đơn giản thì chưa khớp, quá phức tạp thì quá khớp.

Điểm yếu thường gặp nhất không nằm ở thuật toán mà ở *rò rỉ dữ liệu* (data leakage) [62]: thông tin của tập test lọt vào lúc huấn luyện làm điểm số đẹp hơn thực tế. Chương này giữ quy trình ở Hình 2.1 cho cả ba bài: làm sạch không dùng nhãn trước, chia tập, chỉ fit biến đổi trên train, chọn siêu tham số và ngưỡng trên validation, và chỉ đo test một lần.

2.1.2 Hồi quy tuyến tính và hồi quy logistic

Hồi quy tuyến tính dự đoán $\hat{y} = \mathbf{w}^{\top}\mathbf{x} + b$ và cực tiểu sai số bình phương. Ridge [63] thêm phạt $\lambda\|\mathbf{w}\|_2^2$, nghiệm đóng trở thành

$$\mathbf{w}^{*} = (X^{\top}X + \lambda I)^{-1} X^{\top}\mathbf{y}, \tag{2.2}$$



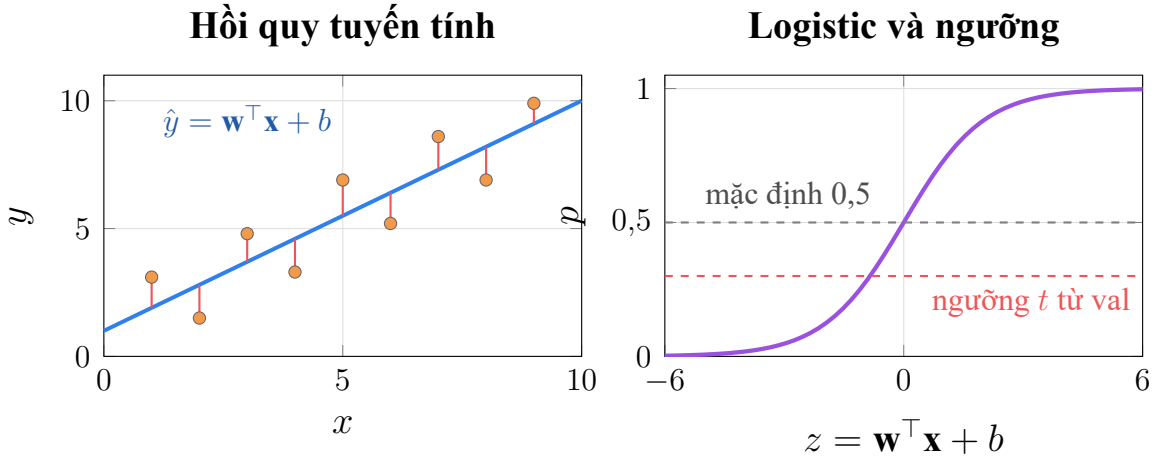
Hình 2.1. Quy trình bảy bước áp dụng cho cả ba dataset; val và test chỉ đi qua biến đổi đã fit trên train

ổn định hơn khi các đặc trưng tương quan mạnh, như các cột one-hot trong bài giá nhà.

Hồi quy logistic [64] đưa tổ hợp tuyến tính qua sigmoid và ước lượng bằng hợp lý cực đại, tương đương cực tiểu entropy chéo. Với K lớp, softmax thay cho sigmoid:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_j \exp(\mathbf{w}_j^\top \mathbf{x})}, \qquad J = -\frac{1}{n} \sum_{i=1}^{n} \log P(y_i \mid \mathbf{x}_i). \tag{2.3}$$

Nhãn được quyết định bằng cách so xác suất với ngưỡng t. Khi lớp lệch, ngưỡng 0,5 thường không tốt nhất, nên t được chọn trên validation (Hình 2.2, phải).



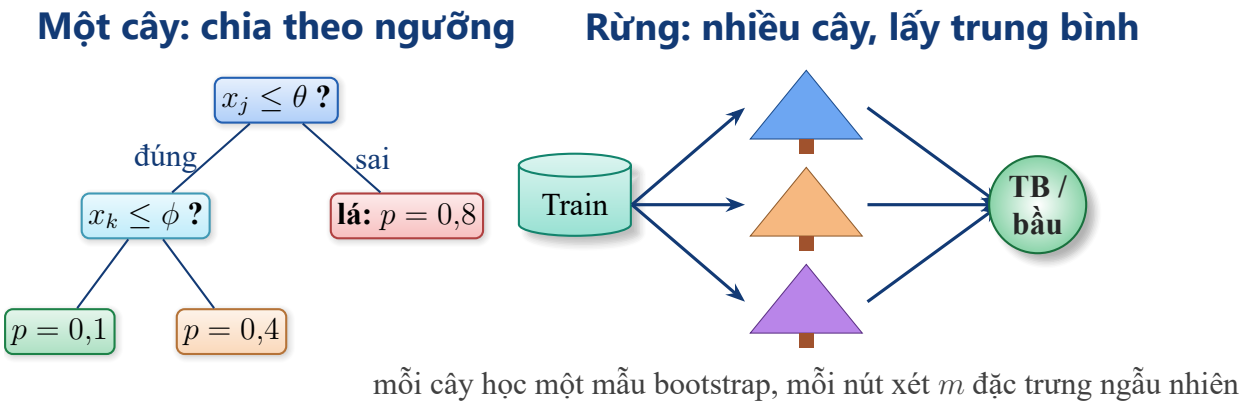
Hình 2.2. Trái: đường hồi quy và phần dư (đoạn đỏ) mà MSE cực tiểu. Phải: sigmoid và ngưỡng quyết định

2.1.3 Cây quyết định và rừng ngẫu nhiên

Cây quyết định [65, 66] chia không gian đặc trưng theo các ngưỡng, ở mỗi nút chọn phép tách làm giảm độ vẩn đục nhiều nhất. Với p_k là tỷ lệ lớp k trong nút t và N_L, N_R là số mẫu hai nhánh con:

$$G(t) = 1 - \sum_k p_k^2, \qquad \Delta G = G(t) - \frac{N_L}{N_t} G(t_L) - \frac{N_R}{N_t} G(t_R). \tag{2.4}$$

Với hồi quy, độ vẩn đục là phương sai của y trong nút. Cây dễ diễn giải nhưng phương sai cao. Random Forest [67] huấn luyện nhiều cây trên mẫu bootstrap, mỗi nút chỉ xét một tập con đặc trưng ngẫu nhiên, rồi lấy trung bình (Hình 2.3). Nếu B cây có phương sai σ^2 và tương quan từng đôi ρ, phương sai của trung bình là $\rho\sigma^2 + (1-\rho)\sigma^2/B$: thêm cây chỉ giảm số hạng sau, còn chọn đặc trưng ngẫu nhiên giảm chính ρ.



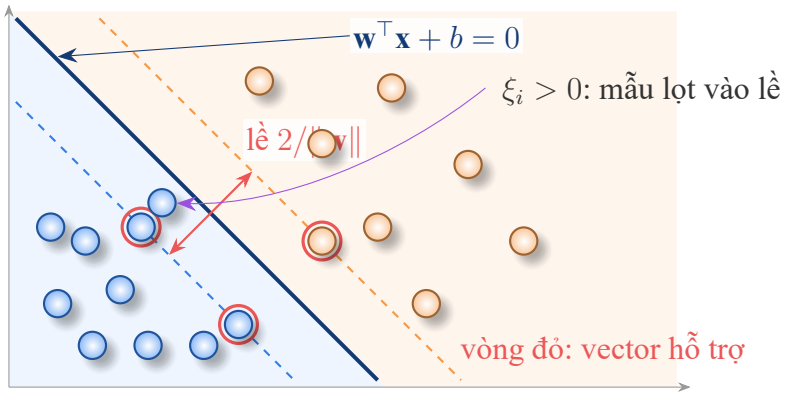
Hình 2.3. Một cây quyết định và rừng tổng hợp nhiều cây trên các mẫu bootstrap

2.1.4 Máy vector hỗ trợ

SVM [68] tìm siêu phẳng có lề lớn nhất, cho phép vi phạm qua biến nới lỏng ξ_i:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1 - \xi_i, \;\; \xi_i \ge 0. \tag{2.5}$$

Chỉ các mẫu nằm trên hoặc trong lề (vector hỗ trợ) quyết định nghiệm (Hình 2.4). Thủ thuật kernel như $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ cho biên phi tuyến nhưng chi phí tăng ít nhất theo bình phương số mẫu, nên thực nghiệm ở đây dùng SVM tuyến tính, hợp với văn bản nhiều chiều [69].



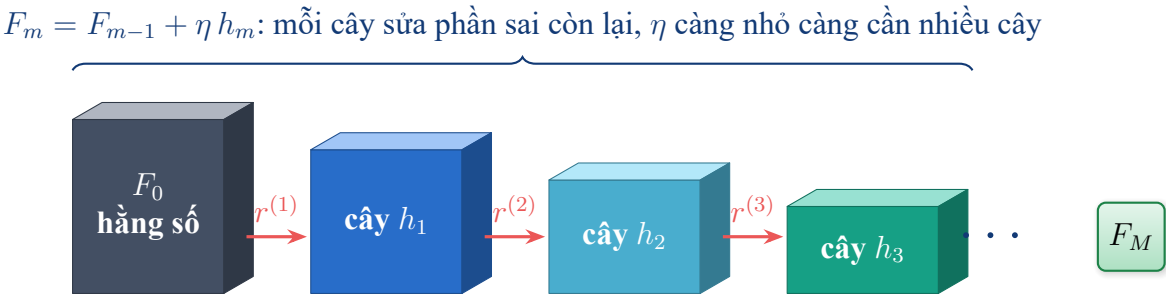
Hình 2.4. SVM lề mềm: siêu phẳng, hai biên lề, vector hỗ trợ và một mẫu lọt vào lề

2.1.5 Gradient boosting

Boosting [70] xây cây tuần tự. Ở vòng m, cây h_m khớp gradient âm của mất mát, với MSE chính là phần dư, rồi được cộng vào mô hình theo tốc độ học η:

$$r_i^{(m)} = -\left.\frac{\partial \ell(y_i, F)}{\partial F}\right|_{F=F_{m-1}(\mathbf{x}_i)}, \qquad F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}). \tag{2.6}$$

Rừng chủ yếu giảm phương sai, còn boosting giảm thiên lệch (Hình 2.5). XGBoost [71] dùng thêm đạo hàm bậc hai và phạt độ phức tạp của cây. Số cây không chọn tay mà dừng sớm theo validation.



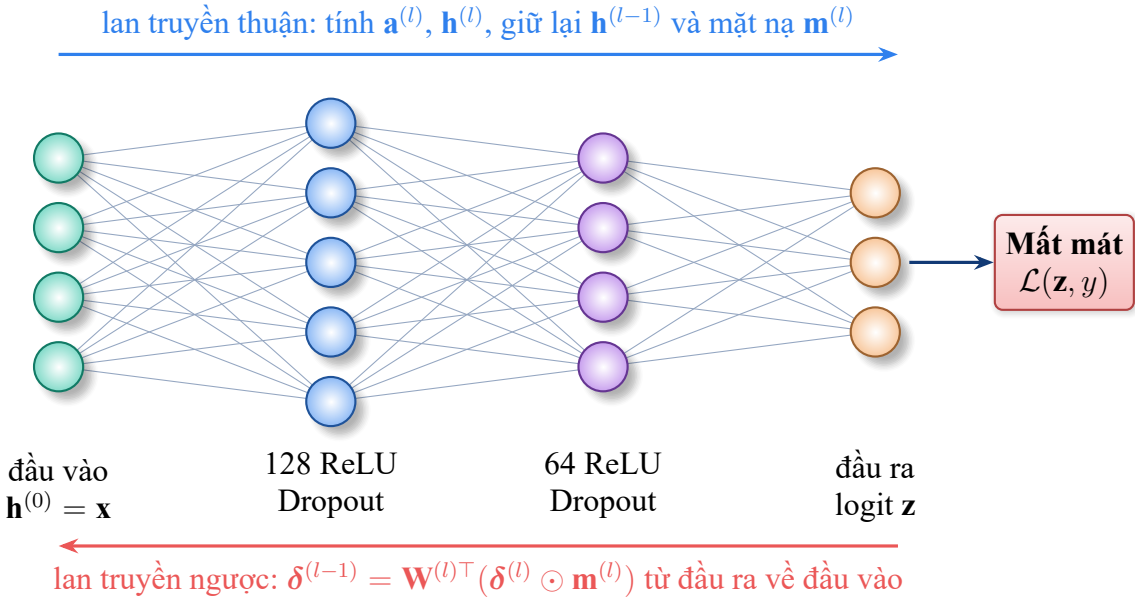
Hình 2.5. Gradient boosting: mỗi cây nông học phần dư $r^{(m)}$ của tổng các cây trước

2.1.6 MLP và lan truyền ngược

Mạng truyền thẳng tính $\mathbf{a}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$ và $\mathbf{h}^{(l)} = \mathbf{a}^{(l)} \odot \mathbf{m}^{(l)}$, trong đó $\mathbf{m}^{(l)}$ là mặt nạ của ReLU nhân với mặt nạ dropout [72]. Lan truyền ngược [73] áp dụng quy tắc chuỗi từ đầu ra về đầu vào. Đặt $\boldsymbol{\delta}^{(l)} = \partial \mathcal{L}/\partial \mathbf{h}^{(l)}$, ta có

$$\boldsymbol{\delta}^{(l-1)} = \mathbf{W}^{(l)\top}\big(\boldsymbol{\delta}^{(l)} \odot \mathbf{m}^{(l)}\big), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \big(\boldsymbol{\delta}^{(l)} \odot \mathbf{m}^{(l)}\big)\,\mathbf{h}^{(l-1)\top}. \tag{2.7}$$

Chi phí tính gradient vì vậy chỉ cỡ một lượt lan truyền thuận, thay vì một lượt cho mỗi tham số như sai phân. Tham số được cập nhật bằng Adam [74]. Hình 2.6 là kiến trúc dùng chung trên NumPy, Keras và PyTorch.



Hình 2.6. MLP hai tầng ẩn và hai chiều truyền: thuận tính đầu ra, ngược nhân chuỗi đạo hàm

Đoạn mã 2.1. Lan truyền ngược viết tay trên NumPy, đúng theo công thức trên

```python
# tlf/ch2_mlp_numpy.py
def backward(self, g, cache):
    """Lan truyền ngược: g = dL/da của tầng cuối, nhân chuỗi về từng tầng trước."""
    grads_W, grads_b = [None] * len(self.W), [None] * len(self.b)
    for k in reversed(range(len(self.W))):
        h, mask = cache[k]
        if mask is not None:
            g = g * mask
        grads_W[k] = h.T @ g / len(h)
        grads_b[k] = g.mean(axis=0)
        g = g @ self.W[k].T
    return grads_W, grads_b
```

Đoạn mã này được đối chiếu với sai phân trung tâm trên 119 tham số của một mạng nhỏ; kết quả ở mục 2.6.

2.1.7 Độ đo đánh giá

Với TP, FP, FN của mỗi lớp, Precision $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, Recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, F1 là trung bình điều hòa của hai đại lượng. **F1 macro** lấy trung bình không trọng

số trên các lớp nên lớp hiếm được tính ngang lớp đông; đây là chỉ số chính của hai bài phân loại. ROC-AUC [75] đo khả năng xếp hạng, không phụ thuộc ngưỡng. Với hồi quy, chỉ số chính là RMSE, kèm MAE và $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$. Mọi chỉ số chính đi kèm khoảng tin cậy 95 % bằng bootstrap trên test (500 lần lấy lại mẫu): hai mô hình có khoảng chồng nhau nhiều thì chưa đủ cơ sở nói mô hình này hơn.

2.2 Thiết lập thực nghiệm

Bảng 2.1 tóm tắt ba bộ dữ liệu. Trên mỗi bộ, Dummy làm mốc, rồi đến LogReg hoặc Ridge, Linear SVM, cây quyết định, Random Forest, XGBoost và MLP trên ba nền tảng. Mọi mô hình huấn luyện trên toàn bộ tập train, không lấy mẫu con. Housing được quét lưới nhỏ, chọn theo validation. CDC và VSFC dùng một cấu hình cố định cho mỗi họ để tiết kiệm tài nguyên, nên kết quả của chúng chưa phải mức tốt nhất có thể; với VSFC, cấu hình rừng được đặt không giới hạn độ sâu vì văn bản thưa cần cây sâu. XGBoost của CDC và VSFC dùng tốc độ học 0,1 và tối đa 500 cây, Housing dùng 0,05 và tối đa 1.500 cây. Các mô hình cổ điển và Keras chạy CPU tối đa 8 luồng; MLP PyTorch chạy GPU.

Bảng 2.1. Tổng quan ba dataset (số liệu gốc trước làm sạch)

Dataset	Bài toán	Số mẫu	Đặc trưng	MB
CDC Diabetes Health Indicators	Phân loại nhị phân	253.680	21	12,87
Vietnam Housing 2024	Hồi quy	30.229	11	3,37
UIT-VSFC	Phân loại ba lớp	16.175	1	0,64

Đoạn mã 2.2. Cấu hình cố định dùng cho CDC và VSFC

```
# scripts/ch2_classic.py
ONE_CONFIG = {"linear": {"C": 1}, "svm": {"C": 0.1}, "tree":
    {"max_depth": 10, "min_samples_leaf": 20},
                "rf": {"max_depth": 20, "min_samples_leaf": 2}, "xgb":
                    {"max_depth": 5, "reg_lambda": 1}}
```

2.3 CDC Diabetes: sàng lọc nguy cơ tiểu đường

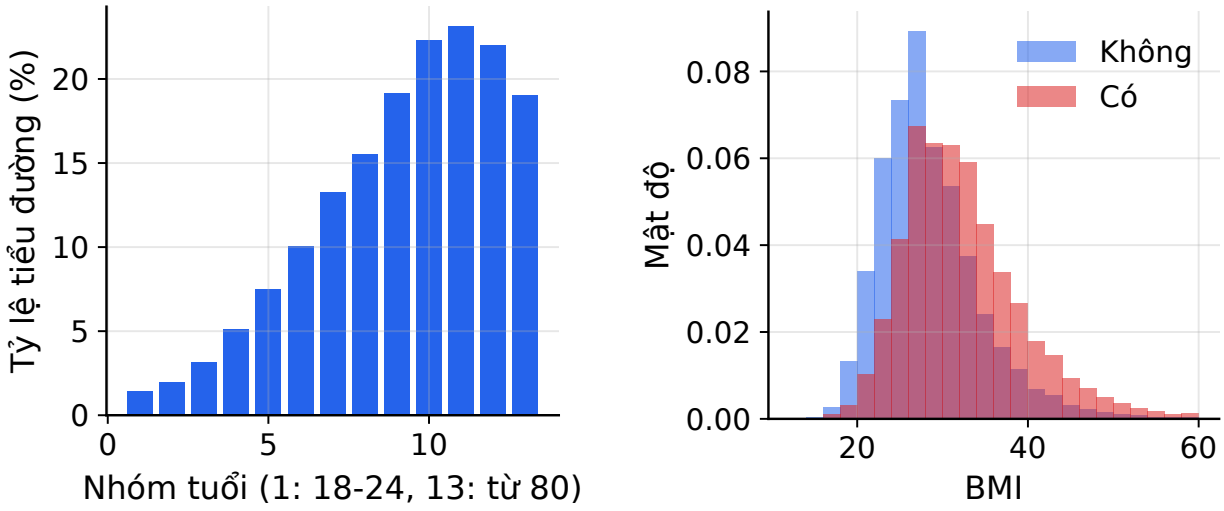
2.3.1 Nguồn và kích cỡ

Bộ *CDC Diabetes Health Indicators* [76] (UCI id 891, liên kết) lấy từ khảo sát qua điện thoại BRFSS 2015 của CDC Hoa Kỳ. Bộ gồm 253.680 hồ sơ, 21 đặc trưng, 12,87 MB. Nhãn `Diabetes_binary` cho biết người trả lời có tiểu đường hoặc tiền tiểu đường. Vì dữ liệu chủ yếu do người trả lời tự khai, kết quả ở đây chỉ mang tính học thuật, không dùng cho chẩn đoán.

Bảng 2.2. Năm hồ sơ thực của CDC Diabetes (một phần các cột)

Nhãn	BMI	HighBP	HighChol	GenHlth	PhysHlth	Age	Income
0	23	1	0	2	0	13	6
1	22	0	0	2	5	9	8
0	26	1	1	2	0	11	6
1	39	1	0	4	0	11	5
0	22	0	0	1	0	10	8

2.3.2 Phân bố và nhận xét



(a) Tỷ lệ bệnh theo nhóm tuổi

(b) BMI theo nhãn

Hình 2.7. CDC Diabetes: nguy cơ tăng theo tuổi, phân bố BMI của nhóm bệnh lệch phải

Nhận xét: Dữ liệu không có ô trống (0) nhưng có 24.206 hồ sơ trùng khít. Sau khi bỏ trùng còn 229.474 hồ sơ, tỷ lệ dương 15,29 %, cao hơn con số 13,93 % trước khi bỏ trùng vì hồ sơ trùng phần lớn là người khỏe. Trung vị BMI của nhóm bệnh là 31, nhóm không bệnh là 27. Dữ liệu không có HbA1c hay đường huyết, nên bài toán khó hơn hẳn các bộ có chỉ số xét nghiệm.

2.3.3 Tiền xử lý chống rò rỉ

Hồ sơ trùng phải bỏ *trước* khi chia, nếu không cùng một hồ sơ có thể nằm ở cả train và test. Chia phân tầng 70/15/15 rồi mới fit `StandardScaler` trên train cho bảy cột có thứ bậc; 14 cột nhị phân giữ nguyên. Ngưỡng quyết định được quét trên validation theo F1 macro.

Đoạn mã 2.3. Chia tập trước, fit bộ chuẩn hóa chỉ trên train

```
# tlf/ch2_data.py
def cdc_split():
    df = load_cdc()
    X, y = df[CDC_SCALE + CDC_BINARY],
     ↪  df[CDC_TARGET].to_numpy(dtype=int)
    X_tr, X_va, X_te, y_tr, y_va, y_te = split_70_15_15(X, y)
    pre = ColumnTransformer([("num", StandardScaler(), CDC_SCALE)],
     ↪  remainder="passthrough",
                            verbose_feature_names_out=False)
    Xs = [pre.fit_transform(X_tr), pre.transform(X_va),
     ↪  pre.transform(X_te)]
    Xs = [x.astype(np.float32) for x in Xs]
    return Split(*Xs, y_tr, y_va, y_te, pre, "clf", ["Không", "Có"],
     ↪  list(pre.get_feature_names_out()))
```

Đoạn mã 2.4. Chọn ngưỡng trên validation; test không tham gia

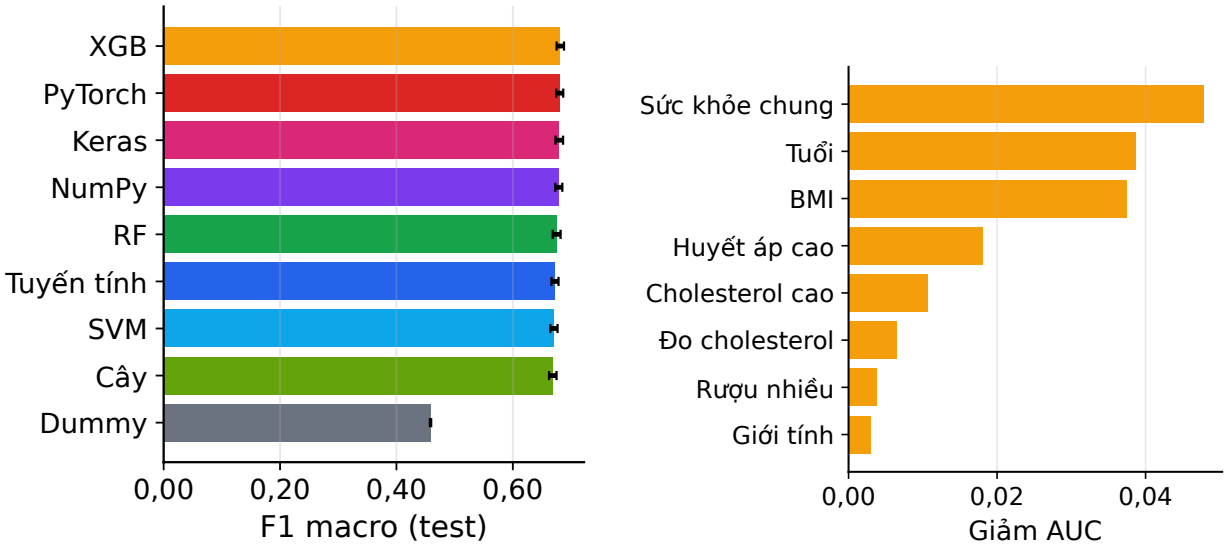
```
# tlf/ch2_metrics.py
def best_threshold(y_va, p_va):
    """Quét ngưỡng trên validation, lấy ngưỡng cho F1 macro cao nhất.
     ↪  Test không tham gia."""
    grid = np.arange(0.05, 0.71, 0.01)
    scores = [f1_score(y_va, p_va >= t, average="macro") for t in grid]
    return float(grid[int(np.argmax(scores))])
```

2.3.4 Kết quả và phân tích lỗi

Nhận xét: Accuracy dễ gây nhầm ở đây: Dummy luôn đoán "không bệnh" đạt accuracy 0,8470, cao hơn XGBoost (0,8247), nhưng F1 macro chỉ 0,4586 và không phát hiện được ca nào. XGBoost cao nhất với F1 macro 0,6815, AUC 0,8223, nhưng khoảng tin cậy của nó chồng với MLP và Random Forest, nên chưa đủ cơ sở nói nó hơn hai mô hình này. LogReg đạt 0,6726, kém chưa tới 0,01. Với dữ liệu khảo sát

Bảng 2.3. CDC Diabetes trên test, ngưỡng chọn trên validation; F1 macro kèm khoảng tin cậy 95 %

Mô hình	F1 macro	AUC	Recall dương	Precision dương	KTC 95 %	Giây
Dummy	0,4586	0,5000	0,0000	0,0000	[0,458; 0,460]	0,0
LogReg/Ridge	0,6726	0,8125	0,5168	0,4114	[0,666; 0,678]	1,8
Linear SVM	0,6709	0,8122	**0,5620**	0,3948	[0,664; 0,677]	8,7
Cây quyết định	0,6689	0,8040	0,5130	0,4049	[0,662; 0,675]	3,3
Random Forest	0,6756	0,8128	0,5151	0,4186	[0,669; 0,682]	51,8
XGBoost	**0,6815**	**0,8223**	0,5041	**0,4367**	[0,675; 0,688]	12,4
MLP NumPy	0,6792	0,8211	0,5474	0,4151	[0,673; 0,685]	76,0
MLP Keras	0,6799	0,8206	0,5413	0,4186	[0,673; 0,686]	30,9
MLP PyTorch	0,6807	0,8219	0,5425	0,4198	[0,674; 0,687]	10,3



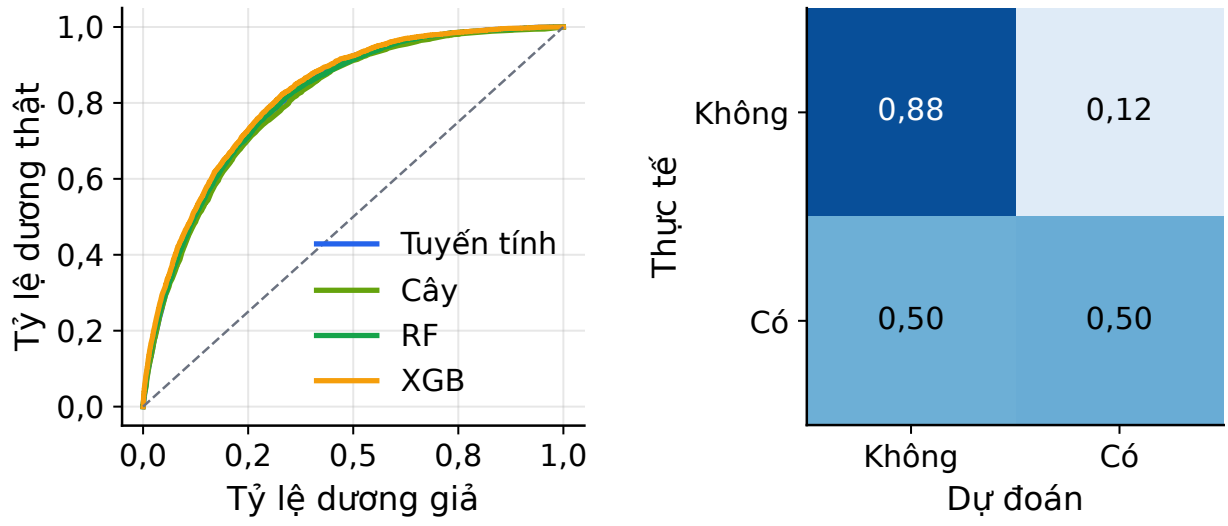
(a) F1 macro trên test

(b) Tầm quan trọng hoán vị

Hình 2.8. CDC Diabetes: so sánh mô hình và các đặc trưng XGBoost dựa vào nhiều nhất

này, các họ mô hình gần như chạm cùng một trần. Riêng ngưỡng lại có tác dụng rõ: cùng XGBoost, ngưỡng 0,5 cho F1 macro 0,5923, recall 0,1689; ngưỡng chọn trên validation cho 0,6815 và recall 0,5041.

Nhận xét: Ở ngưỡng 0,3000, XGBoost có recall 0,5041, precision 0,4367. Trong sàng lọc, bỏ sót ca bệnh thường đắt hơn báo động nhầm, nên ngưỡng thực tế có thể đặt thấp hơn tùy chi phí xét nghiệm. Các đặc trưng hàng đầu ở Hình 2.8b khớp hiểu biết y khoa, nhưng đây là liên hệ thống kê, không phải quan hệ nhân quả.



(a) Đường ROC **(b)** Ma trận nhầm lẫn XGBoost, chuẩn hóa theo hàng

Hình 2.9. CDC Diabetes: bốn đường ROC gần trùng nhau; ở ngưỡng đã chọn, gần một nửa ca dương vẫn bị bỏ sót

2.4 Vietnam Housing: ước lượng giá nhà

2.4.1 Nguồn và kích cỡ

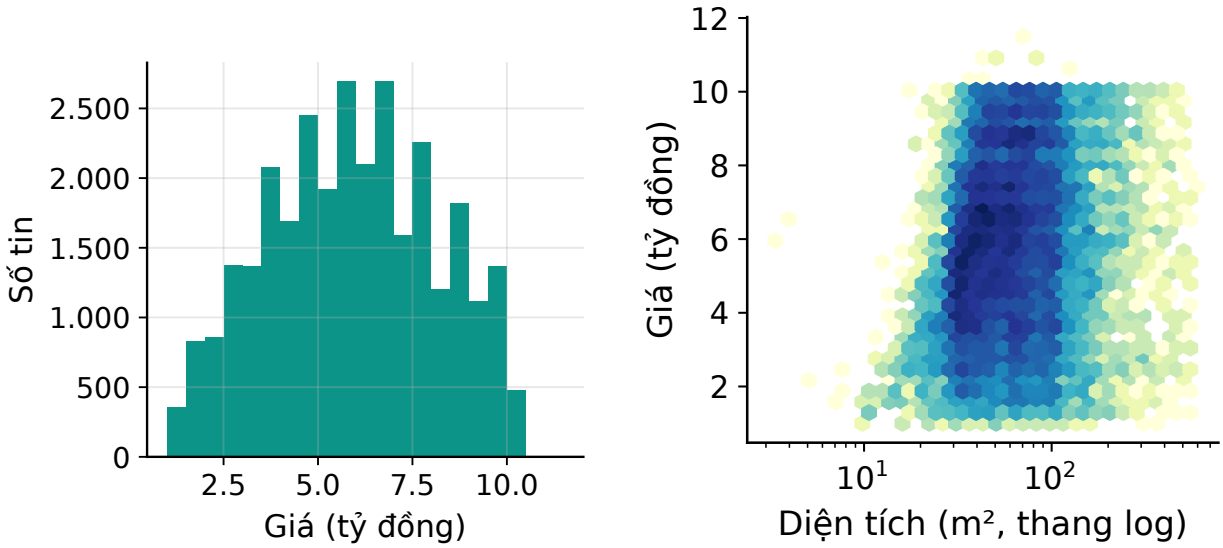
Bộ *House Price Prediction Dataset Vietnam 2024* [77] (Kaggle, giấy phép CC0) gồm 30.229 tin rao bán nhà, 11 đặc trưng, 3,370 MB. Nhãn `Price` tính bằng tỷ đồng.

Bảng 2.4. Năm tin rao thực (PN: phòng ngủ; HĐMB: hợp đồng mua bán; dấu gạch: tin không ghi)

Địa chỉ	m²	Tầng	PN	Pháp lý	Tỷ
Đường Minh Khai, Phường Minh Khai, Hai Bà Trưng, Hà Nội	36	4	2	HĐMB	6,00
11A, Đường Nguyễn Hiền, Phường 4, Quận 3, Hồ Chí Minh	30	2	1	Sổ đỏ	5,60
Đường Phạm Tu, Phường Phước Mỹ, Sơn Trà, Đà Nẵng	56	3	3	Sổ đỏ	6,85
Đường Ngọc Hồi, Xã Ngũ Hiệp, Thanh Trì, Hà Nội	60	5	6	Sổ đỏ	8,30
Phường Bình Chuẩn, Thuận An, Bình Dương	60	–	–	–	3,70

2.4.2 Phân bố và nhận xét

Nhận xét: Giá nằm trọn trong khoảng 1 đến 11,50 tỷ đồng, trung bình 5,872, độ lệch chuẩn 2,212. Khoảng hẹp này cho thấy dữ liệu đã được lọc từ trước, nên mô



(a) Phân bố giá **(b)** Giá theo diện tích

Hình 2.10. Vietnam Housing: giá trải trong khoảng hẹp và có dấu hiệu bị cắt ở hai đầu

hình không dùng được cho nhà ngoài khoảng đó. 71,53 % tin thuộc Hà Nội và TP. Hồ Chí Minh. Dữ liệu thiếu nhiều: 82,65 % tin không ghi hướng ban công, 46,71 % không ghi nội thất. Tương quan Pearson giữa diện tích và giá chỉ 0,0976 (Hình 2.10b): cùng diện tích, giá khác nhau rất xa tùy vị trí, nên mô hình cần kết hợp diện tích với vị trí.

2.4.3 Tiền xử lý chống rò rỉ

Cột `Address` có 10.265 giá trị khác nhau, được tách thành tỉnh, quận, phường và đường. Có hai bẫy rò rỉ. Thứ nhất, cùng một địa chỉ xuất hiện trong nhiều tin, có 1.322 tin gần như trùng lặp, nên chia ngẫu nhiên theo tin sẽ để mô hình thấy gần như chính căn nhà ở test. Vì vậy dữ liệu được chia *theo địa chỉ*: số địa chỉ chung giữa train và test là 0. Thứ hai, mã hóa quận, phường, đường theo giá trung bình [78] dùng chính nhãn, nên dùng `TargetEncoder` với fit chéo năm phần trên train. Cột số được lấy log, điền trung vị và thêm cờ thiếu, vì việc tin không ghi một trường cũng là thông tin.

2.4.4 Kết quả và phân tích lỗi

Nhận xét: XGBoost cho RMSE 1,397, R^2 0,6084, khoảng tin cậy không chồng với Ridge (1,494), nên phần hơn là có thật. Dummy có RMSE 2,235. Mọi mô hình đều giảm được khoảng một phần ba sai số so với mốc, nhưng sai số trung bình 1,055 tỷ đồng vẫn lớn so với giá trung bình 5,872 tỷ. Diện tích đứng đầu về tầm quan trọng

Đoạn mã 2.5. Tách địa chỉ tự do thành năm cột vị trí

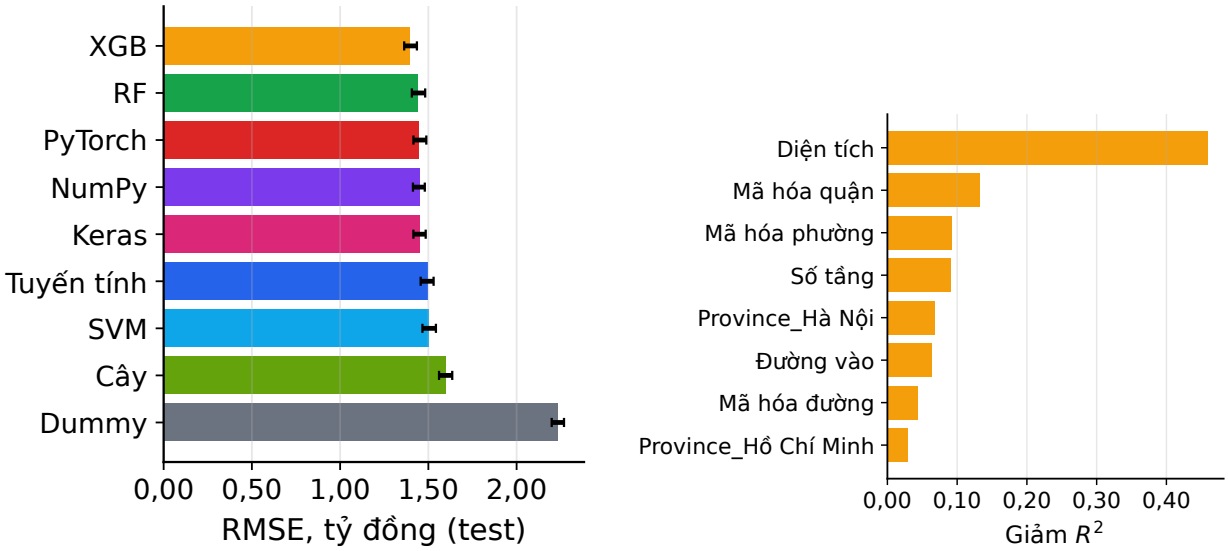
```python
# tlf/ch2_data.py
def parse_address(addr):
    """'Đường X, Phường Y, Quận Z, Tỉnh T' -> loại địa chỉ và bốn cấp vị trí; cấp vắng để rỗng."""
    parts = [p.strip() for p in str(addr).split(",") if p.strip()]
    parts = [""] * (4 - len(parts)) + parts[-4:]
    street, ward, district, province = parts
    if street.startswith("Dự án"):
        kind = "Dự án"
    elif street.startswith(("Đường", "Phố")):
        kind = "Đường phố"
    else:
        kind = "Khác"
    return {"Province": province, "Kind": kind,
            "District": f"{district}|{province}",
            "Ward": f"{ward}|{district}|{province}",
            "Street": f"{street}|{district}|{province}"}
```

Đoạn mã 2.6. Chia theo nhóm địa chỉ: tin cùng địa chỉ nằm trọn một phần

```python
# tlf/ch2_data.py
def group_split_idx(groups, seed=SEED):
    """Chia 70/15/15 theo Address: mọi tin đăng cùng địa chỉ nằm trọn trong một phần."""
    idx = np.arange(len(groups))
    gss = GroupShuffleSplit(n_splits=1, test_size=0.15, random_state=seed)
    a, te = next(gss.split(idx, groups=groups))
    gss = GroupShuffleSplit(n_splits=1, test_size=0.15 / 0.85, random_state=seed)
    i_tr, i_va = next(gss.split(a, groups=groups[a]))
    return a[i_tr], a[i_va], te
```

Bảng 2.5. Vietnam Housing trên test, đơn vị tỷ đồng; RMSE kèm khoảng tin cậy 95 %

Mô hình	MAE	RMSE	R^2	KTC 95 %	Giây
Dummy	1,8573	2,2349	−0,0018	[2,199; 2,269]	0,0
LogReg/Ridge	1,1629	1,4945	0,5521	[1,456; 1,530]	0,0
Linear SVM	1,1590	1,5051	0,5457	[1,466; 1,543]	0,9
Cây quyết định	1,2327	1,5977	0,4880	[1,560; 1,635]	0,1
Random Forest	1,1024	1,4432	0,5823	[1,406; 1,482]	2,6
XGBoost	**1,0551**	**1,3974**	**0,6084**	[1,363; 1,435]	1,4
MLP NumPy	1,1153	1,4491	0,5788	[1,413; 1,480]	36,8
MLP Keras	1,1097	1,4510	0,5777	[1,415; 1,485]	35,3
MLP PyTorch	1,1105	1,4472	0,5799	[1,415; 1,488]	44,0



(a) RMSE trên test

(b) Tầm quan trọng hoán vị

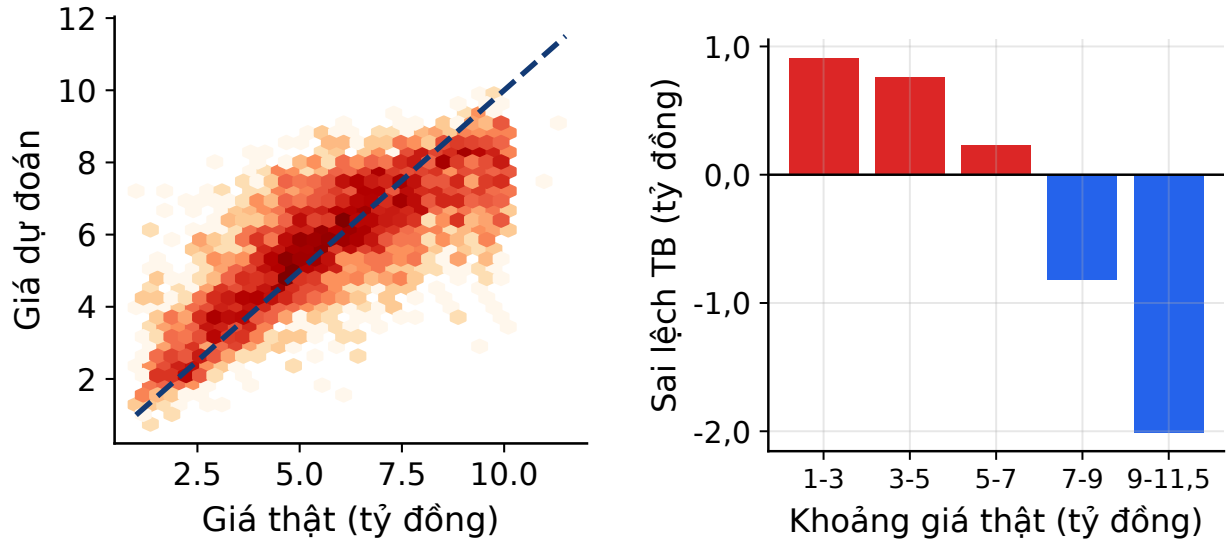
Hình 2.11. Vietnam Housing: so sánh mô hình và các đặc trưng XGBoost dựa vào nhiều nhất

dù tương quan tuyến tính thấp: giá phụ thuộc vào tích diện tích với đơn giá khu vực, kiểu tương tác mà cây bắt được còn Ridge thì không.

Cách chia tập làm đổi kết quả

Cùng cấu hình XGBoost, chia ngẫu nhiên theo tin cho R^2 0,6627, RMSE 1,289; chia theo địa chỉ cho R^2 0,6084, RMSE 1,397. Chênh lệch này là phần lạc quan giả do cùng địa chỉ xuất hiện ở cả hai phía, và số theo địa chỉ mới là ước lượng cho một căn ở địa chỉ chưa từng thấy.

Nhận xét: Hình 2.12 cho thấy mô hình co dự đoán về trung bình: thiếu thông tin như chất lượng nhà và vị trí cụ thể trong ngõ, nó đoán an toàn quanh mức giữa.



(a) Dự đoán so với giá thật **(b)** Sai lệch theo khoảng giá

Hình 2.12. Lỗi của XGBoost: dự đoán co về giữa, nhà rẻ bị đoán cao, nhà đắt bị đoán thấp

Đây là giới hạn của dữ liệu tin rao hơn là của thuật toán, và giải thích vì sao các họ mô hình khác nhau cùng dừng quanh một mức sai số.

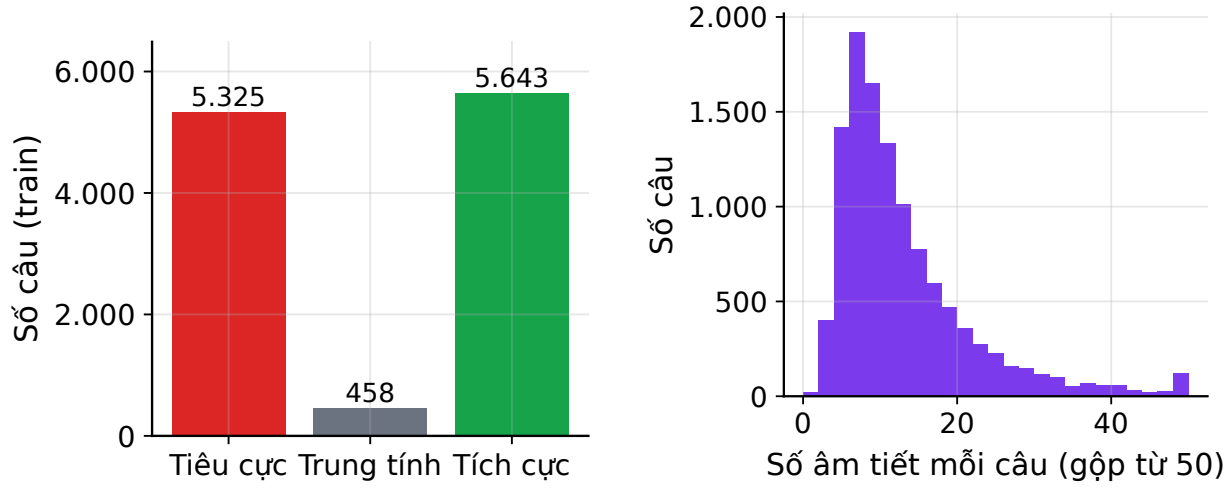
2.5 UIT-VSFC: cảm xúc phản hồi sinh viên

2.5.1 Nguồn và kích cỡ

UIT-VSFC [79] (Hugging Face) gồm 16.175 câu phản hồi của sinh viên về môn học và giảng viên, 0,6400 MB, gán ba nhãn cảm xúc. Thẻ dữ liệu ghi giấy phép *unknown*, nhóm tác giả phát hành cho mục đích nghiên cứu. Giữ nguyên phân chia gốc: 11.426 train, 1.583 validation, 3.166 test.

Bảng 2.6. Năm câu thực trong tập test của UIT-VSFC

Nội dung	Nhãn
vì em đã gần như không có được gì !	Tiêu cực
thầy rất đúng giờ !	Tích cực
nên tăng cường thời gian tiếp xúc các thiết bị thực tế .	Tiêu cực
nhiệt tình , dạy hay .	Tích cực
đôi lúc cô có kể vài câu chuyện hay liên quan đến bài học , hầu hết các tiết học cô luôn mỉm cười nên khi học cảm thấy thoải mái .	Tích cực



(a) Phân bố nhãn **(b)** Độ dài câu

Hình 2.13. UIT-VSFC: lớp trung tính rất hiếm, câu ngắn

2.5.2 Phân bố và nhận xét

Nhận xét: Lớp trung tính chỉ chiếm 4,32 %, nên accuracy không phản ánh được khả năng nhận ra lớp này và F1 macro là chỉ số chính. Câu ngắn, trung vị 10 âm tiết. Tên riêng đã được che bằng các token `wzjwz...` (302 câu). Không câu test nào trùng với train (0).

2.5.3 Tiền xử lý chống rò rỉ

Văn bản được đưa về chữ thường, gom token che tên thành một token, bỏ số và dấu câu, giữ dấu tiếng Việt. Không bỏ từ dừng, vì danh sách từ dừng thường chứa "không" mà từ này đảo cảm xúc. TF-IDF dùng từ đơn và cặp liền kề: tiếng Việt viết tách âm tiết nên cặp giữ được cả "giáo viên" lẫn "không hay". Từ điển và IDF chỉ fit trên train. Mô hình tuyến tính dùng toàn bộ đặc trưng; cây và MLP dùng 5.000 đặc trưng chọn bằng χ^2, cũng chỉ fit trên train. Do lớp lệch, mọi mô hình dùng trọng số lớp nghịch đảo tần suất.

2.5.4 Kết quả và phân tích lỗi

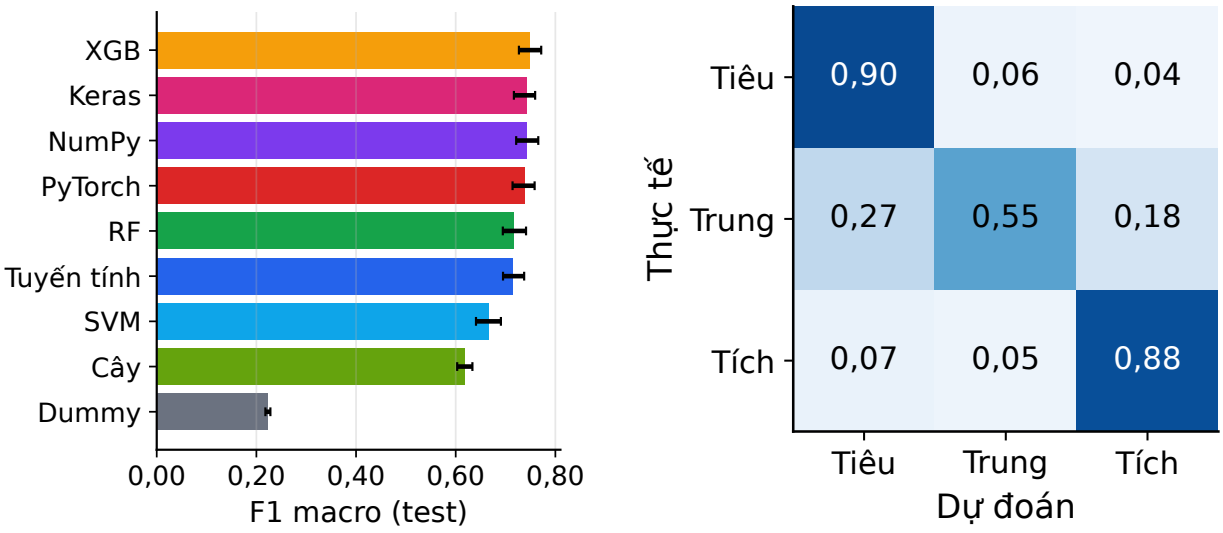
Nhận xét: XGBoost đạt F1 macro 0,7494, Dummy chỉ 0,2229, nhưng khoảng tin cậy chồng với cả ba bản MLP nên nhóm dẫn đầu là ngang nhau. Linear SVM minh họa rõ giới hạn của accuracy: accuracy 0,8834 cao nhất bảng, trong khi F1 macro 0,6654 thấp vì F1 lớp trung tính chỉ 0,1856. Khoảng cách giữa các mô hình gần như nằm hết ở lớp hiếm: F1 của XGBoost cho ba lớp tiêu cực, trung tính, tích cực là 0,8959, 0,4423, 0,9102. Bảng 2.8 cho thấy từ đặc trưng hợp lý: "nhiệt tình", "dễ hiểu" cho lớp tích cực; "khó", "chưa", "không" cho lớp tiêu cực. Bốn câu ở

Đoạn mã 2.7. TF-IDF và chọn đặc trưng fit trên train của phân chia gốc

```
# tlf/ch2_data.py
def vsfc_split(top_k=None):
    """top_k=None: TF-IDF đầy đủ, thưa. top_k=k: giữ k đặc trưng chi2 cao nhất, ma trận đặc cho cây và MLP."""
    tr, va, te = load_vsfc("train"), load_vsfc("validation"), load_vsfc("test")
    steps = [("tfidf", vsfc_vectorizer())]
    if top_k:
        steps.append(("chi2", SelectKBest(chi2, k=top_k)))
    pre = Pipeline(steps)
    y_tr = tr["sentiment"].to_numpy(dtype=int)
    Xs = [pre.fit_transform(tr["sentence"], y_tr), pre.transform(va["sentence"]), pre.transform(te["sentence"])]
    if top_k:
        Xs = [x.toarray().astype(np.float32) for x in Xs]
    return Split(*Xs, y_tr, va["sentiment"].to_numpy(dtype=int), te["sentiment"].to_numpy(dtype=int),
                 pre, "clf", VSFC_LABELS, list(pre.get_feature_names_out()))
```

Bảng 2.7. UIT-VSFC trên test; F1 macro kèm khoảng tin cậy 95 %

Mô hình	Accuracy	F1 macro	KTC 95 %	Giây
Dummy	0,5022	0,2229	[0,218; 0,228]	0,0
LogReg/Ridge	0,8648	0,7151	[0,695; 0,737]	1,2
Linear SVM	**0,8834**	0,6654	[0,641; 0,691]	0,3
Cây quyết định	0,7113	0,6178	[0,603; 0,634]	16,4
Random Forest	0,8418	0,7173	[0,695; 0,741]	29,5
XGBoost	0,8730	**0,7494**	[0,727; 0,772]	214,5
MLP NumPy	0,8809	0,7424	[0,721; 0,766]	19,4
MLP Keras	0,8791	0,7429	[0,717; 0,759]	31,1
MLP PyTorch	0,8742	0,7384	[0,714; 0,758]	4,5



(a) F1 macro trên test **(b)** Ma trận nhầm lẫn XGBoost, chuẩn hóa theo hàng

Hình 2.14. UIT-VSFC: so sánh mô hình và lỗi của XGBoost

Bảng 2.9 chỉ là ví dụ minh họa, không phải thống kê: có câu mô tả không rõ thái độ và câu cần ngữ cảnh mà túi từ và cặp từ không nắm được.

Bảng 2.8. Từ đặc trưng của từng lớp, đọc từ hệ số LogReg

Lớp	Từ nổi bật
Tiêu cực	nên, cần, chưa, khó, hơn, quá, ít, hơi, không, nghỉ
Trung tính	không, dung ôn, ơn, không có, giống, tra thường, chưa có, rất hiền, đủ số, có tương
Tích cực	tốt, rất, hay, dễ hiểu, tình, vui, nhiệt, nhiệt tình, tận, hiểu

Bảng 2.9. Bốn câu XGBoost đoán sai, lấy nguyên văn từ test

Nội dung	Nhãn thật	Dự đoán
giữa lý thuyết từ vựng với trò chơi để dễ tiếp thu .	Tiêu cực	Tích cực
trong trường macbook thầy số hai thì không có máy nào số một .	Tích cực	Tiêu cực
như vậy tụi em sẽ định hướng tốt hơn và tập trung vào những thứ cần thiết .	Trung tính	Tiêu cực
cách mà cô tiếp cận với sinh viên .	Trung tính	Tích cực

2.6 MLP trên NumPy, Keras và PyTorch

Cả ba bản dùng cùng kiến trúc 128–64 với ReLU và dropout, cùng Adam, tốc độ học, cỡ lô và dừng sớm theo validation. Khác nhau chỉ ở bộ khởi tạo, nguồn số ngẫu nhiên và nơi tính toán: NumPy và Keras chạy CPU, PyTorch chạy GPU. Housing và VSFC chạy ba seed, CDC chạy một seed để giữ trần tài nguyên. Trước khi huấn luyện, gradient viết tay được kiểm tra bằng sai phân trung tâm:

Kiểm tra gradient (grad_check)

số tham số: 119
||số học - lan truyền ngược|| / (||số học|| + ||lan truyền ngược||) = $4{,}74 \times 10^{-10}$

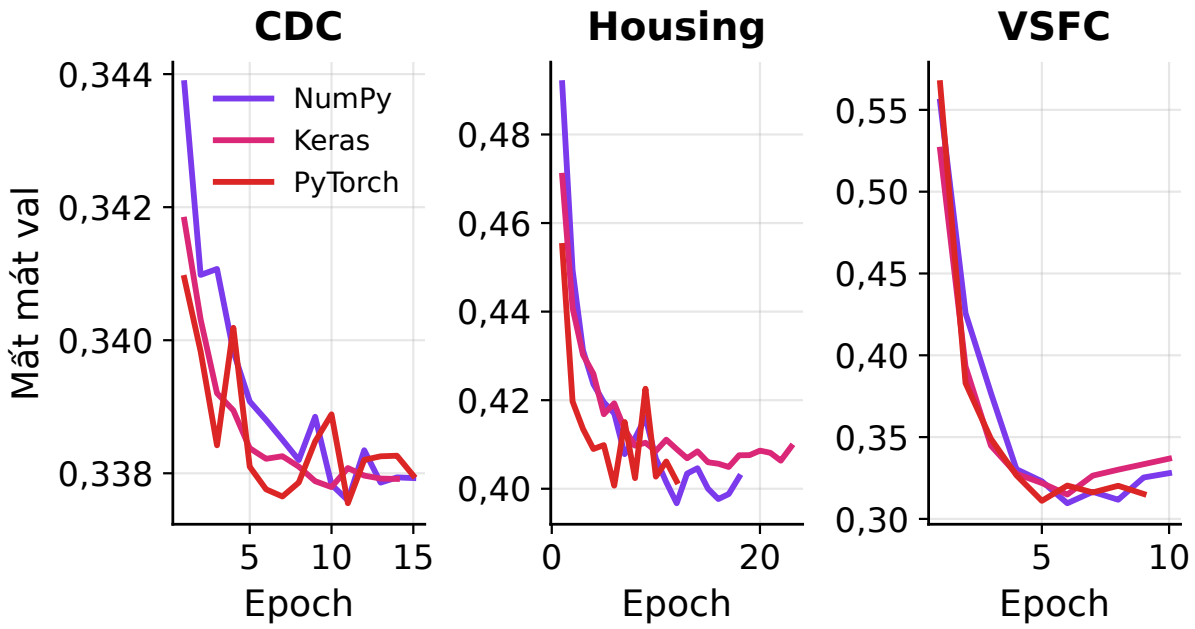
Đoạn mã 2.8. Đối chiếu lan truyền ngược với sai phân trung tâm

```python
# tlf/ch2_mlp_numpy.py
def grad_check(seed=0, eps=1e-6):
    """So đạo hàm lan truyền ngược với sai phân trung tâm trên mạng nhỏ: ||a-b|| / (||a||+||b||), số tham số."""
    rng = np.random.default_rng(seed)
    X, y, w = rng.normal(size=(40, 6)), rng.integers(0, 3, 40), np.ones(40)
    net = NumpyMLP(6, 3, hidden=(8, 5), drop=0.0, seed=seed, dtype=np.float64)
    # Bias khác 0 để sai phân không rơi đúng vào điểm gãy của ReLU tại a = 0.
    net.b = [p + rng.normal(0, 0.1, p.shape) for p in net.b]
    _, gW, gb = net.loss_grad(X, y, w, train=False)
    numeric, exact = [], []
    for P, G in zip(net.W + net.b, gW + gb):
        for i in np.ndindex(P.shape):
            old = P[i]
            P[i] = old + eps
            up = net.loss_grad(X, y, w, train=False)[0]
            P[i] = old - eps
            down = net.loss_grad(X, y, w, train=False)[0]
            P[i] = old
            numeric.append((up - down) / (2 * eps))
            exact.append(G[i])
    a, b = np.array(numeric), np.array(exact)
    return float(np.linalg.norm(a - b) / (np.linalg.norm(a) + np.linalg.norm(b))), len(a)
```

Nhận xét: Sai số tương đối $4{,}74 \times 10^{-10}$ cho thấy lan truyền ngược viết tay khớp

Bảng 2.10. MLP trên ba nền tảng; chỉ số chính là F1 macro (CDC, VSFC) hoặc RMSE (Housing), trung bình theo seed

Dataset	Nền tảng	Chỉ số chính	Độ lệch seed	Epoch TB	Giây
CDC Diabetes	MLP NumPy	0,6792	–	15,0	76,0
CDC Diabetes	MLP Keras	0,6799	–	14,0	30,9
CDC Diabetes	MLP PyTorch	0,6807	–	15,0	10,3
Vietnam Housing	MLP NumPy	1,4491	0,0074	24,3	36,8
Vietnam Housing	MLP Keras	1,4510	0,0015	23,0	35,3
Vietnam Housing	MLP PyTorch	1,4472	0,0048	17,7	44,0
UIT-VSFC	MLP NumPy	0,7424	0,0023	10,0	19,4
UIT-VSFC	MLP Keras	0,7429	0,0027	10,0	31,1
UIT-VSFC	MLP PyTorch	0,7384	0,0023	9,3	4,5



Hình 2.15. Mất mát validation theo epoch của seed cuối trên ba nền tảng

với đạo hàm số. Ba nền tảng cho chỉ số gần như nhau: RMSE Housing từ 1,447 đến 1,451, F1 macro VSFC từ 0,7384 đến 0,7429, F1 macro CDC từ 0,6792 đến 0,6807, các hiệu số đều nhỏ so với khoảng tin cậy. Thư viện không làm đổi kết quả khi cấu hình giống nhau; điều nó đổi là công sức viết mã và thời gian chạy. Cột thời gian chỉ mang tính tham khảo vì máy được chia sẻ với các thực nghiệm khác trong lúc chạy.

2.7 Tổng kết và triển khai

Ba kết luận từ ba bài toán

Chênh lệch giữa các mô hình mạnh nhỏ hơn chênh lệch do xử lý dữ liệu. XGBoost đứng đầu cả ba bài (F1 macro 0,6815 trên CDC, RMSE 1,397 trên Housing, F1 macro 0,7494 trên VSFC) nhưng khoảng tin cậy của nó chồng với các mô hình mạnh khác; chỉ ở Housing nó tách rõ khỏi mô hình tuyến tính. Trong khi đó, chọn ngưỡng và cách chia tập làm đổi kết quả rõ hơn.

Mốc Dummy và khoảng tin cậy là bắt buộc. Không có mốc thì không thấy accuracy 0,8470 của CDC là vô nghĩa; không có khoảng tin cậy thì dễ đọc hiệu số 0,001 thành thắng lợi.

Giới hạn nằm ở dữ liệu. CDC thiếu chỉ số xét nghiệm, Housing thiếu đặc điểm cụ thể của căn nhà, VSFC thiếu mẫu trung tính. Muốn cải thiện đáng kể cần dữ liệu tốt hơn trước khi cần mô hình phức tạp hơn.

Ba mô hình tốt nhất theo validation được lưu thành Pipeline gồm tiền xử lý và mô hình trong `models/ch2/`, phục vụ Web App qua `tlf/serve_ch2.py`. Mô hình nạp lười một lần, chạy CPU và không cần TensorFlow. Đầu vào được kiểm tra miền giá trị, ô số để trống trong bài giá nhà đi qua đúng nhánh trung vị và cờ thiếu như lúc huấn luyện. Kết quả y tế và giá nhà đều kèm ghi chú giới hạn. Ảnh giao diện ở Chương 9.

Đoạn mã 2.9. Dịch vụ dự đoán cảm xúc: nạp lười, kiểm tra đầu vào, trả nhãn và xác suất

```python
# tlf/serve_ch2.py
def predict_sentiment(payload):
    """XGBoost học trên ma trận đặc nên phải đổi sang đặc:
    với ma trận thưa, số 0 bị hiểu là giá trị thiếu và dự đoán sai."""
    text = str(payload.get("text", "")).strip()
    if not text:
        raise ValueError("Chưa nhập câu phản hồi")
    pipe = _model("vsfc")
    x = pipe["pre"].transform([text]).toarray().astype(np.float32)
    p = pipe["model"].predict_proba(x)[0]
    label = VSFC_LABELS[int(np.argmax(p))]
    return {"label": label, "probs": dict(zip(VSFC_LABELS, map(float,
     ↪  p))),
            "model": _best("vsfc")["name"]}
```

Chương 3

Mạng nơ-ron tích chập

Ảnh có cấu trúc không gian: điểm ảnh gần nhau liên quan mạnh, và một nét chữ vẫn là nét chữ dù nằm ở góc nào. Mạng kết nối đầy đủ bỏ qua cấu trúc đó, còn mạng nơ-ron tích chập (CNN) xây nó thành kiến trúc [80]. Chương này đi từ phép tích chập tới các kiến trúc hiện đại, đánh giá chúng trên ba miền ảnh khác nhau: chữ số đường phố, ảnh vệ tinh và mô bệnh học.

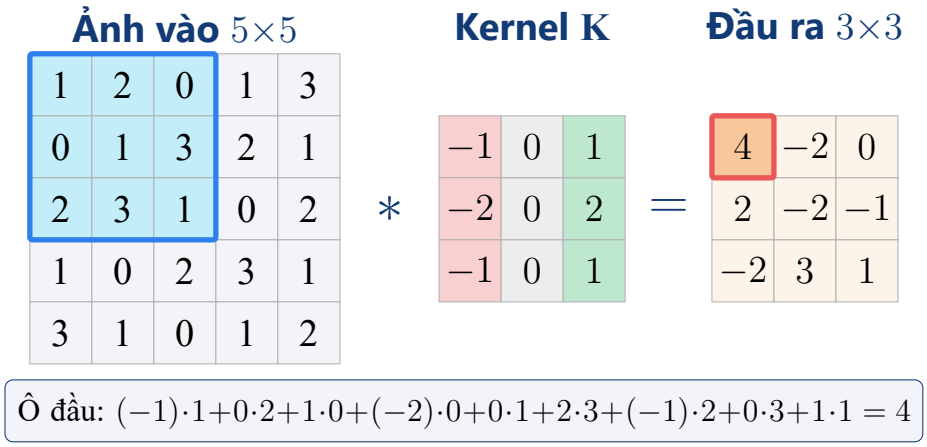
3.1 Cơ sở lý thuyết

3.1.1 Phép tích chập

Với ảnh vào $\mathbf{I}$ và kernel $\mathbf{K}$ cỡ $k\times k$, đặc trưng tại vị trí (i, j) là tổng có trọng số trên một cửa sổ nhỏ:

$$Z_{ij} = \sum_{u=0}^{k-1}\sum_{v=0}^{k-1} K_{uv}\, I_{i+u,\,j+v} + b. \tag{3.1}$$

Thư viện học sâu gọi phép này là tích chập dù về toán học đây là tương quan chéo, vì kernel không bị lật. Hai ý tưởng làm nên hiệu quả của nó: *kết nối cục bộ* (mỗi nơ-ron chỉ nhìn một cửa sổ) và *chia sẻ trọng số* (cùng kernel trượt khắp ảnh). Tầng N kernel $k\times k$ trên đầu vào M kênh chỉ có $(k^2M+1)N$ tham số, không phụ thuộc cỡ ảnh, và đầu ra dịch theo đầu vào. Với đầu vào D điểm, padding p, bước s, cỡ đầu ra là $\lfloor (D+2p-k)/s \rfloor + 1$. Hình 3.1 tính một ví dụ bằng số.

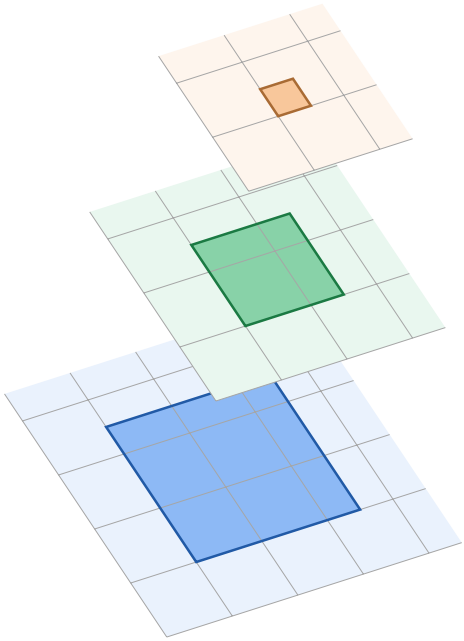


Hình 3.1. Ví dụ tính tích chập không padding: kernel Sobel phản hồi mạnh ở nơi cường độ đổi theo chiều ngang

3.1.2 Pooling và vùng cảm nhận

Max pooling 2×2 bước 2 giữ giá trị lớn nhất mỗi ô, giảm cỡ bản đồ một nửa theo mỗi chiều, không có tham số và chịu được dịch chuyển nhỏ. Global Average Pooling (GAP) lấy trung bình cả một kênh, thay cho Flatten để số tham số không còn phụ thuộc cỡ ảnh.

Vùng cảm nhận là phần ảnh vào ảnh hưởng tới một điểm của tầng đang xét. Với kernel k_ℓ, bước s_ℓ, vùng này tăng theo công thức ở Hình 3.2: mỗi lần giảm mẫu nhân đôi bước nhảy j, nên tầng sau mở rộng vùng nhìn nhanh hơn. Vùng cảm nhận vượt cỡ ảnh 32×32 nghĩa là tầng cuối đã thấy toàn bộ ảnh, dù trọng số thực tế tập trung ở giữa vùng.



Hình 3.2. Vùng cảm nhận lớn dần theo độ sâu; ba con số tính từ đúng dãy tầng của các mô hình trong chương

3.1.3 Chuẩn hóa theo lô

Batch Normalization [81] chuẩn hóa mỗi kênh theo lô rồi cho mạng học lại thang đo: $\hat{x} = \gamma\,(x - \mu_B)/\sigma_B + \beta$. Lúc huấn luyện dùng thống kê của lô, lúc suy luận dùng trung bình trượt đã lưu. Nhờ đó bề mặt mất mát mượt hơn, cho phép tốc độ học lớn, và mạng sâu hội tụ ổn định.

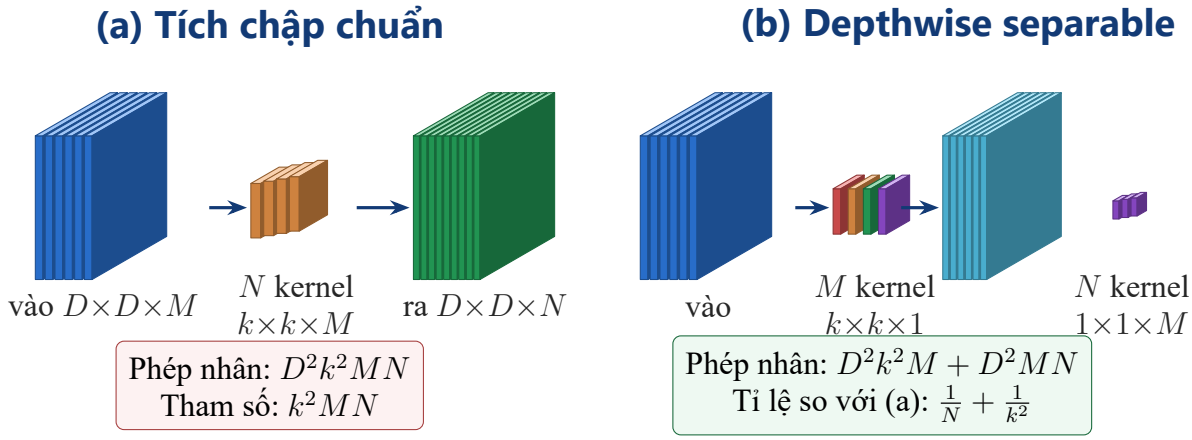
3.1.4 Kết nối tắt

Chồng thêm tầng lên một mạng thẳng có thể làm giảm cả độ đúng trên tập huấn luyện, dù về lý thuyết các tầng thêm có thể học ánh xạ đồng nhất. ResNet [82] đổi

mục tiêu của khối: học phần dư $F(\mathbf{x})$ thay vì toàn bộ ánh xạ, $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$. Khi đó $\partial \mathbf{y}/\partial \mathbf{x} = \partial F/\partial \mathbf{x} + \mathbf{I}$, số hạng đồng nhất giữ cho gradient không tắt dần qua nhiều tầng. Khi đổi kích thước hay số kênh, lối tắt dùng Conv 1×1.

3.1.5 Tích chập tách biệt chiều sâu

MobileNet [83] tách tích chập chuẩn thành hai bước: depthwise lọc riêng từng kênh, pointwise 1×1 trộn các kênh (Hình 3.3). Chẳng hạn $k = 3$, $M = 64$, $N = 128$ cần 73.728 tham số với tích chập chuẩn, còn tách ra chỉ cần $3^2 \cdot 64 + 64 \cdot 128 = 8.768$, ít hơn khoảng 8,4 lần. MobileNetV2 [84] thêm khối *inverted residual*: mở rộng kênh trước khi lọc depthwise, rồi chiếu tuyến tính về ít kênh, vì ReLU trên không gian hẹp làm mất thông tin.



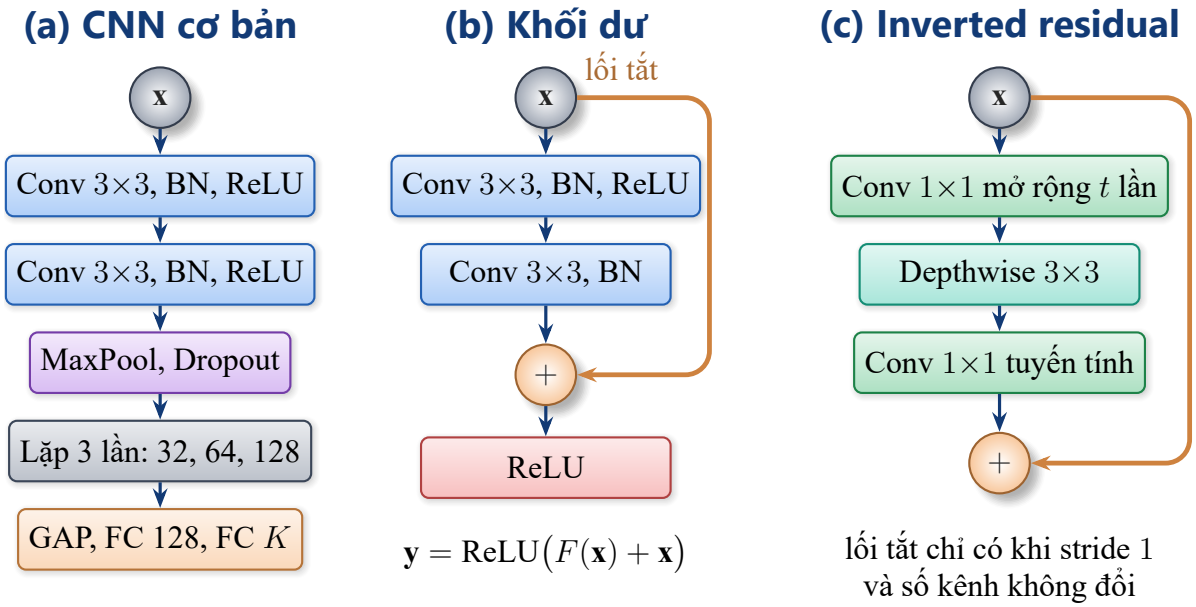
Hình 3.3. Tích chập chuẩn và tích chập tách biệt chiều sâu, cùng kích thước đầu vào và đầu ra

3.1.6 Các kiến trúc thực nghiệm

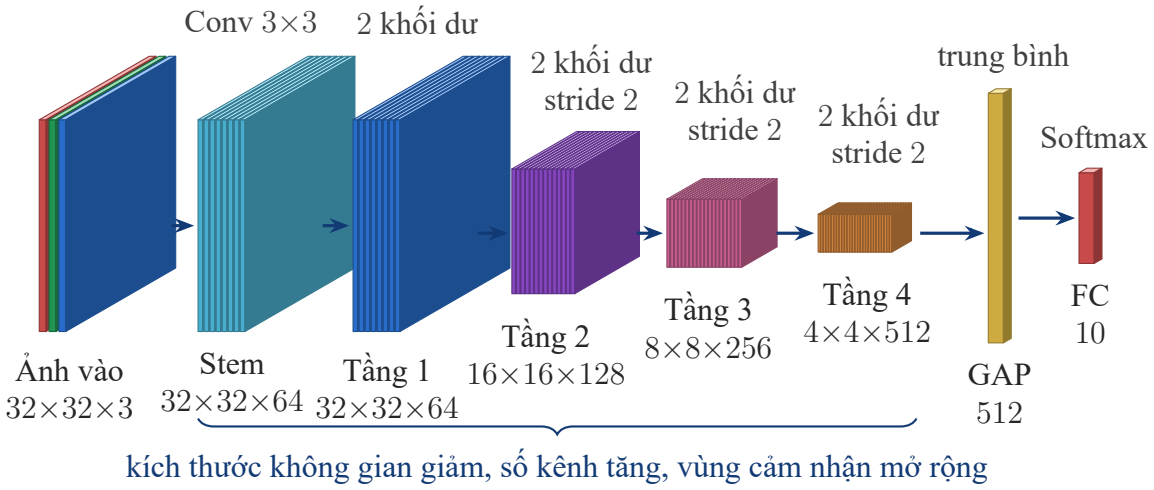
Hình 3.4 so ba khối dựng. **CNN cơ bản** có ba cụm hai lớp Conv kiểu VGG rồi GAP, là mô hình cơ sở với khoảng ba trăm nghìn tham số. **VGG-11** xếp tám lớp Conv có BN, thay ba lớp FC lớn của bản gốc bằng GAP. **ResNet-18** dùng stem bước 1 cho ảnh nhỏ và bốn tầng hai khối dư (Hình 3.5). **MobileNetV2** giữ hệ số mở rộng 6, đổi bước của hai nhóm đầu thành 1. Tất cả đều dùng GAP nên số tham số không đổi theo cỡ ảnh.

3.1.7 Giải thích dự đoán bằng Grad-CAM

Grad-CAM [85] cho biết vùng nào của ảnh khiến mạng chọn lớp c: lấy gradient của điểm lớp theo bản đồ đặc trưng tầng tích chập cuối, trung bình theo không gian được trọng số α_k^c của từng kênh, rồi cộng có trọng số các bản đồ (Hình 3.6). Độ

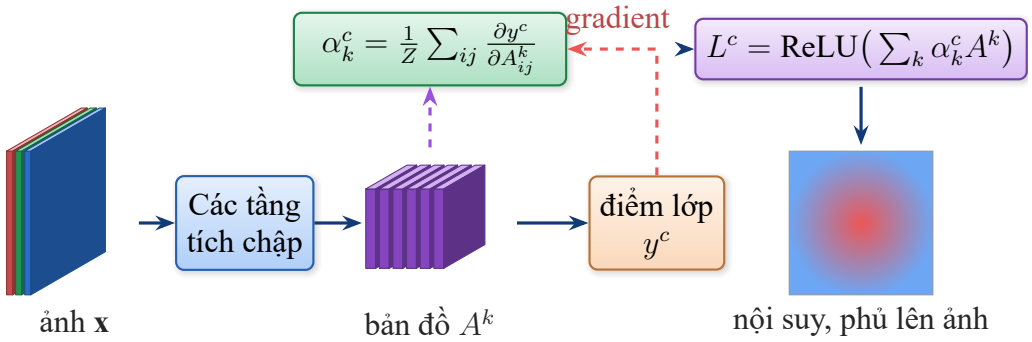


Hình 3.4. Khối dựng của CNN cơ bản, ResNet-18 và MobileNetV2



Hình 3.5. ResNet-18 trên ảnh SVHN: mỗi lát là một kênh đặc trưng, kích thước ghi đúng theo mô hình

phân giải bản đồ nhiệt bằng độ phân giải tầng cuối, nên mạng giảm mẫu nhiều sẽ cho bản đồ thô.



Hình 3.6. Luồng tính Grad-CAM; ô bản đồ nhiệt ở đây chỉ để minh họa

3.2 Dữ liệu

Ba bộ dữ liệu đến từ ba miền khác nhau, được giữ đủ số ảnh và đúng cách chia (Bảng 3.1). SVHN và EuroSAT dùng lại tệp đã chuẩn bị ở Tiểu luận 01 để khớp từng ảnh với dự đoán đã lưu, PathMNIST tải mới từ Zenodo. Mọi đường dẫn đều đã kiểm tra truy cập được ngày 05/11/2026.

Bảng 3.1. Tổng quan ba bộ dữ liệu ảnh

Bộ dữ liệu	Cỡ ảnh	Lớp	Train	Val	Test	Tải về (MB)	Lệch lớp	Giấy phép
SVHN	32×32×3	10	73.257	–	26.032	174,1	3,03	Chỉ dùng phi thương mại
EuroSAT	64×64×3	10	21.600	–	5.400	90,3	1,50	MIT
PathMNIST	28×28×3	9	89.996	10.004	7.180	196,1	1,64	CC BY 4.0

Lệch lớp là tỉ số lớp đông nhất trên lớp ít nhất. Dung lượng là tệp gốc tải về.

3.2.1 SVHN

Nguồn. Street View House Numbers [86], ảnh cắt từ biển số nhà trên Google Street View, bản cropped digits. Trang gốc của Stanford không truy cập được từ máy thực nghiệm; bản công khai: `https://huggingface.co/datasets/ufldl-stanford/svhn`. Giấy phép: Chỉ dùng phi thương mại.

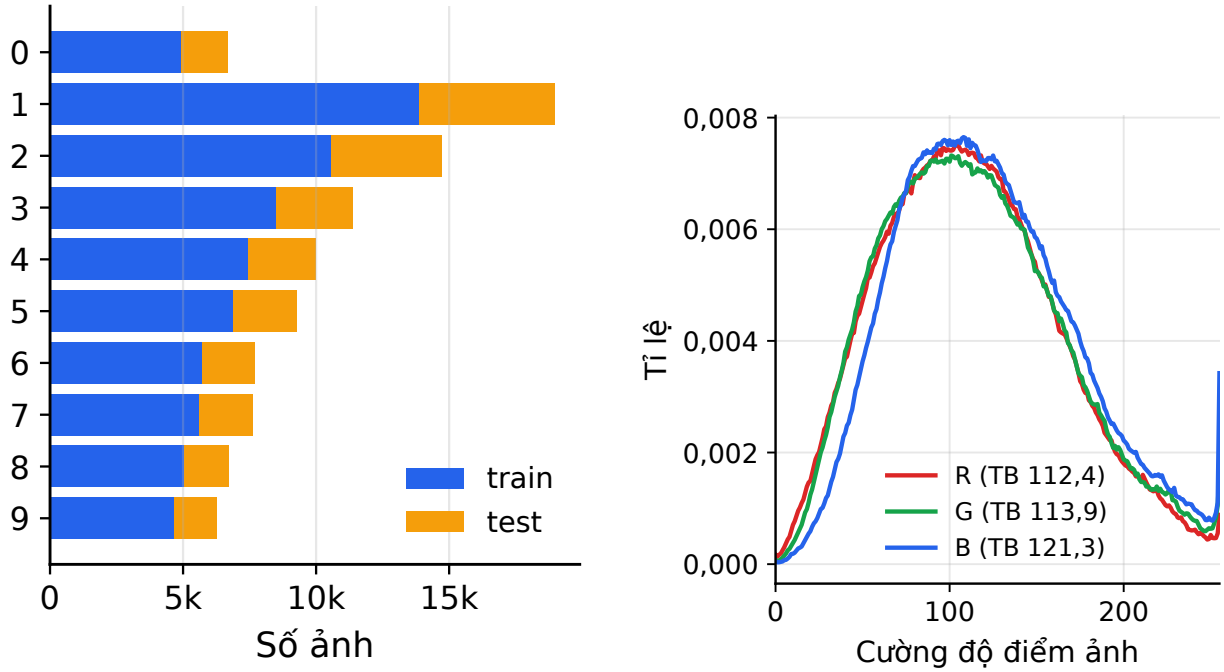
Kích cỡ. 99.289 ảnh màu 32×32, chia sẵn 73.257 ảnh train và 26.032 ảnh test; tệp gốc 174,10 MB.

Nhận xét: Lớp 1 chiếm 19,10%, gấp 3,032 lần lớp 9: đúng quy luật số tự nhiên có chữ số đầu nhỏ thường gặp hơn, nên F1 macro đáng tin hơn độ đúng. Ba kênh gần trùng nhau, màu gần như không mang thông tin về nhãn; khó khăn nằm ở ánh sáng, độ mờ và chữ số nhiễu ở biên.

Tiền xử lý. Chia 255, chuẩn hóa theo trung bình và độ lệch chuẩn từng kênh tính trên tập train; dịch ngẫu nhiên ±4 điểm ảnh. Không lật hay xoay vì 6 và 9, 2



Hình 3.7. Ảnh thật của SVHN, mỗi cột một chữ số. Nhiều ảnh lẫn chữ số bên cạnh ở hai mép



(a) Phân bố lớp

(b) Histogram kênh màu

Hình 3.8. Phân bố lớp và cường độ điểm ảnh của SVHN

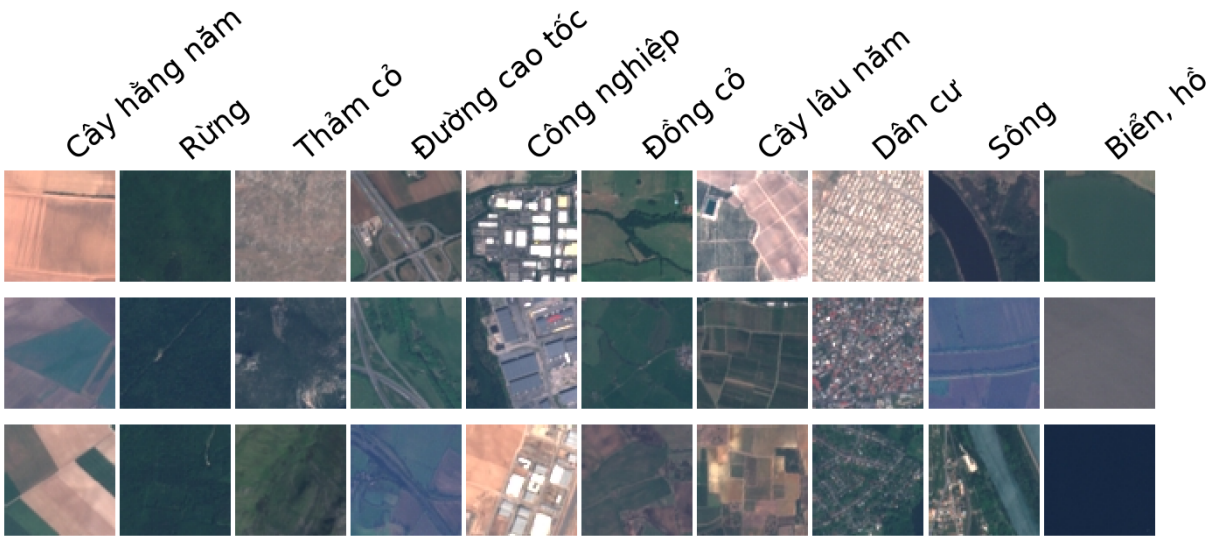
và 5 sẽ đổi nghĩa.

3.2.2 EuroSAT RGB

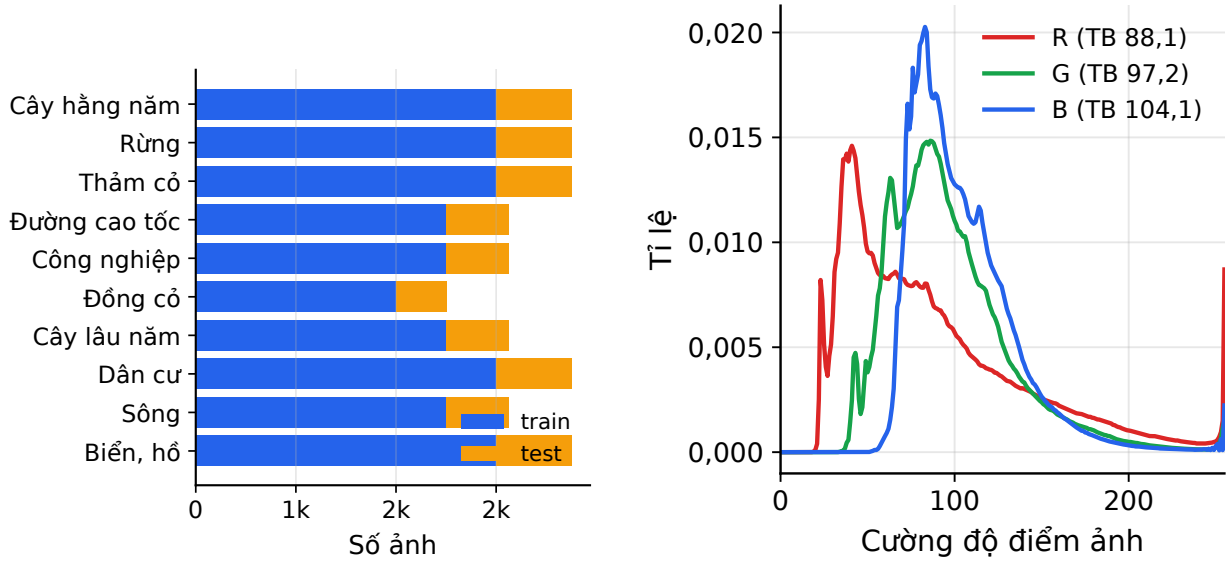
Nguồn. Ảnh vệ tinh Sentinel-2 phủ châu Âu [87], ba băng màu thấy được: `https://zenodo.org/records/7711810`. Giấy phép MIT.

Kích cỡ. 27.000 ảnh 64×64 (độ phân giải mặt đất 10 m), 10 lớp phủ; chia phân tầng 21.600 ảnh train và 5.400 ảnh test; tệp gốc 90,30 MB.

Nhận xét: Dữ liệu khá cân bằng, lệch 1,500 lần. Kênh đỏ trải rộng hơn hẳn hai kênh còn lại vì đất trống và mái nhà đỏ, còn rừng và nước tập trung ở vùng tối. Một



Hình 3.9. Ảnh thật của EuroSAT: lớp phân biệt nhau bằng kết cấu nhiều hơn hình dạng



(a) Phân bố lớp

(b) Histogram kênh màu

Hình 3.10. Phân bố lớp và cường độ điểm ảnh của EuroSAT

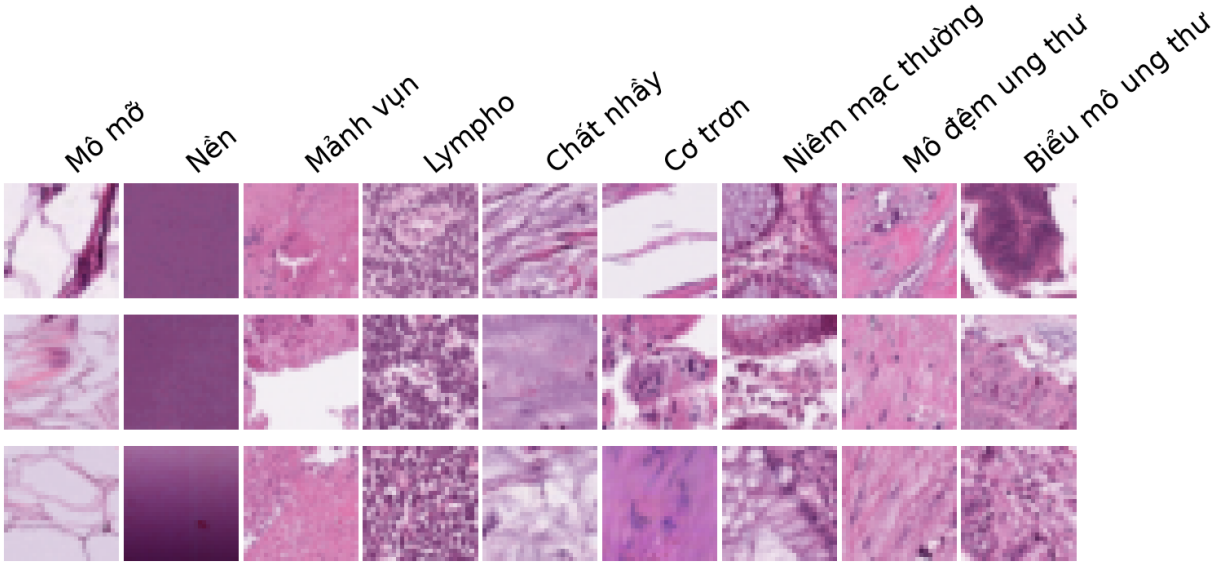
số lớp gần nhau về thị giác (cây hằng năm, cây lâu năm, đồng cỏ) là nguồn lỗi dự kiến.

Tiền xử lý. Chuẩn hóa theo kênh; dịch ± 4 điểm, lật hai chiều và chuyển vị, vì ảnh chụp từ trên xuống không có chiều lên trên.

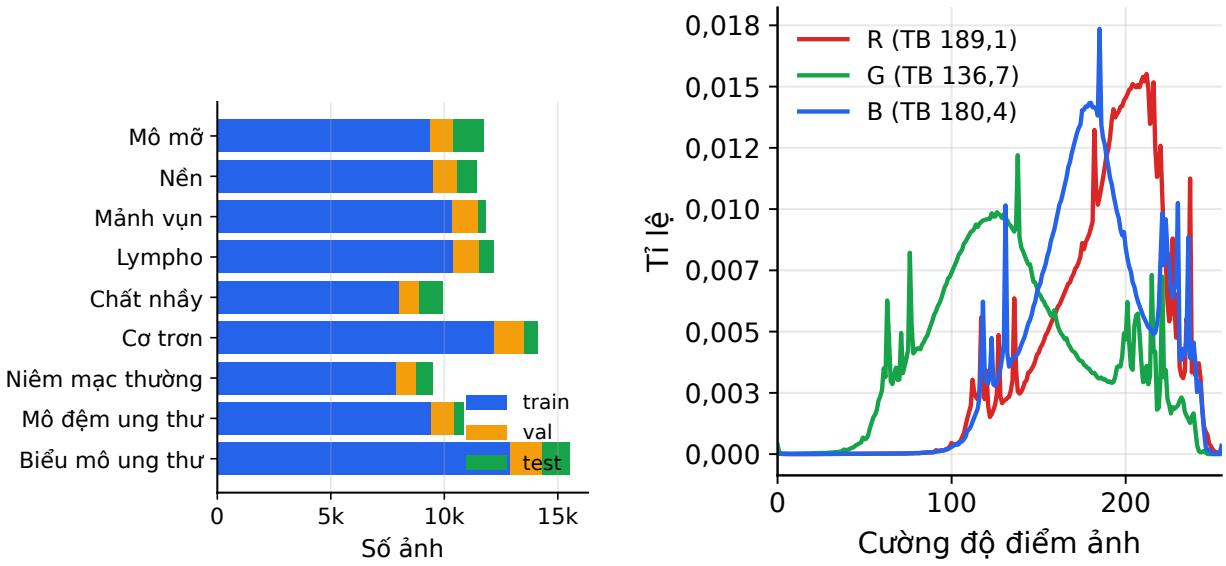
3.2.3 PathMNIST

Nguồn. Tiêu bản mô đại trực tràng nhuộm H&E [88], chuẩn hóa thành 28×28 trong MedMNIST v2 [89]: `https://zenodo.org/records/10519652`. Giấy phép CC BY 4.0. Kết quả chỉ mang tính học thuật, không dùng cho chẩn đoán.

Kích cỡ. 107.180 ảnh màu, 9 lớp mô, giữ đúng cách chia của MedMNIST: 89.996 train, 10.004 val, 7.180 test; tệp gốc 196,10 MB.



Hình 3.11. Ảnh thật của PathMNIST: chín lớp mô cùng tông hồng tím của thuốc nhuộm



(a) Phân bố lớp **(b)** Histogram kênh màu

Hình 3.12. Phân bố lớp và cường độ điểm ảnh của PathMNIST

Nhận xét: Cả bộ lệch 1,636 lần giữa lớp đông nhất và ít nhất. Điểm cần chú ý: tập test đến từ trung tâm khác tập train [88], nên khác biệt thuốc nhuộm và quy trình tạo ra độ lệch miền mà tập val không phản ánh. Đỉnh ở vùng rất sáng của histogram là phần kính không có mô.

Tiền xử lý. Đệm phản chiếu 2 điểm mỗi cạnh thành 32×32 vì VGG-11 có năm lần pool, sau đó chuẩn hóa theo kênh; dịch ±4 điểm, lật và chuyển vị vì tiêu bản

không có hướng cố định.

3.3 Hiện thực trên ba nền tảng

Đoạn mã 3.1 là lõi của bản NumPy: thay bốn vòng lặp lồng nhau bằng việc cắt lát k^2 lần, biến tích chập thành một phép nhân ma trận. Đạo hàm viết tay của mạng thử nhỏ được đối chiếu với sai phân trung tâm, sai số tương đối lớn nhất $2{,}37 \times 10^{-10}$. Keras chạy CPU vì TensorFlow trên Windows không còn hỗ trợ GPU, kết quả của bản này lấy từ Tiểu luận 01.

Đoạn mã 3.1. Duỗi các cửa sổ trượt thành ma trận trong bản NumPy

```python
# tlf/ch3_scratch.py
def im2col(X, kh, kw, stride=1, pad=0):
    """X [N, C, H, W] -> ma trận cột [N·Ho·Wo, C·kh·kw]: mỗi hàng là một cửa sổ đã duỗi thẳng.

    Thay vì 4 vòng lặp lồng nhau, chỉ lặp kh·kw lần (9 lần với kernel 3×3), mỗi lần cắt một lát."""
    N, C, H, W = X.shape
    Ho = (H + 2 * pad - kh) // stride + 1
    Wo = (W + 2 * pad - kw) // stride + 1
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad))) if pad else X
    cols = np.empty((N, C, kh, kw, Ho, Wo), dtype=X.dtype)
    for i in range(kh):
        for j in range(kw):
            cols[:, :, i, j] = Xp[:, :, i:i + stride * Ho:stride, j:j + stride * Wo:stride]
    return cols.transpose(0, 4, 5, 1, 2, 3).reshape(N * Ho * Wo, -1), Ho, Wo
```

Khối inverted residual của MobileNetV2 và Grad-CAM viết bằng PyTorch:

Đoạn mã 3.2. Khối inverted residual: mở rộng, depthwise, chiếu tuyến tính

```python
# tlf/ch3_models.py
class InvertedResidual(nn.Module):
    """1x1 mở rộng kênh -> 3x3 depthwise (groups = số kênh) -> 1x1 chiếu tuyến tính (không ReLU)."""

    def __init__(self, cin, cout, stride, expand):
        super().__init__()
        hidden = cin * expand
        self.use_res = stride == 1 and cin == cout
        layers = [] if expand == 1 else [conv_bn(cin, hidden, 1, act=nn.ReLU6)]
        layers += [conv_bn(hidden, hidden, 3, stride, groups=hidden, act=nn.ReLU6),
                   conv_bn(hidden, cout, 1, act=None)]
        self.conv = nn.Sequential(*layers)

    def forward(self, x):
        return x + self.conv(x) if self.use_res else self.conv(x)
```

Đoạn mã 3.3. Grad-CAM trên tầng tích chập cuối của mọi kiến trúc

```python
# tlf/ch3_torch.py
class GradCAM:
    """Grad-CAM: trọng số kênh là trung bình gradient của điểm lớp theo bản đồ đặc trưng tầng cuối,
    bản đồ nhiệt = ReLU(Σ_k α_k A_k), nội suy về kích thước ảnh và chia cho giá trị lớn nhất."""

    def __init__(self, model):
        self.model, self.acts = model.eval(), None
        cam_layer(model).register_forward_hook(lambda m, i, o: setattr(self, "acts", o))

    def __call__(self, x, cls=None):
        out = self.model(x)
        cls = out.argmax(1) if cls is None else cls
        grads = torch.autograd.grad(out.gather(1, cls.view(-1, 1)).sum(), self.acts)[0]
        alpha = grads.mean(dim=(2, 3), keepdim=True)
        cam = F.relu((alpha * self.acts).sum(1, keepdim=True))
        cam = F.interpolate(cam, size=x.shape[2:], mode="bilinear", align_corners=False)
        cam = cam / (cam.amax(dim=(2, 3), keepdim=True) + 1e-8)
        return cam[:, 0].detach().cpu().numpy(), cls.detach().cpu().numpy()
```

3.4 Đánh giá và so sánh

3.4.1 Thiết lập

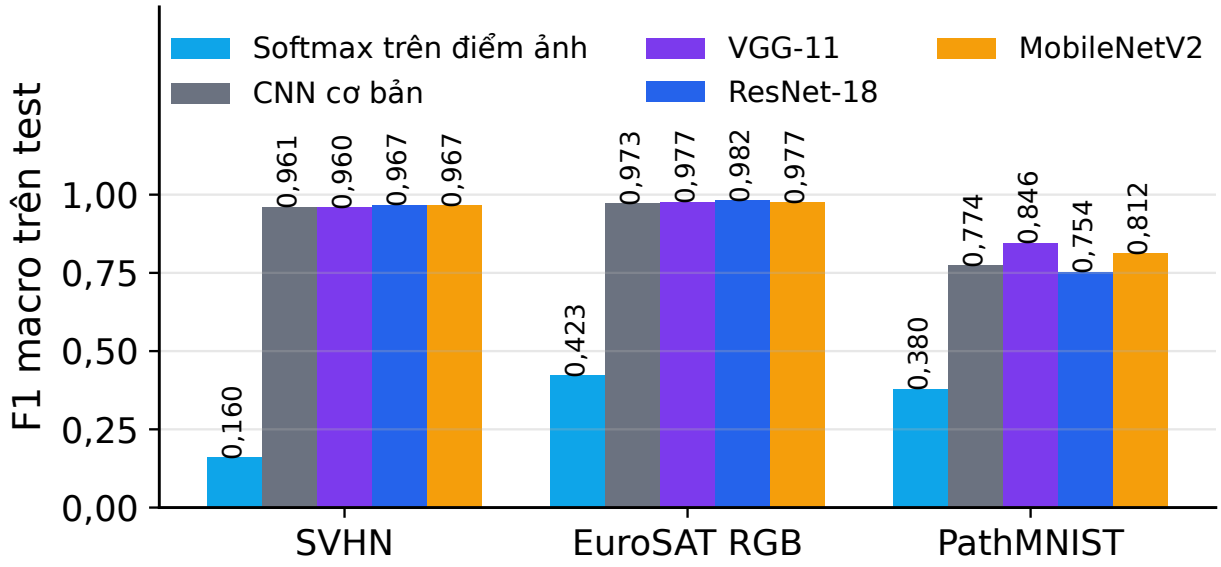
Bốn kiến trúc học cùng một công thức trên PyTorch GPU: SGD Nesterov momentum 0,9, lịch OneCycle đỉnh 0,1, weight decay $5 \cdot 10^{-4}$, lô 256, độ chính xác hỗn hợp bfloat16. Kết quả SVHN (15 epoch) và EuroSAT (30 epoch) lấy lại từ Tiểu luận 01, gồm trọng số, dự đoán và lịch sử; mọi chỉ số được tính lại từ tệp dự đoán và khớp số gốc. PathMNIST huấn luyện mới với đủ 89.996 ảnh train trong 6 epoch, số epoch ít hơn vì GPU dùng chung với các việc khác. Cả ba bộ dùng trọng số epoch cuối, không chọn mô hình theo val hay test. Hai mốc so sánh là đoán lớp đông nhất và hồi quy softmax trên ảnh thu nhỏ.

Bảng 3.2. Kết quả trên tập test của ba bộ dữ liệu

Bộ dữ liệu	Mô hình	Độ đúng	F1 macro	Tham số	Huấn luyện (s)	ms/ảnh CPU	Nguồn
SVHN	Lớp đông nhất	0,1959	0,0328	–	–	–	mới
	Hồi quy softmax	0,2405	0,1604	7.690	53	–	mới
	CNN cơ bản	0,9642	0,9611	305.258	278	10,75	TL01
	VGG-11	0,9632	0,9605	9.228.362	415	23,97	TL01
	ResNet-18	**0,9693**	**0,9674**	11.173.962	772	68,06	TL01
	MobileNetV2	0,9690	0,9666	2.236.682	1.953	60,89	TL01
	CNN cơ bản (Keras)	0,9619	0,9582	306.154	4.203	12,08	TL01
EuroSAT	Lớp đông nhất	0,1111	0,0200	–	–	–	mới
	Hồi quy softmax	0,4328	0,4228	7.690	11	–	mới
	CNN cơ bản	0,9741	0,9733	305.258	404	20,10	TL01
	VGG-11	0,9778	0,9772	9.228.362	622	64,17	TL01
	ResNet-18	**0,9826**	**0,9819**	11.173.962	1.219	195,24	TL01
	MobileNetV2	0,9772	0,9767	2.236.682	1.293	18,46	TL01
	CNN cơ bản (Keras)	0,9428	0,9414	306.154	4.992	2,58	TL01
PathMNIST	Lớp đông nhất	0,1717	0,0326	–	–	–	mới
	Hồi quy softmax	0,5064	0,3800	5.301	55	–	mới
	CNN cơ bản	0,8448	0,7744	305.129	135	11,65	mới
	VGG-11	**0,8914**	**0,8464**	9.227.849	153	75,79	mới
	ResNet-18	0,8010	0,7539	11.173.449	379	88,44	mới
	MobileNetV2	0,8599	0,8121	2.235.401	788	72,78	mới

Số in đậm là tốt nhất trong từng bộ. Nguồn TL01 là kết quả dùng lại từ Tiểu luận 01, mới là chạy cho chương này.

Thứ hạng trên PathMNIST khác hai bộ còn lại: ResNet-18 đứng cuối nhóm CNN, còn VGG-11 dẫn đầu.



Hình 3.13. F1 macro trên test: khoảng cách lớn nhất nằm giữa mô hình tuyến tính và CNN

Kết quả chính

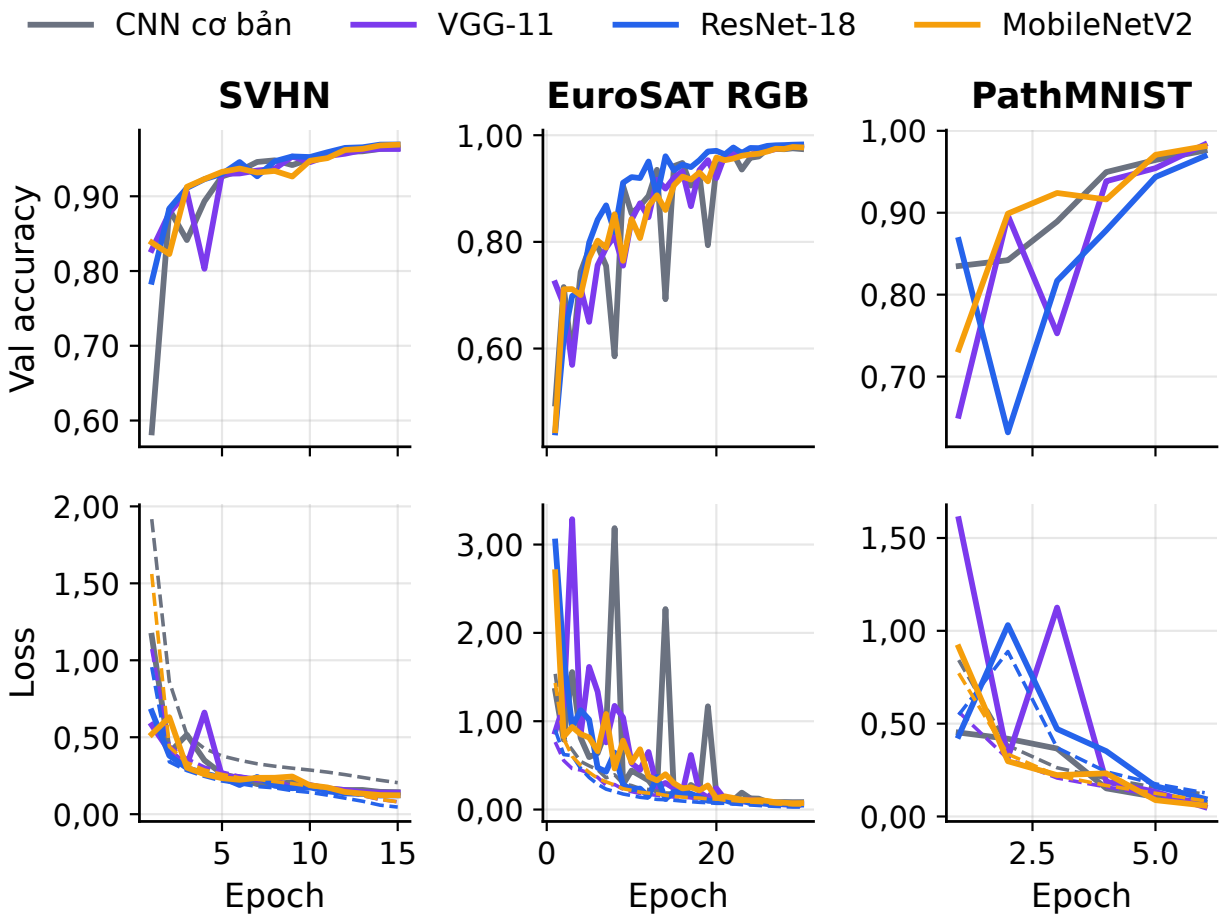
- SVHN: ResNet-18 đạt F1 macro 0,9674, softmax chỉ đạt 0,1604, tức gần như không học được từ điểm ảnh thô.
- EuroSAT: ResNet-18 đạt F1 macro 0,9819, softmax đạt 0,4228 nhờ màu sắc đã mang nhiều thông tin.
- PathMNIST: VGG-11 đạt F1 macro 0,8464, trong khi cùng mô hình đạt độ đúng 0,9833 trên val nhưng chỉ 0,8914 trên test.

3.4.2 Đường cong học

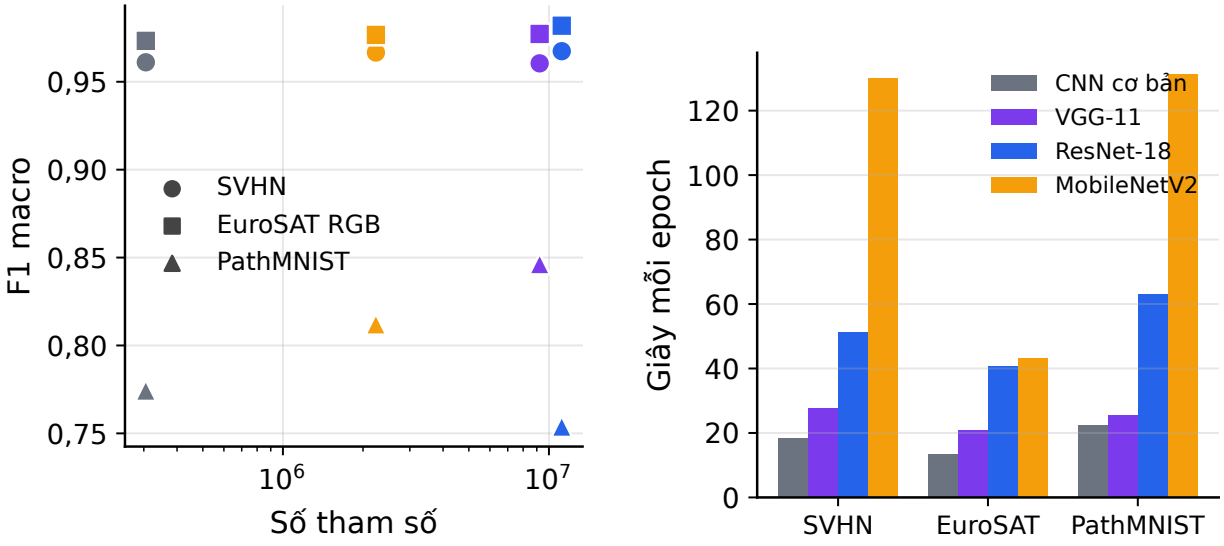
Nhận xét: SVHN và EuroSAT hội tụ gọn, loss train gần loss theo dõi nên không có dấu hiệu quá khớp nặng. Các gai trên đường EuroSAT của CNN cơ bản rơi vào đoạn tốc độ học cao của OneCycle và biến mất khi tốc độ giảm. PathMNIST với 6 epoch vẫn còn đi lên ở epoch cuối, nên các mô hình này chưa hội tụ hẳn trên val.

3.4.3 Đánh đổi độ đúng và chi phí

Nhận xét: Trên SVHN và EuroSAT, tăng từ 305.258 tham số lên 11.173.962 tham số chỉ thêm 0,5032 và 0,8519 điểm phần trăm độ đúng, đổi lại thời gian huấn luyện gấp vài lần. Độ trễ một ảnh trên CPU của ResNet-18 là 195,24 ms ở ảnh 64×64, MobileNetV2 chỉ 18,46 ms vì stem bước 2 giảm cỡ bản đồ ngay từ đầu.



Hình 3.14. Độ đúng và loss theo epoch. SVHN và EuroSAT theo dõi trên test (không dùng để chọn mô hình), PathMNIST theo dõi trên val

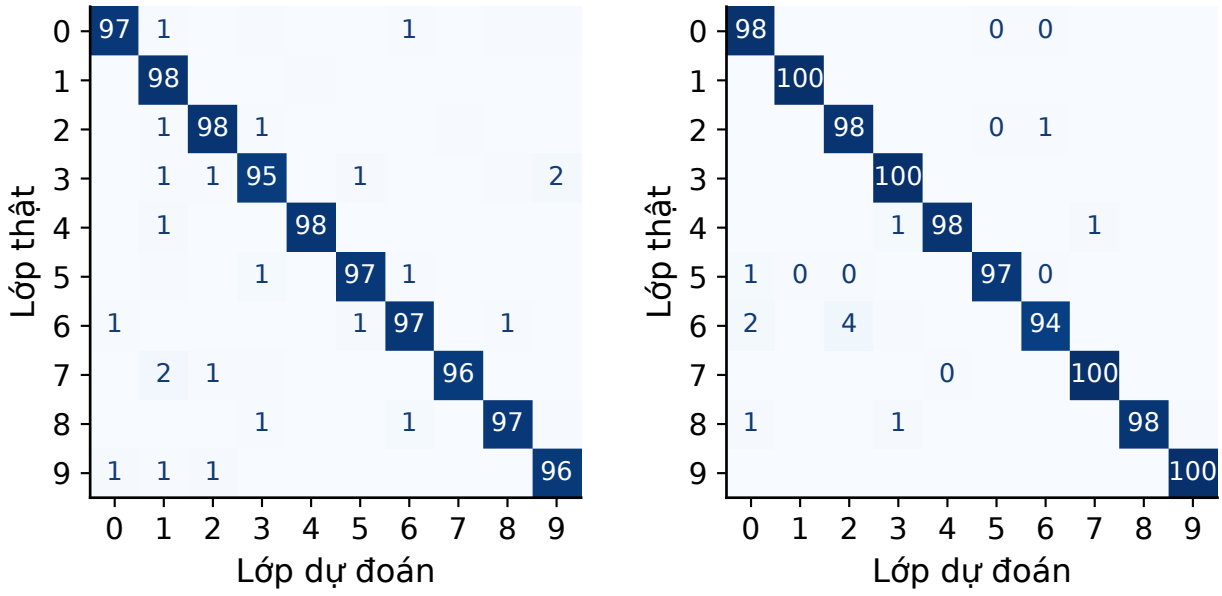


(a) F1 macro theo số tham số

(b) Giây huấn luyện mỗi epoch

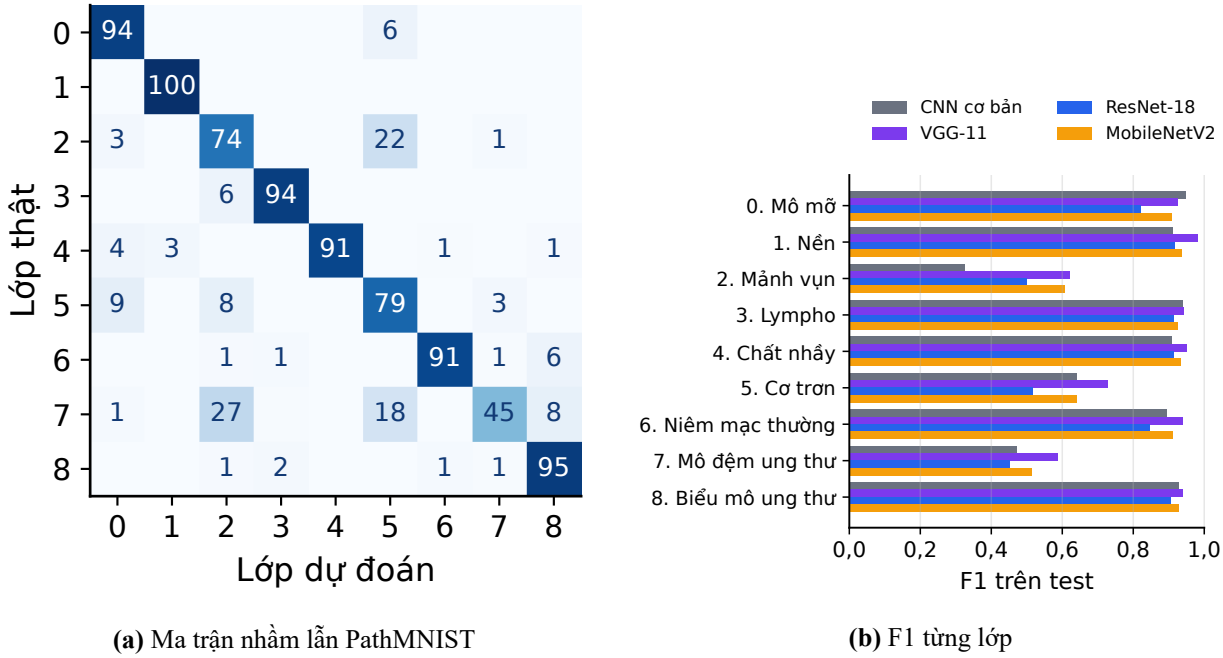
Hình 3.15. Chi phí của từng kiến trúc

3.4.4 Phân tích lỗi



(a) SVHN **(b)** EuroSAT

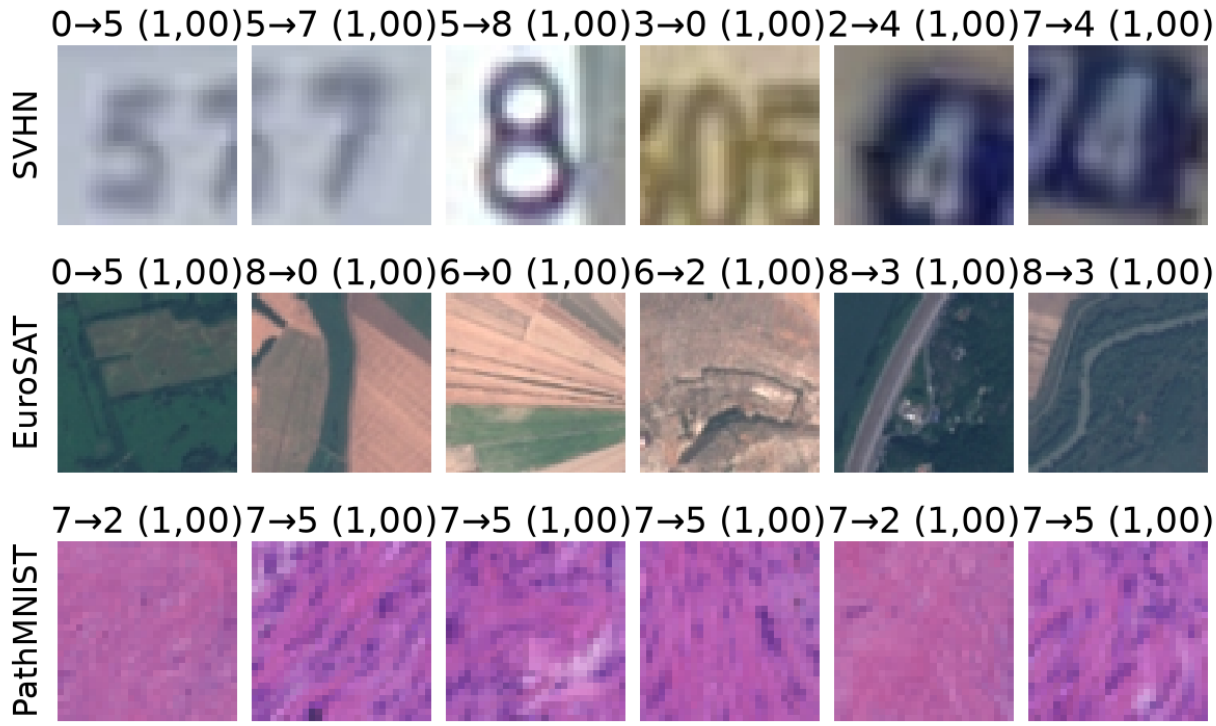
Hình 3.16. Ma trận nhầm lẫn của mô hình tốt nhất, chuẩn hóa theo hàng (%)



(a) Ma trận nhầm lẫn PathMNIST **(b)** F1 từng lớp

Hình 3.17. Lỗi trên PathMNIST tập trung vào vài lớp

Nhận xét: Trên SVHN nhầm lẫn nhiều nhất là 7 thành 1 với 2,48%, ảnh nhiễu nhiều hơn là lỗi hệ thống. EuroSAT nhầm Cây lâu năm thành Thảm cỏ (3,60%), đúng cặp lớp gần nhau về thị giác đã dự đoán ở phần dữ liệu. PathMNIST nghiêm trọng hơn nhiều: Mô đệm ung thư bị đoán thành Mảnh vụn tới 26,84%.



Hình 3.18. Ảnh test bị mô hình tốt nhất đoán sai với độ tin cao nhất, ghi lớp thật→lớp đoán (xác suất). Ba hàng lần lượt là SVHN, EuroSAT, PathMNIST

Lệch miền, không phải quá khớp thông thường

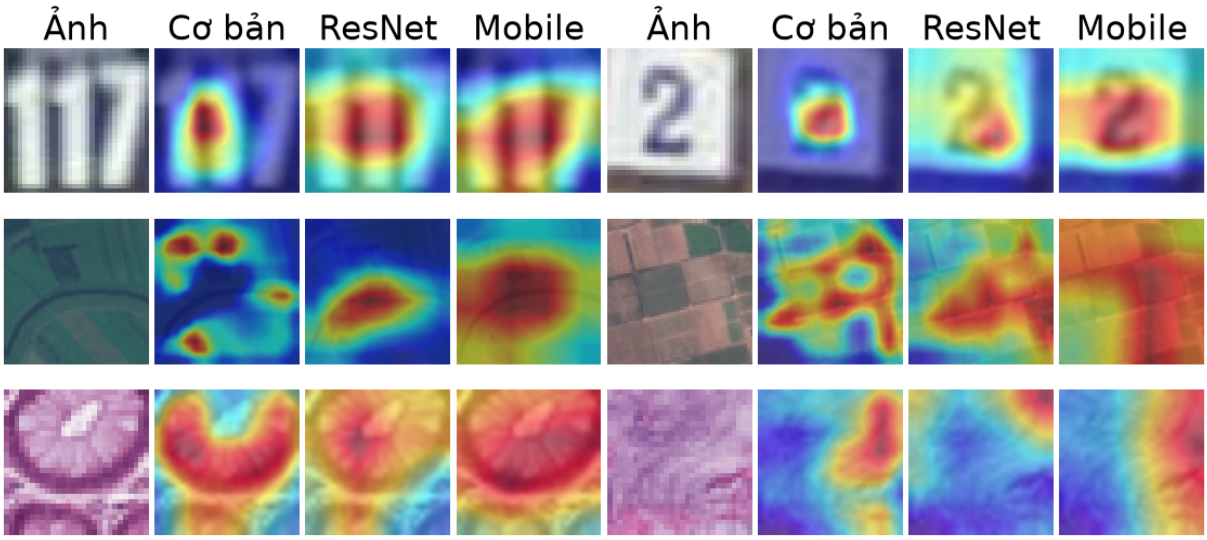
Độ đúng trên val của PathMNIST cao vì val cùng nguồn với train. Tập test đến từ trung tâm khác nên phân phối màu nhuộm đổi, và thứ hạng mô hình cũng đổi. Với ảnh y tế, một con số val đẹp không cho biết mô hình có dùng được ở bệnh viện khác hay không.

3.4.5 Giải thích bằng Grad-CAM

Nhận xét: Trên SVHN cả ba mạng tập trung vào chữ số ở giữa ảnh và bỏ qua chữ số lẫn ở hai mép, đúng cách nhãn được gán. Bản đồ của CNN cơ bản sắc nét hơn vì tầng cuối còn 8×8 điểm so với 4×4 của ResNet-18 và MobileNetV2. Trên ảnh vệ tinh, mạng bám theo ranh giới thửa ruộng và dòng sông. Trên tiêu bản mô, vùng nóng phủ cấu trúc tuyến thay vì vùng kính trắng. Grad-CAM cho biết mô hình nhìn vào đâu, không chứng minh đó là lý do đúng về mặt y học.

3.4.6 So sánh nền tảng

Nhận xét: Cùng kiến trúc, bản Keras đạt 0,9619 trên SVHN, sát PyTorch, nhưng thấp hơn rõ trên EuroSAT (0,9428 so với 0,9741) và tốn thời gian gấp 15,12 đến



Hình 3.19. Grad-CAM trên ảnh test được đoán đúng; mỗi hàng một bộ, hai cụm ảnh, mỗi cụm gồm ảnh gốc và bản đồ của ba kiến trúc

Bảng 3.3. So sánh ba nền tảng: huấn luyện PyTorch và Keras, NumPy suy luận bằng trọng số PyTorch, và hàng cuối là NumPy tự huấn luyện mạng hai tầng

Bộ dữ liệu	Nền tảng	Thiết bị	Việc	Độ đúng	Thời gian (s)
SVHN	PyTorch	GPU	huấn luyện	0,9642	278
	Keras	CPU	huấn luyện	0,9619	4.203
EuroSAT	PyTorch	GPU	huấn luyện	0,9741	404
	Keras	CPU	huấn luyện	0,9428	4.992
SVHN	NumPy	CPU	suy luận 2.000 ảnh	0,9655	33,4 ms/ảnh
	PyTorch	CPU	suy luận 2.000 ảnh	0,9655	3,5 ms/ảnh
SVHN	NumPy	CPU	mạng hai tầng, 3 epoch	0,8534	712,8 s/epoch

12,35 lần vì chạy CPU. Hai bản dùng cùng kiến trúc nhưng khác thư viện tăng cường và khởi tạo mặc định, nên chênh lệch này không nên hiểu là chất lượng của nền tảng. Bản NumPy cho dự đoán trùng PyTorch trên 2.000 ảnh test (tỉ lệ trùng 100,00%), chậm khoảng mười lần; riêng im2col nhanh hơn bốn vòng lặp lồng nhau từ 262,97 xuống 2,276 ms mỗi ảnh ở tầng đầu.

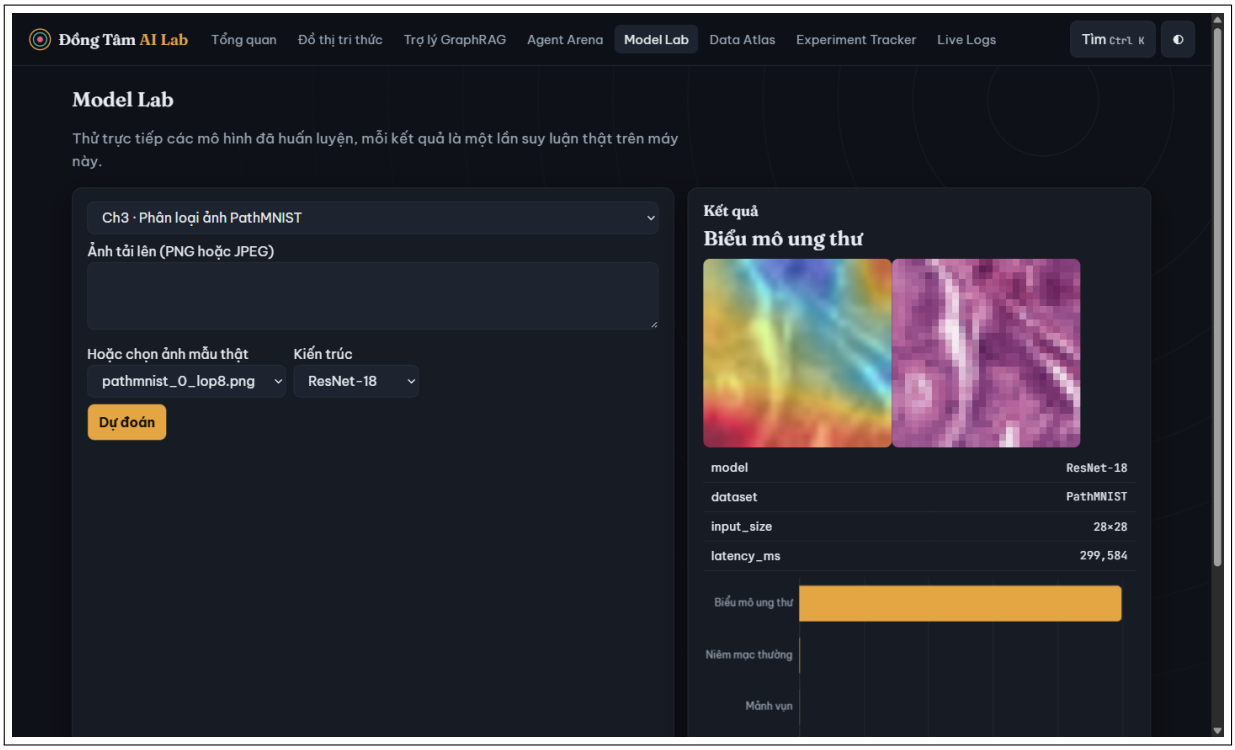
Hàng cuối của bảng là bản NumPy huấn luyện từ đầu bằng lan truyền ngược viết tay, trên đủ 73.257 ảnh SVHN, với mạng hai tầng nhỏ hơn CNN cơ bản và chỉ 3 epoch. Độ đúng đạt 0,8534, F1 macro 0,8377. Con số này chứng minh phần đạo hàm viết tay học được thật, không dùng để xếp hạng với các hàng trên vì kiến trúc và số epoch khác nhau.

Nhật ký huấn luyện PathMNIST

```
pathmnist__custom: acc 0.8448 f1 0.7744 135s params 305129
pathmnist__resnet18: acc 0.8010 f1 0.7539 379s params 11173449
pathmnist__mobilenetv2: acc 0.8599 f1 0.8121 788s params 2235401
pathmnist__vgg11: acc 0.8914 f1 0.8464 153s params 9227849
```

3.5 Triển khai

Mô-đun `tlf/serve_ch3.py` nạp lưới mô hình và chạy trên CPU: người dùng tải ảnh lên hoặc chọn ảnh mẫu thật trong `models/ch3/examples/`, chọn kiến trúc, nhận xác suất top-5 và Grad-CAM dạng PNG base64. Đầu vào được kiểm tra cỡ tệp, cỡ ảnh, và ảnh mẫu chỉ chọn theo tên trong danh sách.



Hình 3.20. Trang phân loại ảnh trên Web App

Nhìn chung, CNN nhỏ đã đủ tốt cho ảnh 32×32; kiến trúc sâu hơn chỉ thêm được ít, còn khả năng tổng quát qua miền dữ liệu mới là thách thức thực sự.

Chương 4

Mạng nơ-ron hồi quy

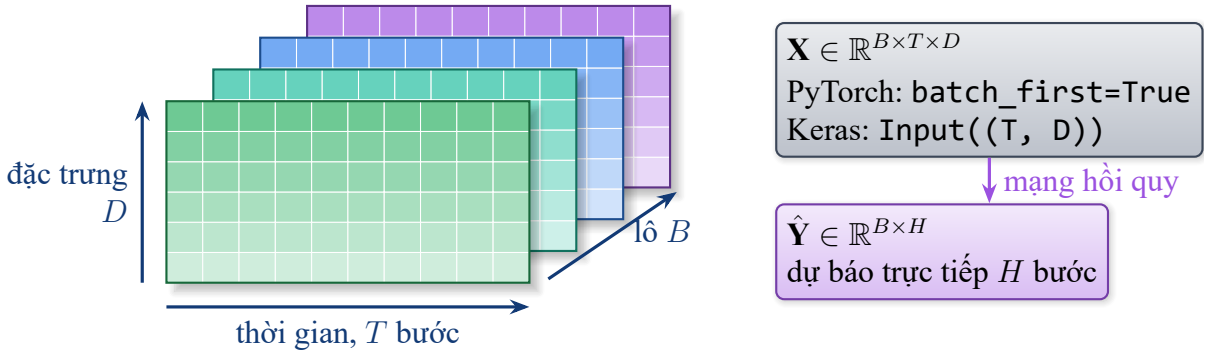
Mạng nơ-ron hồi quy (RNN) xử lý dữ liệu có thứ tự: giá cổ phiếu, thời tiết, tải điện. Chương này đi từ lý thuyết BPTT và tiêu biến gradient đến dự báo thật trên ba nguồn dữ liệu. Thước đo xuyên suốt là một câu hỏi đơn giản: mô hình có thắng được việc lấy lại giá trị vừa quan sát hay không.

4.1 Cơ sở lý thuyết

4.1.1 Tensor chuỗi và RNN cơ bản

Mẫu trong bảng hay ảnh thường được xem là độc lập, còn hai quan sát liền kề của một chuỗi thì không. Một lô cửa sổ là tensor $\mathbf{X} \in \mathbb{R}^{B\times T\times D}$ (Hình 4.1): B mẫu, mỗi mẫu T bước, mỗi bước D đặc trưng. RNN Elman [90] giữ trạng thái ẩn h_t làm bộ nhớ và dùng chung một bộ trọng số ở mọi bước (Hình 4.2):

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad \hat{y} = W_{hy}h_T + b_y. \tag{4.1}$$

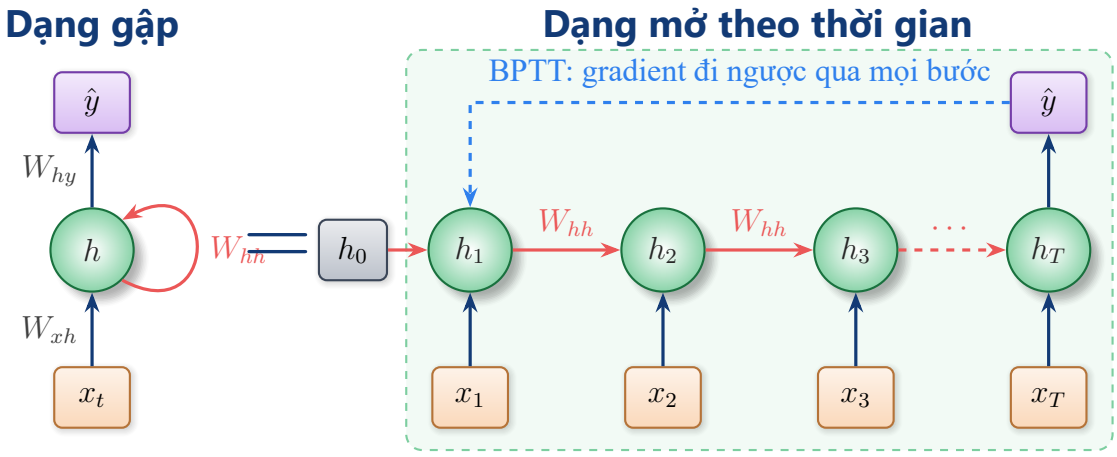


Hình 4.1. Tensor đầu vào $B \times T \times D$ và đầu ra dự báo trực tiếp H bước.

4.1.2 BPTT, tiêu biến và bùng nổ gradient

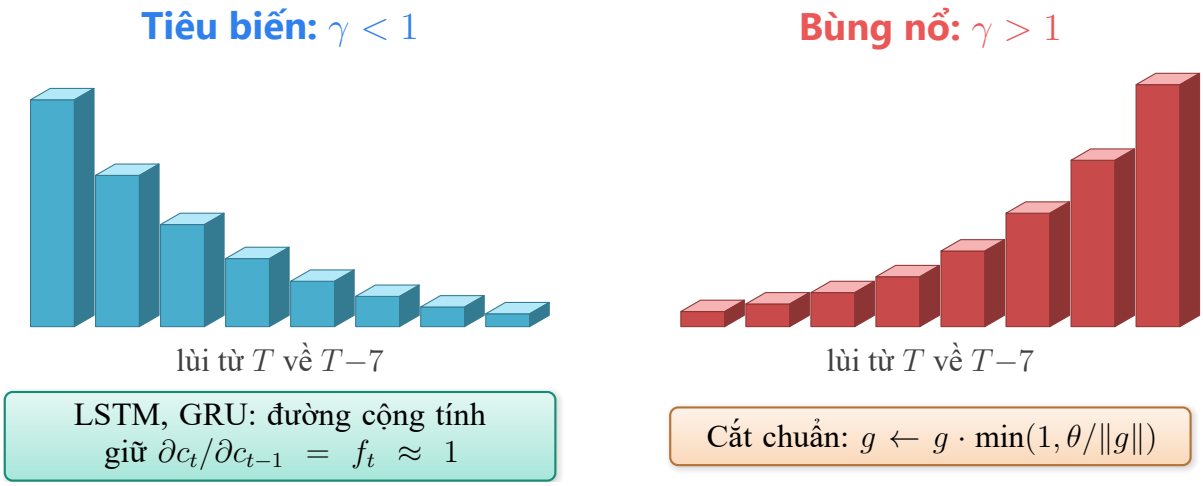
Mở mạng theo thời gian ta có một đồ thị sâu T lớp, và lan truyền ngược trên đồ thị đó gọi là BPTT [91]. Quy tắc dây chuyền cho

$$\frac{\partial L}{\partial W_{hh}} = \sum_{k=1}^{T} \frac{\partial L}{\partial h_T}\Big(\prod_{j=k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}}\Big)\frac{\partial^{+} h_k}{\partial W_{hh}}, \qquad \frac{\partial h_j}{\partial h_{j-1}} = \operatorname{diag}(1-h_j^2)\,W_{hh}^{\top}. \tag{4.2}$$



Hình 4.2. RNN ở dạng gập và dạng mở theo thời gian; trọng số dùng chung ở mọi bước.

Vì $|1-h^2| \le 1$, nếu giá trị kỳ dị lớn nhất của W_{hh} nhỏ hơn 1 thì tích Jacobi co lại theo cấp số mũ của khoảng cách $T-k$: bước xa gần như không nhận được tín hiệu học. Ngược lại, tích có thể tăng vọt [92] (Hình 4.3). Bùng nổ được xử lý bằng cắt chuẩn gradient [93], $g \leftarrow g \cdot \min(1, \theta/\|g\|)$, giữ nguyên hướng và chỉ giới hạn độ dài bước; thực nghiệm dùng $\theta = 1$. Tiêu biến thì phải sửa ở kiến trúc.



Hình 4.3. Tích Jacobi co hoặc giãn theo khoảng cách thời gian, cùng hướng khắc phục.

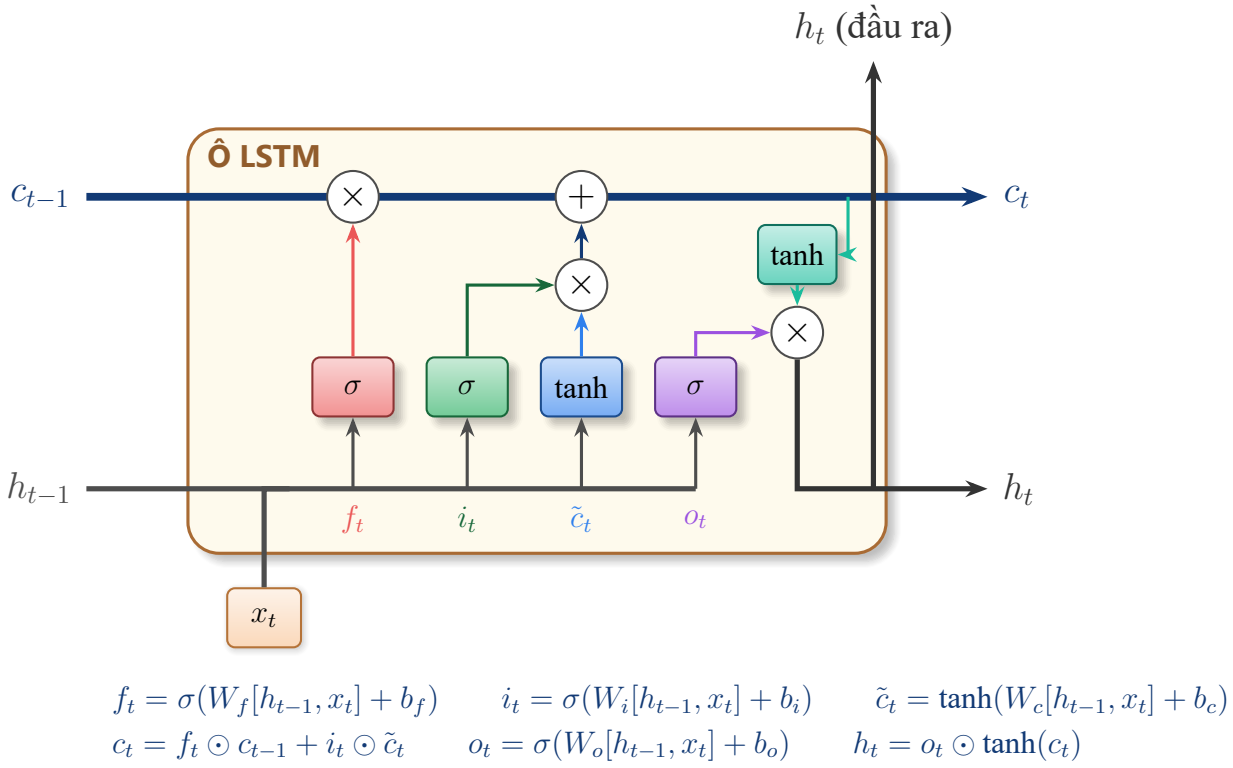
4.1.3 LSTM và GRU

LSTM [94] thêm ô nhớ c_t được cập nhật theo phép cộng, có ba cổng sigmoid điều tiết (Hình 4.4):

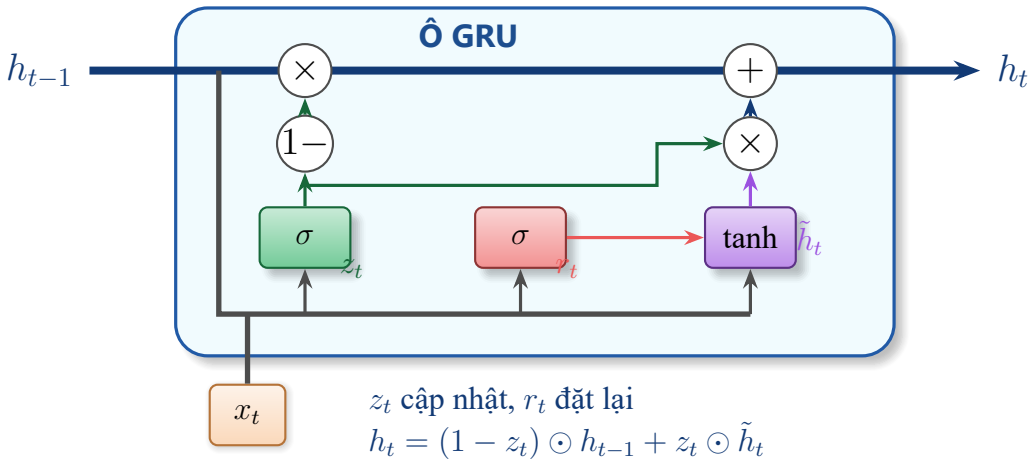
$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \quad & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), \quad & \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \tag{4.3}$$

Khi $f_t \approx 1$ thì $\partial c_t / \partial c_{t-1} \approx 1$, gradient đi qua nhiều bước mà không co lại. GRU [95] gộp ô nhớ với trạng thái ẩn và chỉ dùng hai cổng (Hình 4.5):

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}, x_t]), & r_t &= \sigma(W_r[h_{t-1}, x_t]), \\ \tilde{h}_t &= \tanh(W_h[r_t \odot h_{t-1}, x_t]), & h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \end{aligned} \tag{4.4}$$



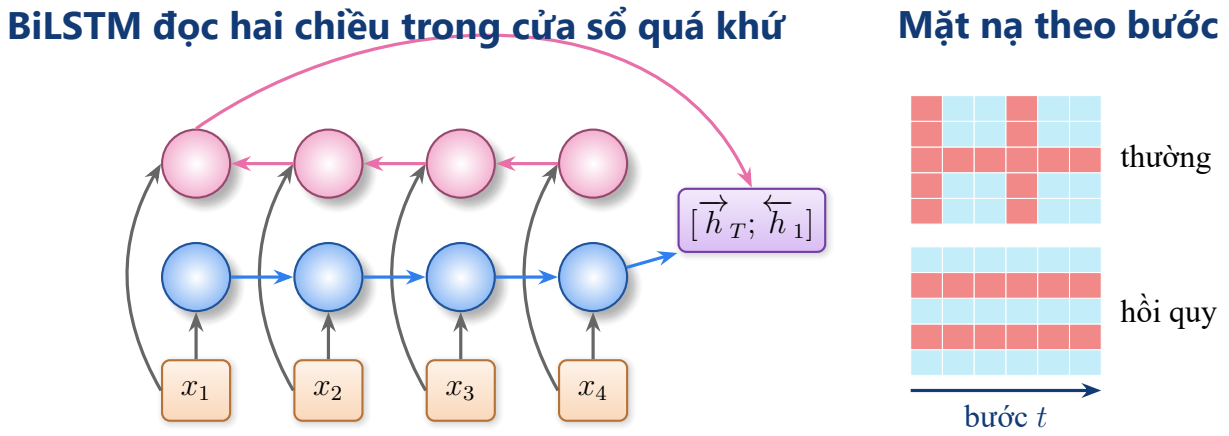
Hình 4.4. Ô LSTM: đường ô nhớ phía trên và ba cổng.



Hình 4.5. Ô GRU: hai cổng, một trạng thái.

4.1.4 BiLSTM và recurrent dropout

BiLSTM [96] chạy một LSTM xuôi và một LSTM ngược rồi nối hai trạng thái. Trong dự báo, chiều ngược chỉ đọc lại cửa sổ đã quan sát, không thấy tương lai thật, nên vẫn hợp lệ. Dropout thường đổi mặt nạ ở mỗi bước và làm nhiễu bộ nhớ; recurrent dropout [97] giữ một mặt nạ cho cả chuỗi (Hình 4.6). Keras có sẵn qua tham số `recurrent_dropout`; `nn.LSTM` của PyTorch không có, nên mô hình PyTorch ở đây chỉ đặt Dropout sau trạng thái ẩn cuối.



Hình 4.6. Trái: BiLSTM trong cửa sổ quá khứ. Phải: mặt nạ dropout thường và mặt nạ recurrent dropout.

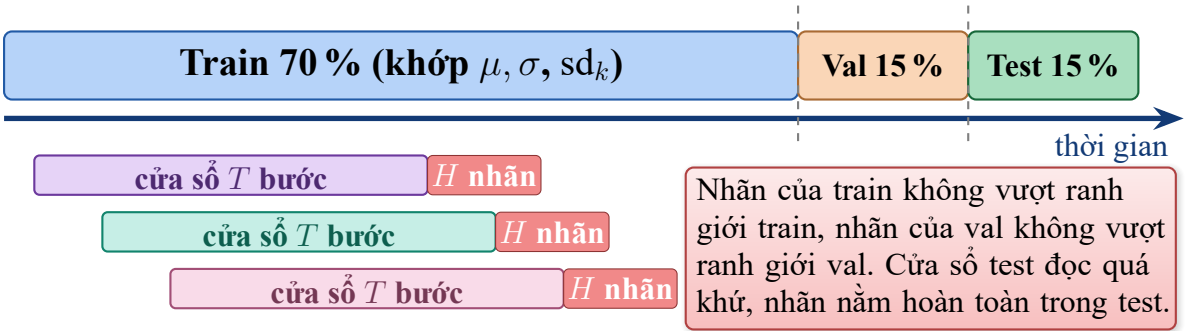
4.2 Dữ liệu và tiền xử lý

4.2.1 Quy trình chung, không rò rỉ

Cả sáu chuỗi đi qua một quy trình (Hình 4.7): chia theo thời gian 70/15/15, mean và std của đặc trưng chỉ khớp trên hàng thuộc train, cửa sổ trượt bước 1, không xáo trộn qua ranh giới. Nhãn là phần thay đổi so với điểm cuối cửa sổ, chia cho độ lệch chuẩn của từng bước trên train:

$$\mathbf{Y}_{t,k} = (y_{t+k} - y_t)/\mathrm{sd}_k, \quad k = 1, \ldots, H. \tag{4.5}$$

Với cổ phiếu y là log giá. Như vậy đầu ra 0 đúng bằng Persistence, mô hình chỉ học phần vượt baseline. Giá 2026 cao gấp nhiều lần giá 2012, nên đặc trưng giá đều là tỉ số, không phải mức.



Hình 4.7. Chia theo thời gian, khớp thống kê trên train và cửa sổ trượt.

4.2.2 VN30: FPT, VCB, HPG, VIC

Nguồn: Yahoo Finance qua `yfinance`, `https://finance.yahoo.com/quote/FPT.VN/history/`, giá đã điều chỉnh cổ tức và chia tách. **Kích cỡ:** 15.329 dòng từ 2012-01-02 đến 2026-11-05 (FPT 3.851, VCB 3.842, HPG 3.851, VIC 3.785 phiên), 1,274 MB, lưu ở `data/ch4/vn30_prices.csv`. Sau khi loại phiên khối lượng 0 và các hàng đầu chưa đủ chỉ báo, FPT còn 3.614 hàng. Tám đặc trưng: lợi suất, gap mở cửa, thân nến, biên độ, khối lượng tương đối, độ lệch so với SMA20, RSI14 và độ biến động 20 phiên. $T = 60$, $H = 10$. Bảng 4.1 là bốn dòng FPT thật, Hình 4.8 là chuỗi, phân bố và ACF.

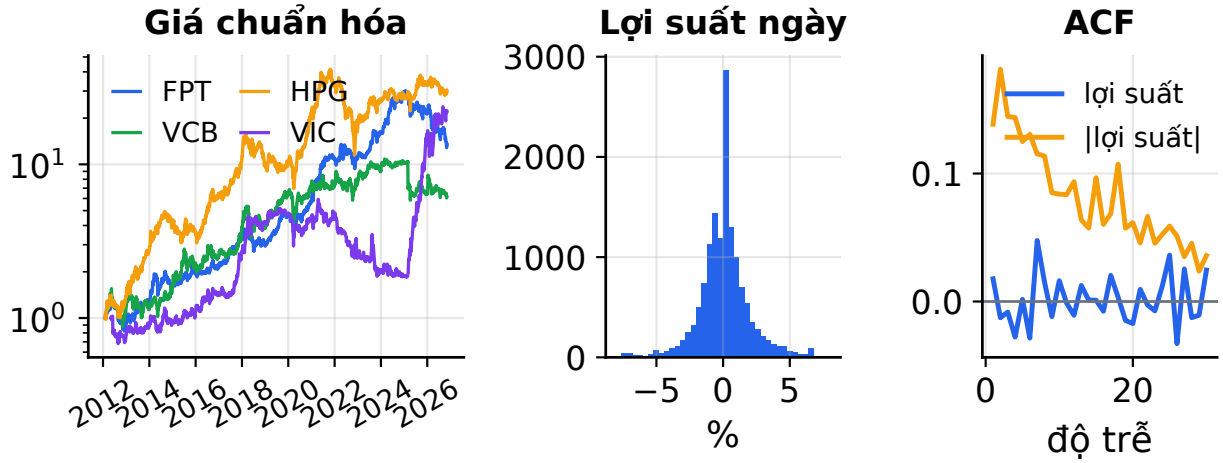
Bảng 4.1. Mẫu thực của FPT (VNĐ, đã điều chỉnh).

Ngày	Open	High	Low	Close	Volume
2012-01-03	3.608	3.752	3.608	3.737	597.072
2016-11-23	8.414	8.570	8.376	8.541	3.605.116
2021-12-10	43.666	43.938	43.213	43.395	2.328.763
2026-11-05	54.500	54.600	52.800	52.900	10.047.731

Nhận xét: Phân bố lợi suất nhọn và có đuôi dày, biên ±7 % của HoSE cắt phần đuôi. ACF của lợi suất dao động sát 0, còn ACF của trị tuyệt đối dương rõ: biến động có quán tính, hướng thì gần như không. Đây là lý do nên chờ đợi Persistence khó bị đánh bại ở một bước.

4.2.3 Thời tiết Hà Nội theo giờ

Nguồn: Open-Meteo Archive API, `https://open-meteo.com/en/docs/historical-weather-api`, tái phân tích ERA5 [98] ở ô lưới gần Hà Nội, không phải trạm đo. **Kích cỡ:** 94.920 giờ từ 2016-01-01 đến 2026-10-29, 4,037 MB, 0 ô thiếu; nhiệt độ từ 4,400 đến 40,20 °C. Mười đặc trưng gồm nhiệt độ, ẩm,

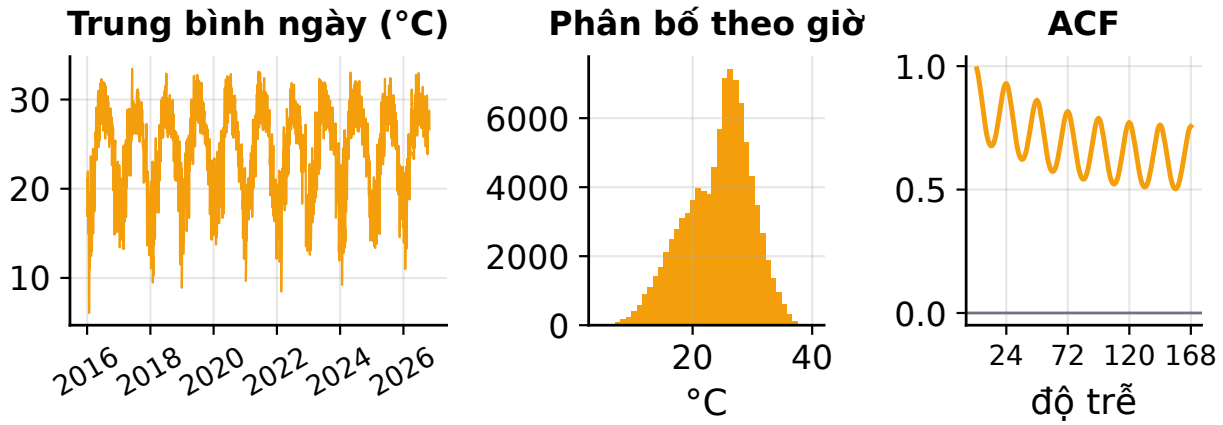


Hình 4.8. VN30: giá chuẩn hóa về 1 ở phiên đầu, lợi suất ngày gộp bốn mã, ACF của FPT.

áp suất, gió, log mưa, mây cùng sin, cos của giờ và ngày trong năm. $T = 72$, $H = 24$.

Bảng 4.2. Mẫu thực của chuỗi thời tiết.

Giờ	Nhiệt độ	Ẩm %	Áp suất	Gió	Mưa
2016-01-01 00:00	15,1	74,0	1.025,0	3,3	0,0
2019-08-11 07:00	28,6	91,0	999,3	4,4	0,0
2023-03-21 15:00	32,7	57,0	999,8	5,9	0,0
2026-10-29 23:00	25,6	90,0	1.016,2	3,5	0,0



Hình 4.9. Thời tiết Hà Nội: trung bình ngày, phân bố, ACF tới 168 giờ.

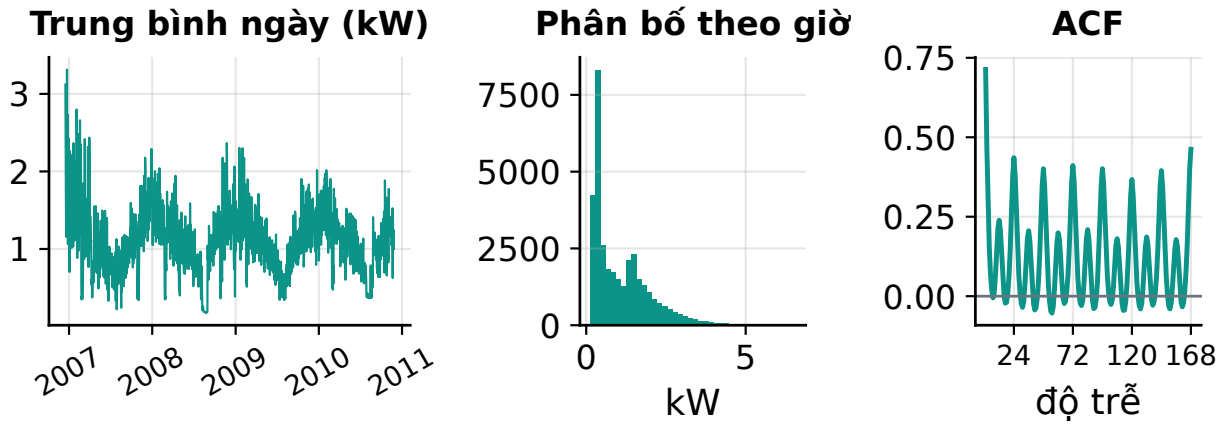
Nhận xét: Chuỗi có chu kỳ năm và chu kỳ ngày rất rõ, ACF giữ ở mức cao và nhấp nhô theo 24 giờ, phân bố có đuôi về phía lạnh. Tín hiệu mạnh, nên kỳ vọng mô hình vượt xa baseline.

4.2.4 Household Power Consumption

Nguồn: UCI id 235 [99], https://archive.ics.uci.edu/dataset/235, một hộ ở Sceaux, Pháp. **Kích cỡ:** 2.075.259 phút đo (126,80 MB khi giải nén, thiếu 1,252 %), gộp đủ thành 34.589 giờ từ 2006-12-16 đến 2010-11-26. Tiền xử lý: 421 giờ không có phút nào đo được; lỗ tới 6 giờ nội suy tuyến tính, lỗ dài lấy cùng giờ tuần trước. Mười đặc trưng gồm công suất tác dụng, phản kháng, điện áp, ba công tơ phụ, sin, cos của giờ và thứ. $T = 72$, $H = 24$.

Bảng 4.3. Mẫu thực của chuỗi điện theo giờ.

Giờ	P (kW)	Q (kW)	U (V)	Điều hòa (Wh)
2006-12-16 17:00	4,22	0,23	234,64	16,86
2008-04-06 20:00	2,68	0,05	236,52	14,57
2009-07-27 15:00	0,34	0,21	241,91	1,63
2010-11-26 21:00	0,93	0,00	239,69	0,00



Hình 4.10. Công suất hộ gia đình: trung bình ngày, phân bố, ACF tới 168 giờ.

Nhận xét: Phân bố lệch phải với nhiều giờ nền thấp và các đỉnh ngắn khi bật thiết bị lớn. ACF có đỉnh tại bội của 24 giờ, và đỉnh 168 giờ cao tương đương đỉnh 24 giờ, cho thấy nhịp tuần. Khác với thời tiết, phần dao động ngẫu nhiên lớn, nên MAPE sẽ cao vì mẫu số nhỏ vào ban đêm.

4.3 Hiện thực

Bốn kiến trúc PyTorch dùng chung một khung (Đoạn mã 4.1): một lớp hồi quy, Dropout, Linear ra H giá trị. Dự báo trực tiếp tránh việc phải đoán các đặc trưng tương lai như kiểu đệ quy. Cổ phiếu dùng 32 đơn vị ẩn, Dropout 0,3, lô 64, weight

decay; chuỗi theo giờ dùng 64 đơn vị, Dropout 0,1, lô 256. Tất cả dùng Adam, cắt chuẩn 1, dừng sớm theo MSE validation, một hạt giống.

Đoạn mã 4.1. Mô hình hồi quy nhiều bước bằng PyTorch.

```python
# tlf/ch4_torch.py
class SeqRegressor(nn.Module):
    """Một lớp hồi quy, Dropout rồi Linear ra H giá trị: dự báo trực tiếp nhiều bước từ trạng thái ẩn cuối."""

    def __init__(self, n_features, horizon, arch="lstm", hidden=64, dropout=0.2):
        super().__init__()
        cell = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU, "bilstm": nn.LSTM}[arch]
        self.rnn = cell(n_features, hidden, batch_first=True, bidirectional=arch == "bilstm")
        self.drop = nn.Dropout(dropout)
        self.head = nn.Linear(hidden * (2 if arch == "bilstm" else 1), horizon)

    def forward(self, x):                          # x: (lô, T, D)
        out, _ = self.rnn(x)                       # out: (lô, T, hidden × số chiều)
        return self.head(self.drop(out[:, -1])) # (lô, H)
```

Trái tim của bản NumPy là hàm `backward` (Đoạn mã 4.2): đi ngược từ T về 1, nhân đạo hàm tanh và W_{hh} đúng như công thức của BPTT, cộng dồn gradient vào trọng số dùng chung.

Kiểm tra bản NumPy trên toàn bộ FPT

Sai số tương đối lớn nhất so với sai phân trung tâm: $8{,}39 \times 10^{-9}$
Tham số: 5.322; dừng ở epoch 16; 20,05 giây CPU
RMSE test một bước: 1.623 VNĐ; Persistence: 1.609 VNĐ

Keras dựng cùng cấu trúc bằng `layers.LSTM` (Đoạn mã 4.3), chạy CPU trên FPT. Bảng 4.4 đối chiếu ba khung trên cùng phép chia.

Bảng 4.4. NumPy và Keras so với PyTorch cùng kiến trúc trên FPT.

Chuỗi	Mô hình	Khung	RMSE	RMSE PyTorch	Giảm RMSE %	Epoch	Giây
FPT	RNN	numpy	1.623	1.619	−0,83	16	20
FPT	LSTM	keras	1.613	1.611	−0,24	8	44

Đoạn mã 4.2. BPTT viết tay trên NumPy.

```python
# tlf/ch4_scratch.py
def backward(self, yhat, y):
    """BPTT: δ_T = ∂L/∂h_T, δ_{t-1} = [(1 - h_t²) ⊙ δ_t] Whhᵀ; cộng dồn gradient của trọng số dùng chung."""
    p, hs, x = self.p, self.hs, self.x
    b, k = y.shape
    g = {name: np.zeros_like(v) for name, v in p.items()}
    dy = 2 * (yhat - y) / (b * k)
    g["Why"], g["by"] = hs[-1].T @ dy, dy.sum(0)
    delta = dy @ p["Why"].T
    self.delta_norms = []
    for t in range(x.shape[1], 0, -1):
        self.delta_norms.append(float(np.linalg.norm(delta) / b))
        da = (1.0 - hs[t] ** 2) * delta
        g["Wxh"] += x[:, t - 1].T @ da
        g["Whh"] += hs[t - 1].T @ da
        g["bh"] += da.sum(0)
        delta = da @ p["Whh"].T
    return g
```

Đoạn mã 4.3. Mô hình Keras tương đương.

```python
# tlf/ch4_keras.py
def build_keras_model(window, n_features, horizon, arch="lstm", hidden=64, dropout=0.2, lr=1e-3, rec_drop=0.0):
    """RNN, LSTM, GRU hoặc BiLSTM → Dropout → Dense(H); rec_drop bật recurrent dropout trên cổng hồi quy."""
    import keras
    from keras import layers
    kw = {"recurrent_dropout": rec_drop}
    rec = {"rnn": lambda: layers.SimpleRNN(hidden, **kw), "lstm": lambda: layers.LSTM(hidden, **kw),
           "gru": lambda: layers.GRU(hidden, **kw),
           "bilstm": lambda: layers.Bidirectional(layers.LSTM(hidden, **kw))}[arch]
    model = keras.Sequential([keras.Input((window, n_features)), rec(), layers.Dropout(dropout), layers.Dense(horizon)])
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0), loss="mse")
    return model
```

4.4 Đánh giá

4.4.1 Kết quả một bước

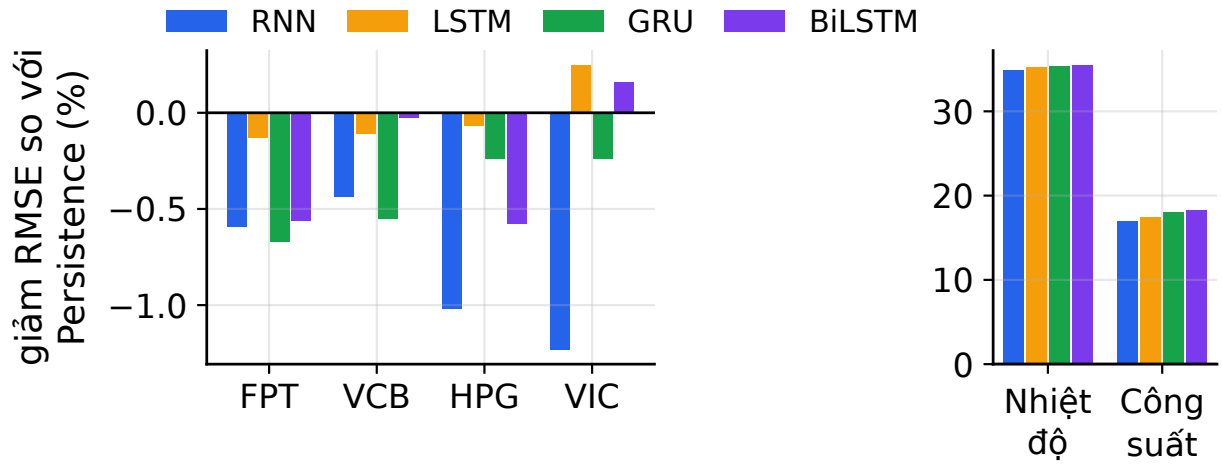
Mô hình được chọn cho mỗi chuỗi có MSE validation thấp nhất, không nhìn test. Kiểm định Diebold-Mariano [100] so sai số bình phương với Persistence; hướng đúng chỉ tính trên các điểm giá thật có đổi.

Bảng 4.5. Tập test, dự báo một bước trên FPT (VNĐ), nhiệt độ (°C), công suất (kW). In đậm là RMSE thấp nhất của chuỗi.

Mô hình	MAE	RMSE	MAPE %	sMAPE %	Hướng %	Giảm RMSE %	DM p
FPT							
Persistence	1.120	**1.609**	1,33	1,33	—	—	—
RNN	1.134	1.619	1,34	1,34	47,3	−0,59	0,028
LSTM	1.129	1.611	1,34	1,34	46,1	−0,13	0,569
GRU	1.138	1.620	1,35	1,35	46,1	−0,67	0,010
BiLSTM	1.136	1.618	1,35	1,34	45,3	−0,56	0,161
Nhiệt độ							
Persistence	0,627	0,864	2,49	2,49	—	—	—
Mùa vụ 24 giờ	1,378	1,944	5,86	5,73	66,0	—	—
RNN	0,363	0,562	1,46	1,45	83,8	+34,91	<0,001
LSTM	0,355	0,559	1,42	1,42	84,6	+35,26	<0,001
GRU	0,357	0,559	1,43	1,42	84,6	+35,35	<0,001
BiLSTM	0,357	**0,558**	1,43	1,42	84,5	+35,44	<0,001
Công suất							
Persistence	0,383	0,582	46,40	38,71	—	—	—
Mùa vụ 24 giờ	0,507	0,756	66,92	49,40	66,5	—	—
RNN	0,346	0,483	47,58	38,80	66,5	+16,97	<0,001
LSTM	0,336	0,480	44,00	36,39	67,8	+17,46	<0,001
GRU	0,343	0,477	47,28	37,95	68,0	+18,02	<0,001
BiLSTM	0,342	**0,476**	47,64	37,85	67,8	+18,24	<0,001

Bảng 4.6. Bốn mã VN30: RMSE một bước theo kiến trúc (VNĐ) và mô hình được chọn. Cột Đa số là tỉ lệ của hướng phổ biến hơn trong test.

Mã	Pers.	RNN	LSTM	GRU	BiLSTM	Chọn	Giảm %	DM p	Hướng %	Đa số %
FPT	1.609	1.619	1.611	1.620	1.618	LSTM	−0,13	0,569	46,1	54,1
VCB	1.646	1.653	1.648	1.655	1.647	LSTM	−0,11	0,381	48,0	51,1
HPG	379	383	379	380	381	LSTM	−0,07	0,821	49,8	53,8
VIC	4.042	4.092	4.032	4.052	4.036	BiLSTM	+0,16	0,613	56,6	56,6



Hình 4.11. Mức giảm RMSE so với Persistence. Hai trục khác thang đo vì độ lớn chênh nhau nhiều lần.

Giá cổ phiếu: không thắng được Persistence

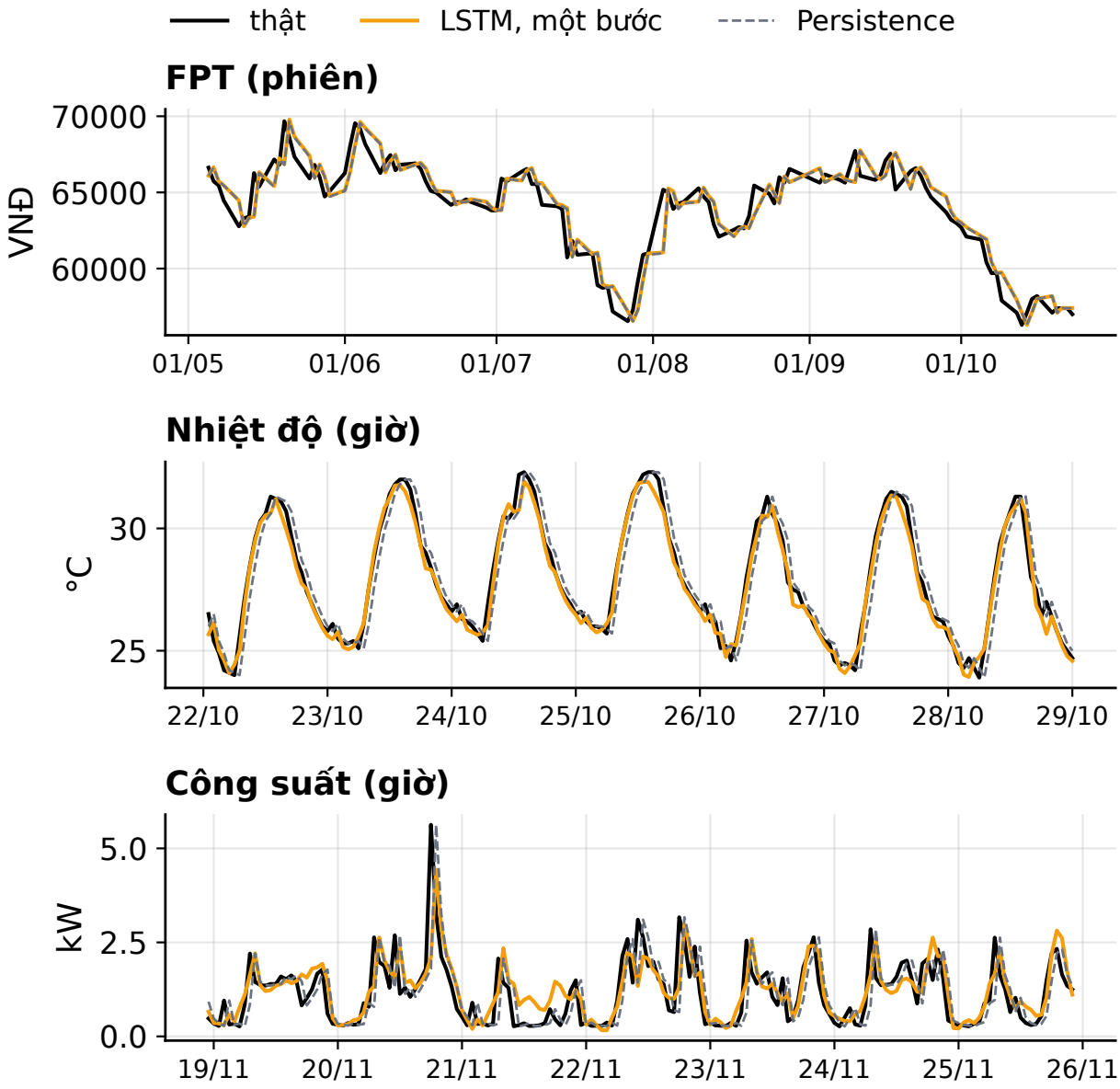
Trên 4 mã, mô hình chọn theo validation chỉ có 1 mã nhích hơn Persistence và 0 mã có ý nghĩa thống kê ở mức 5 %. FPT: LSTM đạt RMSE 1.611 so với 1.609 VNĐ, $p = 0{,}5690$; hướng đúng 46,09 %, thấp hơn mức đoán theo hướng phổ biến 54,12 %. Đường dự báo trông bám sát giá thật (Hình 4.12) chỉ vì nó gần như lặp lại giá hôm trước. Kết quả phù hợp với ACF lợi suất gần 0, và kết quả đối chiếu độc lập của Tiểu luận 02 trên cùng bốn mã (cửa sổ 30, hai lớp, 8 mô hình) cũng có tỉ số RMSE so với baseline từ 0,9998 đến 1,154. Đây là kết quả học thuật, không phải khuyến nghị đầu tư.

Hai chuỗi vật lý thì khác hẳn. Nhiệt độ giảm RMSE 35,35 % so với Persistence (0,5586 so với 0,8639 °C), công suất giảm 16,97 %, cả hai với $p < 0{,}001$ và hướng đúng 84,62 %, 66,45 %. Bốn kiến trúc chênh nhau rất ít so với khoảng cách tới baseline, và RNN thường không thua: với cửa sổ 72 giờ, phần lớn thông tin nằm ở vài giờ cuối.

Hình 4.13 giải thích vì sao: ở FPT, MSE validation chững ngay từ các epoch đầu rồi tăng trong khi train giảm, tức mạng chỉ học thuộc nhiễu; ở chuỗi theo giờ cả hai cùng giảm rồi đi ngang.

4.4.2 Gradient trên mô hình thật

Hình 4.14 đo $\|\partial \hat{y}_1 / \partial x_{T-k}\|$ trên test. Ở trễ 48 giờ của chuỗi nhiệt độ, độ nhạy tương đối của RNN là $5{,}86 \times 10^{-4}$, của LSTM là 0,0060, của GRU là 0,0024: đường RNN rơi nhanh nhất, đúng như lý thuyết. Công suất thì không có trật tự đó (RNN 0,0392, LSTM 0,0146), vì độ nhạy còn phụ thuộc mô hình học được gì, không chỉ

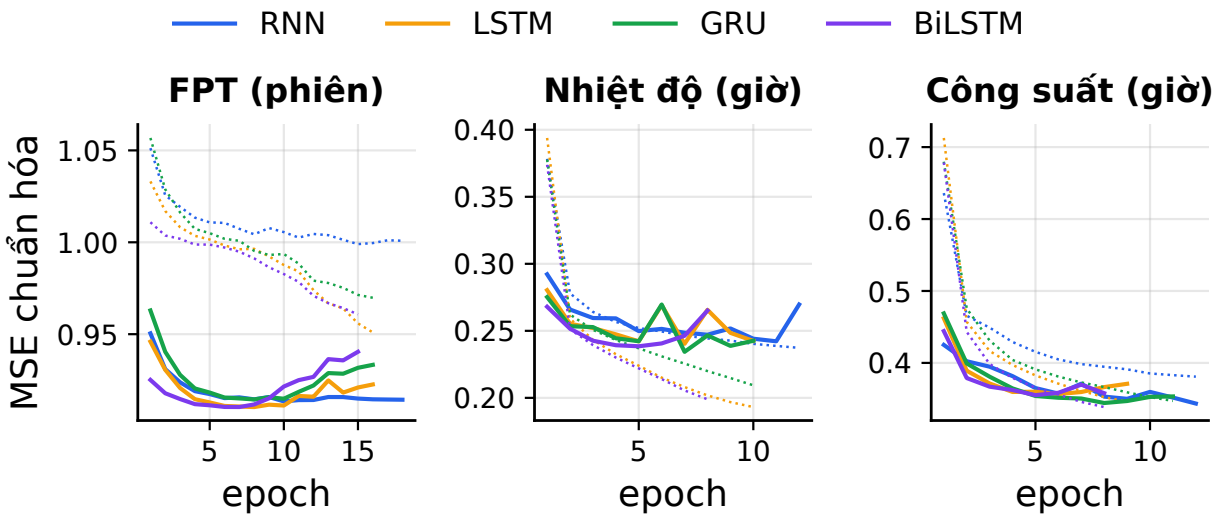


Hình 4.12. Dự báo một bước cuối tập test của mô hình được chọn.

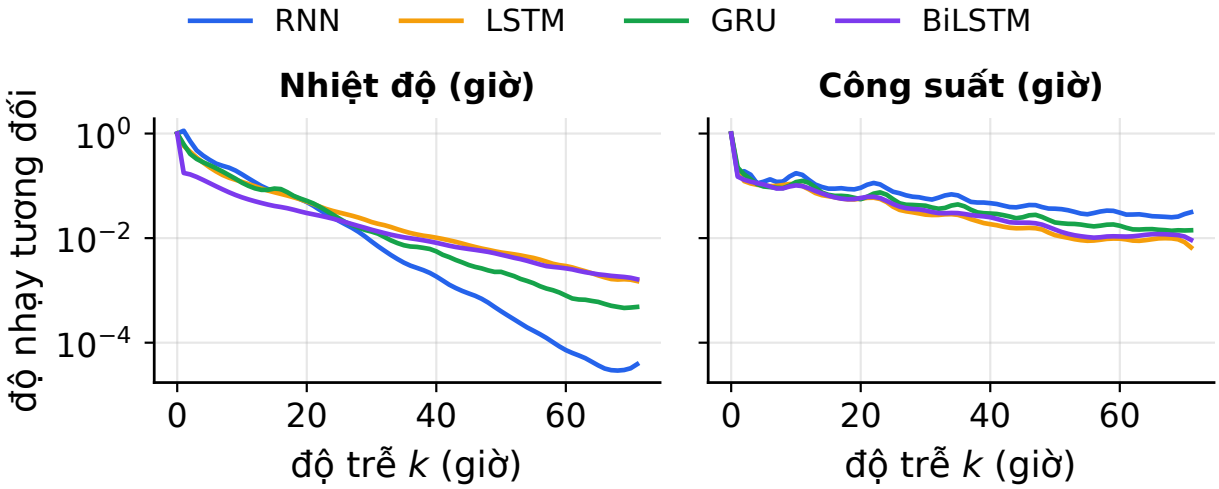
phụ thuộc kiến trúc. Trên bản NumPy, $\|\delta_t\|$ giảm còn 0,0141 lần giá trị ban đầu sau cả cửa sổ (Hình 4.15).

4.4.3 Dự báo nhiều bước và khoảng dao động

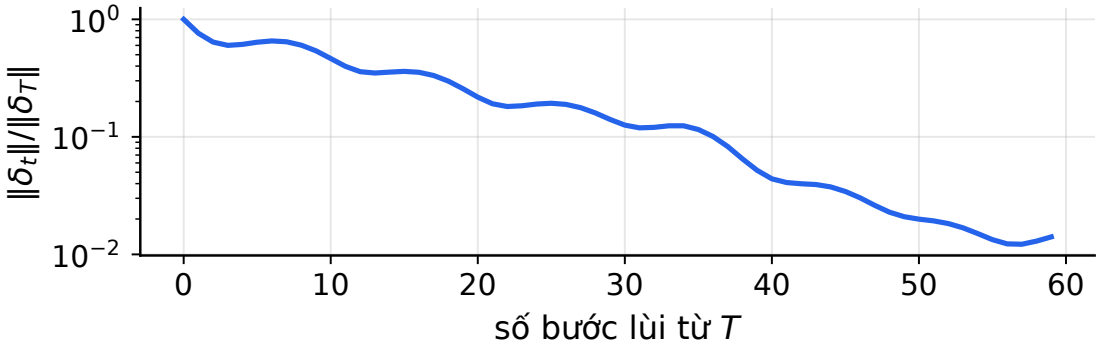
Hình 4.16 cho RMSE theo bước. FPT: bốn kiến trúc bám sát Persistence và nhỉnh hơn về sai số ở tầm xa, không có lợi thế nào. Nhiệt độ: Persistence vọt lên giữa chu kỳ ngày rồi tụt về tại bước 24, baseline mùa vụ giữ ngang, còn cả bốn kiến trúc thấp hơn hai baseline. Công suất có dạng tương tự. Khoảng 90 % lấy từ phân vị sai số validation theo từng bước; độ phủ trên test ở Bảng 4.7. Độ phủ thực của cổ phiếu lệch khỏi 90 % (FPT 82,06 %, VIC 72,82 %) có thể vì biến động giai đoạn test khác giai đoạn validation; chuỗi nhiệt độ đạt 90,23 %.



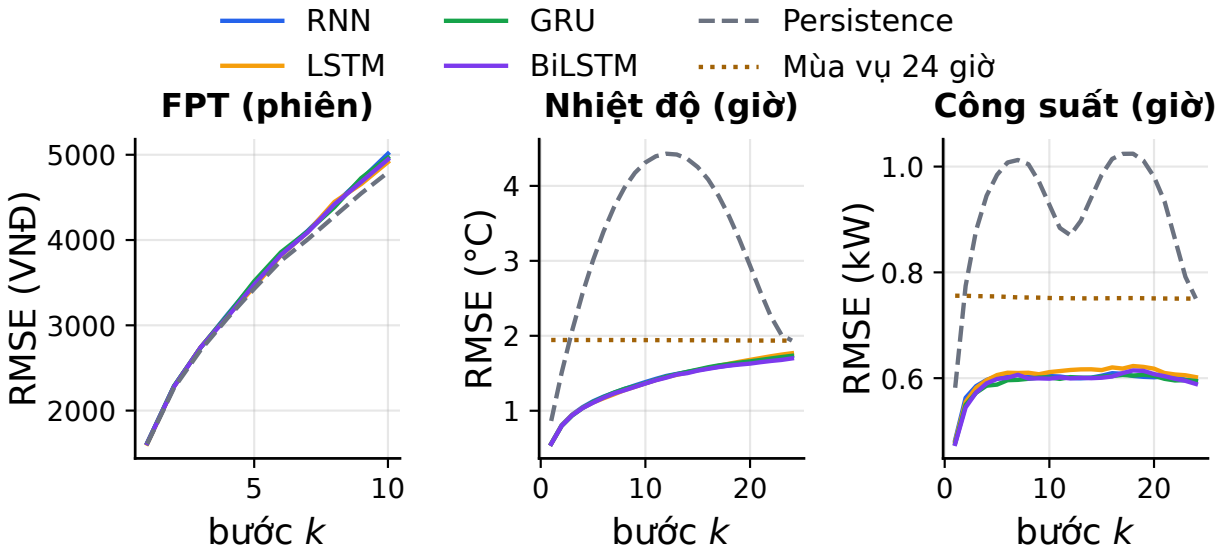
Hình 4.13. MSE chuẩn hóa theo epoch: nét liền là validation, nét chấm là train.



Hình 4.14. Độ nhạy đầu ra theo độ trễ, chuẩn hóa về 1 tại bước cuối.



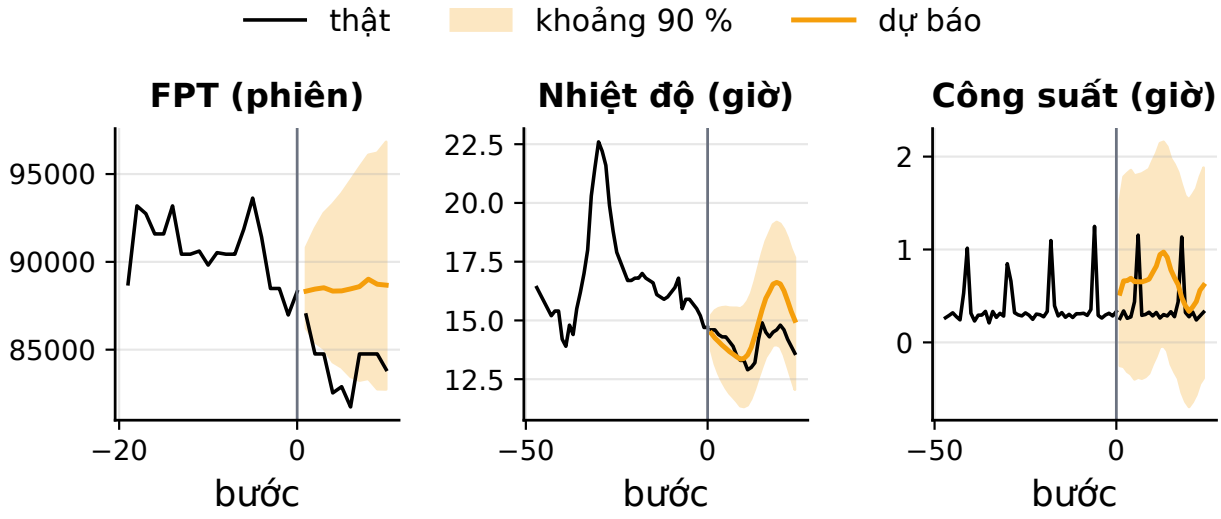
Hình 4.15. Chuẩn của δ_t trong BPTT NumPy khi lùi về đầu cửa sổ, RNN trên FPT.



Hình 4.16. RMSE test theo bước dự báo.

Bảng 4.7. Sai số ở bước cuối H và khoảng 90 % (độ phủ trên test, độ rộng trung bình theo đơn vị gốc).

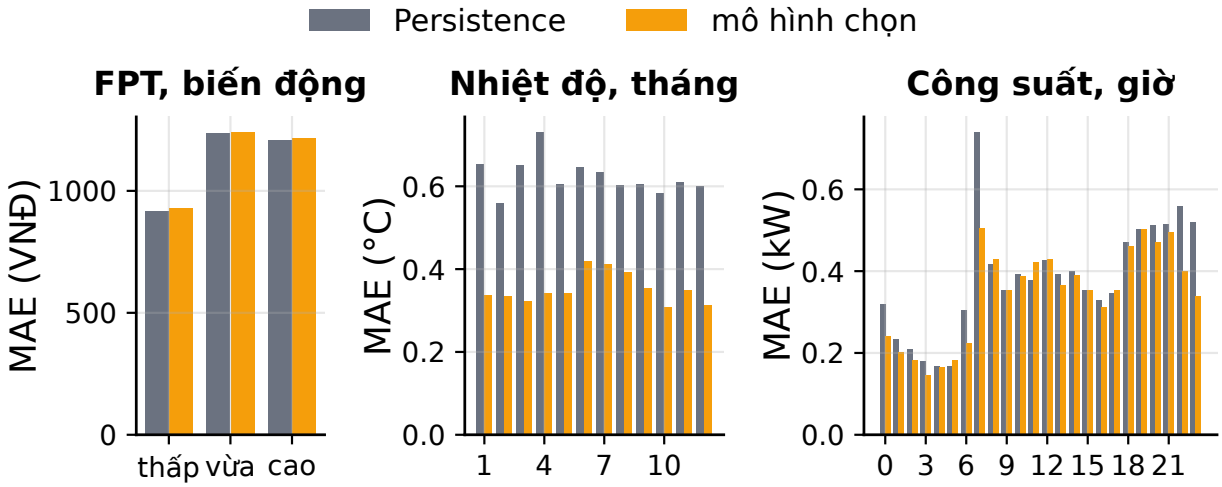
Chuỗi	Mô hình	Pers. bước H	Mô hình bước H	Phủ 90 %	Rộng
FPT	LSTM	4.793	4.914	82,1	9.943
VCB	LSTM	5.018	5.107	87,2	6.785
HPG	LSTM	1.030	1.036	96,1	3.599
VIC	BiLSTM	15.559	15.519	72,8	18.391
Nhiệt độ	GRU	1,936	1,733	90,2	4,319
Công suất	RNN	0,750	0,595	92,8	2,184



Hình 4.17. Một lần dự báo trực tiếp giữa tập test, kèm khoảng 90 %.

4.4.4 Phân tích lỗi

Hình 4.18 tách MAE một bước. FPT: nhóm biến động thấp có sai số nhỏ hơn hai nhóm còn lại, và ở cả ba nhóm mô hình ngang Persistence. Nhiệt độ: lợi thế ổn định qua các tháng. Công suất: lợi thế lớn nhất ở các giờ chuyển trạng thái buổi sáng, khi Persistence trễ một nhịp; buổi tối đỉnh bật tắt thiết bị vẫn khó đoán với cả hai.



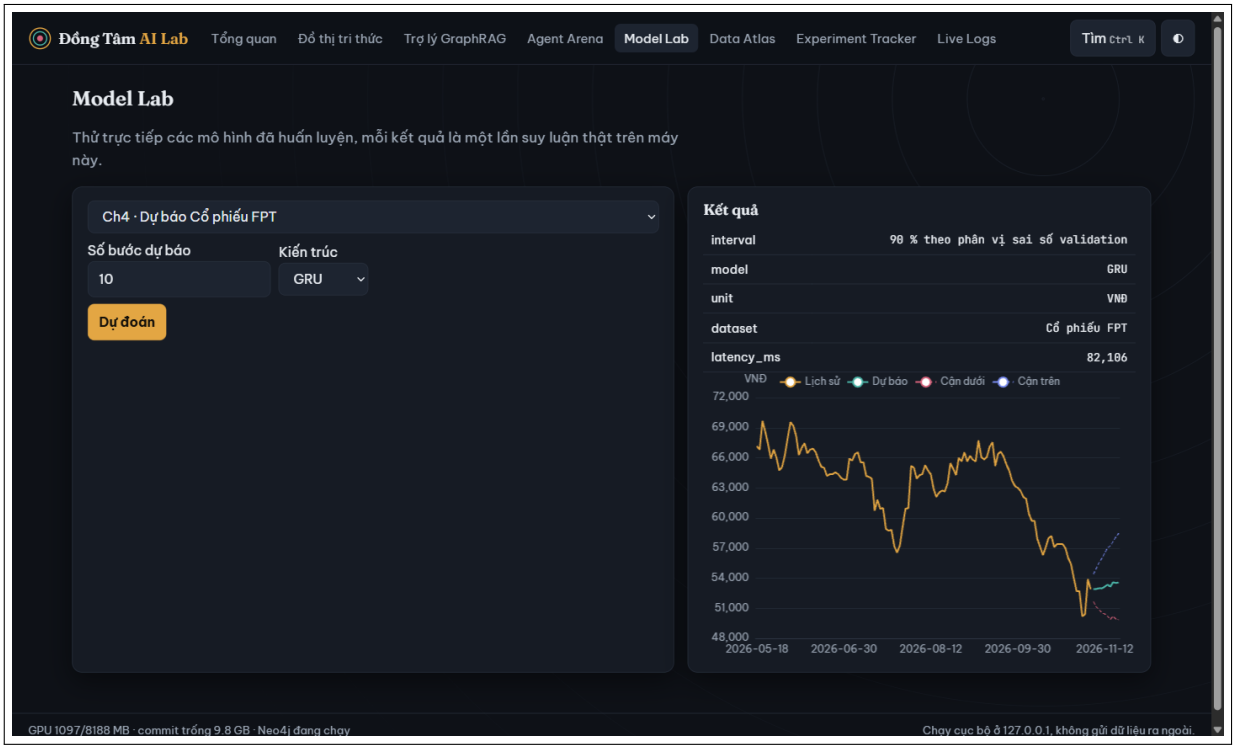
Hình 4.18. MAE một bước theo nhóm của mô hình được chọn và Persistence.

4.5 Triển khai

`tlf/serve_ch4.py` trả chuỗi quá khứ, dự báo N bước và khoảng 90 % cho từng mã và từng chuỗi, nạp mô hình từ `models/ch4/` và chạy CPU. Giao diện in kèm RMSE test của mô hình và của Persistence, để người dùng thấy ngay khi mô hình không thắng baseline.

Giới hạn

Mỗi cấu hình chỉ chạy một hạt giống nên khác biệt nhỏ giữa kiến trúc không đủ để xếp hạng. Keras chỉ chạy LSTM trên FPT, NumPy chỉ chạy RNN trên FPT. Thí nghiệm recurrent dropout chưa chạy, phần đó mới dừng ở lý thuyết. Lịch phiên tương lai trên Web App mới bỏ cuối tuần, chưa trừ ngày lễ.



Hình 4.19. Trang dự báo chuỗi thời gian trên Web App.

Chương 5

Knowledge Graph, Neo4j, RAG và Agent

Chương trước làm việc với chuỗi số. Chương này làm việc với tri thức: thực thể, quan hệ và văn bản, sao cho mô hình ngôn ngữ trả lời được câu hỏi về dữ kiện cụ thể mà không tự nghĩ ra. LLM thuần có ba điểm yếu đã biết: trọng số đóng băng ở thời điểm huấn luyện, ảo giác khi gặp dữ kiện hiếm, và khó nối nhiều mắt xích thông tin. Ba thành phần của chương xử lý ba điểm đó theo thứ tự: đồ thị tri thức trong Neo4j giữ dữ kiện, RAG đưa dữ kiện vào ngữ cảnh của LLM, tác tử ReAct chọn công cụ khi một lần truy xuất không đủ.

Toàn bộ dữ liệu nằm trong một cơ sở dữ liệu Neo4j thật gồm 50.234 nút và 175.010 cạnh, đọc trực tiếp từ máy chủ bằng Cypher. Ba LLM đều chạy cục bộ. Kết quả y tế và đầu tư trong chương chỉ mang tính học thuật.

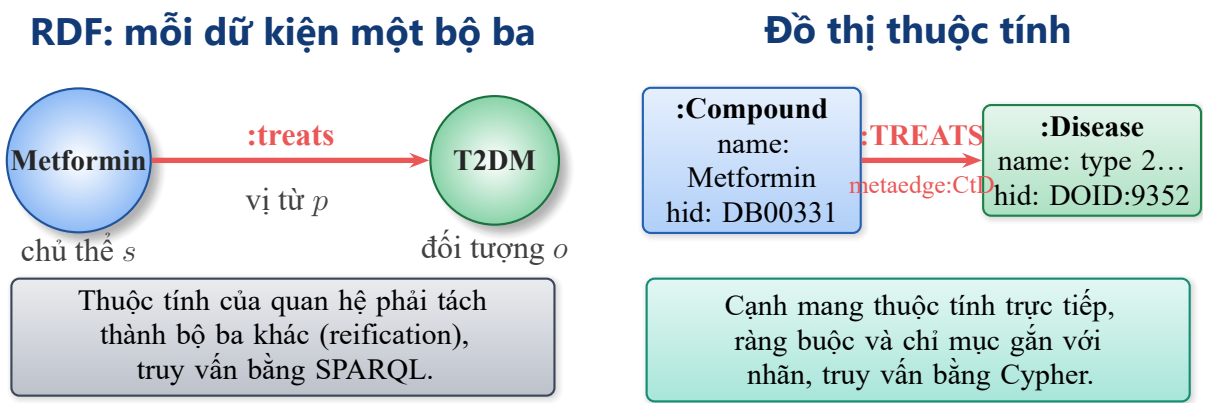
5.1 Biểu diễn tri thức bằng đồ thị

Đồ thị tri thức

Đồ thị tri thức là tập dữ kiện $\mathcal{G} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$, trong đó $\mathcal{E}$ là tập thực thể, $\mathcal{R}$ là tập quan hệ [101]. Mỗi dữ kiện (s, p, o) khẳng định chủ thể s có quan hệ p với đối tượng o.

RDF [102] chuẩn hóa dữ kiện thành bộ ba và định danh mọi thứ bằng IRI, nhờ đó dữ liệu từ nhiều nguồn nối được với nhau và truy vấn bằng SPARQL. Điểm bất tiện là quan hệ không có thuộc tính riêng: muốn ghi hệ số tương quan của một cặp cổ phiếu phải tách thành nhiều bộ ba. Đồ thị thuộc tính (Labeled Property Graph) cho phép cả nút lẫn cạnh mang thuộc tính, gắn nhãn cho nút và kiểu cho cạnh. Neo4j theo mô hình này, truy vấn bằng Cypher [103], một ngôn ngữ khớp mẫu mà câu lệnh trông giống chính đồ thị: `(a)-[:BINDS]->(b)`. Hình 5.1 đặt hai cách biểu diễn cạnh nhau trên cùng một dữ kiện lấy từ Hetionet.

Bản thể học (ontology) là phần lược đồ: khai báo có những lớp nào, quan hệ nào nối lớp nào, ràng buộc gì phải đúng. Ở Neo4j, ontology của chương được thể hiện bằng nhãn, kiểu cạnh và ràng buộc duy nhất. Đồ thị khác cơ sở dữ liệu vector ở bản chất câu hỏi: đồ thị trả lời chính xác câu hỏi có cấu trúc và truy vết được đường đi, còn vector trả lời câu hỏi mơ hồ theo độ gần nghĩa nhưng chỉ cho một con số khoảng cách, không giải thích được. Chương này dùng cả hai trong cùng

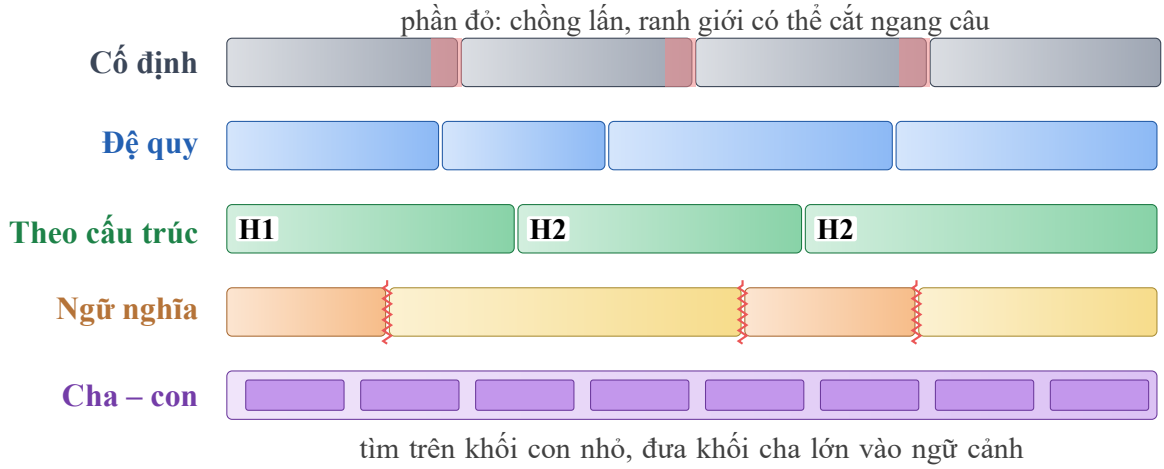


Hình 5.1. Cùng một dữ kiện theo RDF và theo đồ thị thuộc tính

một máy chủ.

5.2 RAG và phân mảnh văn bản

RAG [104] tách tri thức khỏi trọng số mô hình: lập chỉ mục tài liệu, truy xuất đoạn liên quan, ghép vào prompt rồi mới sinh câu trả lời. Muốn đổi tri thức chỉ cần đổi kho tài liệu, không phải huấn luyện lại. Chất lượng phụ thuộc trước hết vào cách cắt văn bản (Hình 5.2). Cắt cố định đơn giản nhưng hay đứt câu. Cắt đệ quy ưu tiên ranh giới mục, đoạn, câu rồi mới tới từ. Cắt theo cấu trúc bám tiêu đề tài liệu. Cắt ngữ nghĩa đặt ranh giới khi cosin giữa hai câu liền kề tụt xuống. Kiểu cha – con tìm trên khối nhỏ cho chính xác nhưng đưa khối lớn vào ngữ cảnh. Với Wikipedia, chương dùng cắt đệ quy 180 từ, chồng lấn 30 từ. Dữ liệu sinh từ đồ thị không cần cắt vì mỗi cặp (nút, quan hệ) đã là một đơn vị nghĩa.



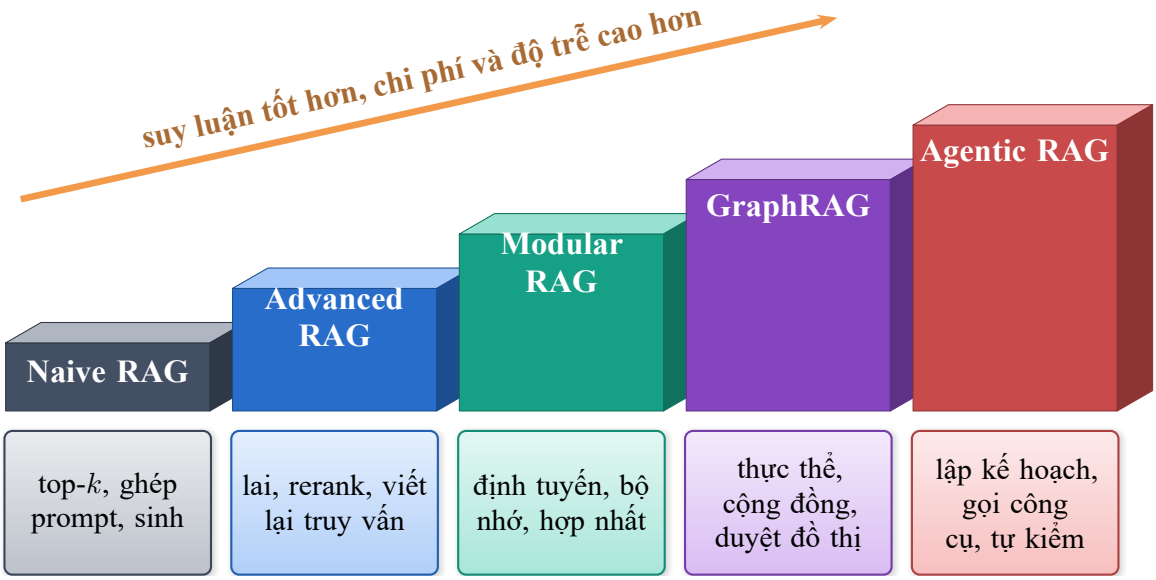
Hình 5.2. Năm chiến lược phân mảnh trên cùng một văn bản

Ba bộ truy xuất cơ sở chấm điểm khác nhau. BM25 [105] khớp mặt chữ, với $f(t,d)$ là tần suất từ t trong tài liệu, $|d|$ là độ dài tài liệu:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,|d|/\overline{|d|}\right)}, \qquad k_1 = 1{,}5,\ b = 0{,}75. \tag{5.1}$$

Dense nhúng câu hỏi và tài liệu bằng multilingual-e5-base [106] (768 chiều) rồi lấy cosin, tìm gần đúng bằng đồ thị HNSW [107] trong chỉ mục vector của Neo4j. Hybrid hợp nhất các danh sách bằng Reciprocal Rank Fusion [108], chỉ dùng thứ hạng nên không cần chuẩn hóa điểm:

$$\mathrm{RRF}(d) = \sum_{r} \frac{1}{c + \mathrm{hạng}_r(d)}, \qquad c = 60. \tag{5.2}$$



Hình 5.3. Năm thế hệ RAG, từ truy xuất đơn lẻ tới tác tử

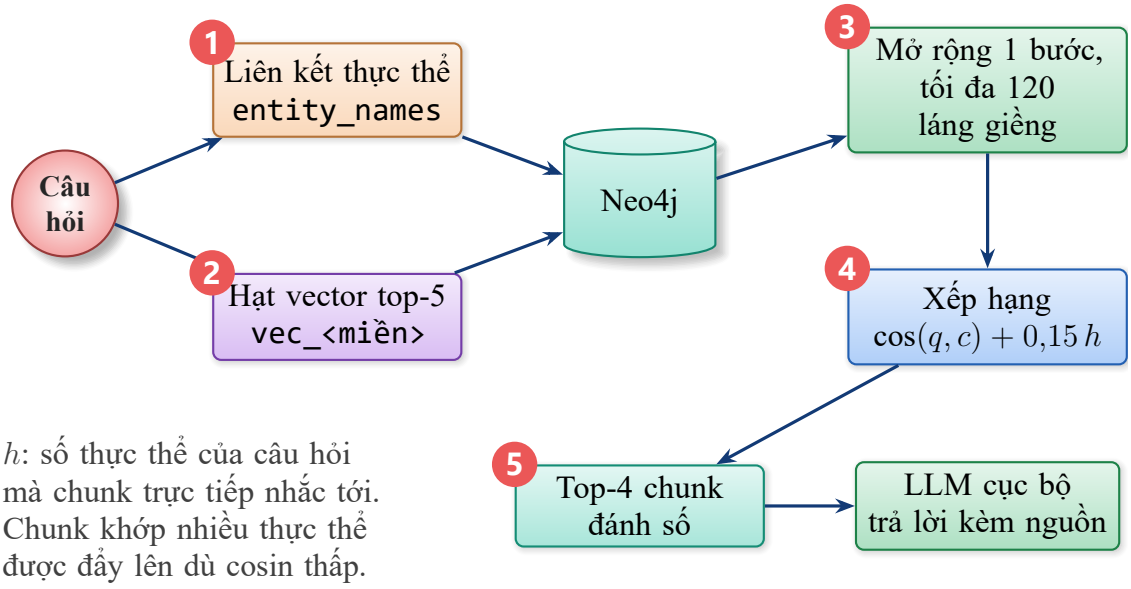
5.3 GraphRAG và tác tử ReAct

Naive RAG thất bại khi đáp án nằm rải trên hai tài liệu mà tài liệu thứ hai không giống câu hỏi. GraphRAG lấy đồ thị làm đường đi giữa các tài liệu. Phiên bản gốc [109] dựng cộng đồng thực thể rồi tóm tắt bằng LLM, phù hợp câu hỏi toàn cục nhưng tốn nhiều lượt gọi LLM lúc lập chỉ mục. Chương này chọn hướng tìm kiếm cục bộ, không cần LLM khi dựng chỉ mục (Hình 5.4): tìm thực thể trong câu hỏi bằng chỉ mục toàn văn, thêm thực thể từ năm chunk gần nhất theo vector, mở rộng một bước trên đồ thị, rồi xếp hạng các chunk gắn với tập nút vừa thu được:

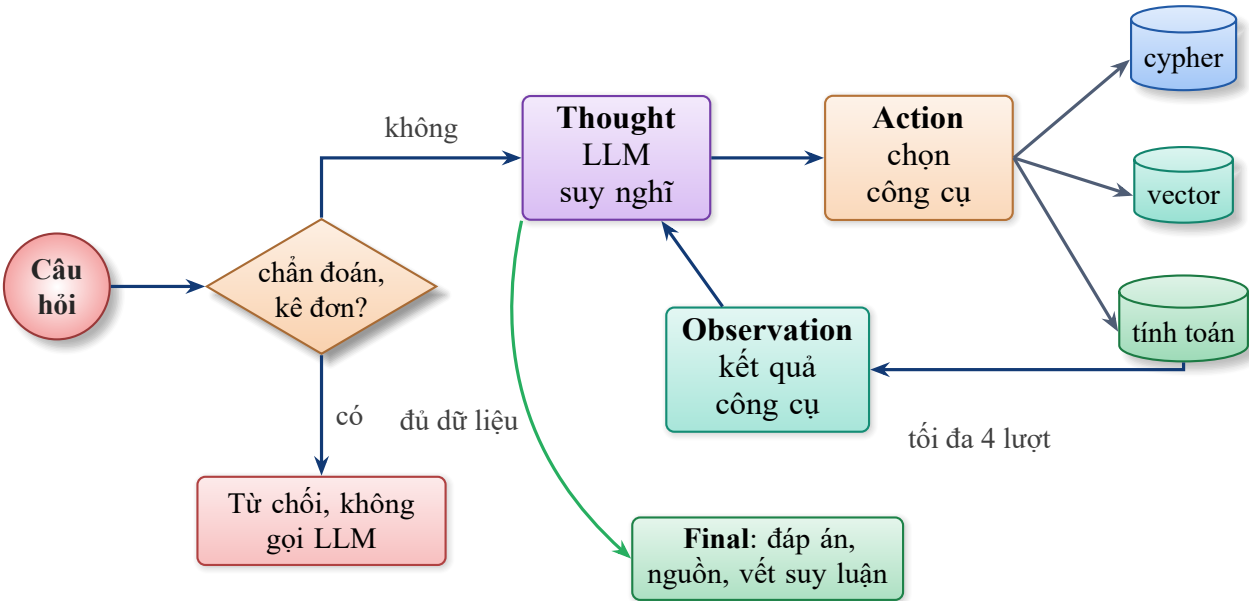
$$\text{score}(c) = \cos(\mathbf{q}, \mathbf{c}) + \lambda\, h(c), \quad \lambda = 0{,}15, \tag{5.3}$$

trong đó $h(c)$ là số thực thể liên kết từ câu hỏi mà chunk c trực tiếp gắn với. Hệ số λ đặt tay và giữ cố định, không dò trên tập đánh giá.

ReAct [110] xen kẽ suy nghĩ và hành động: ở mỗi lượt LLM viết một Thought rồi chọn Action, chương trình chạy công cụ và trả Observation vào ngữ cảnh, $S_{t+1} = S_t \cup \{\text{Thought}_t, \text{Action}_t, \text{Observation}_t\}$, cho tới khi LLM viết Final. Tác tử trong chương có ba công cụ: truy vấn Cypher chỉ đọc, tra vector và máy tính. Trước vòng lặp có một bộ lọc biểu thức chính quy chặn câu xin chẩn đoán hoặc kê đơn, chạy ngoài LLM nên không phụ thuộc vào việc mô hình có nghe lời hay không (Hình 5.5).



Hình 5.4. Luồng GraphRAG cục bộ trên Neo4j

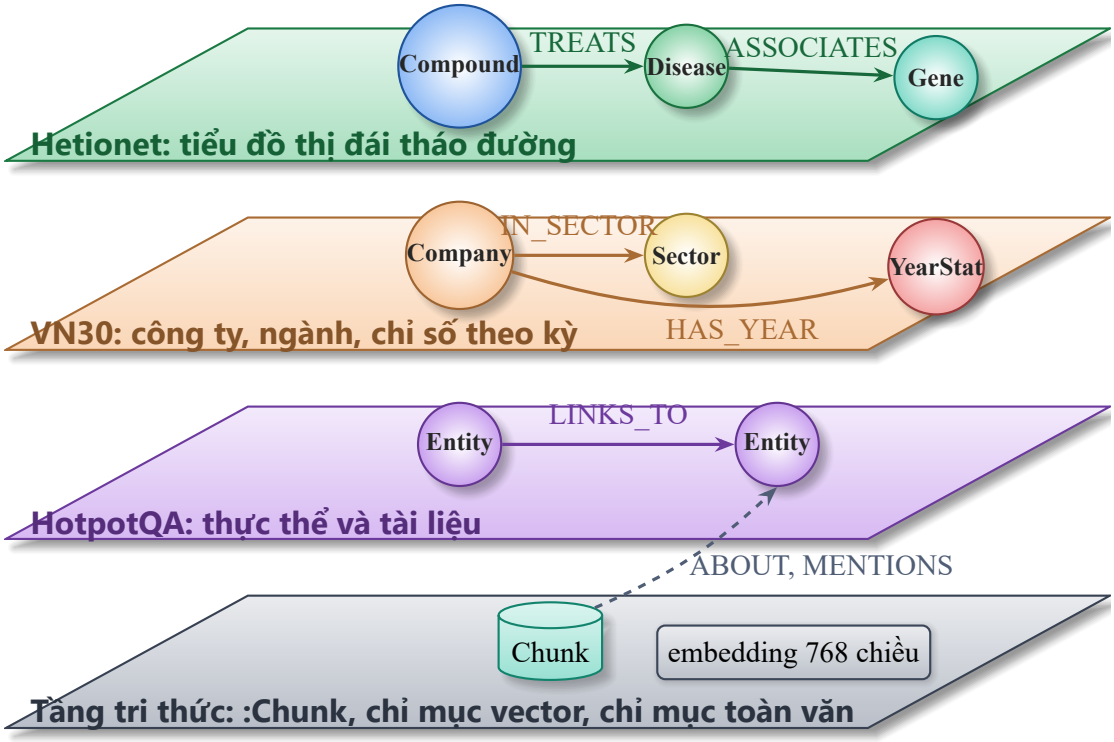


Hình 5.5. Vòng lặp ReAct có bộ lọc an toàn đặt trước LLM

5.4 Thiết kế đồ thị trong Neo4j

Neo4j 5.26 Community chạy trong Docker, heap 768 MB, page cache 512 MB. Mọi miền dùng chung một cơ sở dữ liệu vì bản Community chỉ có một cơ sở dữ liệu người dùng. Nút tri thức đều có nhãn chung `Entity` kèm nhãn riêng và thuộc tính `dataset`, nhờ đó một ràng buộc duy nhất và một chỉ mục toàn văn phủ cả ba miền (Hình 5.6, Bảng 5.1). Mỗi miền có một nhãn chunk và một chỉ mục vector riêng, vì chỉ mục vector của Neo4j lọc sau khi tìm, dùng chung sẽ làm mất ứng viên.

Dữ liệu nạp theo lô tối đa 2.000 dòng để giao dịch không vượt heap. Mọi lệnh



Hình 5.6. Ba miền tri thức và tầng chunk trong cùng một đồ thị Neo4j

Bảng 5.1. Ontology của đồ thị, số nút đọc từ Neo4j

Miền	Nhãn	Nghĩa	Số nút	Thuộc tính	Quan hệ đi ra
Hetionet	`Disease`	Bệnh	130	name, hid	ASSOCIATES, LOCALIZES, PRESENTS, RESEMBLES
Hetionet	`Compound`	Hợp chất, thuốc	967	name, hid	TREATS, PALLIATES, BINDS, CAUSES
Hetionet	`Gene`	Gen	6.027	name, hid	INTERACTS, PARTICIPATES
Hetionet	`Pathway`	Đường truyền	1.146	name, hid	(từ Gene) PARTICIPATES
Hetionet	`SideEffect`	Tác dụng phụ	974	name, hid	(từ Compound) CAUSES
VN30	`Company`	Công ty niêm yết	30	ticker, name	IN_SECTOR, MEMBER_OF, CORRELATED
VN30	`Sector`	Ngành	11	name	(từ Company) IN_SECTOR
VN30	`YearStat`	Chỉ số năm	253	ret_pct, vol_pct, high, low	OF_YEAR, (từ Company) HAS_YEAR
VN30	`MonthStat`	Chỉ số tháng	2.958	month, ret_pct	OF_MONTH, (từ Company) HAS_MONTH
HotpotQA	`TextEntity`	Thực thể văn bản	18.672	name	LINKS_TO
Cả ba	`Chunk`	Đoạn tri thức	17.218	text, source, embedding	ABOUT, MENTIONS

Đoạn mã 5.1. Ràng buộc, chỉ mục toàn văn và chỉ mục vector

```
CREATE CONSTRAINT entity_id IF NOT EXISTS FOR (e:Entity) REQUIRE e.id
  ↪  IS UNIQUE;
CREATE CONSTRAINT chunk_id IF NOT EXISTS FOR (c:Chunk) REQUIRE c.id IS
  ↪  UNIQUE;
CREATE CONSTRAINT company_ticker IF NOT EXISTS FOR (c:Company) REQUIRE
  ↪  c.ticker IS UNIQUE;
CREATE FULLTEXT INDEX entity_names IF NOT EXISTS FOR (e:Entity) ON EACH
  ↪  [e.name, e.ticker];
CREATE VECTOR INDEX vec_hetionet IF NOT EXISTS
FOR (c:ChunkHetionet) ON (c.embedding)
OPTIONS {indexConfig: {`vector.dimensions`: 768,
                       `vector.similarity_function`: 'cosine'}};
```

Đoạn mã 5.2. Nạp cạnh theo lô UNWIND, kiểu cạnh động qua APOC

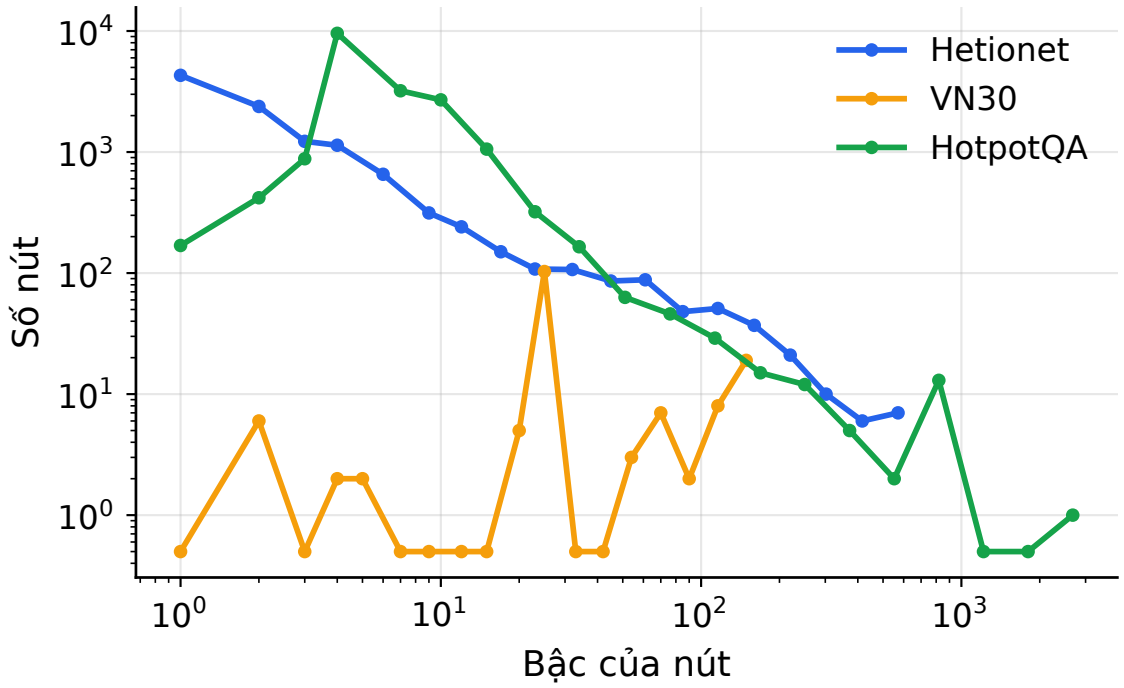
```
UNWIND $rows AS r
MATCH (a:Entity {id: r.source}), (b:Entity {id: r.target})
CALL apoc.merge.relationship(a, r.type, {}, r.props, b, {}) YIELD rel
RETURN count(rel) AS n
```

ghi dùng MERGE nên chạy lại không sinh trùng. Bảng 5.2 là số thật đọc từ máy chủ sau khi nạp.

Bảng 5.2. Quy mô đồ thị theo miền, đọc từ Neo4j

Miền	Thực thể	Cạnh thực thể	Chunk	Bậc TB	Trung vị	Bậc lớn nhất	Nút trung tâm
Hetionet	10.976	38.659	6.727	7,66	2	784	hypertension
VN30	157	398	722	52,38	31	191	Tập đoàn Vingroup
HotpotQA	18.672	55.817	9.769	9,51	6	3.973	American

Nhận xét: Hetionet và HotpotQA có phân bố bậc đuôi dài: trung vị bậc lần lượt 2,000 và 6,000 trong khi bậc lớn nhất là 784 và 3.973. Nút trung tâm của HotpotQA là từ “American”, một cụm viết hoa ít giá trị suy luận. Vì vậy truy vấn mở rộng phải đặt trần 120 láng giềng cho mỗi nút, nếu không một nút như vậy kéo theo gần như cả đồ thị. VN30 có 30 công ty với mạng tương quan dày nên không có đuôi dài.



Hình 5.7. Phân bố bậc của ba đồ thị, trục log – log

5.5 Ba bộ dữ liệu

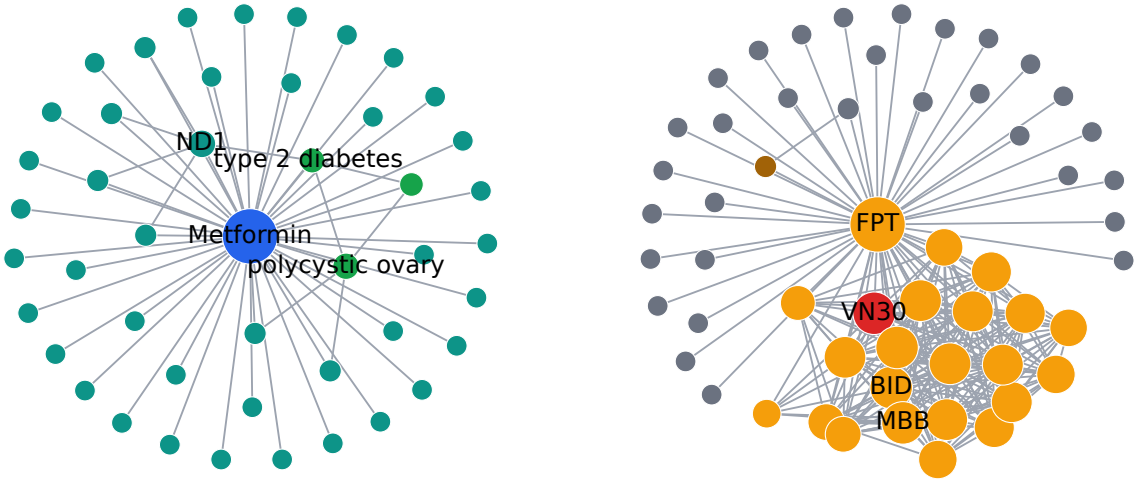
5.5.1 Hetionet: tiểu đồ thị đái tháo đường

Nguồn: Hetionet v1.0 [111], tệp TSV và SIF trên `https://github.com/hetio/hetionet`, giấy phép CC0. **Kích cỡ:** bản đầy đủ có 2.250.197 cạnh; tiểu đồ thị giữ 38.659 cạnh và 10.976 nút. **Tiền xử lý:** lấy hai bệnh hạt đái tháo đường type 1 và type 2 cùng 505 nút trong một bước quanh chúng, giữ mọi cạnh giữa các nút này, rồi giữ thêm các cạnh đi ra thuộc nhóm hữu ích cho câu hỏi thuốc và bệnh. Quá trình sinh học (GpBP, hơn 40 nghìn cạnh) và biểu hiện mô đi ra ngoài bị bỏ để vừa heap. Mỗi cặp (nút, quan hệ) sinh một chunk tiếng Việt liệt kê láng giềng, chia phần khi quá 70 tên.

Mẫu chunk Hetionet

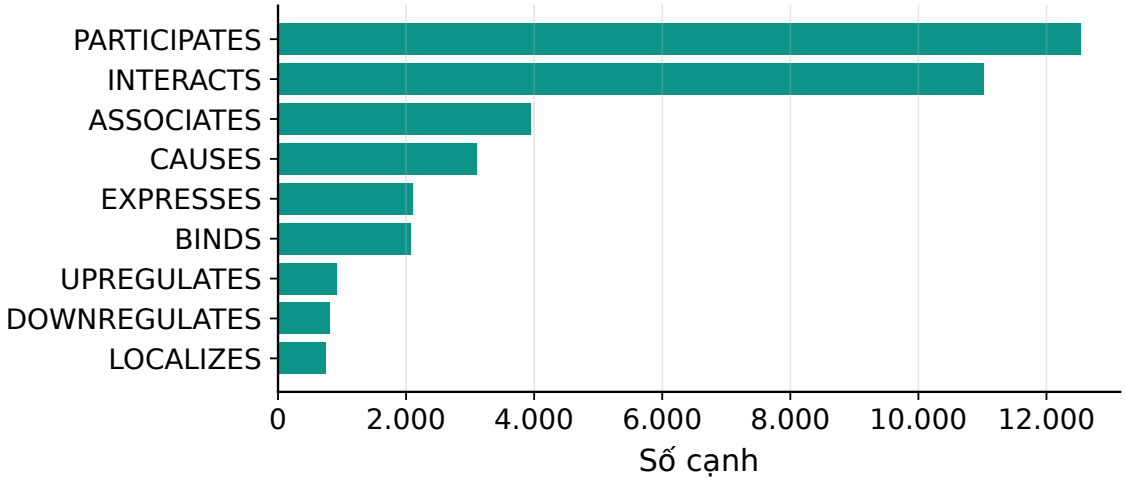
```
het:Compound::DB00331:CtD:0:0: Hợp chất Metformin điều trị:
metabolic syndrome X, polycystic ovary syndrome, type 2 diabetes
mellitus. ...
het:Disease::DOID:9352:DpS:0:0: Bệnh type 2 diabetes mellitus có
triệu chứng: Acute Coronary Syndrome, Albuminuria, Birth Weight,
Body Weight, Flushing, Gastroparesis, Microvascular Angina, Obesity,
Obesity, Abdominal, Overweight, Proteinuria ...
```

Nhận xét: Hai kiểu cạnh PARTICIPATES và INTERACTS chiếm hơn một nửa



(a) Quanh Metformin **(b)** Quanh FPT

Hình 5.8. Đồ thị con đọc từ Neo4j bằng `subgraph()`



Hình 5.9. Số cạnh theo kiểu trong tiểu đồ thị Hetionet

số cạnh, trong khi TREATS rất ít. Câu hỏi về điều trị vì vậy dựa trên số ít cạnh có giá trị cao. Nút bậc cao nhất trong tiểu đồ thị không phải đái tháo đường mà là hypertension, do các gen dùng chung kéo cả các bệnh lân cận vào đồ thị.

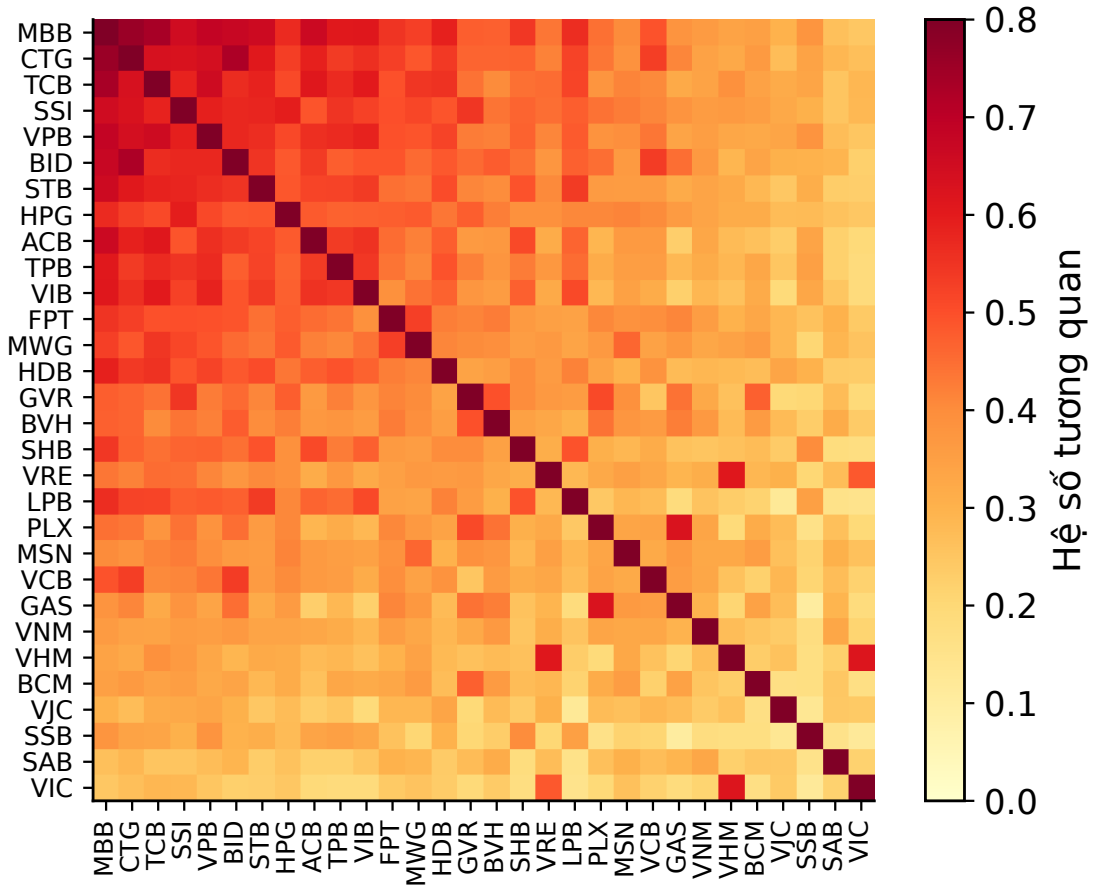
5.5.2 VN30: công ty, ngành, chỉ số và văn bản

Nguồn: giá phiên từ Yahoo Finance qua `yfinance`, mã có hậu tố `.VN`; bài viết từ API của Wikipedia tiếng Việt. **Kích cỡ:** 64.149 dòng phiên của 30 mã từ 2018-01-01 tới 2026-10-30, 3.211 nút chỉ số, 722 chunk. Danh sách 30 mã và ngành do người viết khai báo để minh họa, không phải ảnh chụp chính thức của rổ VN30 tại một ngày. Một số mã niêm yết muộn nên chỉ có dữ liệu từ năm 2020 hoặc 2021.

Tiền xử lý: tính lợi suất kỳ, độ biến động năm hóa, cao thấp theo năm và tháng; năm dưới 100 phiên bị bỏ. Từ 435 cặp mã, giữ 232 cặp có hệ số tương quan lợi suất ngày từ 0,35 trở lên làm cạnh `CORRELATED`. Wikipedia cắt đệ quy, mỗi chunk giữ URL và mã phiên bản bài. Hai ngoại lệ: VRE không có bài riêng nên bỏ, MSN dùng bài Masan Consumer là công ty con vì tìm kiếm trả về bài đó.

Mẫu chunk VN30

```
year:FPT:2023: Năm 2023, cổ phiếu FPT (Tập đoàn FPT, ngành Công
nghệ) có lợi suất +47,34%, độ biến động năm hóa 19,38%, giá cao nhất
66.073 đồng, thấp nhất 42.697 đồng, đóng cửa cuối kỳ 62.992 đồng,
260 phiên. Lợi suất theo t ...
profile:FPT: FPT là mã của Tập đoàn FPT, thuộc rổ VN30, ngành Công
nghệ. Cùng ngành: không có mã nào khác. ...
```



Hình 5.10. Ma trận tương quan lợi suất ngày của 30 mã, xếp theo tương quan trung bình

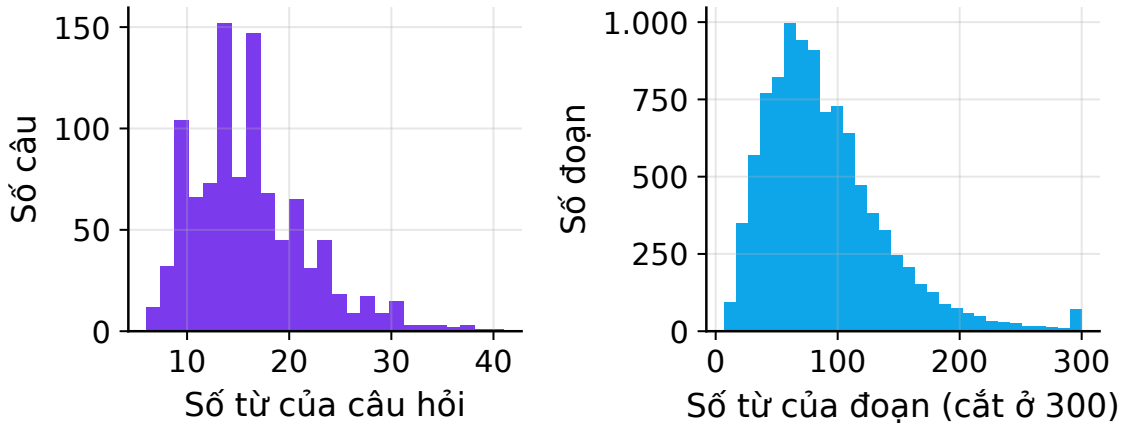
Nhận xét: Các mã ngân hàng và SSI tập trung thành khối tương quan cao ở góc trên trái. SAB và VIC nằm ở cuối ma trận, tức biến động ít đồng pha với phần còn lại. Tương quan chỉ mô tả quá khứ, không hàm ý nhân quả.

5.5.3 HotpotQA: truy xuất đa bước

Nguồn: HotpotQA [112], cấu hình distractor, https://huggingface.co/datasets/hotpotqa/hotpot_qa. **Kích cỡ:** dùng 1.000 câu đầu của tập validation; gộp 10 đoạn của mỗi câu theo tiêu đề được 9.769 đoạn. **Tiền xử lý:** mỗi tiêu đề là một thực thể, mỗi đoạn là một tài liệu; cụm viết hoa xuất hiện ít nhất hai đoạn cũng thành thực thể, tạo cạnh `MENTIONS` và `LINKS_TO`. Đây là trích xuất luật, không dùng LLM.

Mẫu chunk HotpotQA

```
hot:Scott Derrickson: Scott Derrickson. Scott Derrickson (born July
16, 1966) is an American director, screenwriter and producer. He
lives in Los Angeles, California. He is best known for directing
horror films such as ``Sinister'', ...
hot:Ed Wood: Ed Wood. Edward Davis Wood Jr. (October 10, 1924
– December 10, 1978) was an American filmmaker, actor, writer,
producer, and director. ...
```



Hình 5.11. Độ dài câu hỏi và đoạn văn của HotpotQA

Đoạn mã 5.3. Đường đi cầu nối giữa hai tài liệu qua một thực thể

```
MATCH (a:Chunk)-[:MENTIONS]->(e:Entity)<-[:ABOUT]-(b:Chunk)
WHERE a.dataset = 'hotpot' AND a <> b
RETURN a.title AS from_doc, e.name AS bridge, b.title AS to_doc
LIMIT $k
```

Nhận xét: Mỗi câu có đúng hai đoạn vàng, tám đoạn còn lại là đoạn nhiễu được chọn vì giống câu hỏi, nên đây là tập khó cho truy xuất một bước. Ngữ cảnh cắt ở 170 từ mỗi đoạn là đủ cho phần lớn các đoạn.

5.6 Cài đặt truy xuất trên Neo4j

Câu hỏi trên đồ thị được viết thẳng bằng Cypher. Truy vấn dưới đây trả lời một câu hai bước mà tìm kiếm văn bản khó làm được: gen nào vừa liên quan tới đái tháo đường type 2 vừa là đích của thuốc điều trị bệnh đó. Đáp án của bộ câu hỏi đánh giá được tính bằng đúng các truy vấn dạng này.

Đoạn mã 5.4. Giao hai đường đi: thuốc điều trị bệnh và gen liên quan tới bệnh

```
MATCH (c:Compound)-[:TREATS]->(d:Disease {id: $disease})
MATCH (c)-[:BINDS]->(g:Gene)<-[:ASSOCIATES]-(d)
RETURN c.name AS compound, g.name AS gene
ORDER BY compound, gene
```

Năm dòng đầu của kết quả thật

```
compound=Bromocriptine, gene=ADRA2A
compound=Bromocriptine, gene=CYP1A2
compound=Bromocriptine, gene=CYP3A4
compound=Bromocriptine, gene=HTR2C
compound=Chlorpropamide, gene=ABCC8
```

Đoạn mã 5.5. Mã có lợi suất cao nhất của mỗi ngành trong một năm

```
MATCH (c:Company)-[:IN_SECTOR]->(s:Sector)
MATCH (c)-[:HAS_YEAR]->(y:YearStat {year: $year})
WITH s, c, y ORDER BY y.ret_pct DESC
WITH s, collect({ticker: c.ticker, ret: y.ret_pct})[0] AS top
RETURN s.name AS sector, top.ticker AS ticker, top.ret AS ret_pct
ORDER BY ret_pct DESC
```

Năm dòng đầu của kết quả thật, năm 2023

```
sector=Chứng khoán, ticker=SSI, ret_pct=92.61
sector=Ngân hàng, ticker=HDB, ret_pct=78.87
sector=Nguyên vật liệu, ticker=GVR, ret_pct=56.38
sector=Công nghệ, ticker=FPT, ret_pct=47.34
sector=Năng lượng, ticker=PLX, ret_pct=10.75
```

GraphRAG gồm hai truy vấn. Truy vấn đầu dùng chỉ mục toàn văn để liên kết thực thể. Truy vấn sau mở rộng một bước từ tập hạt, gom chunk gắn với tập nút và

tính điểm ngay trong Cypher bằng `vector.similarity.cosine`.

Đoạn mã 5.6. Liên kết thực thể bằng chỉ mục toàn văn

```
CALL db.index.fulltext.queryNodes('entity_names', $lucene)
YIELD node AS e, score
WHERE e.dataset = $ds
RETURN e.id AS id, e.name AS name, score
ORDER BY score DESC LIMIT $n
```

Đoạn mã 5.7. Mở rộng đồ thị và xếp hạng chunk

```
UNWIND $seeds AS sid
MATCH (s:Entity {id: sid})
CALL (s) {
  MATCH (s)-[r]-(m:Entity)
  RETURN m ORDER BY type(r), m.id LIMIT $fanout
}
WITH collect(DISTINCT s) + collect(DISTINCT m) AS nodes
UNWIND nodes AS n
MATCH (c:Chunk)-[:ABOUT|MENTIONS]->(n)
WITH c, count(DISTINCT CASE WHEN n.id IN $linked THEN n END) AS hits
WITH c, hits, vector.similarity.cosine(c.embedding, $vec) AS sim
RETURN c.id AS id, sim + $boost * hits AS score, hits, sim
ORDER BY score DESC LIMIT $k
```

Đoạn mã 5.8. Ghép ba bước của GraphRAG trong Python

```
# tlf/ch5_retrieval.py
def search(self, q, vec, k=10):
    v = [float(x) for x in vec]
    linked = [r["id"] for r in self.link(q)]
    top = N.cypher(S.VECTOR_SEARCH, {"index": self.index, "k":
     ↪  self.n_vec, "vec": v})
    extra = N.cypher(S.SEED_ENTITIES, {"chunks": [r["id"] for r in
     ↪  top]})
    seeds = list(dict.fromkeys(linked + [r["id"] for r in extra]))
    rows = N.cypher(S.GRAPH_RETRIEVE, {"seeds": seeds, "linked":
     ↪  linked, "vec": v, "fanout": self.fanout,
                                       "boost": self.boost, "k": k})
    return [(r["id"], r["score"]) for r in rows]
```

Đoạn mã 5.9. Hợp nhất RRF

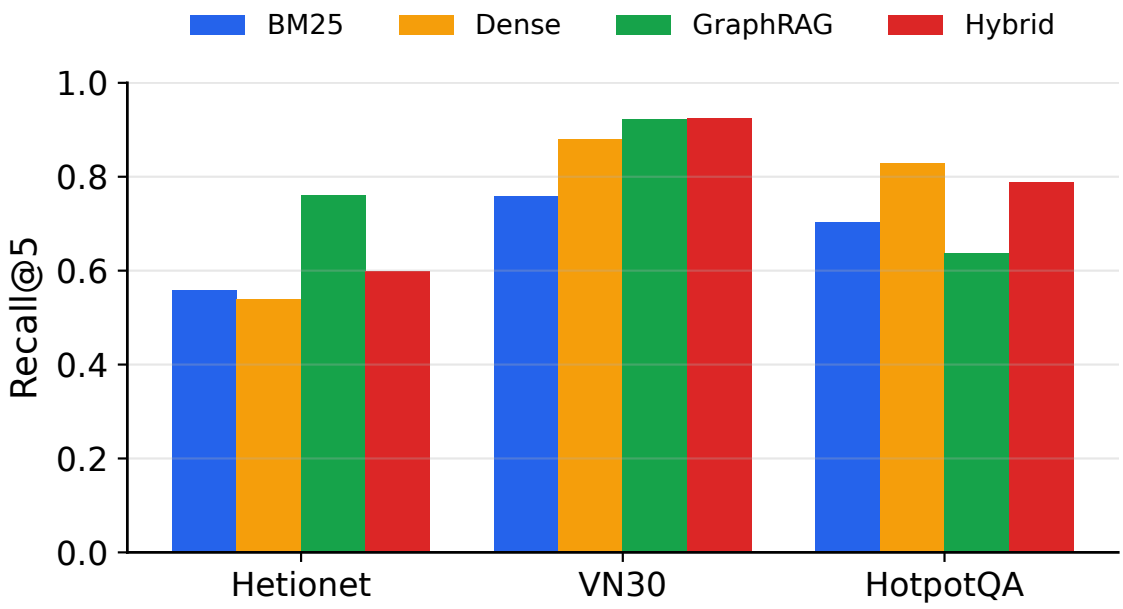
```python
# tlf/ch5_retrieval.py
def search(self, q, vec, k=10):
    fused = {}
    for r in self.parts:
        for rank, (cid, _) in enumerate(r.search(q, vec, self.depth),
        ↪  1):
            fused[cid] = fused.get(cid, 0.0) + 1.0 / (self.c + rank)
    return sorted(fused.items(), key=lambda x: -x[1])[:k]
```

5.7 Đánh giá truy xuất

Câu hỏi Hetionet và VN30 sinh bằng khuôn, đáp án tính từ dữ liệu bằng Cypher. Câu HotpotQA giữ nguyên câu gốc và đoạn vàng do bộ dữ liệu cung cấp. Vì sinh bằng khuôn nên hai bộ đầu đều đặn hơn câu hỏi thật, kết quả không nên suy rộng cho người dùng thật. Mọi câu hỏi trong Bảng 5.3 đều được chạy, không lấy mẫu con. Với G là tập chunk vàng và rel_i cho biết kết quả hạng i có vàng hay không:

$$\mathrm{Recall@}k = \frac{|G \cap \text{top-}k|}{|G|}, \quad \mathrm{nDCG@10} = \frac{\sum_{i=1}^{10} \mathrm{rel}_i / \log_2(i+1)}{\mathrm{IDCG}}. \tag{5.4}$$

MRR là trung bình nghịch đảo hạng của kết quả đúng đầu tiên, tính trong top-10.



Hình 5.12. Recall@5 theo miền và bộ truy xuất

Bảng 5.3. Bộ câu hỏi đánh giá theo miền và loại

Miền	Loại	Số câu	Ví dụ	Đáp án
Hetionet	Đếm 1 bước	43	Trong đồ thị con, hợp chất Doxepin liên kết với bao nhiêu gen?	5
Hetionet	Lọc 2 bước	43	Trong các thuốc điều trị type 2 diabetes mellitus, thuốc nào gây tác dụng	Bromocriptine
Hetionet	Giao 2 bước	48	Gen nào liên quan tới type 2 diabetes mellitus và tham gia đường truyền Se	AANAT
Hetionet	Tra 1 bước	19	Hợp chất Chlorpropamide thuộc nhóm dược lý nào?	Sulfonylurea Compo
VN30	Tổng hợp 2 bước	27	Trong ngành Ngân hàng, mã nào có lợi suất cao nhất năm 2021?	SSB
VN30	So sánh 2 bước	31	Năm 2023, cổ phiếu HPG hay VNM có lợi suất cao hơn?	HPG
VN30	Tra 1 bước	58	Mã nào có tương quan lợi suất ngày cao nhất với PLX?	GAS
VN30	Số liệu kỳ	92	Tháng 5 năm 2026, lợi suất của cổ phiếu VHM là bao nhiêu phần trăm?	6,85
HotpotQA	Cầu nối (bridge)	807	What government position was held by the woman who portrayed Corliss Arche	Chief of Protocol
HotpotQA	So sánh (comparison)	193	Were Scott Derrickson and Ed Wood of the same nationality?	yes

Bảng 5.4. Chất lượng truy xuất của bốn phương pháp

Miền	Bộ truy xuất	R@1	R@3	R@5	R@10	MRR	nDCG	ms
Hetionet (n=153)	BM25	0,528	0,554	0,558	0,558	0,691	0,581	52,2
	Dense	0,516	0,539	0,539	0,549	0,672	0,567	10,1
	GraphRAG	**0,679**	**0,718**	**0,760**	**0,834**	**0,974**	**0,818**	263,2
	Hybrid	0,526	0,585	0,598	0,633	0,721	0,618	205,6
VN30 (n=208)	BM25	0,522	0,721	0,760	0,873	0,694	0,724	6,1
	Dense	0,748	0,863	0,880	0,889	0,882	0,854	11,9
	GraphRAG	0,808	0,909	0,923	0,925	0,926	0,910	103,2
	Hybrid	**0,812**	**0,921**	**0,925**	**0,930**	**0,938**	**0,920**	172,2
HotpotQA (n=1000)	BM25	0,393	0,626	0,703	0,838	0,855	0,743	120,8
	Dense	**0,433**	**0,758**	**0,829**	0,888	**0,909**	**0,830**	46,5
	GraphRAG	0,277	0,522	0,637	0,740	0,689	0,613	163,6
	Hybrid	0,406	0,689	0,788	**0,906**	0,883	0,799	842,7

ms: trung vị thời gian một truy vấn, chưa tính thời gian mã hóa câu hỏi. In đậm: tốt nhất trong miền.

Bảng 5.5. Recall@5 theo loại câu hỏi

Miền	Loại câu hỏi	n	BM25	Dense	GraphRAG	Hybrid
Hetionet	Đếm 1 bước	43	0,977	0,953	**1,000**	0,977
Hetionet	Lọc 2 bước	43	0,163	0,430	**0,500**	0,465
Hetionet	Giao 2 bước	48	0,361	0,104	**0,684**	0,219
Hetionet	Tra 1 bước	19	**1,000**	0,947	**1,000**	**1,000**
VN30	Tổng hợp 2 bước	27	0,333	0,389	0,407	**0,426**
VN30	So sánh 2 bước	31	**1,000**	0,758	**1,000**	**1,000**
VN30	Tra 1 bước	58	0,448	0,983	**1,000**	**1,000**
VN30	Số liệu kỳ	92	**1,000**	**1,000**	**1,000**	**1,000**
HotpotQA	Cầu nối (bridge)	807	0,685	**0,800**	0,587	0,755
HotpotQA	So sánh (comparison)	193	0,777	**0,948**	0,850	0,925

Không có bộ truy xuất thắng ở mọi miền

GraphRAG dẫn đầu rõ ở Hetionet: Recall@5 0,7603 so với 0,5577 của BM25 và 0,5392 của Dense. Ở câu giao hai bước, Dense chỉ đạt 0,1042 còn GraphRAG đạt 0,6840: chunk chỉ liệt kê tên nên vector không phân biệt được, trong khi đồ thị đi đúng cạnh.

HotpotQA cho kết quả ngược lại: Dense đạt 0,8290, GraphRAG chỉ 0,6375. Đồ thị ở miền này dựng bằng luật từ cụm viết hoa nên nhiễu, các nút như "American" đẩy nhầm chunk qua số hạng $\lambda h(c)$. GraphRAG chỉ tốt khi đồ thị tốt.

Nhận xét: Hybrid thắng ở VN30 (MRR 0,9377) và có Recall@10 cao nhất ở HotpotQA (0,9065), nhưng ở Hetionet lại kém GraphRAG (0,5980 so với 0,7603): RRF bầu theo số đông nên một nguồn tốt bị hai nguồn yếu kéo xuống. Câu tổng hợp theo ngành của VN30 là loại khó nhất, Recall@5 không quá 0,4259, vì đáp án phải so sánh nhiều mã chứ không nằm ở một chunk. Loại câu này cần Cypher, phần tác tử sẽ kiểm tra điều đó. Độ trễ trung vị của Hybrid cao nhất, 842,67 ms ở HotpotQA, vì phải chạy cả ba nguồn tuần tự.

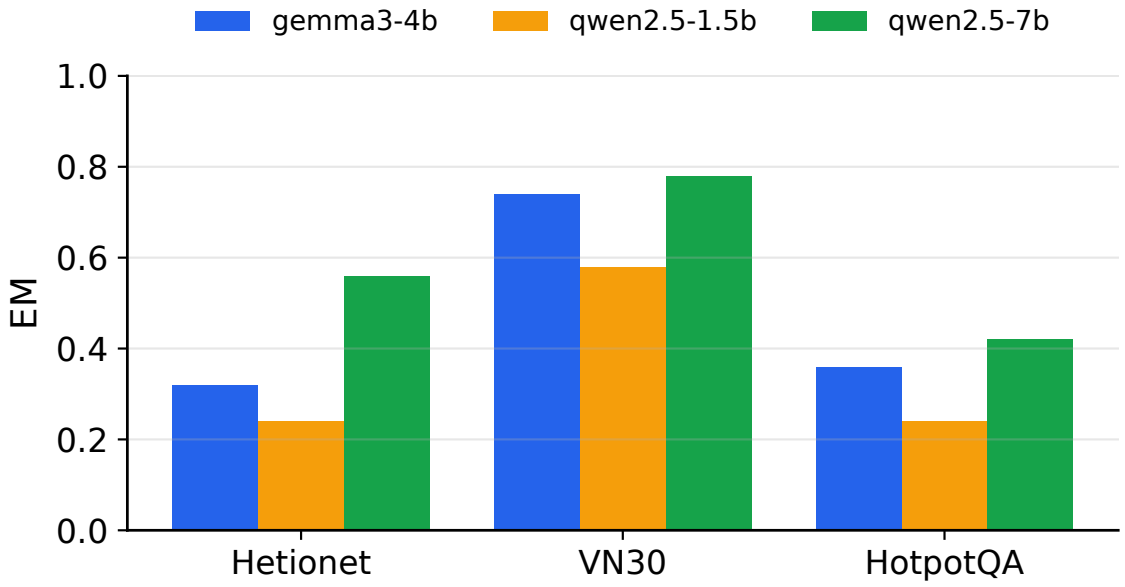
5.8 Đánh giá sinh với ba LLM

Ba mô hình chạy cục bộ qua Ollama trên RTX 4060: Qwen2.5 7B, Gemma 3 4B và Qwen2.5 1.5B. Cả ba nhận cùng ngữ cảnh Hybrid top-4, mỗi chunk cắt ở 170 từ, nhiệt độ 0, tối đa 48 token. Mỗi mô hình trả lời 50 câu đầu của mỗi miền, tổng cộng 450 lượt sinh. EM là tỷ lệ khớp chính xác sau chuẩn hóa, đáp án số được phép

lệch 0,5%. F1 tính trên token. Độ bám ngữ cảnh là tỷ lệ token của câu trả lời có mặt trong ngữ cảnh, phản ánh mức độ không tự nghĩ ra nhưng không khẳng định câu trả lời đúng. Câu trả lời "Không đủ dữ liệu" được tính là bám ngữ cảnh.

Bảng 5.6. Chất lượng và độ trễ sinh của ba LLM trên ba miền

Miền	LLM	EM	F1	Độ bám	Trễ p50 (s)
Hetionet (n=50)	qwen2.5-7b	**0,560**	**0,560**	**0,960**	0,36
	gemma3-4b	0,320	0,320	0,760	0,42
	qwen2.5-1.5b	0,240	0,256	0,855	0,09
VN30 (n=50)	qwen2.5-7b	**0,780**	0,727	**0,980**	0,95
	gemma3-4b	0,740	**0,740**	**0,980**	0,82
	qwen2.5-1.5b	0,580	0,604	**0,980**	0,25
HotpotQA (n=50)	qwen2.5-7b	**0,420**	**0,506**	0,640	0,71
	gemma3-4b	0,360	0,448	**0,760**	0,71
	qwen2.5-1.5b	0,240	0,349	0,750	0,15



Hình 5.13. EM của ba LLM theo miền

Nhận xét: Qwen2.5 7B có EM cao nhất cả ba miền, cao nhất ở VN30 (0,7800) và thấp nhất ở HotpotQA (0,4200). Gemma 3 4B theo sát ở VN30 (0,7400) nhưng kém rõ ở Hetionet (0,3200 so với 0,5600). Qwen2.5 1.5B yếu nhất ở cả ba miền nhưng nhanh nhất, trung vị 0,2548 giây ở VN30 so với 0,9498 giây của 7B. Độ bám ngữ cảnh ở VN30 của cả ba đều xấp xỉ 0,9800 vì đáp án là mã hoặc số lấy thẳng từ ngữ cảnh. Ở HotpotQA, 7B có độ bám thấp hơn hai mô hình còn lại dù EM cao hơn,

nên độ bám không thay thế được độ chính xác. Với 50 câu mỗi ô, sai lệch một vài câu đã đổi EM vài điểm phần trăm, nên chỉ các chênh lệch lớn mới có ý nghĩa.

Phân tích lỗi của Qwen2.5 7B. Hybrid đưa được ít nhất một chunk vàng vào top-4 ở 89,33% số câu, nên phần lớn lỗi nằm ở bước đọc. Có ba nhóm. Một là không ghép được hai chunk: 16/150 câu trả lời "Không đủ dữ liệu", như câu hỏi thuốc điều trị đái tháo đường gây một tác dụng phụ cụ thể, có đáp án Ramipril. Hai là đọc nhầm con số trong chunk dày số: hỏi lợi suất VIC tháng 1/2024, đáp án $-4{,}71$, mô hình trả $-17{,}16$. Ba là lỗi do cách chấm: "Lee Hazlewood" so với đáp án vàng "Barton Lee Hazlewood" bị tính sai EM dù đúng người, nên EM ở HotpotQA thấp hơn chất lượng thực tế.

5.9 Tác tử ReAct và so sánh ba hệ

Tác tử dùng Qwen2.5 7B, tối đa 4 lượt. Công cụ Cypher chỉ cho phép câu lệnh đọc: từ khóa ghi và thủ tục ghi bị từ chối trước khi tới máy chủ, câu lệnh có giới hạn thời gian 10 giây và 20 dòng kết quả. Bộ lọc lâm sàng được thử trên 20 câu tự viết: chặn 10/10 câu xin chẩn đoán hoặc kê đơn và chặn nhầm 0/10 câu tra cứu học thuật. Bộ lọc dựa trên từ khóa nên cách diễn đạt né từ khóa vẫn có thể lọt qua, con số trên chỉ áp dụng cho bộ thử này.

Bảng 5.7. Naive RAG, GraphRAG và Agent trên cùng LLM Qwen2.5 7B

Miền	Hệ thống	EM	F1	Độ bám	Trễ p50 (s)	Số bước
Hetionet (n=30)	Naive RAG	0,433	0,433	**0,900**	0,38	1,00
	GraphRAG	**0,600**	**0,622**	0,767	0,41	1,00
	Agent	0,333	0,333	0,433	3,08	2,20
VN30 (n=30)	Naive RAG	0,633	0,633	0,933	0,82	1,00
	GraphRAG	**0,767**	**0,733**	**0,967**	0,62	1,00
	Agent	0,633	0,467	0,900	4,03	2,57
HotpotQA (n=30)	Naive RAG	**0,533**	**0,645**	0,583	0,72	1,00
	GraphRAG	0,200	0,310	0,483	0,62	1,00
	Agent	0,467	0,611	**0,633**	5,33	2,80

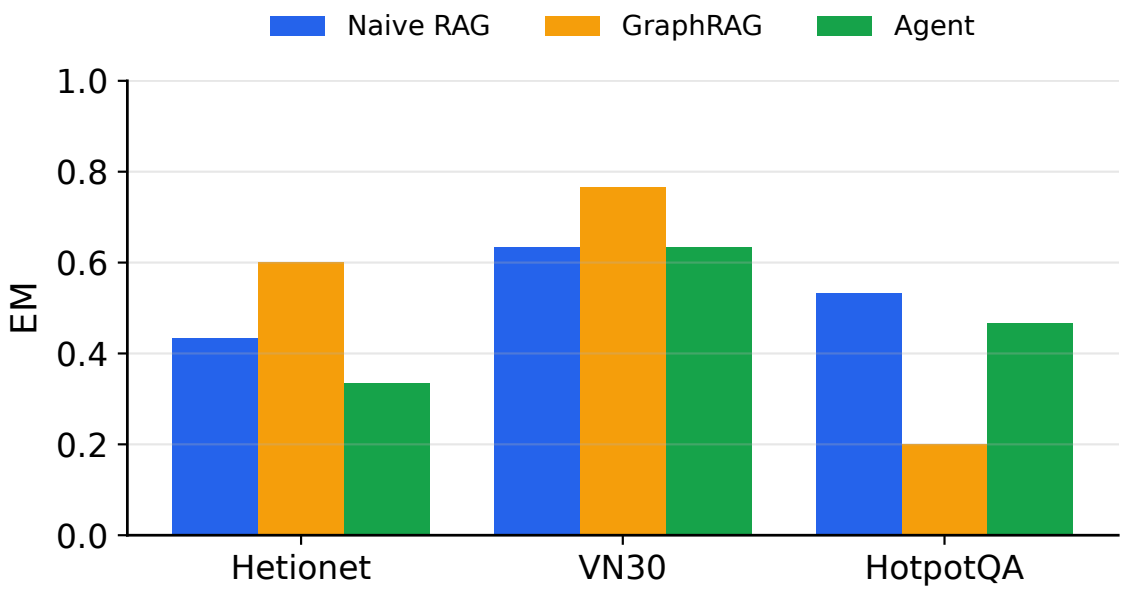
Vết suy luận đúng, câu hỏi VN30

CÂU HỎI Tháng 8 năm 2019, lợi suất của cổ phiếu HPG là bao nhiêu phần trăm?

THOUGHT Cần tìm cổ phiếu HPG có liên kết HAS_MONTH với tháng 8/2019 và lấy giá trị ret_pct.

Đoạn mã 5.10. Vòng lặp ReAct: LLM viết, chương trình chạy công cụ

```python
# tlf/ch5_agent.py
def run_agent(llm, question, ds, max_new_tokens=120):
    """Generator phát từng bước {"kind","text"}, cuối cùng phát {"kind":"final",...}."""
    if G.is_clinical(question):
        yield {"kind": "final", "text": G.REFUSAL, "sources": [], "blocked": True, "steps": 0, "forced": False}
        return
    tools, log, seen = Tools(ds), [], set()
    history = f"{SCHEMA_HINT[ds]}\n\nCâu hỏi: {question}\n"
    for step in range(1, MAX_STEPS + 1):
        text = llm.chat(history, SYSTEM_REACT, max_new_tokens)
        kind, tool, arg = parse_step(text)
        thought = re.search(r"Thought:\s*(.+)", text)
        if thought:
            log.append({"kind": "thought", "text": thought[1].strip()})
            yield log[-1]
        if kind == "final":
            yield {"kind": "final", "text": arg, "sources": tools.sources, "blocked": False, "steps": step, "forced": False}
            return
        if kind == "none" or (tool, arg) in seen:
            break
        seen.add((tool, arg))
        log.append({"kind": "action", "text": f"{tool}({arg})"})
        yield log[-1]
        obs = tools.run(tool, arg)[:OBS_CHARS]
        log.append({"kind": "observation", "text": obs})
        yield log[-1]
        history += f"Thought: {thought[1].strip() if thought else ''}\nAction: {tool}\nInput: {arg}\nObservation: {obs}\n"
    yield {"kind": "final", "text": force_final(llm, question, log, ds), "sources": tools.sources, "blocked": False,
           "steps": MAX_STEPS, "forced": True}
```



Hình 5.14. EM của ba hệ theo miền

ACTION	cypher(MATCH (c:Company {ticker:'HPG'})-[:HAS_MONTH]->(m:MonthStat {month:'2019-08'}) RETURN m.ret_pct)
OBSERV.	[{"m.ret_pct": -1.77}]
THOUGHT	Đã tìm được lợi suất của cổ phiếu HPG vào tháng 8/2019.
FINAL	-1.77

Vết suy luận sai, câu hỏi Hetionet

CÂU HỎI	Trong các thuốc điều trị type 2 diabetes mellitus, thuốc nào gây tác dụng phụ Fibrosis?
THOUGHT	Cần tìm thuốc (compound) điều trị type 2 diabetes mellitus và gây tác dụng phụ Fibrosis.
ACTION	cypher(MATCH (c:Compound)-[:TREATS|PALLIATES]->(:Disease {name:'type 2 diabetes mellitus'})-[:CAUSES]->(se:SideEffect {name:'Fibrosis'}) RETURN c.name LIMIT 1)
OBSERV.	Không có dòng nào.
THOUGHT	Không có kết quả trả về, có thể không có thuốc nào gây cả hai tác dụng.
FINAL	No

Tác tử không thắng toàn cục, nhưng thắng đúng chỗ dự đoán

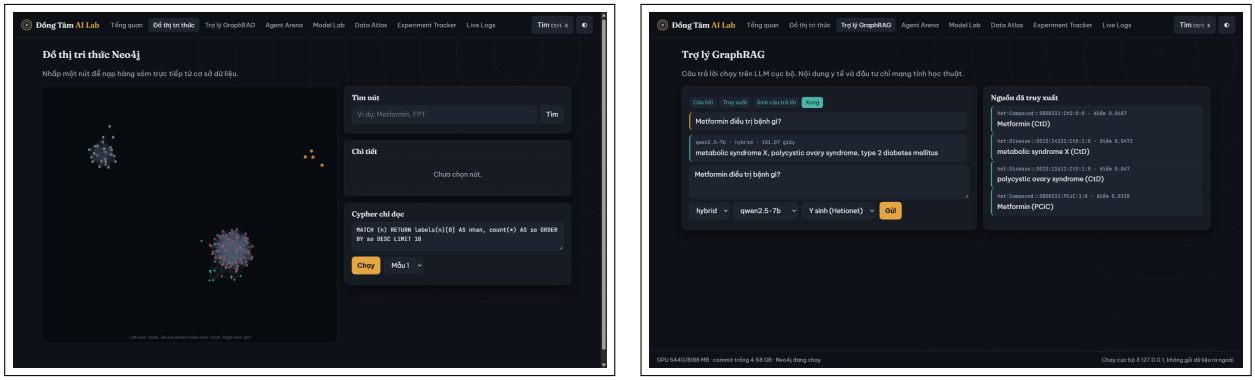
Tính chung, GraphRAG một lượt có EM cao hơn Agent ở Hetionet (0,6000 so với 0,3333) và VN30 (0,7667 so với 0,6333). Ở câu tổng hợp theo ngành của VN30, nơi đáp án không nằm trong một chunk, tình hình đảo ngược: Agent đạt 1,000, GraphRAG chỉ 0,3333. Ở HotpotQA câu cầu nối, Agent cũng hơn

GraphRAG (0,3636 so với 0,0909), nhưng ở đây GraphRAG vốn yếu do đồ thị nhiễu và Naive RAG mới là hệ tốt nhất (0,5333).

Nhận xét: Nguyên nhân Agent thua nằm ở câu Cypher do LLM viết, không nằm ở dữ liệu. Ở Hetionet, trong các lượt gọi Cypher chỉ 10 lượt có kết quả, 16 lượt rỗng và 6 lượt lỗi. Lỗi rỗng thường do sai chiều cạnh, như viết `(c)-[:INCLUDES]->(pc)` trong khi lược đồ là chiều ngược lại, hoặc nối sai kiểu cạnh. Khi thấy rỗng, mô hình thường kết luận luôn thay vì sửa truy vấn. Ở VN30 có 18 lượt lỗi; một phần do mô hình bọc câu lệnh trong khối mã nhiều dòng mà bộ phân tích của tác tử chỉ đọc một dòng. Đây là điểm yếu của cài đặt, chưa được sửa trong lần đo này. Cái giá của vòng lặp là độ trễ: trung vị Agent 4,032 giây ở VN30, so với 0,6226 giây của GraphRAG. Với mẫu 30 câu mỗi miền, chênh lệch dưới khoảng 10 điểm phần trăm không nên xem là khác biệt chắc chắn.

5.10 Triển khai và kết luận chương

`tlf/serve_ch5.py` cung cấp `ask(question, mode, model)`, một generator phát các sự kiện bước, chunk truy xuất, token và kết quả cuối để Web App truyền qua SSE, cùng `subgraph()` và `stats()` cho trang đồ thị. Câu trả lời ở miền y tế và tài chính kèm ghi chú chỉ mang tính học thuật. Lần hỏi đầu tiên phải nạp e5-base lên CPU; lúc máy bận, bước này đo được 106,70 giây, nên giao diện báo tiến độ và `unload()` giải phóng bộ nhúng khi không dùng.



(a) Khám phá đồ thị Neo4j **(b)** Trợ lý hỏi đáp

Hình 5.15. Web App: trang đồ thị và trang trợ lý

Ba kết luận rút ra từ số đo. Thứ nhất, cách dựng đồ thị quyết định GraphRAG: đồ thị có cấu trúc thật của Hetionet cho Recall@5 cao hơn hẳn văn bản thuần, đồ thị trích bằng luật của HotpotQA lại làm kết quả tệ đi. Thứ hai, không có một bộ truy xuất tốt nhất, và RRF không phải lúc nào cũng an toàn. Thứ ba, tác tử đáng

dùng cho câu hỏi tổng hợp nhưng trên LLM 7B chất lượng câu Cypher là nút thắt: cung cấp sẵn các truy vấn có tham số an toàn hơn để LLM tự viết. Giới hạn của đánh giá: câu hỏi Hetionet và VN30 sinh bằng khuôn, không có câu hỏi trả lời từ phần Wikipedia, và GraphRAG không dùng tóm tắt cộng đồng. Chương 6 đi tiếp sang kiến trúc nhiều tác tử.

Chương 6

Hệ đa tác tử

Chương 5 dừng ở một tác tử ReAct dùng công cụ. Chương này đặt câu hỏi tiếp theo: chia việc cho nhiều tác tử có làm hệ thống đúng hơn không, và phải trả bao nhiêu token cho phần đúng thêm đó? Lý thuyết đi trước, sau đó ba kiến trúc được tự cài trên cùng một LLM cục bộ và đo trên ba dataset, có kiểm định cặp vì cỡ mẫu nhỏ.

6.1 Mô hình hình thức của hệ đa tác tử

6.1.1 Dec-POMDP

Khi nhiều tác tử cùng theo đuổi một mục tiêu nhưng mỗi tác tử chỉ thấy một phần sự việc, khung chuẩn là quá trình quyết định Markov quan sát một phần phi tập trung (Dec-POMDP) [113, 114]:

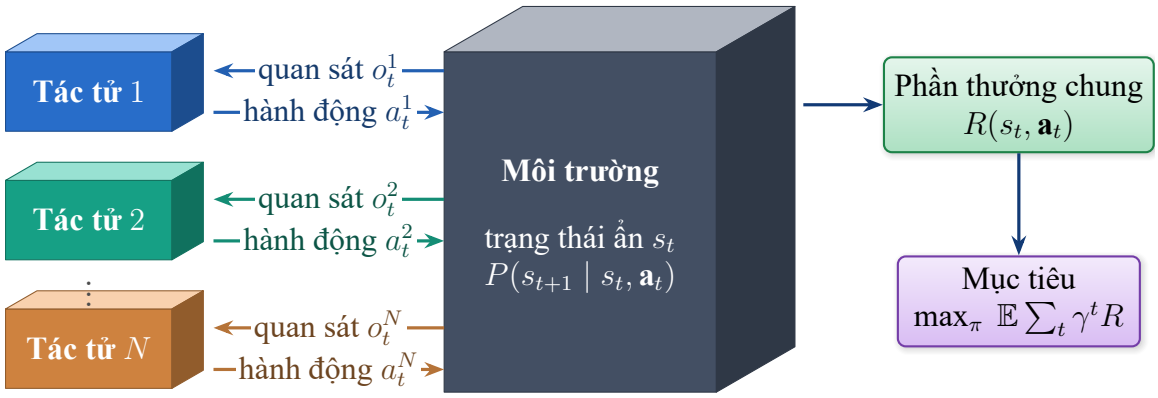
$$\mathcal{M} = \langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}^i\}, P, R, \{\Omega^i\}, O, \gamma \rangle. \tag{6.1}$$

Ở đây $\mathcal{I}$ là tập N tác tử, $\mathcal{S}$ là không gian trạng thái, $\mathcal{A}^i$ là tập hành động của tác tử i. Hành động chung $\mathbf{a}_t = (a_t^1, \ldots, a_t^N)$ đưa hệ sang trạng thái mới theo $P(s_{t+1} \mid s_t, \mathbf{a}_t)$. Mọi tác tử nhận chung phần thưởng $R(s_t, \mathbf{a}_t)$, còn quan sát riêng $o_t^i \in \Omega^i$ sinh từ $O(\mathbf{o}_t \mid s_{t+1}, \mathbf{a}_t)$. Chính sách π^i chỉ được dựa trên lịch sử riêng h_t^i, và mục tiêu là

$$\pi^* = \arg\max_{\pi} \; \mathbb{E}\Big[\sum_{t=0}^{T} \gamma^t R(s_t, \mathbf{a}_t)\Big], \qquad \pi = (\pi^1, \ldots, \pi^N), \;\; a_t^i \sim \pi^i(\cdot \mid h_t^i). \tag{6.2}$$

Với chân trời hữu hạn và từ hai tác tử trở lên, bài toán này thuộc lớp NEXP-đầy đủ [113], khó hơn hẳn POMDP một tác tử. Đó là lý do hệ thực tế không tìm chính sách tối ưu mà áp đặt cấu trúc phối hợp cố định, như các kiến trúc ở mục 6.2.

Áp vào tác tử LLM, phép ánh xạ khá thẳng: trạng thái ẩn là đáp án đúng, quan sát riêng là phần ngữ cảnh mỗi tác tử được nhìn thấy, hành động là văn bản hoặc lời gọi công cụ, phần thưởng là đúng hay sai ở cuối. Khác biệt là chính sách không học được ở đây: nó được cố định bởi system prompt cộng trọng số LLM, nên thiết kế kiến trúc thay cho việc tối ưu π.



Hình 6.1. Dec-POMDP: mỗi tác tử chỉ thấy quan sát riêng, cùng tác động lên trạng thái ẩn và chia chung phần thưởng.

6.1.2 Markov game và cân bằng Nash

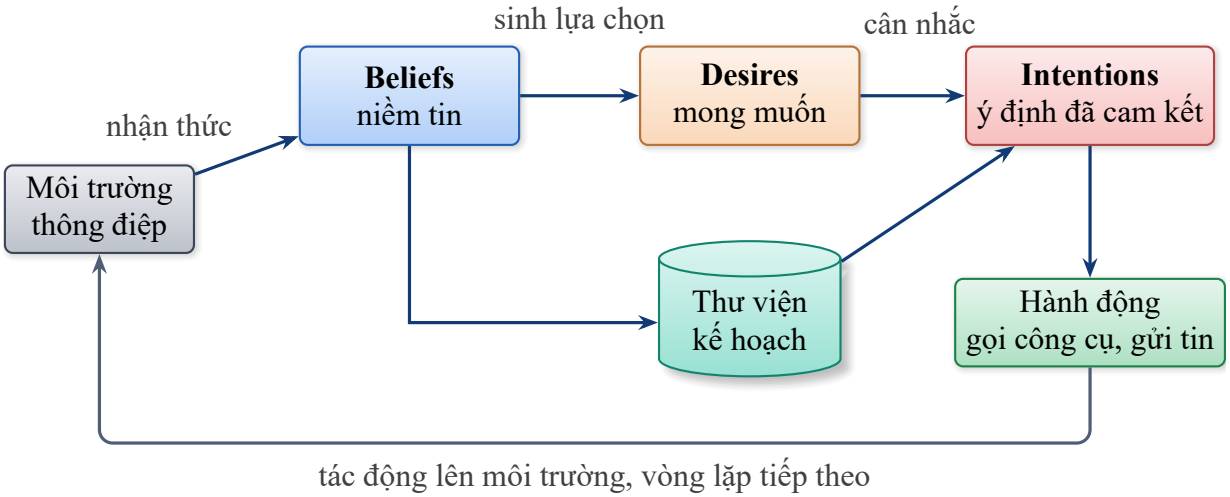
Khi mỗi tác tử có phần thưởng riêng R^i, ta có Markov game [115]: hợp tác hoàn toàn là trường hợp $R^1 = \cdots = R^N$, đối kháng tổng không là $R^1 = -R^2$. Không còn một tối ưu chung, nên lời giải là cân bằng Nash [116], nơi không tác tử nào được lợi khi đơn phương đổi chính sách:

$$J^i(\pi^{i*}, \pi^{-i*}) \;\geq\; J^i(\pi^i, \pi^{-i*}) \qquad \forall \pi^i,\ \forall i. \tag{6.3}$$

Debate mượn hình thức đối kháng này nhưng mục tiêu cuối vẫn là hợp tác: hai bên phản biện nhau để trọng tài có đáp án tốt hơn, không phải để một bên thắng.

6.1.3 Kiến trúc nhận thức BDI

Dec-POMDP mô tả hệ nhìn từ ngoài; BDI mô tả một tác tử nhìn từ trong. Xuất phát từ lý thuyết lý trí thực tiễn của Bratman [117] và được Rao, Georgeff đưa thành kiến trúc [118], mô hình gồm niềm tin (điều tác tử tin là đúng), mong muốn (trạng thái muốn đạt) và ý định (kế hoạch đã cam kết). Điểm đáng giá là cam kết: tác tử không cân nhắc lại mọi thứ ở mỗi bước, trừ khi niềm tin thay đổi đủ để bỏ ý định. Vòng ReAct [119] gần với mô hình này: lịch sử Observation đóng vai niềm tin, system prompt đóng vai mong muốn, Action là ý định một bước.



Hình 6.2. Chu trình BDI: nhận thức cập nhật niềm tin, cân nhắc chọn ý định, ý định sinh hành động.

6.2 Kiến trúc phối hợp và giao thức

6.2.1 Bốn cấu trúc phối hợp

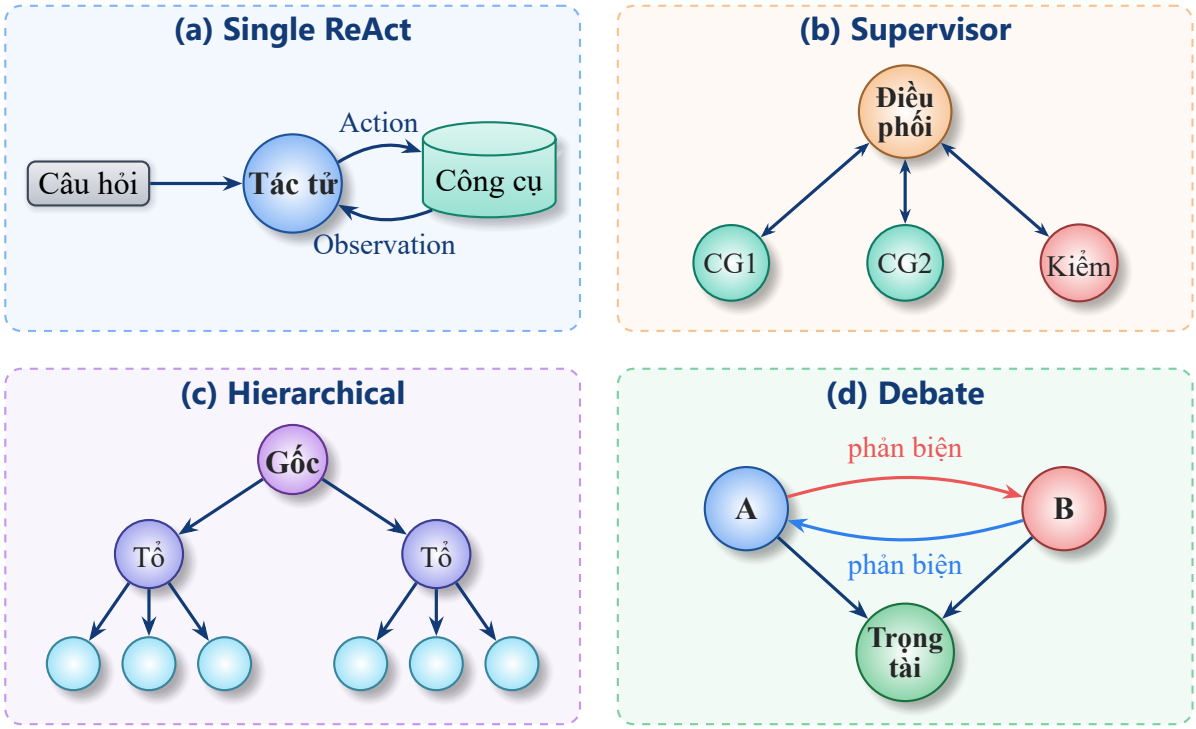
Hình 6.3 so sánh các cấu trúc chương này bàn tới.

- **Single ReAct:** một tác tử, ít chi phí phối hợp nhất, nhưng không ai kiểm tra lại nó.
- **Supervisor:** một tác tử điều phối giao việc cho chuyên gia và tổng hợp. Luồng dễ kiểm soát; cái giá là điều phối sai thì cả đội sai theo.
- **Hierarchical:** Supervisor nhiều tầng, gốc giao cho trưởng nhóm, trưởng nhóm giao cho người làm. Hợp với bài toán lớn; mỗi tầng thêm một lần tóm tắt và một lần mất thông tin. Chương không đo kiến trúc này vì bài toán ở đây không đủ lớn để cần tới hai tầng.
- **Debate:** các tác tử ngang hàng giải độc lập rồi đọc chéo [120, 121]. Sai sót của một bên có cơ hội bị bên kia bắt, với điều kiện hai bên không sai giống nhau.

Điều kiện cuối là mấu chốt. Hai tác tử chạy cùng một LLM chỉ khác prompt thì sai số tương quan cao, nên lợi ích của biểu quyết nhỏ hơn nhiều so với hai nguồn độc lập.

6.2.2 Giao thức trao đổi

FIPA-ACL [122] chuẩn hóa thông điệp theo lý thuyết hành vi ngôn ngữ: mỗi thông điệp mang một *performative* như `request`, `inform`, `propose`, kèm người gửi, người nhận, nội dung và bản thể học dùng để hiểu nội dung. Tác tử LLM hiện đại đổi phần nội dung hình thức đó lấy văn bản tự do cộng vài dòng có cấu trúc. Hệ ở đây theo hướng tối giản: thông điệp là một JSON phẳng `{type, from, to,`



Hình 6.3. Bốn cấu trúc phối hợp; chương này cài (a), (b), (d).

`content, tokens}`, và các hành vi được quy ước bằng từ khóa trong nội dung (`ASSIGN`, `PASS`, `FAIL`, `Action`, `Final Answer`).

Mỗi lượt gọi LLM phải gửi lại toàn bộ ngữ cảnh mà tác tử đó cần, vì LLM không giữ trạng thái. Tổng chi phí một câu hỏi là

$$T = \sum_{c \in \text{lượt gọi}} \left(|\text{prompt}_c| + |\text{output}_c| \right), \tag{6.4}$$

nên thêm tác tử làm tăng token gần như tuyến tính theo số lượt gọi, bất kể lượt đó có đóng góp gì.

6.2.3 Ba framework phổ biến

Bảng 6.1 chỉ ghi điều đọc được trực tiếp trong README chính thức của từng dự án, truy cập ngày 05/11/2026. Hệ của chương không dùng framework nào: tự cài để đếm được từng token và giữ mã trong phạm vi trích được vào báo cáo.

Bảng 6.1. Đặc điểm đã kiểm chứng từ README của ba framework.

Tiêu chí	LangGraph [123]	AutoGen [124]	CrewAI [125]
Tư tưởng	Điều phối cấp thấp cho tác tử có trạng thái, lấy cảm hứng từ Pregel và Apache Beam	Tác tử phân tầng: Core API truyền thông điệp, AgentChat cho hội thoại nhóm	Crews theo vai trò cộng Flows hướng sự kiện
Phối hợp	Tự định nghĩa bằng đồ thị	Hội thoại hai tác tử hoặc nhóm	Quy trình tuần tự hoặc phân cấp có quản lý
Bền vững và con người	Durable execution, human-in-the-loop, bộ nhớ ngắn và dài hạn	Hoạt động tự chủ hoặc cùng con người	Flows kiểm soát luồng chính xác
Tình trạng	Đang phát triển, dùng được không cần LangChain	Đã lưu trữ, không còn bảo trì; khuyên chuyển sang Microsoft Agent Framework	Đang phát triển

6.3 Dữ liệu

Bảng 6.2 tóm tắt ba dataset, bảng 6.3 cho một mẫu thực của mỗi tập. GSM8K [126] là toán lời văn bậc phổ thông, đáp án là một số. PubMedQA [127] là câu hỏi nghiên cứu y sinh kèm tóm tắt bài báo, nhãn yes, no hoặc maybe do chuyên gia gán. Ca lâm sàng là dữ liệu **tổng hợp** do chính báo cáo sinh ra.

Bảng 6.2. Ba dataset của Chương 6. Hai tập công khai có giấy phép MIT.

Dataset	Tổng mẫu	Dùng đo	Nhiệm vụ	Nguồn
GSM8K (test)	1319	50	toán lời văn, đáp án số	Hugging Face openai/gsm8k
PubMedQA pqa_labeled	1000	50	yes/no/maybe từ tóm tắt	Hugging Face qiaojin/PubMedQA
Ca lâm sàng tổng hợp	200	50	đơn thuốc đúng/sai theo luật	Tự sinh bằng tlf/ch6_rules.py

Dữ liệu tổng hợp, không phải tư vấn y tế

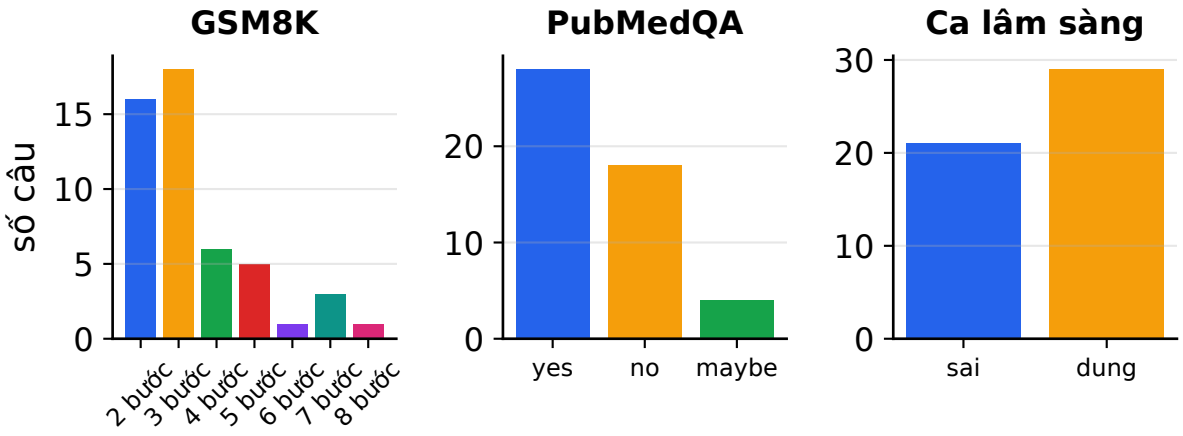
Ca lâm sàng được sinh từ 7 luật cứng về thuốc (Metformin theo eGFR và rượu, Lisinopril, Spironolactone, Ibuprofen, Propranolol, Hydrochlorothiazide, Amlodipine). Ngưỡng được rút gọn cho mục đích đo tác tử, nhãn đúng hay sai là đầu ra của hàm luật, không có bác sĩ nào duyệt. Không dùng cho bất kỳ quyết định y khoa nào.

Tiền xử lý. Mỗi tập được xáo bằng hạt giống cố định rồi lấy các câu đầu, nên

Bảng 6.3. Mẫu đầu ra thực của từng dataset (đã cắt ngắn).

Dataset	Mẫu đầu ra thực	Nhãn
GSM8K	Jared is trying to increase his typing speed. He starts with 47 words per minute (WPM). After some lessons the next time he tests his typing speed it has increased to 52 WPM. If he continues...	52
PubMedQA	Does open access endoscopy close the door to an adequately informed patient?...	yes
Ca lâm sàng	Bệnh nhân nữ 60 tuổi, tăng huyết áp. eGFR 62 mL/phút/1,73m2, kali 5,4 mmol/L, huyết áp 142/70 mmHg, nhịp tim 46 lần/phút, uống 0 ly rượu/ngày. Đơn đề xuất: Propranolol 40 mg, 2 lần/ngày....	sai

mọi kiến trúc gặp đúng cùng câu hỏi và so sánh theo cặp được. Đáp án GSM8K lấy số sau dấu ####; đáp án của mô hình lấy số cuối cùng sau `Final Answer`. Với PubMedQA, các đoạn tóm tắt được nối lại và đưa nguyên vào prompt. Ca lâm sàng chứa đủ mọi chỉ số dù thuốc chỉ liên quan vài chỉ số, các giá trị lấy mẫu dày quanh ngưỡng, hai lớp cân bằng. Phân bố ở hình 6.4.



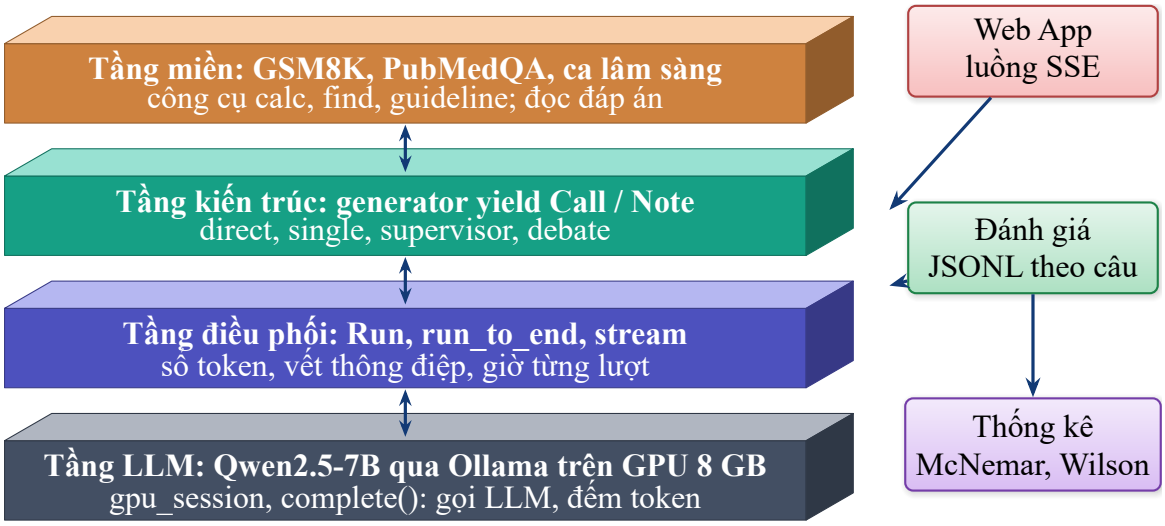
Hình 6.4. Phân bố trên tập con dùng đo: số bước lời giải GSM8K, nhãn PubMedQA và nhãn ca tổng hợp.

Nhận xét: PubMedQA lệch lớp nên độ đúng phải đọc cùng F1 macro và lớp đa số. GSM8K không có lớp đa số, baseline của nó là một lượt gọi LLM.

6.4 Cài đặt

6.4.1 Tác tử là generator

Hình 6.5 là các tầng của hệ. Điểm thiết kế chính: mỗi kiến trúc là một generator Python, `yield Call` để xin LLM trả lời, `yield Note` để ghi thông điệp không tốn token. Tác tử không tự gọi LLM nên không tự khai được token của mình. Lớp `Run` giữ sổ, vì vậy cùng một mã kiến trúc chạy được theo hai cách: `run_to_end` khi đánh giá, `stream` phát từng thông điệp cho Web App qua SSE. Mô hình là Qwen2.5-7B chạy qua Ollama trên GPU; token ra lấy từ Ollama, token vào đếm lại bằng tokenizer của mô hình.



Hình 6.5. Các tầng của hệ đa tác tử tự cài.

6.4.2 Ba kiến trúc

Single ReAct (đoạn mã 6.1) lặp Thought, Action, Observation. Mỗi miền có một công cụ: `calc` tính biểu thức qua cây cú pháp nên không chạy được mã tùy ý, `find` tìm câu trong tóm tắt, `guideline` trả luật của thuốc. Luật không nằm trong đề, nên trên ca lâm sàng tác tử buộc phải gọi công cụ mới có căn cứ. Hết lượt mà chưa kết luận thì tác tử bị buộc viết một dòng `Final Answer`; vẫn chưa có nhãn thì bị hỏi lại đúng nhãn, các lượt này cũng được tính token. Luật đọc nhãn nằm một chỗ (`conclusion_word`), dùng chung lúc chạy và lúc chấm. Trên ca tiếng Việt, mọi lệnh cho tác tử cũng bằng tiếng Việt.

Đoạn mã 6.1. Vòng ReAct, dùng lại làm thân của mọi chuyên gia và tác tử tranh luận.

```
# tlf/ch6_arch.py
```

```python
def react(item, dom, agent="ReAct", to=USER, persona="", brief="", max_steps=4):
    """Vòng Thought-Action-Observation; hết lượt thì buộc kết luận và hỏi lại nhãn nếu còn thiếu."""
    system = f"{dom.task} {persona}".strip() + "\n" + say(dom, "format")
    log, asked = dom.prompt(item) + (f"\n\n{brief}" if brief else ""), set()
    for _ in range(max_steps):
        text = yield Call(agent, to, system, log, dom.max_new)
        done, act = FINAL.search(text), ACTION.search(text)
        fresh = act and act.group(1) in dom.tools and (act.group(1), act.group(2).lower()) not in asked
        if fresh:     # tác tử viết cả Action lẫn Final Answer: kết luận chưa dựa trên kết quả công cụ nên bỏ
            name, arg = act.group(1), act.group(2)
            asked.add((name, arg.lower()))
            obs = dom.tools[name](arg, item)
            yield Note(f"Công cụ {name}", agent, f"{arg} -> {obs}")
            log += f"\n{text[:act.end()].strip()}\nObservation: {obs}"
        elif done and dom.parse(done.group(1)) is not None:
            return done.group(1).strip()
        else:
            log += f"\n{say(dom, 'again') if act else text}\n{say(dom, 'stop')}"
    text = yield Call(agent, to, system, f"{log}\n{say(dom, 'stop')}", CLOSE_TOKENS)
    text = text.split("Final Answer:")[-1].strip()
    return (yield from settle(item, dom, agent, to, text))
```

Supervisor (đoạn mã 6.2): điều phối đọc đề và ghi dòng `ASSIGN`, tối đa hai chuyên gia chạy ReAct với cùng công cụ, kiểm chứng độc lập trả `PASS` hoặc `FAIL` kèm lỗi cụ thể, bị bác thì làm lại một lần.

Đoạn mã 6.2. Kiến trúc Supervisor.

```python
# tlf/ch6_arch.py
def supervisor(item, dom, max_revise=1):
    """Điều phối giao việc, chuyên gia làm, kiểm chứng duyệt; bị bác thì làm lại tối đa max_revise lần."""
    team = "; ".join(f"{k}: {v}" for k, v in dom.roster.items())
    system = say(dom, "sup", team=team)
    plan = yield Call("Supervisor", "Đội chuyên gia", system,
                      f"{dom.prompt(item)}\n\n{say(dom, 'assign')}", 80)
    names, brief, rounds = _assign(plan, dom.roster), say(dom, "brief", plan=plan), 1
    while True:
        reports = {}
```

```
        for name in names:
            role = say(dom, "role", role=dom.roster[name])
            reports[name] = yield from react(item, dom, name,
             ↪  "Supervisor", role, brief, 3)
        draft = next(iter(reports.values()))
        if len(reports) > 1:
            lines = "\n".join(f"{k}: {v}" for k, v in reports.items())
            ask = f"{dom.prompt(item)}\n\n{say(dom,
             ↪  'reports')}\n{lines}\n{say(dom, 'give')}"
            draft = yield Call("Supervisor", "Verifier", system, ask,
             ↪  dom.max_new // 2)
            draft = yield from settle(item, dom, "Supervisor",
             ↪  "Verifier", draft)
        ask = f"{dom.prompt(item)}\n\n{say(dom,
         ↪  'draft')}\n{draft}\n\n{say(dom, 'check')}"
        check = yield Call("Verifier", "Supervisor", say(dom,
         ↪  "verifier"), ask, 90)
        if check.strip().upper().startswith("PASS") or rounds >
         ↪  max_revise:
            break
        rounds, brief = rounds + 1, say(dom, "redo", draft=draft,
         ↪  check=check)
    yield Note("Supervisor", USER, f"Kết luận: {draft}")
    return {"answer": dom.parse(draft), "rounds": rounds, "text":
     ↪  draft, "verifier": check[:200]}
```

Debate (đoạn mã 6.3): tác tử A giải trực tiếp, tác tử B được dặn hoài nghi và giải theo đường khác để giảm tương quan sai số. Chỉ hai nhãn đọc được và trùng nhau mới là đồng thuận, khi đó dừng ngay, không tốn lượt trọng tài; còn lại thì đọc chéo một vòng, vẫn chưa đồng thuận thì trọng tài quyết.

Đoạn mã 6.3. Kiến trúc Debate.

```
# tlf/ch6_arch.py
def debate(item, dom, max_rounds=2):
    """Hai tác tử giải độc lập, đọc chéo nhau. Chỉ hai nhãn đọc được và
     ↪  trùng nhau mới là đồng thuận;
    còn lại, kể cả khi một bên không ra nhãn, trọng tài quyết."""
    pro, con = say(dom, "pro"), say(dom, "con")
    a = yield from react(item, dom, "Tác tử A", "Tác tử B", pro,
     ↪  max_steps=3)
    b = yield from react(item, dom, "Tác tử B", "Tác tử A", con,
     ↪  max_steps=3)
    rounds = 1

    def agreed():
        return dom.parse(a) is not None and dom.parse(a) ==
         ↪  dom.parse(b)

```

```
    while not agreed() and rounds < max_rounds:
        rounds += 1
        a = yield from _rebut(item, dom, "Tác tử A", "Tác tử B", pro,
        ↪  a, b)
        b = yield from _rebut(item, dom, "Tác tử B", "Tác tử A", con,
        ↪  b, a)
    if agreed():
        yield Note("Trọng tài", USER, f"Hai bên đồng thuận:
        ↪  {dom.parse(a)}")
        return {"answer": dom.parse(a), "rounds": rounds, "text": a,
        ↪  "consensus": True}
    ask = f"{dom.prompt(item)}\n\n{say(dom, 'decide', a=a, b=b)}"
    verdict = yield Call("Trọng tài", USER, say(dom, "judge"), ask,
    ↪  dom.max_new)
    verdict = yield from settle(item, dom, "Trọng tài", USER, verdict)
    return {"answer": dom.parse(verdict), "rounds": rounds, "text":
    ↪  verdict, "consensus": False}
```

6.5 Đánh giá

6.5.1 Thiết lập đo

Mọi kiến trúc dùng cùng mô hình `qwen2.5-7b`, nhiệt độ 0, trên cùng 50 câu GSM8K, 50 câu PubMedQA và 50 ca tổng hợp. Thêm một baseline một lượt gọi, không công cụ, không suy luận. Cỡ mẫu nhỏ nên độ đúng kèm khoảng tin cậy Wilson 95%; so sánh hai kiến trúc dùng kiểm định McNemar chính xác trên các câu hai bên bất đồng, hiệu độ đúng có khoảng bootstrap, và p được hiệu chỉnh Holm vì có chín phép so sánh. Toàn bộ thông điệp nằm ở `data/ch6/runs/*.jsonl`, kết quả chấm lại được từ các tệp này.

6.5.2 Phân tích

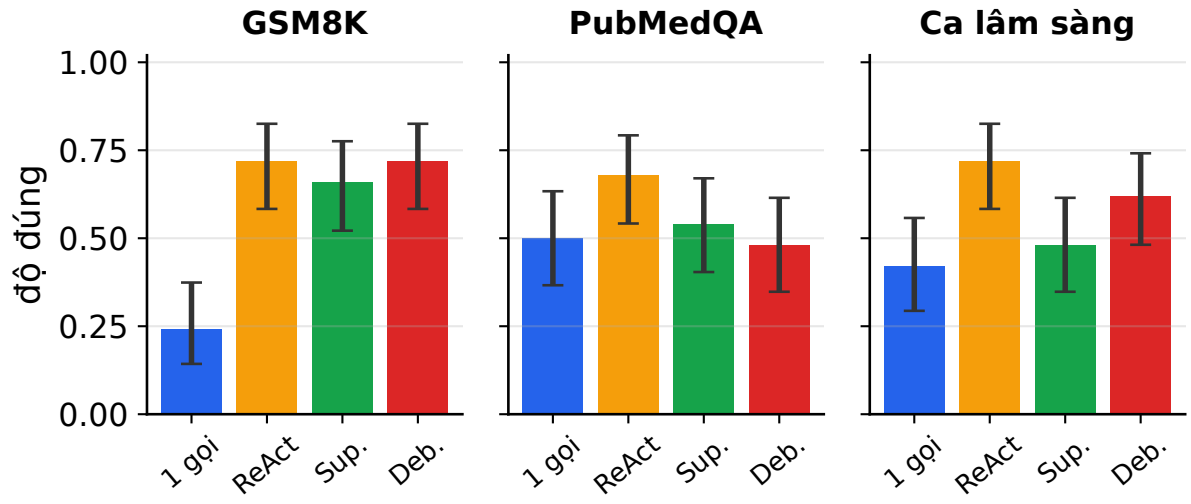
Đa tác tử không thắng. Single ReAct cao nhất hoặc ngang hàng cao nhất trên cả ba dataset: GSM8K 72,00% (Supervisor 66,00%, Debate 72,00%), PubMedQA 68,00% (54,00%, 48,00%), ca lâm sàng 72,00% (48,00%, 62,00%). Tuy vậy, chưa so sánh nào đạt mức 5% sau hiệu chỉnh Holm. Giá trị p nhỏ nhất là 0,0063 (Holm 0,0571), ứng với Debate so với ReAct trên PubMedQA: Debate thắng 1 câu, thua 11 câu. Với 50 câu mỗi tập, kết luận vững là không thấy lợi ích của đa tác tử trên mô hình này; chưa đủ để nói đa tác tử chắc chắn kém hơn.

Chi phí thì rõ ràng. So với ReAct, Supervisor tốn gấp 3,2 lần token trên GSM8K, 4,0 lần trên PubMedQA và 4,6 lần trên ca lâm sàng; Debate tốn gấp 3,1, 3,7 và 3,1 lần. Bước nhảy đáng giá nhất không nằm ở số tác tử mà ở công cụ: so

Bảng 6.4. So sánh các kiến trúc trên ba dataset (qwen2.5-7b).

Dataset	Kiến trúc	Đúng %	KTC 95%	F1	Token	Gọi LLM	Vòng	Trễ (s)
GSM8K	Một lượt gọi	24,0	14,3–37,4	–	79	1,0	1,00	0,4
GSM8K	Single ReAct	72,0	58,3–82,5	–	689	2,7	1,00	7,0
GSM8K	Supervisor	66,0	52,2–77,6	–	2.196	8,6	1,76	22,7
GSM8K	Debate	72,0	58,3–82,5	–	2.154	7,1	1,36	20,4
PubMedQA	Một lượt gọi	50,0	36,6–63,4	0,449	344	1,0	1,00	0,7
PubMedQA	Single ReAct	68,0	54,2–79,2	0,593	1.059	2,2	1,00	3,0
PubMedQA	Supervisor	54,0	40,4–67,0	0,496	4.281	8,9	1,68	21,2
PubMedQA	Debate	48,0	34,8–61,5	0,464	3.950	7,4	1,66	23,4
Ca lâm sàng	Một lượt gọi	42,0	29,4–55,8	0,296	150	1,0	1,00	0,4
Ca lâm sàng	Single ReAct	72,0	58,3–82,5	0,718	784	2,0	1,00	6,8
Ca lâm sàng	Supervisor	48,0	34,8–61,5	0,403	3.593	8,4	1,96	30,7
Ca lâm sàng	Debate	62,0	48,2–74,1	0,601	2.456	5,6	1,26	25,5

Lớp đa số: GSM8K 0,0%, PubMedQA 56,0%, Ca lâm sàng 58,0%. Độ trễ là trung vị mỗi câu, các lượt gọi chạy tuần tự.



Hình 6.6. Độ đúng và khoảng tin cậy Wilson 95%.

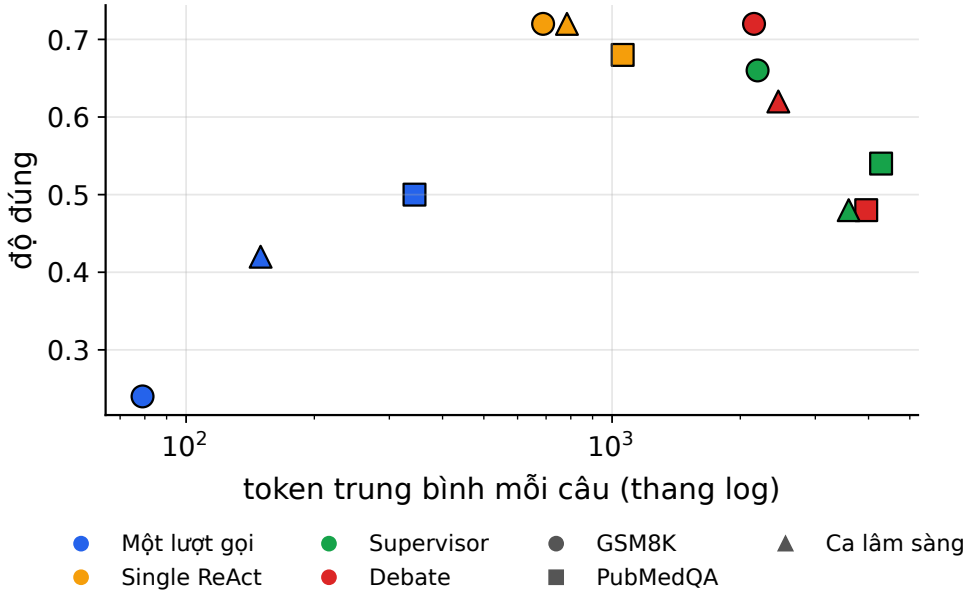
với một lượt gọi, vòng ReAct nâng độ đúng từ 24,00% lên 72,00% trên GSM8K và từ 42,00% lên 72,00% trên ca lâm sàng.

Kiểm chứng bác gần như mọi thứ. Verifier trả FAIL ở 86,36% lượt trên GSM8K, 70,24% trên PubMedQA và 96,94% trên ca lâm sàng. Nó không có công cụ, chỉ suy luận lại bằng chính LLM đó. Tác dụng của lượt làm lại tùy miền: GSM8K sửa đúng 5 câu, làm hỏng 1; PubMedQA sửa 12, hỏng 3; ca lâm sàng chỉ sửa 1 mà hỏng 10. Như vậy lượt làm lại có ích ở hai miền đầu nhưng Supervisor vẫn dưới ReAct, còn trên ca lâm sàng nó là nguyên nhân chính của khoảng cách.

Tranh luận ít khi xảy ra đúng chỗ. Hai tác tử bất đồng ở 20,00% câu GSM8K,

Bảng 6.5. Kiểm định cặp McNemar; thắng là số câu kiến trúc bên phải đúng còn bên trái sai.

Dataset	So sánh		Thắng	Thua	Hiệu (%)	p	p Holm
GSM8K	Single ReAct	so với Supervisor	4	7	−6,0 [−18,0; 6,0]	0,549	1,000
GSM8K	Single ReAct	so với Debate	3	3	+0,0 [−10,0; 10,0]	1,000	1,000
GSM8K	Supervisor	so với Debate	5	2	+6,0 [−4,0; 16,0]	0,453	1,000
PubMedQA	Single ReAct	so với Supervisor	3	10	−14,0 [−28,0; 0,0]	0,092	0,554
PubMedQA	Single ReAct	so với Debate	1	11	−20,0 [−32,0; −8,0]	0,006	0,057
PubMedQA	Supervisor	so với Debate	5	8	−6,0 [−20,0; 8,0]	0,581	1,000
Ca lâm sàng	Single ReAct	so với Supervisor	5	17	−24,0 [−42,0; −8,0]	0,017	0,135
Ca lâm sàng	Single ReAct	so với Debate	6	11	−10,0 [−26,0; 6,0]	0,332	1,000
Ca lâm sàng	Supervisor	so với Debate	9	2	+14,0 [2,0; 26,0]	0,065	0,458



Hình 6.7. Đánh đổi giữa token và độ đúng.

42,00% câu PubMedQA và 12,00% ca lâm sàng. Trên PubMedQA, khi hai bên đồng thuận thì độ đúng chỉ là 41,38%: cùng một LLM nên sai giống nhau, hoặc một bên nhận sai dù đang đúng. Trọng tài chọn đúng 12 trong 21 ca PubMedQA, nhưng 0 trong 6 ca lâm sàng.

Trên ca lâm sàng, ReAct và Debate vượt lớp đa số 58,00%, Supervisor thì không. Lỗi còn lại chủ yếu là so số: vết ở mục sau có tác tử cho rằng eGFR 62 nằm trong khoảng 30–45. Đây là giới hạn của mô hình, không sửa được bằng cách thêm tác tử cùng mô hình.

Giới hạn của phép đo

- Mỗi dataset 50 câu, khoảng tin cậy còn rộng; một LLM, nhiệt độ 0, nên các tác tử không độc lập thật sự.
- Lượt đo đầu có lỗi cài đặt và đã bị loại khỏi báo cáo: lượt ép kết luận quá ngắn khiến 63,33% câu của Supervisor trên ca lâm sàng không có nhãn; lệnh tiếng Anh trên đề tiếng Việt làm 37,50% lượt trôi sang tiếng Trung; hai bên cùng không có nhãn bị xem là đồng thuận. Sau khi sửa và đo lại, mọi câu đều có nhãn đọc được, tỉ lệ trôi ngôn ngữ còn 0,28%. Vết cũ giữ ở `data/ch6/runs_n30`.
- Verifier không có công cụ; cho nó tra được luật có thể đổi kết quả của Supervisor, chương chưa đo phương án này.

Tiểu luận 03 từng đo GSM8K với mô hình khác, Qwen2.5-1.5B: hệ đa tác tử đạt 57,14% trên 35 câu, một tác tử suy luận từng bước đạt 73,33% trên 60 câu. Hai tập câu khác nhau nên không gộp số, nhưng chiều kết quả giống ở đây.

6.6 Kịch bản minh họa: hội chẩn đa chuyên khoa

Vết dưới đây lấy nguyên từ lượt đánh giá, không dựng riêng. Ca tổng hợp là: *Bệnh nhân nữ 47 tuổi, đái tháo đường típ 2, hen phế quản. eGFR 62 mL/phút/1,73m2, kali 5,4 mmol/L, huyết áp 98/90 mmHg, nhịp tim 62 lần/phút, uống 3 ly rượu/ngày. Đơn đề xuất: Metformin 1000 mg, 2 lần/ngày.* Nhãn theo luật: **sai** (`dung` là đơn hợp lệ). Nội dung dài bị cắt ở `[...]`, chữ Hán mô hình tự sinh được thay bằng nhãn `[chữ Hán]`, bản đầy đủ ở `outputs/metrics/ch6_scenario.json`. Ca này không phải tư vấn y tế.

Supervisor: điều phối, chuyên khoa, kiểm chứng

Supervisor	ASSIGN: Đơn thuốc Đơn thuốc SAI do vi phạm luật về kali máu cao. (265 token)
Thận	Thought: cần kiểm tra xem liệu liều metformin có phù hợp với eGFR và kali máu không. Action: guideline[metformin] Trước khi t [...] (359 token)
Công cụ	metformin -> Metformin: chống chỉ định khi eGFR < 30. Khi 30 <= eGFR < 45, tổng liều tối đa 1000 mg/ngày. Không dùng khi uống [...] (0 token)
Thận	Final Answer: SAI Lý do: eGFR của bệnh nhân là 62 mL/phút/1,73m2, nằm trong khoảng 30-45, do đó tổng liều metformin tối đa là [...] (498 token)
Verifier	FAIL Lý do: Mặc dù eGFR của bệnh nhân nằm trong khoảng 30-45 mL/phút/1,73m2, do đó liều metformin tối đa là 1000 mg/ngày, như [...] (372 token)

Thận	Thought: cần tra luật của metformin để kiểm tra xem việc uống 3 ly rượu/ngày có ảnh hưởng đến việc sử dụng metformin không. A [...] (608 token)
Thận	Final Answer: SAI Lý do: mặc dù eGFR của bệnh nhân nằm trong khoảng 30-45 mL/phút/1,73m2, do đó liều metformin tối đa là 1000 [...] (690 token)
Verifier	FAIL Lý do: Đơn thuốc đề xuất metformin 1000 mg, 2 lần/ngày cho bệnh nhân, nhưng theo luật metformin, liều tối đa cho bệnh nh [...] (397 token)
Supervisor	Kết luận: SAI Lý do: mặc dù eGFR của bệnh nhân nằm trong khoảng 30-45 mL/phút/1,73m2, do đó liều metformin tối đa là 1000 mg/ [...] (0 token)

Debate: hai tác tử và trọng tài

Tác tử A	Thought: Metformin thường được sử dụng trong điều trị đái tháo đường típ 2. Tuy nhiên, cần xem xét eGFR và nồng độ kali trong [...] (425 token)
Công cụ	metformin -> Metformin: chống chỉ định khi eGFR < 30. Khi 30 <= eGFR < 45, tổng liều tối đa 1000 mg/ngày. Không dùng khi uống [...] (0 token)
Tác tử A	Final Answer: SAI Lý do: Nồng độ kali 5,4 mmol/L cao có thể gây độc cho metformin, và bệnh nhân uống rượu 3 ly/ngày cũng vi p [...] (504 token)
Tác tử B	Thought: Metformin thường được sử dụng trong điều trị đái tháo đường típ 2. Tuy nhiên, cần xem xét các yếu tố khác như eGFR v [...] (453 token)
Tác tử B	Final Answer: SAI Lý do: eGFR của bệnh nhân là 62 mL/phút/1,73m2 (trong phạm vi 30-45), nên tổng liều tối đa metformin là 100 [...] (522 token)
Trọng tài	Hai bên đồng thuận: sai (0 token)

Cả ba kiến trúc đều ra nhãn đúng ở ca này: 831 token với ReAct, 3.189 với Supervisor, 1.904 với Debate. Nhưng nhãn đúng không có nghĩa là lý do đúng: theo luật, đơn sai vì bệnh nhân uống từ 3 ly rượu/ngày, trong khi chuyên khoa Thận và tác tử B lại xếp eGFR 62 vào khoảng 30–45, tác tử A viện dẫn kali, còn Verifier trả FAIL cả hai lần với một nhãn đúng. Hình thức hội chẩn không tự đảm bảo chất lượng suy luận. Web App phát trực tiếp các thông điệp kiểu này qua `tlf/serve_ch6.py`, hàm `run(arch, question, model)`.

6.7 Tóm tắt chương

Chương đã trình bày Dec-POMDP, Markov game, BDI, các cấu trúc phối hợp và tự cài ba kiến trúc trên một LLM cục bộ. Kết quả đo thật đi ngược kỳ vọng phổ biến: với Qwen2.5-7B, thêm tác tử làm token tăng nhiều lần mà độ đúng không tăng, và Single ReAct có công cụ vẫn cao nhất hoặc ngang hàng cao nhất. Đa tác tử chỉ có cơ hội khi các tác tử thực sự khác nhau về thông tin hoặc công cụ, và khi bên kiểm chứng có căn cứ độc lập thay vì chỉ suy luận lại.

Chương 7

Deep Graph và mạng nơ-ron đồ thị

Các chương trước làm việc trên bảng, lưới ảnh và chuỗi, nơi mỗi mẫu có vị trí cố định. Bài báo trích dẫn nhau, sản phẩm được mua cùng nhau hay tiền chạy qua các giao dịch thì không có thứ tự như vậy: số láng giềng thay đổi tùy nút, và đánh số lại nút không làm đồ thị khác đi. Chương này xây GCN, GraphSAGE, GAT trên ba đồ thị thật, đo chúng với hai đường cơ sở, rồi dự đoán liên kết trên đồ thị Hetionet đã nạp vào Neo4j ở Chương 5.

7.1 Cơ sở toán học

Đồ thị vô hướng $G = (V, E)$ có N nút được mô tả bởi ma trận kề A ($A_{ij} = 1$ khi có cạnh), ma trận bậc đường chéo $D_{ii} = \sum_j A_{ij}$ và ma trận đặc trưng $X \in \mathbb{R}^{N \times F}$ (Hình 7.1). Laplacian $L = D - A$ và dạng chuẩn hóa đối xứng $L_{\text{sym}} = I - D^{-1/2} A D^{-1/2}$ đo độ gồ ghề của tín hiệu x trên đồ thị:

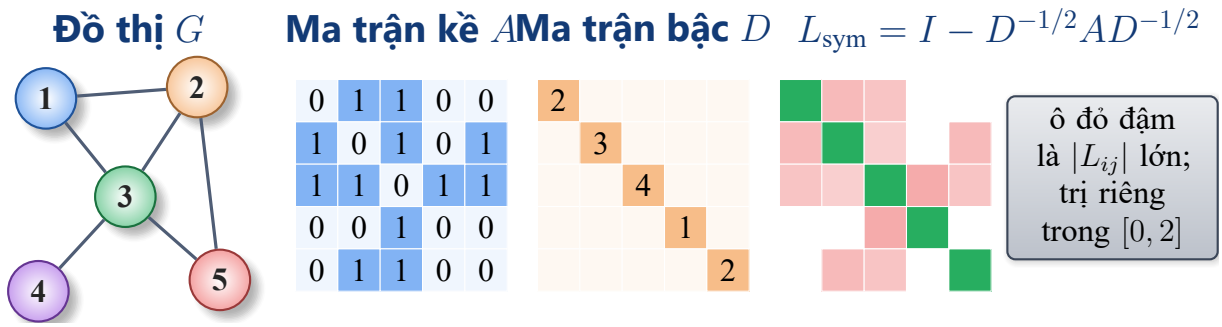
$$x^{\top} L_{\text{sym}}\, x = \sum_{(i,j)\in E} \left(\frac{x_i}{\sqrt{D_{ii}}} - \frac{x_j}{\sqrt{D_{jj}}} \right)^2 . \tag{7.1}$$

L_{sym} đối xứng nửa xác định dương, trị riêng nằm trong $[0, 2]$, số trị riêng 0 bằng số thành phần liên thông. Lọc tín hiệu theo phổ cần phân rã trị riêng $O(N^3)$; Kipf và Welling xấp xỉ bậc một rồi thêm tự khuyên $\tilde{A} = A + I$, $\tilde{D} = D + I$, được phép truyền một lớp chỉ tốn $O(|E|)$ [128]:

$$H^{(\ell+1)} = \sigma\!\left(\hat{A}\, H^{(\ell)} W^{(\ell)} \right), \qquad \hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}. \tag{7.2}$$

Chuẩn hóa hai phía làm trọng số cạnh bằng $1/\sqrt{\tilde{d}_i \tilde{d}_j}$: nút trung tâm nhiều cạnh không áp đảo láng giềng, còn tự khuyên giữ lại thông tin của chính nút.

Truyền thông điệp (MPNN). Gilmer và cộng sự gộp phần lớn GNN vào một khuôn [129] (Hình 7.2): mỗi cạnh sinh thông điệp ψ, mỗi nút gộp thông điệp bằng phép $\bigoplus$ không phụ thuộc thứ tự (tổng, trung bình, cực đại), rồi cập nhật trạng thái bằng ϕ. Sau L lớp, một nút nghe được vùng cách nó L bước. Ba kiến trúc khác



Hình 7.1. Bốn cách nhìn một đồ thị 5 nút: cấu trúc, ma trận kề, ma trận bậc và Laplacian chuẩn hóa.

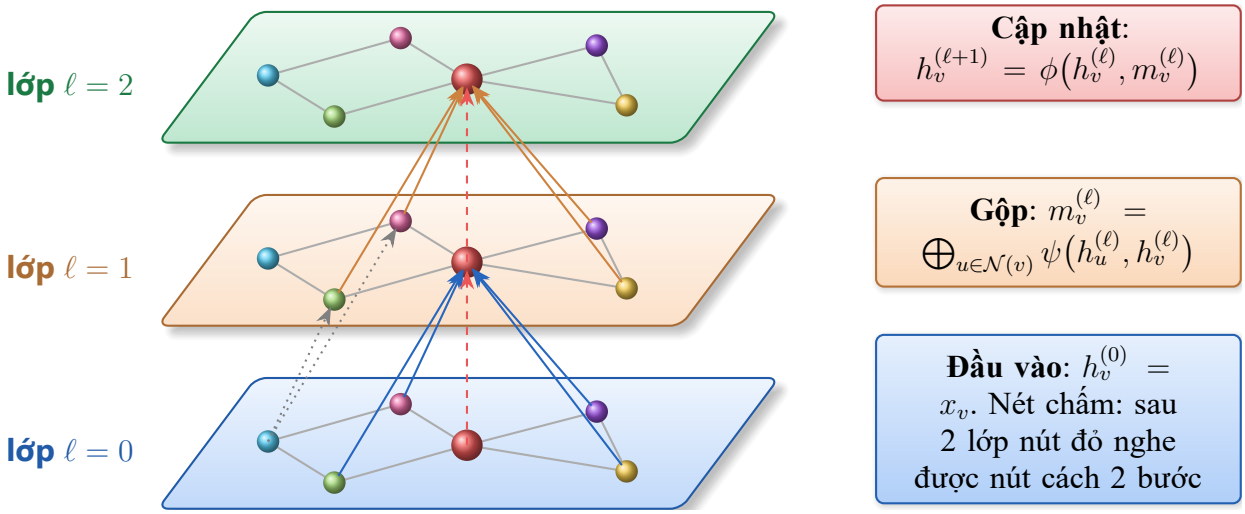
nhau ở cách gán trọng số cho láng giềng:

$$\text{GCN:} \quad h'_v = \sigma\Big(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} W h_u \Big), \tag{7.3}$$

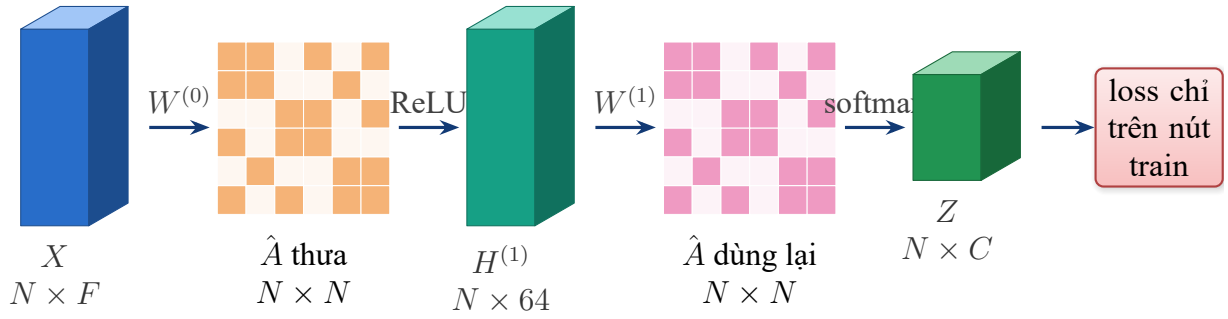
$$\text{GraphSAGE:} \quad h'_v = \sigma\big(W_1 h_v + W_2 \,\text{mean}\{h_u : u \in \mathcal{S}(v)\} \big), \tag{7.4}$$

$$\text{GAT:} \quad h'_v = \Big\|_{k=1}^{K} \sigma\Big(\sum_{u} \alpha_{vu}^{k} W^k h_u \Big). \tag{7.5}$$

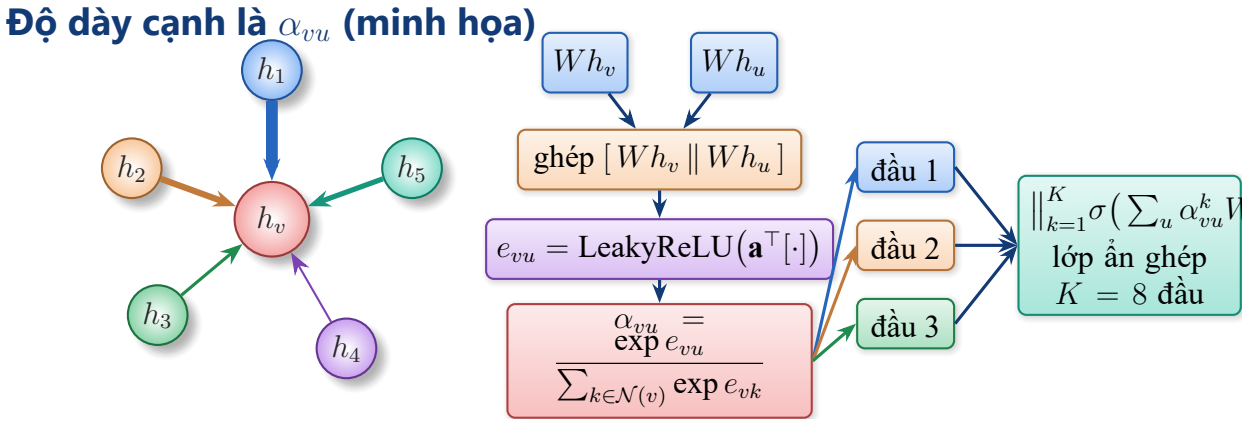
GCN cố định trọng số theo bậc. GraphSAGE [130] tách riêng trọng số của chính nút và lấy mẫu tập láng giềng $\mathcal{S}(v)$ lúc học, nên áp được cho nút chưa từng thấy. GAT [131] học hệ số chú ý α_{vu} từ chính đặc trưng hai đầu cạnh (Hình 7.4).



Hình 7.2. Truyền thông điệp qua ba lớp: cùng một đồ thị, nút đỏ gộp thông tin láng giềng ở lớp trước.



Hình 7.3. GCN hai lớp cho phân loại nút: $Z = \hat{A}\,\mathrm{ReLU}(\hat{A}XW^{(0)})W^{(1)}$.



Hình 7.4. Tính hệ số chú ý của GAT và ghép nhiều đầu chú ý.

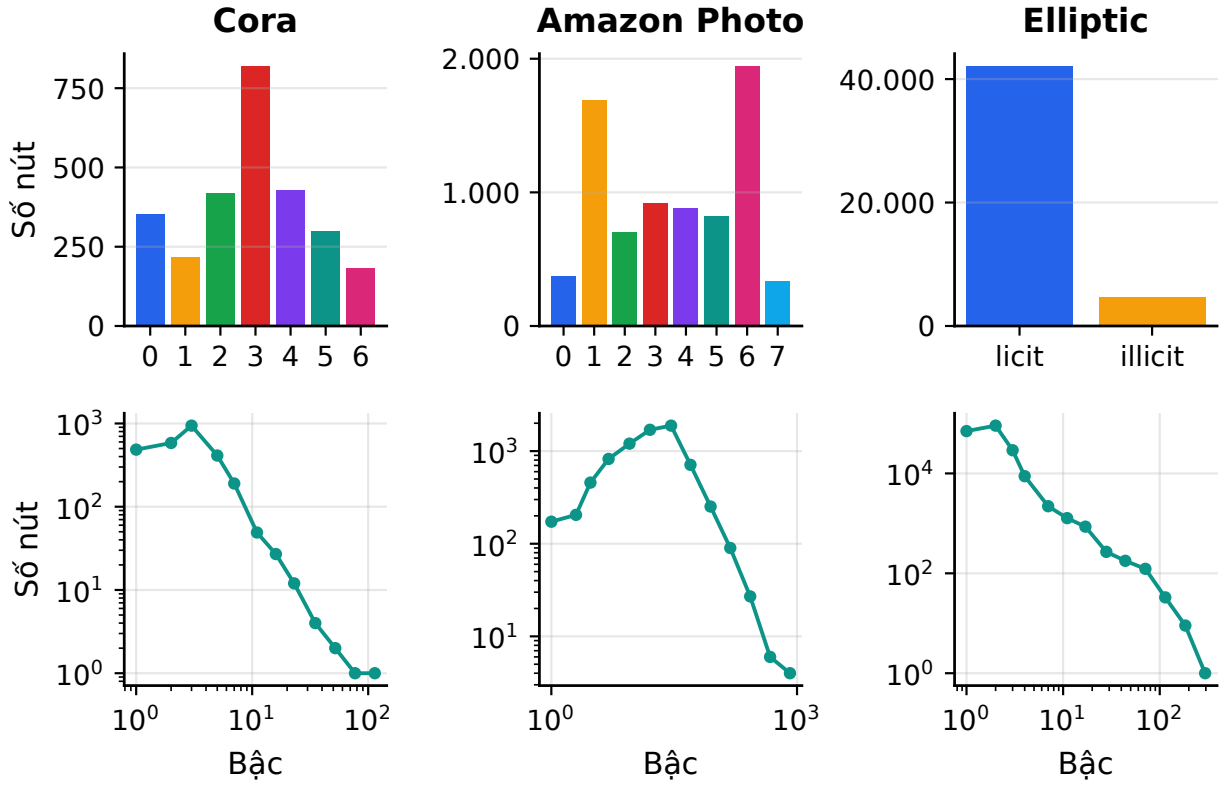
7.2 Ba đồ thị thực nghiệm

Nguồn. *Cora* lấy từ kho Planetoid [132] (`https://github.com/kimiyoung/planetoid`, bản dữ liệu dùng lại từ Tiểu luận 03): nút là bài báo, cạnh là trích dẫn, đặc trưng là túi từ nhị phân. *Amazon Photo* của Shchur và cộng sự [133] (`https://github.com/shchur/gnn-benchmark`): nút là sản phẩm, cạnh nối hai sản phẩm hay được mua cùng nhau; bộ gốc không công bố tên lớp nên ghi Lớp 0 đến Lớp 7. *Elliptic Bitcoin* [134], tải bản sao của PyG (`https://data.pyg.org/datasets/elliptic/`): nút là giao dịch, cạnh là dòng tiền, đặc trưng là 165 thuộc tính đã ẩn danh. Tôi chọn Elliptic thay ogbn-arxiv vì bộ này có chiều thời gian thật và lệch lớp nặng, tức hai cái bẫy đánh giá mà Cora không có. Cả ba dùng toàn bộ đồ thị. Bảng 7.1 là số đo thật trên dữ liệu đã tải.

Nhận xét: Đồng dạng cạnh (homophily) của Cora và Photo lần lượt là 0,8100 và 0,8272, nên trung bình láng giềng là phép lọc có lợi. Đuôi bậc dài: Cora có nút bậc 168, Photo có nút bậc 1.434 cùng 115 nút cô lập mà GNN chỉ thấy đặc trưng của chính chúng. Elliptic có đồng dạng thô 0,9537 nhưng chỉ 0,5162 sau hiệu chỉnh, vì 9,245 giao dịch licit mới có một illicit nên hai đầu cạnh trùng nhãn phần nhiều do tình cờ. Chỉ 46.564 trên 203.769 nút có nhãn. Đồ thị này tách thành đúng 49 thành phần liên thông ứng với 49 bước thời gian, số cạnh vắt qua hai bước là 0.

Bảng 7.1. Kích cỡ và tính chất đồ thị. Bậc, đồng dạng tính sau khi đối xứng hóa cạnh; đồng dạng trên Elliptic chỉ tính cạnh có nhãn ở hai đầu.

Thuộc tính	Cora	Amazon Photo	Elliptic
Số nút	2.708	7.650	203.769
Số cạnh vô hướng	5.278	119.081	234.355
Đặc trưng mỗi nút	1.433	745	165
Số lớp	7	8	2
Nút có nhãn	2.708	7.650	46.564
Bậc trung bình	3,90	31,13	2,30
Bậc lớn nhất	168	1.434	473
Nút cô lập	0	115	0
Thành phần liên thông	78	136	49
Đồng dạng cạnh	0,810	0,827	0,954
Đồng dạng hiệu chỉnh	0,771	0,785	0,516
Lệch lớp (lớn/nhỏ)	4,5	5,9	9,2
Dung lượng tệp gốc (MB)	0,6	16,3	665,1



Hình 7.5. Phân bố lớp (hàng trên) và phân bố bậc thang log-log (hàng dưới). Elliptic chỉ đếm nút có nhãn.

Bảng 7.2. Nút mẫu thật của Cora và Amazon Photo: nút đầu tiên của hai lớp đầu.

Đồ thị	Nút	Nhãn	Bậc	Số từ	Láng giềng	Nhãn láng giềng đầu
Cora	3	Case_Based	1	21	2544	Case_Based
Cora	18	Genetic_Algorithms	5	5	139, 1560	Genetic_Algorithms
Amazon Photo	16	Lớp 0	1	54	7505	Lớp 0
Amazon Photo	12	Lớp 1	8	69	880, 2282	Lớp 1

Bảng 7.3. Giao dịch mẫu thật của Elliptic. Đặc trưng đã z-score bằng thống kê các bước 1 đến 29.

txId	Nhãn	Bước	Bậc	Đặc trưng 1	Đặc trưng 2
232629023	illicit	1	1	−0,192	0,593
232438397	licit	1	161	0,132	6,974
230425980	chưa biết	1	2	−0,191	−0,183

Tiền xử lý. Cạnh được đối xứng hóa, bỏ tự khuyên. Cora, Photo chia mỗi hàng đặc trưng cho tổng của hàng, phép biến đổi theo từng nút nên không học gì từ test. Elliptic z-score bằng trung bình, độ lệch của riêng các bước 1 đến 29. Cora dùng phép chia chuẩn 140/500/1.000. Photo không có phép chia chuẩn nên lấy 20 nút mỗi lớp cho train, 30 cho val, còn lại test, mỗi seed một phép chia. Elliptic chia theo thời gian: 26.381 nút có nhãn ở bước 1–29, 3.513 ở bước 30–34, 16.670 ở bước 35–49, kèm trọng số lớp nghịch đảo tần suất trong hàm mất mát.

7.3 Hiện thực: GCN từ đầu và PyTorch thuần

Bản NumPy không dùng autograd hay ma trận thưa: $\hat{A}$ là mảng dày $N \times N$, đạo hàm suy tay theo quy tắc chuỗi. Với $G = \partial \mathcal{L}/\partial Z$ ta có $\partial \mathcal{L}/\partial W^{(1)} = H^{(1)\top} \hat{A}^{\top} G$, rồi đi ngược qua dropout và ReLU để được $\partial \mathcal{L}/\partial W^{(0)}$ (Đoạn mã 7.1). Hàm `grad_check` so đạo hàm này với sai phân trung tâm trên đồ thị ngẫu nhiên 12 nút, sai lệch lớn nhất $2{,}74 \times 10^{-9}$.

Đầu ra thực của GCN NumPy trên Cora: ba seed đầu và trung bình 5 seed

```
numpy 0 0.817 26.7 s
numpy 1 0.822 25.7 s
numpy 2 0.814 25.8 s
trung bình độ đúng 0,8166, F1 macro 0,8059
```

Đoạn mã 7.1. Lan truyền ngược viết tay của GCN NumPy.

```python
# tlf/ch7_numpy.py
def loss_grads(self, A, X, y, mask, train=True):
    """Lan truyền ngược bằng tay: G = ∂L/∂Z, rồi đi ngược từng phép nhân ma trận."""
    logits, (h0, z1, h1, k1) = self.forward(A, X, train)
    p = softmax(logits)
    n = mask.sum()
    loss = -np.log(p[mask, y[mask]] + 1e-12).mean() + 0.5 * self.wd * (self.W[0] ** 2).sum()
    one_hot = np.eye(p.shape[1], dtype=np.float32)[y]
    g = (p - one_hot) * mask[:, None] / n        # ∂L/∂Z, n × c
    at_g = A.T @ g
    grad1 = h1.T @ at_g                          # ∂L/∂W₁ = H¹ᵀ Âᵀ G
    g_h1 = (at_g @ self.W[1].T) * k1             # ngược qua dropout H¹
    g_z1 = g_h1 * (z1 > 0)                       # ngược qua ReLU
    grad0 = h0.T @ (A.T @ g_z1) + self.wd * self.W[0]   # ∂L/∂W₀
    return loss, [grad0, grad1]
```

Đoạn mã 7.2. Tính $\hat{A}$ thưa bằng PyTorch thuần, dùng cho mọi lớp GCN.

```python
# tlf/ch7_gnn.py
def gcn_norm_adj(edge_index, n):
    """Â = D̃^(-1/2)(A + I)D̃^(-1/2), thưa."""
    ei = self_loops(edge_index, n)
    one = torch.ones(ei.size(1), device=ei.device)
    deg = torch.zeros(n, device=ei.device).index_add_(0, ei[1], one)
    w = deg.pow(-0.5)
    w = w[ei[0]] * w[ei[1]]
    return torch.sparse_coo_tensor(ei.flip(0), w, (n, n)).coalesce()
```

Đoạn mã 7.3. Lớp GAT: điểm cạnh, softmax theo nút đích, gộp có trọng số.

```
# tlf/ch7_gnn.py
def forward(self, x, edge_index):
    n = x.size(0)
    ei = self_loops(edge_index, n)
    src, dst = ei
    h = self.lin(x).view(n, self.heads, self.d_out)
    e = (h * self.a_src).sum(-1)[src] + (h * self.a_dst).sum(-1)[dst]
    alpha = scatter_softmax(F.leaky_relu(e, 0.2), dst, n)
    self.last_alpha = (ei.detach(), alpha.detach())
    drop = F.dropout(alpha, self.dropout, self.training)
    out = torch.zeros_like(h).index_add_(0, dst, h[src] *
     ↪  drop.unsqueeze(-1))
    out = out.reshape(n, -1) if self.concat else out.mean(1)
    return out + self.bias
```

7.4 Đánh giá ba mô hình trên ba đồ thị

7.4.1 So sánh với đường cơ sở

MLP chỉ dùng đặc trưng; Label Propagation (LP) chỉ dùng cạnh, lặp $F \leftarrow \alpha SF + (1-\alpha)Y$ [135] với α chọn trên val. Mô hình hai lớp, ẩn 64 chiều (Elliptic 128), Adam, chọn trọng số theo F1 macro của val, dừng sớm; riêng test chỉ đo một lần ở cuối. GraphSAGE lấy mẫu tối đa 10 láng giềng (Photo 25) khi học. Các siêu tham số Photo, Elliptic được chỉnh trên val. Mỗi cấu hình chạy 5 seed trên GPU.

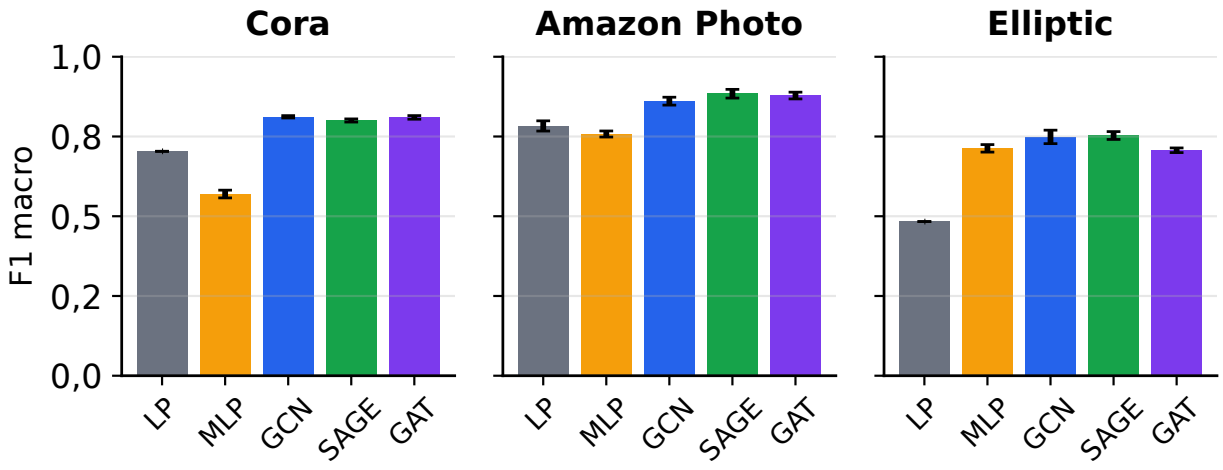
Bảng 7.4. Phân loại nút, trung bình trên các seed. Elliptic đo bằng chia thời gian, cột F1 illicit là thước đo thật của bài toán.

Đồ thị	Mô hình	Độ đúng	F1 macro	F1 illicit	Seed	Giây/lượt
Cora	Label Propagation	0,703 ± 0,000	0,704 ± 0,000	–	5	0,0
Cora	MLP	0,590 ± 0,013	0,570 ± 0,012	–	5	1,6
Cora	GCN	0,824 ± 0,005	**0,812 ± 0,004**	–	5	5,9
Cora	GraphSAGE	0,804 ± 0,006	0,800 ± 0,005	–	5	6,1
Cora	GAT	0,819 ± 0,008	0,810 ± 0,006	–	5	6,1
Cora	GCN NumPy	0,817 ± 0,004	0,806 ± 0,005	–	5	28,5
Amazon Photo	Label Propagation	0,802 ± 0,020	0,783 ± 0,016	–	5	0,5
Amazon Photo	MLP	0,776 ± 0,008	0,758 ± 0,010	–	5	3,9
Amazon Photo	GCN	0,878 ± 0,015	0,861 ± 0,012	–	5	32,8
Amazon Photo	GraphSAGE	0,897 ± 0,017	**0,884 ± 0,014**	–	5	38,5
Amazon Photo	GAT	0,895 ± 0,014	0,878 ± 0,011	–	5	51,8
Elliptic	Label Propagation	0,935 ± 0,000	0,483 ± 0,000	0,000	5	4,3
Elliptic	MLP	0,918 ± 0,007	0,713 ± 0,012	0,470	5	15,9
Elliptic	GCN	0,939 ± 0,004	0,749 ± 0,021	0,530	5	48,2
Elliptic	GraphSAGE	0,933 ± 0,006	**0,753 ± 0,012**	0,542	5	42,0
Elliptic	GAT	0,917 ± 0,002	0,707 ± 0,007	0,459	5	95,4

Trung bình ± độ lệch chuẩn. Giây là trên GPU RTX 4060 dùng chung, GCN NumPy và LP chạy CPU.

Đọc kết quả

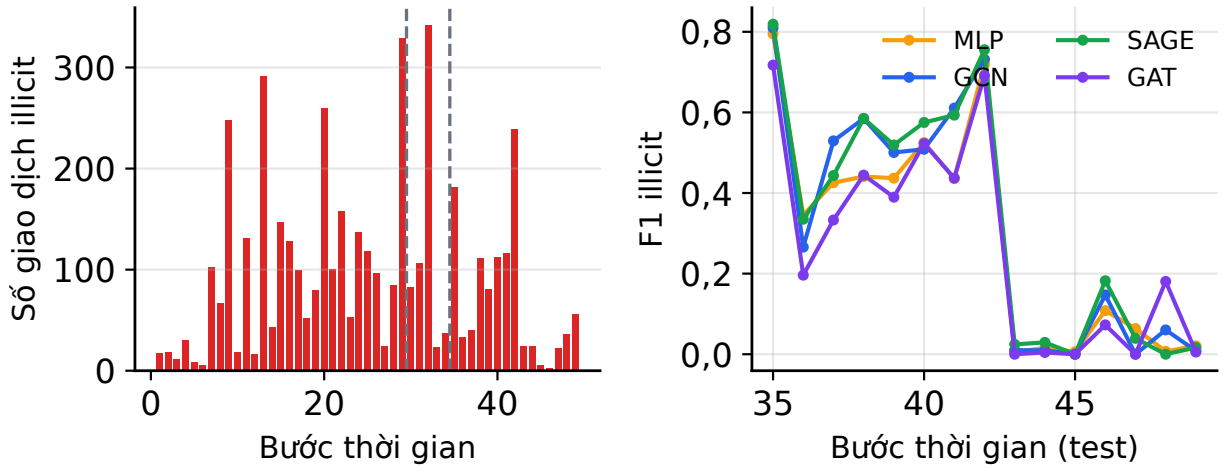
Trên Cora, GCN đạt F1 macro 0,8117, hơn MLP 0,2421; bản NumPy đạt 0,8059, khớp bản PyTorch, cho thấy đạo hàm viết tay đúng. Trên Photo, GraphSAGE dẫn đầu với 0,8841, hơn MLP 0,1263. LP không dùng đặc trưng mà vẫn đạt 0,7036 trên Cora và 0,7831 trên Photo, nghĩa là riêng cấu trúc đã mang nhiều thông tin nhãn. Trên Elliptic, LP cho F1 illicit bằng 0,0000 vì nhãn lan tới 0,0000 phần nút test: không cạnh nào nối hai bước thời gian, nên nhãn quá khứ



Hình 7.6. F1 macro của năm phương pháp trên ba đồ thị, thanh sai số là độ lệch chuẩn giữa các seed.

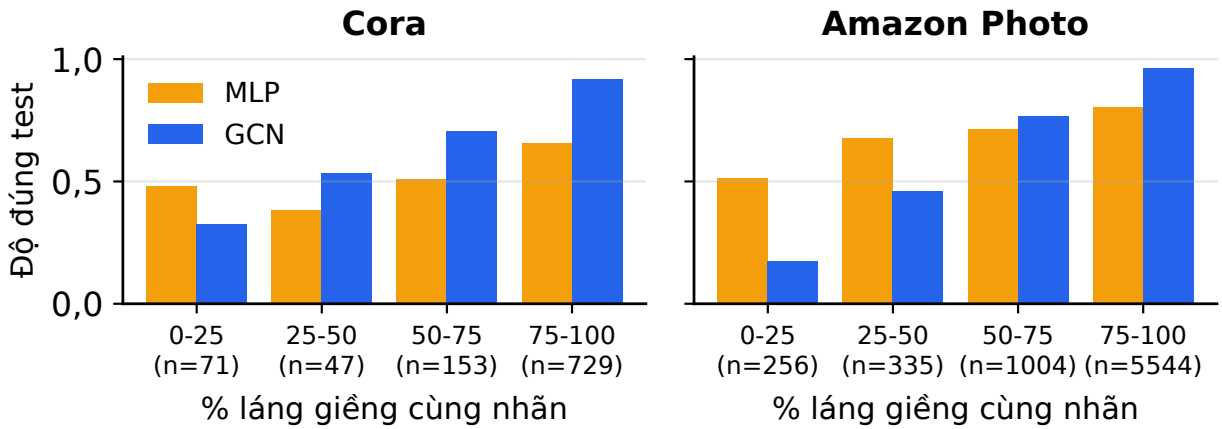
không có đường đi tới tương lai, và độ đúng 0,9350 của nó chỉ là tỷ lệ lớp đa số.

Trên Elliptic, mọi GNN đều học quy nạp vì đồ thị test không dính với đồ thị train. Tốt nhất là GraphSAGE, F1 illicit 0,5424; GAT chỉ đạt 0,4589, thấp hơn cả MLP 0,4704 dù tốn thời gian nhất. Hình 7.7 cho biết lý do tổng thể không cao: F1 trung bình của GCN là 0,5677 ở các bước 35–42 rồi rơi xuống 0,0337 từ bước 43. Weber và cộng sự cũng ghi nhận cú rơi tại đây, trùng thời điểm một chợ đen lớn đóng cửa [134]: kiểu gian lận đổi, và không mô hình nào trong năm mô hình theo kịp.



Hình 7.7. Trái: số giao dịch illicit theo bước, nét đứt ngăn train, val, test. Phải: F1 illicit trên test theo bước, trung bình các seed.

Nhận xét: Phân tích lỗi cho thấy GNN giỏi ở đâu và hỏng ở đâu (Hình 7.8). Ở nhóm nút có trên 75 % láng giềng cùng nhãn, GCN đạt 0,9177 trên Cora, so với

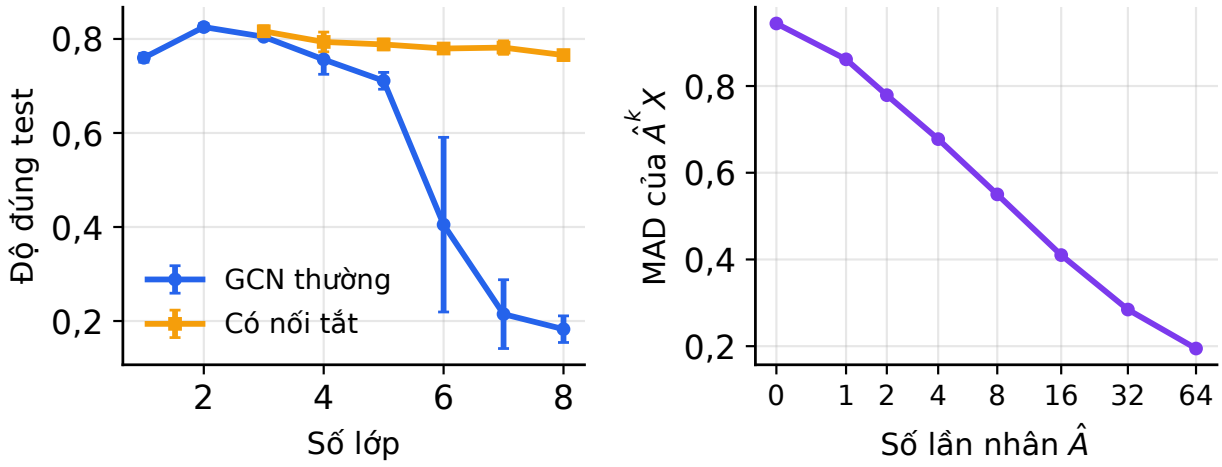


Hình 7.8. Phân tích lỗi: độ đúng test theo tỷ lệ láng giềng cùng nhãn của nút (seed 0).

MLP 0,6571. Ở nhóm dưới 25 %, tình thế đảo ngược: GCN 0,3239 so với MLP 0,4789 trên 71 nút Cora, và 0,1719 so với 0,5117 trên 256 nút Photo. Láng giềng khác nhãn kéo biểu diễn về lớp sai, đúng cái giá phải trả của phép làm trơn.

7.4.2 Over-smoothing theo độ sâu

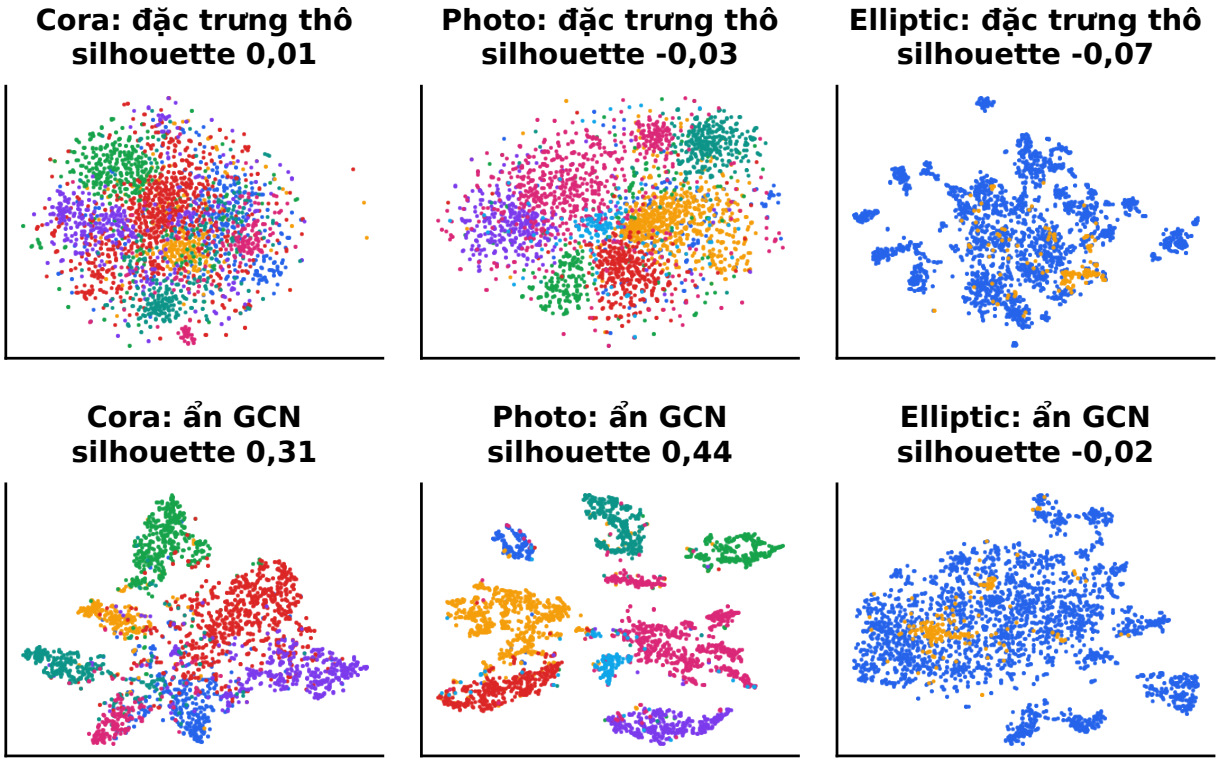
Số liệu độ sâu trên Cora lấy lại nguyên từ Tiểu luận 03 (cùng dữ liệu, GCN ẩn 64, 3 seed), không chạy lại. Hai lớp đạt 0,8253; tám lớp còn 0,1827, còn nối tắt giữ được 0,7657. Bỏ hẳn việc học, nhân $\hat{A}$ lặp lại làm khoảng cách cosine trung bình MAD [136] giữa các nút tụt từ 0,9442 xuống 0,1946: $\hat{A}^k$ hội tụ về vector riêng trội, nên mọi nút cùng thành phần liên thông dần trùng nhau (Hình 7.9).



Hình 7.9. Over-smoothing trên Cora: độ đúng theo số lớp, và MAD của $\hat{A}^k X$ không qua huấn luyện.

7.4.3 Biểu diễn học được: t-SNE

Hình 7.10 chiếu đặc trưng thô và lớp ẩn GCN xuống hai chiều (3.000 nút với Photo, Elliptic; Cora lấy từ Tiểu luận 03). Silhouette tính trên vector gốc theo nhãn thật: Cora tăng từ 0,0087 lên 0,3074, Photo từ −0,0266 lên 0,4383, Elliptic từ −0,0696 lên −0,0202.



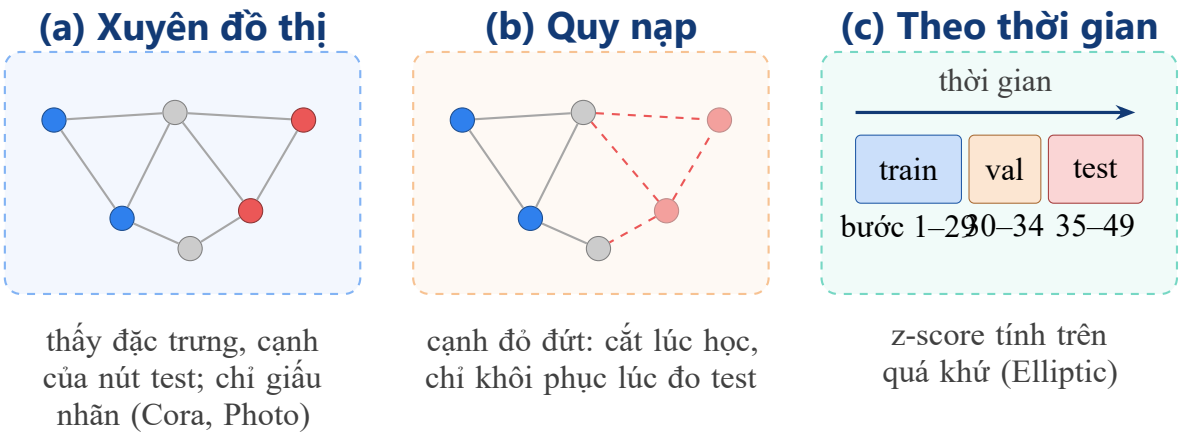
Hình 7.10. t-SNE của đặc trưng thô (trên) và biểu diễn ẩn GCN (dưới), màu là nhãn thật.

7.5 Chống rò rỉ dữ liệu trên đồ thị

Đồ thị có ba kiểu rò rỉ mà dữ liệu bảng không có (Hình 7.11). Học xuyên đồ thị cho mô hình thấy đặc trưng và cạnh của nút test lúc truyền tin; đây là giả định hợp lệ khi đồ thị đóng, nhưng lạc quan nếu nút mới đến sau. Học quy nạp cắt mọi cạnh chạm nút giữ lại, do `graph_views` thực hiện (Đoạn mã 7.4): train không thấy val, test; val không thấy test. Với dữ liệu có thời gian, chia ngẫu nhiên cho mô hình học từ tương lai.

Mức lạc quan của từng phép chia

Trên đồ thị đóng, cắt cạnh tới nút chưa thấy làm F1 macro giảm 0,0244 với GCN trên Cora và tới 0,1274 với GraphSAGE trên Photo. Elliptic nặng hơn



Hình 7.11. Ba cách dựng phép chia: xuyên đồ thị, quy nạp và theo thời gian.

Đoạn mã 7.4. Ba góc nhìn đồ thị cho học quy nạp.

```python
# tlf/ch7_gnn.py
def graph_views(kind, g, split, inductive, device):
    """Ba góc nhìn đồ thị. Quy nạp: train không thấy val, test; val
    ↪ không thấy test; test thấy tất cả."""
    ei = g.edge_index.to(device)
    full = graph_input(kind, ei, g.n)
    if not inductive:
        return full, full, full
    hv, ht = split.hold_val.to(device), split.hold_test.to(device)
    train = graph_input(kind, keep_nodes(ei, hv | ht), g.n)
    return train, graph_input(kind, keep_nodes(ei, ht), g.n), full
```

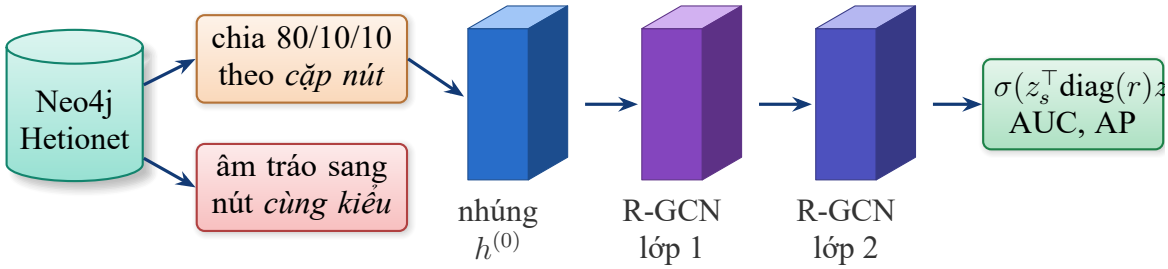
Bảng 7.5. Đo rò rỉ. Cora, Photo: mức 1 là xuyên đồ thị, mức 2 là quy nạp. Elliptic: mức 1 là chia theo thời gian, mức 2 là chia ngẫu nhiên.

Đồ thị	Mô hình	Phép so	Mức 1	Mức 2
Cora	GCN	F1 macro: xuyên đồ thị / quy nạp	0,812	0,787 ± 0,006
Cora	GraphSAGE	F1 macro: xuyên đồ thị / quy nạp	0,800	0,770 ± 0,006
Amazon Photo	GCN	F1 macro: xuyên đồ thị / quy nạp	0,861	0,802 ± 0,033
Amazon Photo	GraphSAGE	F1 macro: xuyên đồ thị / quy nạp	0,884	0,757 ± 0,072
Elliptic	MLP	F1 illicit: chia thời gian / ngẫu nhiên	0,470	0,825 ± 0,007
Elliptic	GCN	F1 illicit: chia thời gian / ngẫu nhiên	0,530	0,794 ± 0,013

nhiều: chia ngẫu nhiên cho MLP F1 illicit 0,8247 nhưng chia theo thời gian chỉ còn 0,4704, GCN từ 0,7938 còn 0,5303. Với dữ liệu gian lận, con số từ phép chia ngẫu nhiên không nói được gì về khả năng dùng thật.

7.6 Dự đoán liên kết trên Hetionet từ Neo4j

Làn Chương 5 đã nạp tiểu đồ thị Hetionet [137] quanh đái tháo đường vào Neo4j, nên tôi đọc thẳng cạnh bằng Cypher (nguồn: neo4j), không tải lại tệp. Đồ thị có 10.976 nút thuộc 10 kiểu, 38.659 bộ ba thuộc 23 quan hệ. Bài toán: cho (s, r, t), cạnh này có thật không?



Hình 7.12. Quy trình dự đoán liên kết từ Neo4j tới AUC, AP.

Tách cạnh là chỗ dễ rò rỉ nhất. Hai nút có thể nối bằng nhiều quan hệ, nên tôi chia theo *cặp nút* (Đoạn mã 7.5): 30.923/3.862/3.874 bộ ba, đồ thị truyền tin chỉ gồm cạnh train. Mẫu âm tráo đầu hoặc đuôi sang nút cùng kiểu và loại mọi bộ ba đã biết; tráo tự do sẽ dễ hơn nhiều vì chỉ cần nhận ra kiểu nút. R-GCN [138] gán mỗi quan hệ một vector đường chéo d_r, giải mã bằng DistMult.

Bảng 7.6. Dự đoán liên kết, 3 seed, âm cùng kiểu. Dòng cuối cố ý để cạnh test lọt vào đồ thị truyền tin khi đo.

Phương pháp	AUC	AP	Giây/lượt
Adamic-Adar	0,642 ± 0,002	0,641 ± 0,004	3,6
Tích bậc	0,725 ± 0,002	0,744 ± 0,002	0,0
DistMult (không truyền tin)	0,839 ± 0,007	0,843 ± 0,006	212,8
GCN + DistMult	0,767 ± 0,008	0,792 ± 0,008	181,9
R-GCN + DistMult	0,799 ± 0,007	0,816 ± 0,001	160,4
R-GCN, cạnh test lọt vào đồ thị	0,919 ± 0,004	0,905 ± 0,004	–

Đoạn mã 7.5. Tách cạnh theo cặp nút không thứ tự.

```python
# tlf/ch7_link.py
def split_pairs(triples, seed, val=0.1, test=0.1):
    """Chia 80/10/10 theo cặp nút không thứ tự, chống rò rỉ qua cạnh song song và cạnh ngược."""
    pair = np.minimum(triples[:, 0], triples[:, 2]) * 100000 + np.maximum(triples[:, 0], triples[:, 2])
    uniq, inv = np.unique(pair, return_inverse=True)
    order = np.random.default_rng(seed).permutation(uniq.size)
    fold = np.zeros(uniq.size, dtype=np.int8)
    fold[order[:int(test * uniq.size)]] = 2
    fold[order[int(test * uniq.size):int((test + val) * uniq.size)]] = 1
    f = fold[inv]
    return triples[f == 0], triples[f == 1], triples[f == 2]
```

Mô hình đơn giản thắng, và rò rỉ cho số đẹp giả

DistMult không truyền tin đạt AUC 0,8390, cao hơn R-GCN 0,7990. Với số cạnh train hiện có nhúng nút tự do đã đủ học; hai lớp truyền tin trên đồ thị không có đặc trưng nút chưa đem lại lợi ích với số epoch ngắn này. Heuristic Tích bậc đạt 0,7248, tức một phần lớn tín hiệu chỉ là nút nào nhiều cạnh. Cùng mô hình R-GCN đã huấn luyện, chỉ đưa cạnh test vào đồ thị truyền tin khi đo, AUC nhảy lên 0,9186, tăng 0,1196: con số đẹp nhất bảng là con số sai.

AUC theo quan hệ dao động mạnh: cao nhất là gen tham gia đường truyền (GpPW), thấp nhất là hợp chất điều trị bệnh (CtD), quan hệ có ít cạnh test nhất. Khi xếp hạng hợp chất cho *đái tháo đường type 2* bằng mô hình seed 0, trong tám ứng viên đầu ngoài cạnh train có Sitagliptin và Miglitol, đúng là hai cạnh Hetionet đã giấu đi trong val, test; những ứng viên khác như Sildenafil hay Drospirenone chỉ là giả thuyết của mô hình. Kết quả này chỉ mang tính học thuật, không phải khuyến nghị y khoa.

7.7 Triển khai và tóm tắt

`tlf/serve_ch7.py` nạp đồ thị và trọng số GCN, GAT lười trên CPU, không cần PyG. Với một nút, nó trả nhãn thật, xác suất dự đoán, đồ thị con 1 đến 2 bước, trọng số mỗi cạnh (α của GAT, $\hat{A}_{vu}$ của GCN) và danh sách thông điệp vào nút để Web App vẽ truyền thông điệp (ảnh ở Chương 9).

Chương này xác nhận GNN vượt đường cơ sở khi cạnh đồng dạng và phép đo

trung thực, đồng thời chỉ ra ba giới hạn đo được: nút lạc giữa láng giềng khác nhãn, kiểu gian lận đổi theo thời gian, và đồ thị thưa không đặc trưng nơi mô hình nông thắng. Phép chia, không phải kiến trúc, quyết định con số có đáng tin hay không.

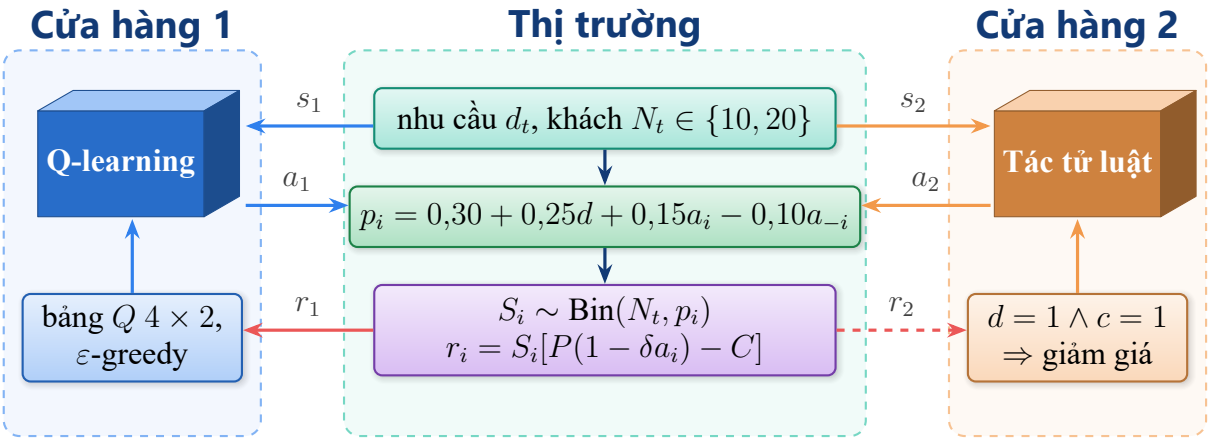
Chương 8

Tác tử học tăng cường và tác tử dựa trên luật

Chương này cô đọng bài tập Agent 02: hai cửa hàng cùng quyết định giữ giá hay giảm giá, một bên dùng Q-learning, một bên dùng luật viết tay. Môi trường là bộ mô phỏng theo đề bài [139], mọi quan sát đều là **dữ liệu tổng hợp**. Số liệu dưới đây được chép nguyên từ lượt chạy của bài tập đó, không có thí nghiệm mới.

8.1 Mô hình hóa

Ở mỗi bước t, cửa hàng i chọn đồng thời $a_i \in \{0, 1\}$ (0: giữ giá, 1: giảm giá) sau khi quan sát $s_i = (d_t, c_i)$, với $d_t \in \{0, 1\}$ là nhu cầu và $c_i = a_{-i}(t-1)$ là hành động trước của đối thủ. Số khách N_t là 10 hoặc 20; số đơn bán được $S_i \sim \text{Binomial}(N_t, p_i)$ với p_i ở Hình 8.1. Phần thưởng là lợi nhuận với $P = 10$, $C = 5$, $\delta = 0{,}20$; mỗi episode có 20 bước. Hai tác tử không trao đổi thông điệp mà ảnh hưởng nhau qua thị trường chung.



Hình 8.1. Một bước tương tác: hai tác tử chọn hành động đồng thời, thị trường tính xác suất chuyển đổi, doanh số và phần thưởng.

Tác tử luật giữ giá khi nhu cầu thấp; khi nhu cầu cao, nó chỉ giảm giá nếu đối thủ vừa giảm. Tác tử RL giữ bảng Q tám ô và cập nhật theo Q-learning [140] với $\alpha = 0{,}10$, $\gamma = 0{,}90$, $\varepsilon_e = \max(0{,}02;\ 0{,}20 \cdot 0{,}995^e)$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big]. \tag{8.1}$$

Đoạn mã 8.1. Cập nhật Q-learning theo (8.1).

```
# tlf/ch8_agents.py
def learn(self, obs, action, reward, next_obs, done):
    """Q(s,a) ← Q(s,a) + α[r + γ·max_a' Q(s',a') − Q(s,a)]; ở bước cuối
    ↪ mục tiêu chỉ còn r."""
    target = reward if done else reward + self.gamma *
    ↪ self.q[next_obs].max()
    td_error = target - self.q[obs][action]
    self.q[obs][action] += self.alpha * td_error
    self.visits[obs][action] += 1
    return td_error
```

Vì mô hình nhỏ, lợi nhuận kỳ vọng tính được chính xác (Bảng 8.1). Với $\delta = 20\%$, giữ giá tốt hơn ở ba trên bốn ô, nên không bao giờ giảm giá là chính sách tối ưu với giá trị 35,00 mỗi bước, và tác tử luật vốn đã làm đúng như vậy. Với $\delta = 5\%$, tối ưu chuyển thành luôn giảm giá (37,35).

Bảng 8.1. Lợi nhuận kỳ vọng một bước theo (d, b); b là hành động hiện tại của đối thủ.

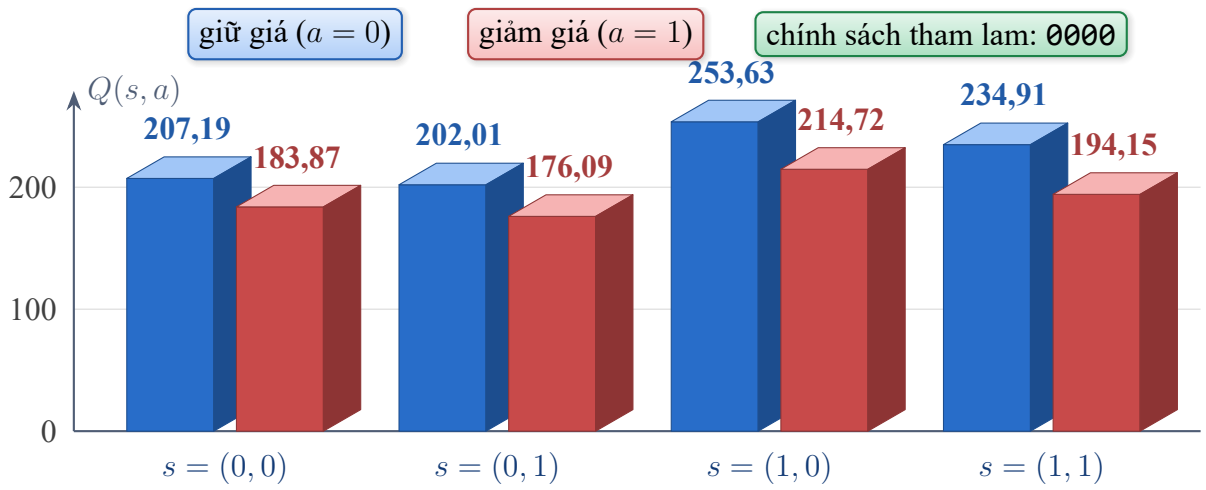
(d, b)	p giữ	p giảm	$\mathbb{E}[r]$ giữ	giảm, δ 20%	Tốt hơn	giảm, δ 5%	Tốt hơn
(0, 0)	0,30	0,45	15,00	13,50	giữ	20,25	giảm
(0, 1)	0,20	0,35	10,00	10,50	giảm	15,75	giảm
(1, 0)	0,55	0,70	55,00	42,00	giữ	63,00	giảm
(1, 1)	0,45	0,60	45,00	36,00	giữ	54,00	giảm

8.2 Thí nghiệm A–E

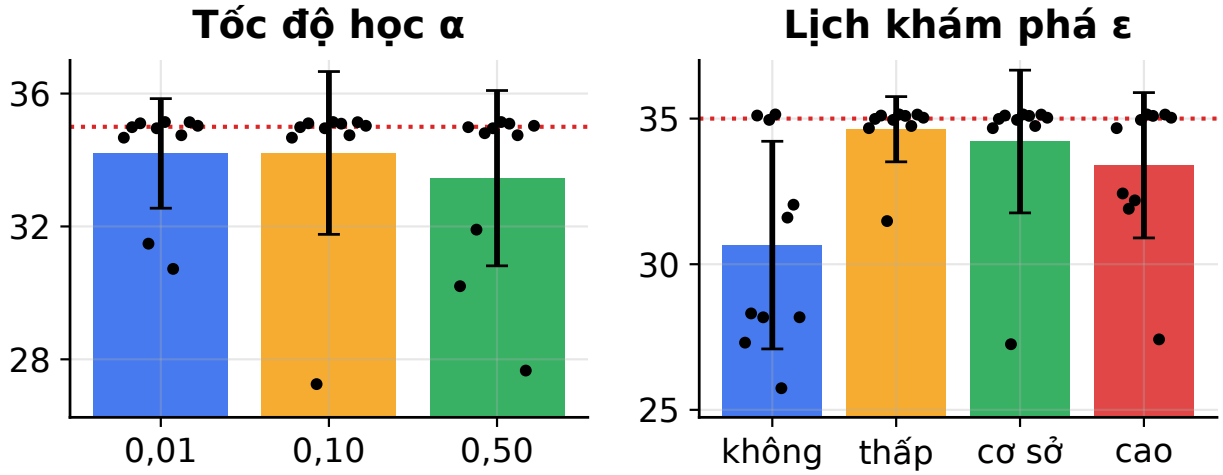
Sau huấn luyện 5.000 episode, bảng Q đóng băng, $\varepsilon = 0$, đánh giá 10.000 bước với seed riêng. Tác tử luật được đặt vào đúng ghế của RL trên cùng chuỗi ngẫu nhiên để so cặp. Ngoài thưởng lấy mẫu, mỗi chính sách còn có giá trị chính xác, tính bằng lan truyền phân phối trạng thái.

A, B. Lượt tham chiếu đạt 34,80 mỗi bước, tỉ lệ giảm giá 0,00%, trùng khít tác tử luật trên cùng chuỗi ngẫu nhiên; giá trị chính xác là 35,00 (Hình 8.2). Thưởng huấn luyện tăng từ 33,36 (500 episode đầu) lên 33,84 (500 episode cuối). Đường học rất nhiễu vì nhiễu Binomial lớn hơn mức cải thiện do học (Hình 8.4, trái).

C, D. Bảng 8.2 và Hình 8.3: $\alpha = 0{,}50$ cho thưởng thấp nhất và chính sách đổi 1.752 lần, vì mỗi mẫu nhiễu chiếm một nửa giá trị cũ; $\alpha = 0{,}01$ ổn định nhất nhưng thưởng huấn luyện chỉ 31,34. Không khám phá là cấu hình tệ nhất (3/10 seed tối



Hình 8.2. Bảng Q của lượt tham chiếu: ở cả bốn trạng thái, cột giữ giá cao hơn cột giảm giá.



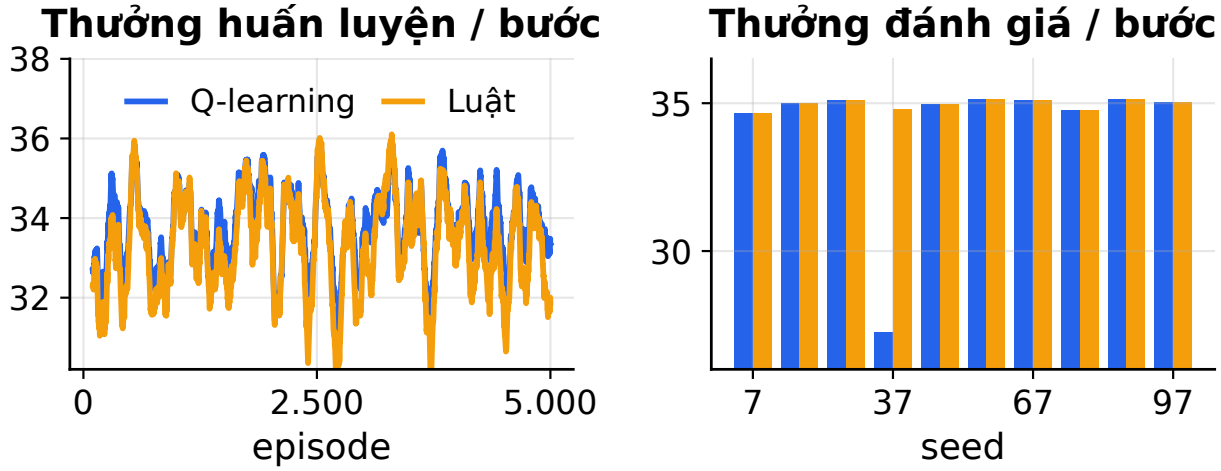
Hình 8.3. Thưởng đánh giá theo α và theo lịch khám phá; chấm đen là từng seed, nét đỏ là mức tối ưu 35.

ưu), vì hành động đầu tiên có thưởng dương được giữ mãi; lịch thấp tốt nhất vì bảng chỉ có tám ô.

Bảng 8.2. Thí nghiệm C, D, E: 10 seed, đánh giá tham lam; "Đổi CS" là số lần chính sách đổi khi huấn luyện.

TN	Cấu hình	Thưởng đánh giá	Seed tối ưu	Đổi CS	KTC 95% RL − luật
C	$\alpha = 0{,}01$	$34{,}20 \pm 1{,}65$	8/10	16,3	$[-1{,}94;\ 0{,}40]$
A, E	$\alpha = 0{,}10$ (cơ sở)	$34{,}21 \pm 2{,}45$	9/10	557,2	$[-2{,}46;\ 0{,}95]$
C	$\alpha = 0{,}50$	$33{,}45 \pm 2{,}64$	7/10	1.752,2	$[-3{,}41;\ 0{,}38]$
D	$\varepsilon = 0$	$30{,}66 \pm 3{,}56$	3/10	0,0	$[-6{,}82;\ -1{,}80]$
D	lịch ε thấp	$34{,}64 \pm 1{,}12$	9/10	297,8	$[-1{,}08;\ 0{,}42]$
D	lịch ε cao	$33{,}40 \pm 2{,}49$	6/10	1.395,1	$[-3{,}30;\ 0{,}16]$

E. Trên 10 seed, RL đạt 34,21 $\pm$ 2,450, luật đạt 34,97; chênh lệch cặp $-0{,}7553$, khoảng tin cậy 95% $[-2{,}464;\ 0{,}9532]$ chứa 0, nên chưa đủ bằng chứng nói bên nào hơn. 9/10 seed đạt tối ưu; một seed học hỏng kéo trung bình xuống (Hình 8.4, phải).



Hình 8.4. Trái: đường học lượt tham chiếu, trung bình trượt 100 episode. Phải: thưởng đánh giá trên mười seed.

8.3 Trả lời mười câu hỏi

1. **Mô hình hóa.** Tác tử là hai cửa hàng, môi trường là thị trường, trạng thái (d, c), hành động giữ hoặc giảm, thưởng là lợi nhuận. Đây là hệ đa tác tử vì thưởng phụ thuộc hành động đối thủ qua p_i và qua c_i.
2. **Học và luật.** RL thích nghi được nhưng tốn khám phá và phụ thuộc seed; luật tất định, dễ đọc nhưng cứng. Ở tham chiếu luật đã tối ưu nên RL chỉ có thể hòa.
3. **Một bước cập nhật.** Với $Q = 2{,}800$, $r = 25{,}00$, $\max Q' = 4{,}252$: mục tiêu $25{,}00 + 0{,}9000 \times 4{,}252 = 28{,}83$, sai số TD 26,03, giá trị mới 5,403, khớp nhật ký chương trình.
4. **Khám phá.** Cần ít nhưng không thể bằng 0: $\varepsilon = 0$ đạt 30,66, lịch thấp 34,64, lịch cao 33,40.
5. **Phần thưởng.** Giảm giá mất 40% biên lợi nhuận để đổi lấy 15 điểm phần trăm chuyển đổi. Huấn luyện theo doanh thu làm lợi nhuận tụt còn 28,19: tác tử tối ưu đúng cái được thưởng.
6. **Đánh giá.** Phải đóng băng bảng Q, tắt khám phá, so cặp trên cùng chuỗi ngẫu nhiên và báo cáo qua nhiều seed.
7. **Tốc độ học.** α lớn bám mẫu gần nhất (đổi 1.752 lần), α nhỏ mượt nhưng chậm (16,30 lần).

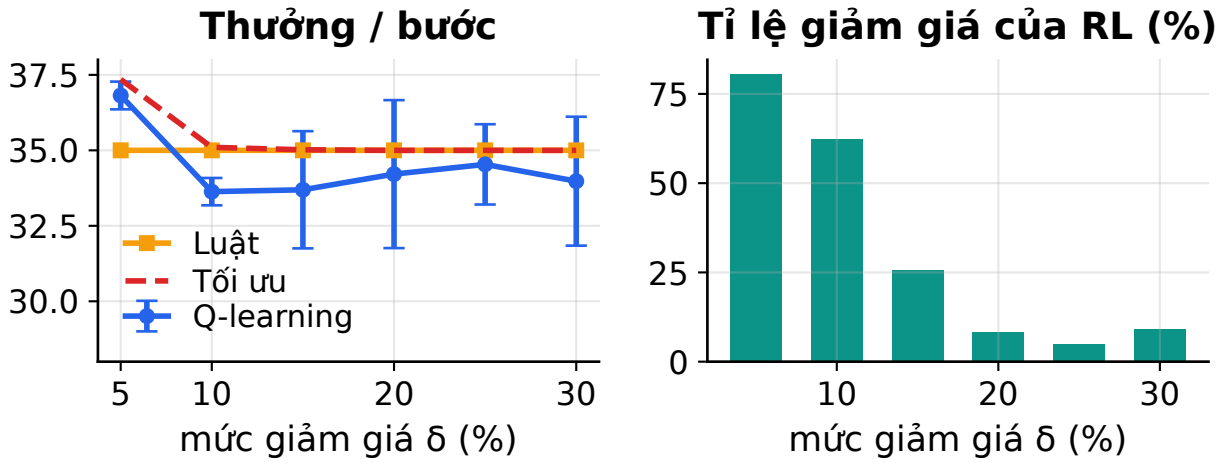
8. **Phi dừng.** Khi đối thủ cũng học, tỉ lệ giảm giá huấn luyện lên 24,93% (so với 16,43%), không seed nào cả hai về `0000`; môi trường đổi theo thời gian [141].
9. **Giới hạn.** Bốn trạng thái, không tồn kho, và quan sát thiếu hành động trước của chính tác tử, nên hậu quả "giảm hôm nay, đối thủ giảm ngày mai" bị ẩn. Mục 8.4 đo tác động.
10. **Tác tử tốt hơn.** Tác tử dựa trên mô hình ở Mục 8.5 đạt tối ưu ở cả năm kịch bản.

8.4 Thay đổi môi trường

Bảng 8.3 và Hình 8.5 đổi mức giảm δ, tính bền của nhu cầu và cách thưởng. Ở $\delta = 5\%$ RL vượt luật, khoảng tin cậy $[1{,}599;\ 2{,}101]$. Ở $\delta = 10\%$ RL lại thua rõ $[-1{,}676;\ -0{,}9959]$: nó giảm giá khi nhu cầu thấp vì lợi trước mắt mà không thấy phản ứng của đối thủ ở bước sau. "RL hơn luật" vì vậy không phải mệnh đề chung.

Bảng 8.3. Các biến thể môi trường (10 seed; đánh giá luôn theo lợi nhuận).

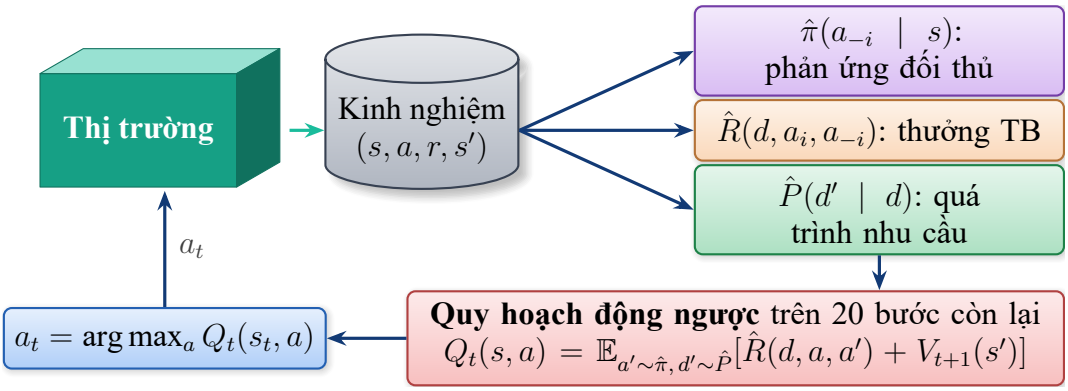
Biến thể	RL đánh giá	Luật	Tối ưu	Giảm giá	Seed tối ưu	KTC 95% RL − luật
$\delta = 5\%$	$36{,}82 \pm 0{,}46$	35,00	37,35	80,5%	2/10	[1,60; 2,10]
$\delta = 10\%$	$33{,}63 \pm 0{,}45$	35,00	35,10	62,3%	1/10	[−1,68; −1,00]
$\delta = 15\%$	$33{,}69 \pm 1{,}94$	35,00	35,02	25,5%	6/10	[−2,62; 0,07]
$\delta = 20\%$	$34{,}21 \pm 2{,}45$	35,00	35,00	8,2%	9/10	[−2,46; 0,95]
$\delta = 25\%$	$34{,}54 \pm 1{,}33$	35,00	35,00	5,0%	9/10	[−1,41; 0,55]
$\delta = 30\%$	$33{,}98 \pm 2{,}14$	35,00	35,00	9,1%	8/10	[−2,49; 0,51]
Nhu cầu bền	$34{,}04 \pm 1{,}92$	35,00	35,00	18,0%	6/10	[−2,48; 0,26]
Thưởng = doanh thu	$28{,}19 \pm 2{,}52$	35,00	35,00	62,3%	1/10	[−8,60; −4,95]



Hình 8.5. Theo mức giảm δ: thưởng của RL, của luật và mức tối ưu; tỉ lệ giảm giá của RL khi đánh giá.

8.5 Tác tử cải tiến

Hai đề xuất nhắm vào lỗ hổng quan sát: Q-learning mở rộng thêm $a_i(t-1)$ vào trạng thái; tác tử dựa trên mô hình ước lượng phản ứng đối thủ, thưởng và nhu cầu từ kinh nghiệm rồi lập kế hoạch đúng 20 bước (Hình 8.6). Tác tử này đạt tối ưu ở 10/10 seed với $\delta = 5\%$, trong khi Q-learning gốc chỉ đạt 2/10 (Bảng 8.4, Hình 8.7). Đổi lại, nó phải biết cấu trúc trạng thái chung và giả định đối thủ dừng.

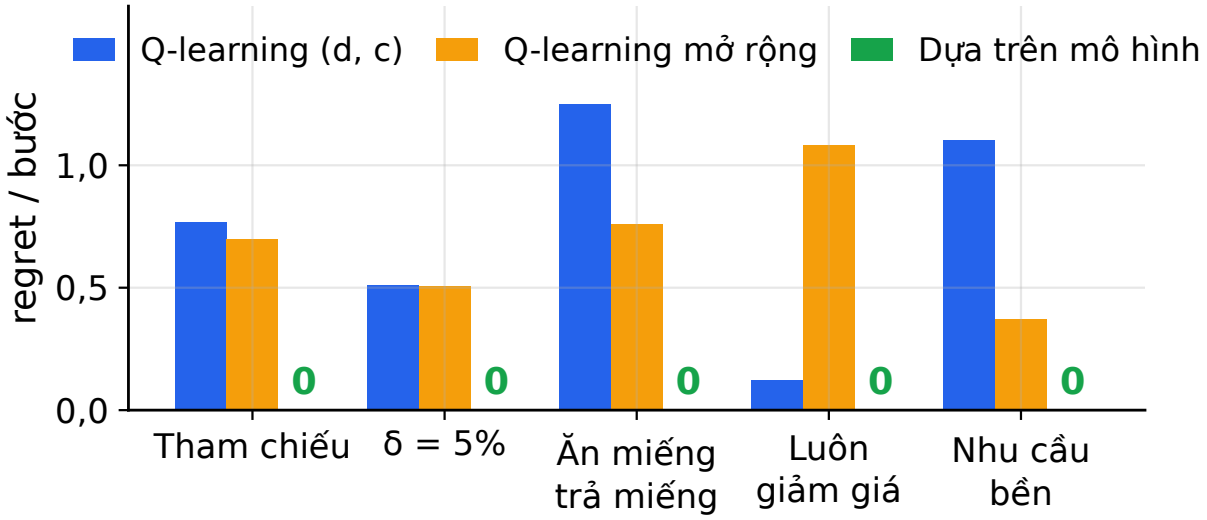


Hình 8.6. Tác tử dựa trên mô hình: thu kinh nghiệm, cập nhật ba mô hình đếm tần suất, quy hoạch động ngược, chọn hành động.

Bảng 8.4. Ba tác tử trong năm kịch bản.

Kịch bản	Tối ưu	Luật	Q-learning	Q mở rộng	Mô hình
Tham chiếu	35,00	35,00	34,23 (9)	34,30 (7)	**35,00** (10)
$\delta = 5\%$	37,35	35,00	36,84 (2)	36,84 (2)	**37,35** (10)
Đối thủ ăn miếng trả miếng	35,00	35,00	33,75 (7)	34,24 (8)	**35,00** (10)
Đối thủ luôn giảm giá	27,75	23,23	27,63 (3)	26,67 (0)	**27,75** (10)
Nhu cầu bền	35,00	35,00	33,90 (6)	34,63 (7)	**35,00** (10)

Giá trị chính xác trung bình mười seed; trong ngoặc là số seed đạt mức tối ưu trên chính sách dừng.



Hình 8.7. Regret mỗi bước của ba tác tử (trung bình mười seed; thấp hơn là tốt hơn).

Phạm vi kết luận

Các kết quả chỉ có giá trị trong mô hình mô phỏng này, không phải kết luận kinh tế.

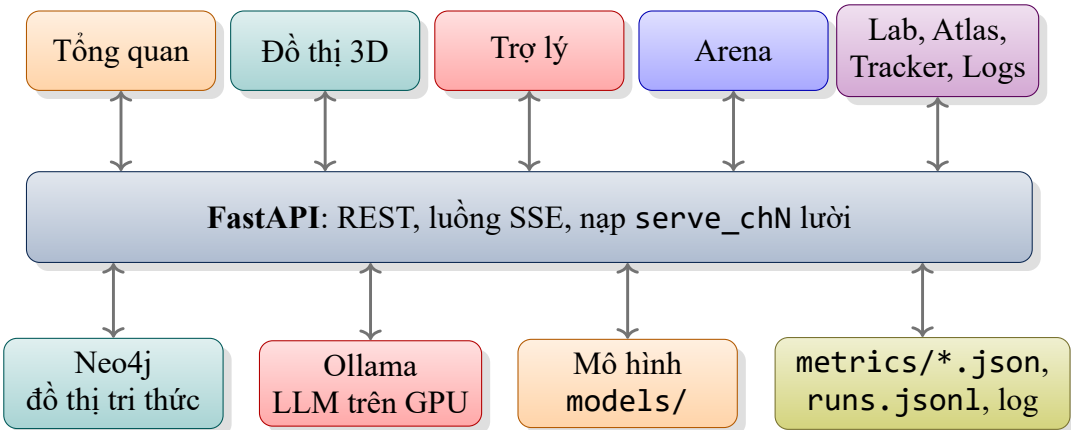
Chương 9

Sản phẩm Web App tổng hợp

Đồng Tâm AI Lab là ứng dụng web chạy ở `127.0.0.1:8090`, ghép kết quả của các chương vào một nơi. Trang không có số liệu viết tay: mọi bảng và biểu đồ đọc trực tiếp từ `outputs/metrics/*.json`, `outputs/runs.jsonl` và Neo4j, nên con số trên web và trong báo cáo cùng một nguồn. Thiếu dữ liệu thì trang báo thiếu, không điền số thay thế.

9.1 Kiến trúc

Máy chủ FastAPI cung cấp REST và luồng SSE, giao diện là các mô-đun JavaScript thuần, không cần bước build. Khởi động không nạp LLM, PyTorch hay TensorFlow; mỗi mô hình chỉ nạp ở yêu cầu đầu tiên của trang cần nó, nhờ vậy ứng dụng chạy được cả khi máy đang huấn luyện.



Hình 9.1. Kiến trúc ba tầng của Đồng Tâm AI Lab.

Trang Trợ lý và Agent Arena nhận sự kiện theo thời gian thực. Bộ phát SSE chạy LLM ở luồng riêng, chuyển mọi lỗi thành sự kiện để người dùng đọc được nguyên văn, và từ chối lần chạy thứ hai khi GPU đang bận.

Đoạn mã 9.1. Bọc lần chạy LLM thành luồng SSE, đóng trang thì dừng generator.

```
# web/api/stream.py
async def sse(start, chapter, request):
    """Bọc start() thành luồng SSE; đóng trang thì đặt cờ dừng để
    ↪ generator ngừng ở sự kiện kế tiếp."""
    if not _busy.acquire(blocking=False):
        data = {"type": "error", "message": "Đang có một lần chạy LLM
        ↪ khác, hãy chờ nó xong."}
        yield {"event": "ev", "data": json.dumps(data,
        ↪ ensure_ascii=False)}
        return
    loop, queue, stop = asyncio.get_running_loop(), asyncio.Queue(),
    ↪ threading.Event()

    def emit(ev):
        loop.call_soon_threadsafe(queue.put_nowait, ev)
    threading.Thread(target=_run, args=(start, chapter, emit, stop),
    ↪ daemon=True).start()
    try:
        while True:
            ev = await queue.get()
            if ev is None:
                break
            yield {"event": "ev", "data": json.dumps(jsonable(ev),
            ↪ ensure_ascii=False)}
            if await request.is_disconnected():
                break
    finally:
        stop.set()
```

9.2 Các trang chức năng

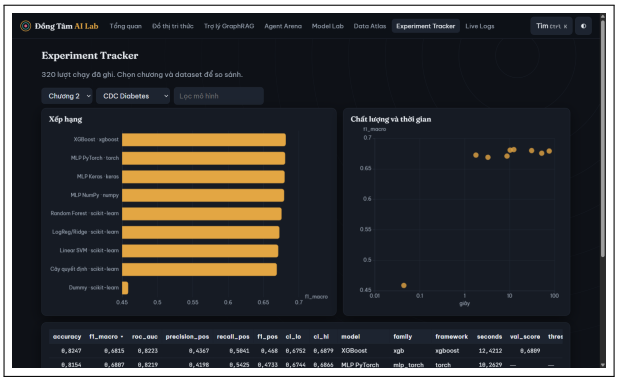
- **Tổng quan:** vòng đồng tâm, mỗi vòng là một chương, độ dài cung theo số lượt chạy thật.
- **Đồ thị tri thức:** đồ thị 3D lấy từ Neo4j, nhấp một nút để nạp hàng xóm, kèm bàn điều khiển Cypher chỉ đọc.
- **Trợ lý GraphRAG:** dải đèn pipeline sáng theo từng sự kiện thật, hiện nguồn đã truy xuất và chọn được LLM.
- **Agent Arena:** xem ba kiến trúc đa tác tử trao đổi trực tiếp, hoặc phát lại vết thông điệp đã ghi mà không cần nạp LLM.
- **Model Lab:** biểu mẫu sinh tự động từ `SPECS` của từng chương, thêm mô hình mới không phải sửa trang.
- **Data Atlas:** thẻ dataset với nguồn, kích cỡ, phân bố và mẫu dữ liệu.

- **Experiment Tracker:** xếp hạng mô hình, biểu đồ đánh đổi chất lượng và thời gian, bảng lọc và sắp xếp được.
- **Live Logs:** nhật ký huấn luyện phát qua SSE, kèm đồng hồ GPU, commit và trạng thái Neo4j.

Ngoài ra có bảng lệnh Ctrl+K tìm trang, dataset, mô hình; chế độ sáng tối lưu trong `localStorage`; và giao diện co theo màn hình điện thoại.

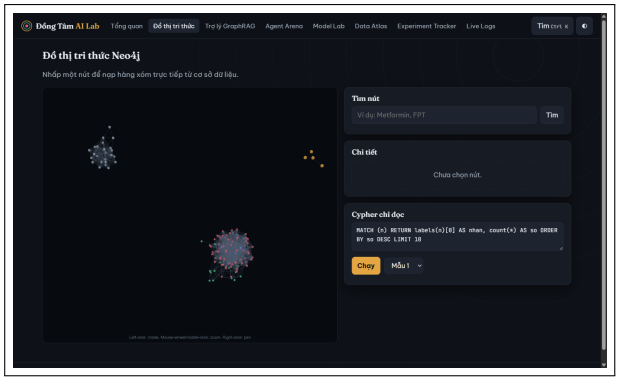


(a) Tổng quan

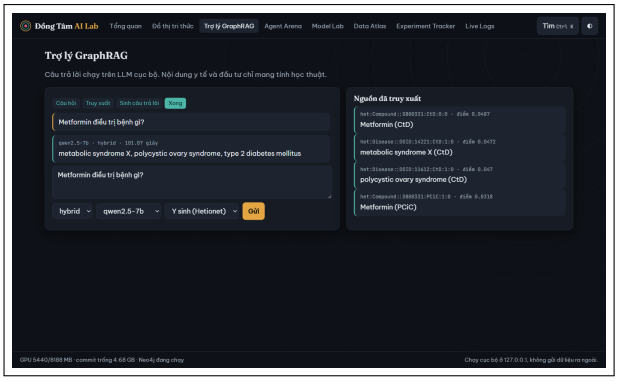


(b) Experiment Tracker

Hình 9.2. Trang tổng quan và trang theo dõi thực nghiệm.

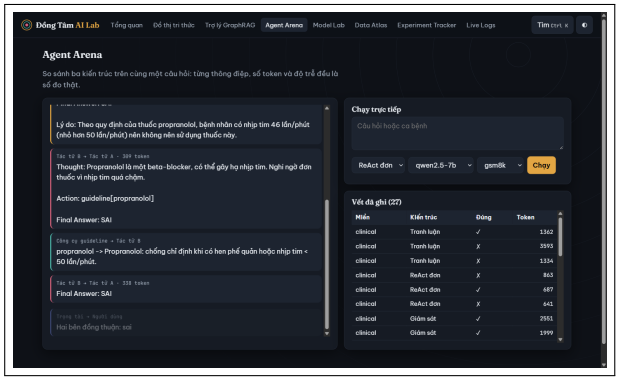


(a) Đồ thị tri thức



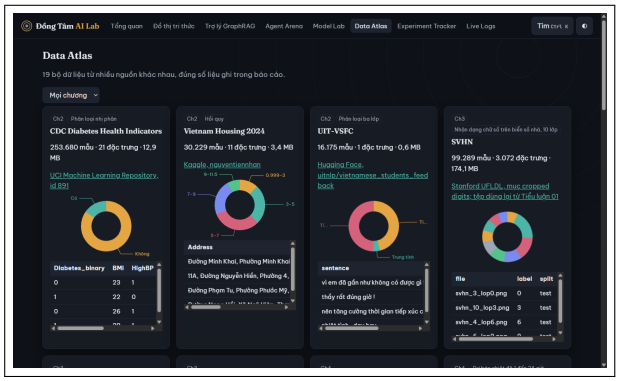
(b) Trợ lý GraphRAG

Hình 9.3. Đồ thị Neo4j và trợ lý truy xuất.



(a) Agent Arena **(b)** Model Lab

Hình 9.4. Đấu trường tác tử và phòng thử mô hình.



(a) Data Atlas **(b)** Live Logs

Hình 9.5. Bản đồ dữ liệu và nhật ký trực tiếp.



Hình 9.6. Trang tổng quan trên khung điện thoại 390 px.

9.3 Kiểm thử

Các điểm cuối chính được gọi thử bằng yêu cầu thật, mỗi trang được mở bằng trình duyệt tự động ở khung desktop, trang tổng quan và theo dõi thực nghiệm được thử thêm ở khung điện thoại 390 px, đều kiểm tra lỗi console. Trợ lý và Agent Arena được chạy với LLM thật. Cypher qua web chỉ chấp nhận câu đọc, câu ghi bị từ chối trước khi tới Neo4j. Ứng dụng chỉ nghe ở địa chỉ cục bộ, LLM chạy trên máy nên câu hỏi không rời khỏi thiết bị.

Phạm vi sử dụng

Nội dung y tế và đầu tư trên trang chỉ mang tính học thuật, không phải lời khuyên chuyên môn.

Kết luận

Kết quả đạt được

Tiểu luận đã đi hết chuỗi kỹ thuật của một hệ thống thông minh: lược sử AI, học máy trên dữ liệu bảng và văn bản, CNN trên ảnh, RNN trên chuỗi thời gian, đồ thị tri thức Neo4j với RAG và tác tử chạy LLM cục bộ, hệ đa tác tử, mạng nơ-ron đồ thị, học tăng cường, và một Web App ghép các phần lại. Bảng 9.1 lấy trực tiếp từ tệp kết quả của từng chương, nêu mô hình tốt nhất theo chỉ số chính trên mỗi bộ dữ liệu.

Bảng 9.1. Giá trị tốt nhất trên tập kiểm thử của từng bộ dữ liệu, chỉ để đối chiếu; mô hình đại diện của mỗi chương được chọn trên tập kiểm định.

Ch.	Bộ dữ liệu	Mô hình tốt nhất	Chỉ số chính	Giá trị
2	CDC Diabetes	XGBoost	F1 macro	0,6815
2	Vietnam Housing	XGBoost	RMSE	1,397
2	UIT-VSFC	XGBoost	F1 macro	0,7494
3	SVHN	ResNet-18	F1 macro	0,9674
3	EuroSAT	ResNet-18	F1 macro	0,9819
3	PathMNIST	VGG-11	F1 macro	0,8464
4	FPT	Persistence	RMSE	1.609
4	VCB	Persistence	RMSE	1.646
4	HPG	Persistence	RMSE	378,64
4	VIC	LSTM	RMSE	4.032
4	Nhiệt độ Hà Nội	BiLSTM	RMSE	0,5578
4	Công suất điện	BiLSTM	RMSE	0,4757
5	Hetionet	GraphRAG	EM	0,6000
5	VN30	Qwen2.5-7B	EM	0,7800
5	HotpotQA	Naive RAG	EM	0,5333
6	GSM8K	Single ReAct, Debate	Độ đúng	0,7200
6	PubMedQA	Single ReAct	Độ đúng	0,6800
6	Ca lâm sàng	Single ReAct	Độ đúng	0,7200
7	Cora	GCN	F1 macro	0,8117
7	Amazon Photo	GraphSAGE	F1 macro	0,8841
7	Elliptic Bitcoin	GraphSAGE	F1 macro	0,7531
8	Tham chiếu	Luật, Dựa trên mô hình	Giá trị đúng	35,00

Ở Chương 8, tác tử dựa trên mô hình đạt giá trị 35,00 mỗi bước ở cấu hình tham chiếu và 37,35 khi $\delta = 5\%$, bằng mức tối ưu ở cả năm kịch bản, trong khi

Q-learning gốc đạt 2/10 seed tối ưu ở $\delta = 5\%$.

Bài học

- **Đường cơ sở quyết định cách đọc kết quả.** Một con số chỉ có nghĩa khi đặt cạnh mô hình đơn giản nhất hoặc mức tối ưu tính được. Ở Chương 8, nếu thiếu mốc tối ưu 35,00, không thể biết tác tử luật đã tối ưu sẵn và RL chỉ có thể hòa.
- **Đánh giá phải tách khỏi huấn luyện.** Chỉ chọn mô hình trên tập kiểm định, dùng tập kiểm thử một lần, và lặp nhiều seed khi kết quả phụ thuộc vào ngẫu nhiên.
- **Chỉ số phải khớp mục tiêu.** Huấn luyện theo doanh thu làm lợi nhuận tụt còn 28,19. Hệ thống tối ưu đúng cái được đo, không phải cái người thiết kế muốn.
- **Giới hạn phần cứng là một phần của thiết kế.** Với GPU 8 GB, việc chọn kích thước mô hình, cách xếp hàng GPU và mức dùng bộ nhớ quyết định việc gì chạy được.

Hạn chế

Mô hình được huấn luyện trên đủ dữ liệu, nhưng phần đánh giá LLM chỉ dùng 50 câu mỗi miền ở Chương 5 và Chương 6 nên sai số mẫu còn lớn; nhiều thực nghiệm chỉ chạy một seed với số epoch vừa đủ, nên chưa nêu được độ lệch giữa các lần chạy. LLM cục bộ (mặc định Qwen2.5-7B qua Ollama) nhỏ hơn nhiều so với mô hình trên đám mây nên kết quả ở Chương 5 và 6 chỉ phản ánh cỡ mô hình này. Dữ liệu ở Chương 8 và ca lâm sàng ở Chương 6 là dữ liệu tổng hợp. Các kết quả về y tế và đầu tư chỉ mang tính học thuật.

Hướng phát triển

Có ba hướng rõ ràng: thêm tồn kho và nhiều mức giá cho mô hình thị trường, đối thủ có trí nhớ dài hơn; đưa quan sát từ đồ thị tri thức vào tác tử để đánh giá trên câu hỏi đa bước dài hơn; và theo dõi mô hình sau triển khai trên Web App để phát hiện suy giảm chất lượng.

Tài liệu tham khảo

[1] Nils J. Nilsson. *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press, 2010.

[2] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[3] Alan M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society*. 2nd ser. 42.1 (1937), pp. 230–265.

[4] Warren S. McCulloch and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity". In: *Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.

[5] Claude E. Shannon. "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423.

[6] Norbert Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press, 1948.

[7] Alan M. Turing. "Computing Machinery and Intelligence". In: *Mind* 59.236 (1950), pp. 433–460.

[8] John McCarthy et al. "A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955". In: *AI Magazine* 27.4 (2006), pp. 12–14.

[9] Allen Newell and Herbert A. Simon. "The Logic Theory Machine: A Complex Information Processing System". In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 61–79.

[10] Frank Rosenblatt. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6 (1958), pp. 386–408.

[11] Arthur L. Samuel. "Some Studies in Machine Learning Using the Game of Checkers". In: *IBM Journal of Research and Development* 3.3 (1959), pp. 210–229.

[12] Joseph Weizenbaum. "ELIZA—A Computer Program for the Study of Natural Language Communication between Man and Machine". In: *Communications of the ACM* 9.1 (1966), pp. 36–45.

[13] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[14] Rodney A. Brooks. "Intelligence without Representation". In: *Artificial Intelligence* 47.1–3 (1991), pp. 139–159.

[15] Automatic Language Processing Advisory Committee. *Language and Machines: Computers in Translation and Linguistics*. Washington, DC: National Academy of Sciences, National Research Council, 1966.

[16] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[17] James Lighthill. "Artificial Intelligence: A General Survey". In: *Artificial Intelligence: A Paper Symposium*. London: Science Research Council, 1973.

[18] Edward H. Shortliffe. *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.

[19] John McDermott. "R1: A Rule-Based Configurer of Computer Systems". In: *Artificial Intelligence* 19.1 (1982), pp. 39–88.

[20] John J. Hopfield. "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8 (1982), pp. 2554–2558.

[21] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning Representations by Back-Propagating Errors". In: *Nature* 323.6088 (1986), pp. 533–536.
[22] Corinna Cortes and Vladimir Vapnik. "Support-Vector Networks". In: *Machine Learning* 20.3 (1995), pp. 273–297.
[23] Leo Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32.
[24] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.
[25] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
[26] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
[27] Murray Campbell, A. Joseph Hoane, and Feng-hsiung Hsu. "Deep Blue". In: *Artificial Intelligence* 134.1–2 (2002), pp. 57–83.
[28] David Ferrucci, Eric Brown, Jennifer Chu-Carroll, et al. "Building Watson: An Overview of the DeepQA Project". In: *AI Magazine* 31.3 (2010), pp. 59–79.
[29] Jia Deng et al. "ImageNet: A Large-Scale Hierarchical Image Database". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255.
[30] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7 (2006), pp. 1527–1554.
[31] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning". In: *Nature* 521.7553 (2015), pp. 436–444.
[32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012, pp. 1097–1105.
[33] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778.
[34] David Silver et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587 (2016), pp. 484–489.
[35] David Silver et al. "Mastering the Game of Go without Human Knowledge". In: *Nature* 550.7676 (2017), pp. 354–359.
[36] Ian Goodfellow et al. "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.
[37] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. "Sequence to Sequence Learning with Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.
[38] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations*. 2015.
[39] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.
[40] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.
[41] John Jumper, Richard Evans, Alexander Pritzel, et al. "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596.7873 (2021), pp. 583–589.
[42] Long Ouyang, Jeff Wu, Xu Jiang, et al. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 27730–27744.
[43] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.

[44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023).
[45] OpenAI. "OpenAI o1 System Card". In: *arXiv preprint arXiv:2412.16720* (2024).
[46] DeepSeek-AI. "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948* (2025).
[47] The Nobel Foundation. *The Nobel Prize in Physics 2024*. 2024. URL: https://www.nobelprize.org/prizes/physics/2024/summary/ (visited on 11/05/2026).
[48] The Nobel Foundation. *The Nobel Prize in Chemistry 2024*. 2024. URL: https://www.nobelprize.org/prizes/chemistry/2024/summary/ (visited on 11/05/2026).
[49] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.
[50] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023.
[51] Anthropic. *Introducing the Model Context Protocol*. 2024. URL: https://www.anthropic.com/news/model-context-protocol (visited on 11/05/2026).
[52] Google. *Announcing the Agent2Agent Protocol (A2A)*. 2025. URL: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/ (visited on 11/05/2026).
[53] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation". In: *arXiv preprint arXiv:2308.08155* (2023).
[54] Khanh-Tung Tran et al. "Multi-Agent Collaboration Mechanisms: A Survey of LLMs". In: *arXiv preprint arXiv:2501.06322* (2025).
[55] Thomas Kwa, Ben West, Joel Becker, et al. "Measuring AI Ability to Complete Long Software Tasks". In: *arXiv preprint arXiv:2503.14499* (2025).
[56] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L series. 2024.
[57] Thủ tướng Chính phủ. *Quyết định 127/QĐ-TTg ban hành Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030*. Hà Nội, ngày 26/01/2021. 2021.
[58] Yoshua Bengio et al. *International AI Safety Report*. Tech. rep. DSIT 2025/001. UK Department for Science, Innovation and Technology, 2025.
[59] François Chollet. "On the Measure of Intelligence". In: *arXiv preprint arXiv:1911.01547* (2019).
[60] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
[61] Stuart Geman, Elie Bienenstock, and René Doursat. "Neural networks and the bias/variance dilemma". In: *Neural Computation* 4.1 (1992), pp. 1–58.
[62] Shachar Kaufman et al. "Leakage in Data Mining: Formulation, Detection, and Avoidance". In: *ACM Transactions on Knowledge Discovery from Data* 6.4 (2012).
[63] Arthur E. Hoerl and Robert W. Kennard. "Ridge regression: Biased estimation for nonorthogonal problems". In: *Technometrics* 12.1 (1970), pp. 55–67.
[64] David R. Cox. "The regression analysis of binary sequences". In: *Journal of the Royal Statistical Society: Series B* 20.2 (1958), pp. 215–242.
[65] Leo Breiman et al. *Classification and Regression Trees*. Wadsworth, 1984.
[66] J. Ross Quinlan. "Induction of decision trees". In: *Machine Learning* 1.1 (1986), pp. 81–106.
[67] Leo Breiman. "Random forests". In: *Machine Learning* 45.1 (2001), pp. 5–32.
[68] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine Learning* 20.3 (1995), pp. 273–297.

[69] Thorsten Joachims. "Text categorization with support vector machines: Learning with many relevant features". In: *European Conference on Machine Learning*. 1998, pp. 137–142.

[70] Jerome H. Friedman. "Greedy function approximation: A gradient boosting machine". In: *Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[71] Tianqi Chen and Carlos Guestrin. "XGBoost: A scalable tree boosting system". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.

[72] Nitish Srivastava et al. "Dropout: A simple way to prevent neural networks from overfitting". In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.

[73] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536.

[74] Diederik P. Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *International Conference on Learning Representations*. 2015.

[75] Tom Fawcett. "An introduction to ROC analysis". In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.

[76] Centers for Disease Control and Prevention. *CDC Diabetes Health Indicators, BRFSS 2015*. UCI Machine Learning Repository, id 891. URL: https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators (visited on 11/05/2026).

[77] Tien Nhan Nguyen. *House Price Prediction Dataset Vietnam - 2024*. Kaggle. 2024. URL: https://www.kaggle.com/datasets/nguyentiennhan/vietnam-housing-dataset-2024 (visited on 11/05/2026).

[78] Daniele Micci-Barreca. "A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems". In: *ACM SIGKDD Explorations Newsletter* 3.1 (2001), pp. 27–32.

[79] Kiet Van Nguyen et al. "UIT-VSFC: Vietnamese Students' Feedback Corpus for Sentiment Analysis". In: *International Conference on Knowledge and Systems Engineering (KSE)*. IEEE, 2018.

[80] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[81] Sergey Ioffe and Christian Szegedy. "Batch normalization: accelerating deep network training by reducing internal covariate shift". In: *Proceedings of the 32nd International Conference on Machine Learning*. 2015, pp. 448–456.

[82] Kaiming He et al. "Deep residual learning for image recognition". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778.

[83] Andrew G. Howard, Menglong Zhu, Bo Chen, et al. *MobileNets: efficient convolutional neural networks for mobile vision applications*. arXiv:1704.04861. 2017.

[84] Mark Sandler et al. "MobileNetV2: inverted residuals and linear bottlenecks". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2018, pp. 4510–4520.

[85] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, et al. "Grad-CAM: visual explanations from deep networks via gradient-based localization". In: *IEEE International Conference on Computer Vision*. 2017, pp. 618–626.

[86] Yuval Netzer, Tao Wang, Adam Coates, et al. "Reading digits in natural images with unsupervised feature learning". In: *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*. 2011.

[87] Patrick Helber et al. "EuroSAT: a novel dataset and deep learning benchmark for land use and land cover classification". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 12.7 (2019), pp. 2217–2226.

[88] Jakob Nikolas Kather, Johannes Krisam, Pornpimol Charoentong, et al. "Predicting survival from colorectal cancer histology slides using deep learning: a retrospective multicenter study". In: *PLOS Medicine* 16.1 (2019), e1002730.

[89] Jiancheng Yang, Rui Shi, Donglai Wei, et al. "MedMNIST v2: a large-scale lightweight benchmark for 2D and 3D biomedical image classification". In: *Scientific Data* 10 (2023), p. 41.

[90] Jeffrey L. Elman. "Finding Structure in Time". In: *Cognitive Science* 14.2 (1990), pp. 179–211.

[91] Paul J. Werbos. "Backpropagation Through Time: What It Does and How to Do It". In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.

[92] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[93] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *Proceedings of ICML*. 2013, pp. 1310–1318.

[94] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[95] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". In: *Proceedings of EMNLP*. 2014, pp. 1724–1734.

[96] Mike Schuster and Kuldip K. Paliwal. "Bidirectional Recurrent Neural Networks". In: *IEEE Transactions on Signal Processing* 45.11 (1997), pp. 2673–2681.

[97] Yarin Gal and Zoubin Ghahramani. "A Theoretically Grounded Application of Dropout in Recurrent Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.

[98] Hans Hersbach et al. "The ERA5 Global Reanalysis". In: *Quarterly Journal of the Royal Meteorological Society* 146.730 (2020), pp. 1999–2049.

[99] Georges Hébrail and Alice Bérard. *Individual Household Electric Power Consumption*. UCI Machine Learning Repository. 2006. URL: https://archive.ics.uci.edu/dataset/235.

[100] Francis X. Diebold and Roberto S. Mariano. "Comparing Predictive Accuracy". In: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[101] Aidan Hogan, Eva Blomqvist, Michael Cochez, et al. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37.

[102] W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. 2014.

[103] Nadime Francis, Alastair Green, Paolo Guagliardo, et al. "Cypher: An Evolving Query Language for Property Graphs". In: *Proceedings of SIGMOD*. 2018, pp. 1433–1445.

[104] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33*. 2020.

[105] Stephen E Robertson and Steve Walker. "Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval". In: *Proceedings of SIGIR*. 1994, pp. 232–241.

[106] Liang Wang, Nan Yang, Xiaolong Huang, et al. *Text Embeddings by Weakly-Supervised Denoising Autoencoder*. arXiv:2202.08904. 2022.

[107] Yu A Malkov and Dmitry A Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836.

[108] Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of SIGIR*. 2009, pp. 758–759.

[109] Darren Edge, Ha Trinh, Newman Cheng, et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16963. 2024.

[110] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023.

[111] Daniel Scott Himmelstein et al. "Systematic integration of biomedical knowledge prioritizes drugs for repurposing". In: *eLife* 6 (2017), e26726.

[112] Zhilin Yang, Peng Xiang, Tao Zhang, et al. "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering". In: *Advances in Neural Information Processing Systems 31*. 2018.

[113] Daniel S. Bernstein et al. "The Complexity of Decentralized Control of Markov Decision Processes". In: *Mathematics of Operations Research* 27.4 (2002), pp. 819–840.

[114] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.

[115] Michael L. Littman. "Markov Games as a Framework for Multi-Agent Reinforcement Learning". In: *Proceedings of the 11th International Conference on Machine Learning*. 1994, pp. 157–163.

[116] John Nash. "Non-Cooperative Games". In: *Annals of Mathematics* 54.2 (1951), pp. 286–295.

[117] Michael E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, 1987.

[118] Anand S. Rao and Michael P. Georgeff. "BDI Agents: From Theory to Practice". In: *Proceedings of the First International Conference on Multiagent Systems*. 1995, pp. 312–319.

[119] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023.

[120] Yilun Du et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv:2305.14325. 2023.

[121] Tian Liang et al. *Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate*. arXiv:2305.19118. 2023.

[122] Foundation for Intelligent Physical Agents. *FIPA ACL Message Structure Specification, SC00061G*. 2002.

[123] LangChain Inc. *LangGraph, README*. URL: https://github.com/langchain-ai/langgraph (visited on 11/05/2026).

[124] Microsoft. *AutoGen, README*. URL: https://github.com/microsoft/autogen (visited on 11/05/2026).

[125] CrewAI Inc. *CrewAI, README*. URL: https://github.com/crewAIInc/crewAI (visited on 11/05/2026).

[126] Karl Cobbe et al. *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168. 2021.

[127] Qiao Jin et al. *PubMedQA: A Dataset for Biomedical Research Question Answering*. arXiv:1909.06146. 2019.

[128] Thomas N. Kipf and Max Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations*. 2017. URL: https://arxiv.org/abs/1609.02907.

[129] Justin Gilmer et al. "Neural Message Passing for Quantum Chemistry". In: *International Conference on Machine Learning*. 2017. URL: https://arxiv.org/abs/1704.01212.

[130] William L. Hamilton, Rex Ying, and Jure Leskovec. "Inductive Representation Learning on Large Graphs". In: *Advances in Neural Information Processing Systems*. 2017. URL: https://arxiv.org/abs/1706.02216.

[131] Petar Veličković et al. "Graph Attention Networks". In: *International Conference on Learning Representations*. 2018. URL: https://arxiv.org/abs/1710.10903.

[132] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. "Revisiting Semi-Supervised Learning with Graph Embeddings". In: *International Conference on Machine Learning*. 2016. URL: https://arxiv.org/abs/1603.08861.

[133] Oleksandr Shchur et al. *Pitfalls of Graph Neural Network Evaluation*. 2018. URL: https://arxiv.org/abs/1811.05868.

[134] Mark Weber et al. *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*. 2019. URL: https://arxiv.org/abs/1908.02591.

[135] Dengyong Zhou et al. "Learning with Local and Global Consistency". In: *Advances in Neural Information Processing Systems*. 2004.

[136] Deli Chen et al. "Measuring and Relieving the Over-Smoothing Problem for Graph Neural Networks from the Topological View". In: *AAAI Conference on Artificial Intelligence*. 2020.

[137] Daniel Scott Himmelstein et al. "Systematic integration of biomedical knowledge prioritizes drugs for repurposing". In: *eLife* 6 (2017).

[138] Michael Schlichtkrull et al. "Modeling Relational Data with Graph Convolutional Networks". In: *European Semantic Web Conference*. 2018.

[139] Học phần Phát triển các Hệ thống Thông minh. *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*. Học viện Công nghệ Bưu chính Viễn thông. 2026.

[140] Christopher J. C. H. Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292.

[141] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. "A Comprehensive Survey of Multiagent Reinforcement Learning". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C* 38.2 (2008), pp. 156–172.